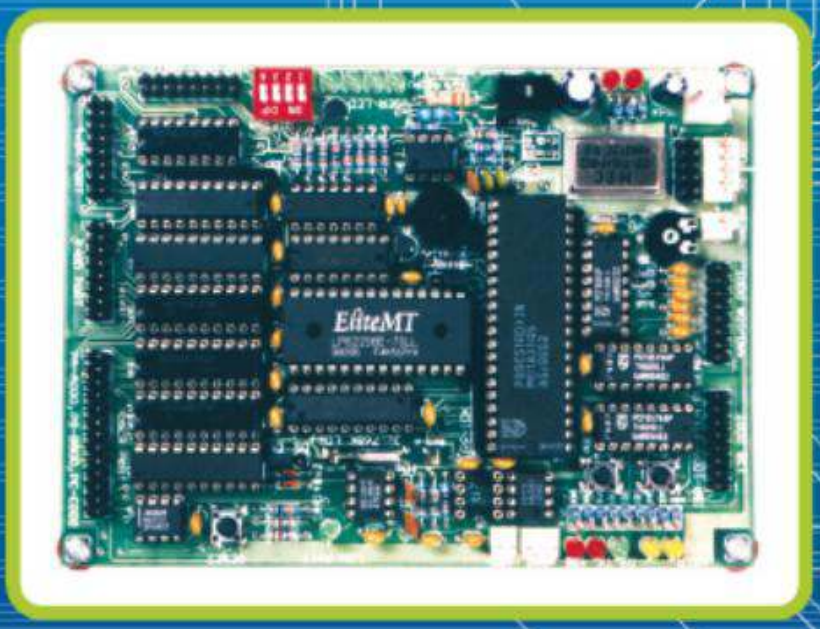
ภาษาแอสเซมบลี
สำหรับ
MCS-51



โดย... รศ.ธีรวัฒน์ ประกอบผล

สำนักพิมพ์ ส.ส.ท.
สมาคมส่งเสริมเทคโนโลยี (ไทย-ญี่ปุ่น)

บทที่ 1
ไมโครคอนโทรลเลอร์ MCS-51

ในปัจจุบันเทคโนโลยีระบบฝังตัว (embedded system) ได้เข้ามามีบทบาทอย่างมากอุปกรณ์ไฟฟ้าอิเล็กทรอนิกส์ต่างๆ ได้มีการเพิ่มหน่วยประมวลผลเข้าภายในตัวมัน ในขณะที่เทคโนโลยีของหน่วยประมวลผลได้มีการพัฒนาอยู่ตลอดเวลา โดยทั่วไปแล้ววงจรควบคุมการทำงานของอุปกรณ์อิเล็กทรอนิกส์ต่าง ๆ อาจอยู่ในรูปของวงจรดิจิตอลทั่ว ๆ ไปก็ได้ แต่ในปัจจุบันจะพบว่าระบบฝังตัวจะใช้ไมโครคอนโทรลเลอร์มากขึ้น เนื่องจากมีความยืดหยุ่นสูง ทำให้การควบคุมการทำงานจะเน้นไปที่การพัฒนาทางซอฟต์แวร์ (software) แทนการพัฒนาทางด้านฮาร์ดแวร์ (hardware) เช่นในอดีต ส่วนใหญ่ผู้พัฒนาเทคโนโลยีระบบฝังตัวจะให้ความสำคัญกับไมโครโพรเซสเซอร์ หรือไมโครคอนโทรลเลอร์เป็นหลัก การแบ่งระดับความซับซ้อนของระบบฝังตัวสามารถแบ่งได้หลายระดับ ระบบฝังตัวที่มีความซับซ้อนมากๆ จะใช้ไมโครโพรเซสเซอร์ที่มีความสามารถสูงเป็นหน่วยประมวลผลกลาง เช่น ตระกูล x86 ของบริษัทอินเทล สำหรับระบบฝังตัวขนาดเล็กจะใช้ไมโครคอนโทรลเลอร์ที่มีการประมวลผลแบบ 4 บิต หรือ 8 บิต ไมโครคอนโทรลเลอร์ที่เหมาะสมกับการศึกษาในเบื้องต้นตัวหนึ่งก็คือไมโครคอนโทรลเลอร์ตระกูล MCS-51

1.1 ภาพรวมของ MCS-51

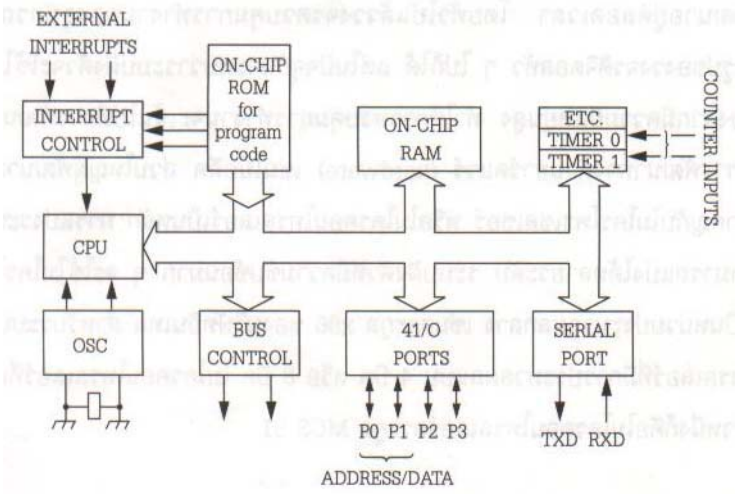
ในปี ค.ศ. 1981 บริษัทอินเทลได้เปิดตัวไมโครคอนโทรลเลอร์แบบ 8 บิต โดยให้ชื่อว่า 8051 โดยไมโครคอนโทรลเลอร์ตัวนี้จะมีหน่วยความจำประเภท RAM ขนาด 128 ไบต์ หน่วยความจำประเภท ROM ขนาด 4 กิโลไบต์ มีพอร์ตขนาด 8 บิต จำนวน 4 พอร์ต มีไทเมอร์ 2 ตัว และพอร์ตอนุกรม 1 พอร์ต โดยทั้งหมดจะรวมอยู่ภายในชิปเดียว ไดอะแกรมภายในตัวมันแสดงได้ดังรูปที่ 1.1 และเนื่องจากหน่วยความจำโปรแกรมเป็นแบบ ROM ที่อยู่ภายในตัวมัน ดังนั้นการโปรแกรมการทำงานต้องโปรแกรมมาจากโรงงานที่ผลิตชิปนี้ออกมาโดยตรง นอกจากนี้บริษัทอินเทลยังได้ผลิตไมโครคอนโทรลเลอร์ออกมาอีกหลายเบอร์ที่มีโครงสร้างภายในใกล้เคียงกับ 8051 โดยเรียกรวม ๆ ว่าตระกูล MCS-51 หรือตระกูล 51 โดยการมีการปรับปรุงโครงสร้างภายใน และเพิ่มองค์ประกอบต่าง ๆ เข้าไปอีก และยังมีบริษัทอื่น ๆ ผลิตตามออกมาด้วยแต่จะใช้เบอร์ที่เรียกแตกต่างกันออกไป

8052

ไมโครคอนโทรลเลอร์เบอร์ 8052 จะมีลักษณะต่างๆ ที่เป็นมาตรฐานเหมือนกับ 8051 แต่จะเพิ่มหน่วยความจะ RAM ภายในเข้าไปอีก 128 ไบต์ และเพิ่มไทเมอร์อีกหนึ่งตัว ทำให้ 8052 มีหน่วยความจำ RAM ภายในทั้งหมด 256 ไบต์ และมีไทเมอร์ 3 ตัว นอกจากนี้ยังมีหน่วยความจำ ROM ภายในเพิ่มเป็น 8 กิโลไบต์ ดังตารางที่ 1.1

8031

ไมโครคอนโทรลเลอร์เบอร์ 8031 จะคล้ายกับ 8051 แต่จะไม่มีหน่วยความจำ ROM ภายในชิป โดยโปรแกรมควบคุมการทำงานจะอยู่ในหน่วยความจำภายนอกเท่านั้น



รูปที่ 1.1 ไดอะแกรมภายในของ 8051

ตารางที่ 1.1

ส่วนประกอบ	8051	8052	8031
หน่วยความจำ ROM ภายใน	4k	8k	0
หน่วยความจำ RAM ภายใน (byte)	128	256	128
Timer	2	3	2
I/O (บิต)	32	32	32
Serial Port	1	1	1
Interrupts Source	6	8	6

8751

จัดเป็นไมโครคอนโทรลเลอร์ตระกูล MCS-51 ตัวหนึ่ง แต่จะมีหน่วยความจำโปรแกรมแบบ UV-EPROM ขนาด 4k byte อยู่ภายในชิป โดยผู้พัฒนาสามารถโปรแกรมลงไปในชิปได้โดยใช่เครื่องโปรแกรม และสามารถแก้ไขโปรแกรมได้โดยการลบโปรแกรมเก่าออกโดยใช้แสง UV

AT89C51

เป็นไมโครคอนโทรลเลอร์ตระกูล MCS-51 ที่ผลิตโดย Atmel Corporation ตัวนี้จะมีหน่วยความจำภายในเป็นแบบ flash memory ทำให้ใช้งานสะดวกกว่า 8751 มาก เนื่องจาก AT89C51 สามารถโปรแกรมและลบดปรแกรมได้ด้วยกระแสไฟฟ้าบนชิปที่ต่ออยู่ในวงจรได้เลย

P89C51RD2

เป็นไมโครคอนโทรลเลอร์ตระกูล MCS-51 ของ Philips Semiconductor โครงสร้างภายในจะคล้ายกับ MCS-51 มาตรฐาน แต่จะมีส่วนเพิ่มเติมขึ้นมาเช่น มีหน่วยความจำภายในเป็นแบบเฟลช และมีการบรรจุ

โปรแกรมบูต รอม (Boot ROM) เข้าไปภายในตัวมัน ทำให้สามารถเขียนหรือลบโปรแกรมได้โดยตรงโดยไม่ต้องถอนชิปออกจากวงจร ที่เรียกว่าการโปรแกรมแบบ ISP (ISP-System Programming)

ในปัจจุบันมีผู้ผลิตไมโครคอนโทรลเลอร์ตระกูล MCS-51 นี้ออกมาหลายบริษัท และมีเบอร์ที่ใช้เรียกแตกต่างกันไป ขึ้นอยู่กับโครงสร้างภายในและเทคโนโลยีที่ใช้ในการสร้างชิป โดยสามารถสรุปได้ดังนี้

803x: ไม่มีหน่วยความจำโปรแกรมภายใน เช่น 8031, 8032 เป็นต้น

805x: มีหน่วยความจำโปรแกรมประเภท ROM ภายใน

8xxx: ใช้เทคโนโลยีแบบ NMOS

8xCxx: ใช้เทคโนโลยีแบบ CMOS

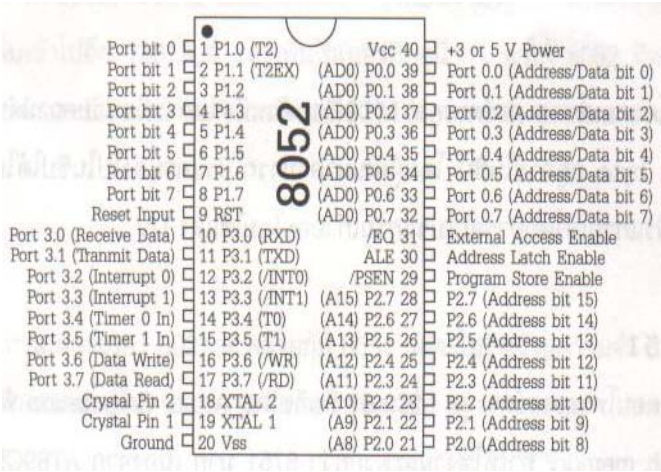
87xx: มีหน่วยความจำโปรแกรมภายในเป็นแบบ EPROM

89xx: มีหน่วยความจำโปรแกรมภายในเป็นแบบ flash EPROM

8xx1: มีหน่วยความจำโปรแกรมภายใน 4 กิโลไบต์ และมี RAM ภายใน 128 ไบต์

8xx2: มีหน่วยความจำโปรแกรมภายใน 8 กิโลไบต์ และมี RAM ภายใน 256 ไบต์

อย่างไรก็ตามแม้ว่าไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะมีอยู่หลายเบอร์ หลายรุ่น แต่ลักษณะการจัดขาของตัวชิปไอซีอาจแตกต่างกันไปบ้าง บางเบอร์มี 40 ขา โดยผู้สนใจสามารถหาศึกษาได้จากคู่มือโดยตรง โดยตำราเล่มนี้กล่าวถึงเทคนิคการเขียนโปรแกรม และการประยุกต์ใช้ และจะใช้ MCS-51 ตัวมาตรฐานเป็นหลักในการอธิบายต่าง ๆ สำหรับลักษณะการจักขาของไมโครคอนโทรลเลอร์เบอร์ 8052 แสดงได้ดังรูปที่ 1.2



รูปที่ 1.2 ลักษณะการจัดขาของ 8052

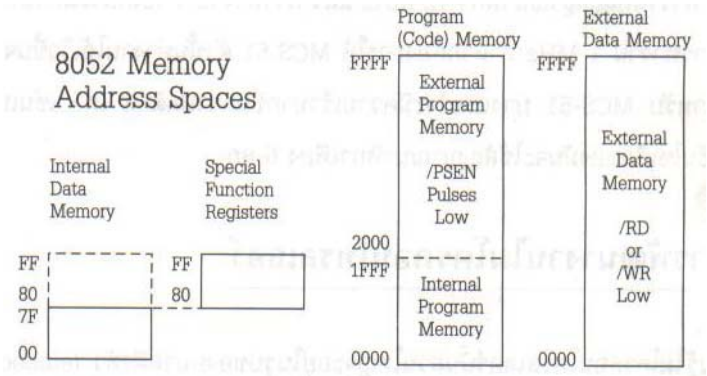
1.2 โครงสร้างหน่วยความจำของ ภาพรวมของ MCS-51

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 ได้ออกแบบการจัดหน่วยความจำเป็นสองส่วน คือ หน่วยความจำโปรแกรม (Program memory) และหน่วยความจำข้อมูล (data memory) โดยทั้งสองส่วนนี้จะมีแอดเดรสแยกออกจากกัน ดังรูป 1.3 สำหรับ MCS-51 บางเบอร์จะมีหน่วยความจำโปรแกรมอยู่ภายในชิป บางเบอร์ต้องต่อเพิ่มภายนอกชิป จากรูป 1.3 จะเห็นว่าแอดเดรสของหน่วยความจำจะมีแอดเดรสตำแหน่งตรงกัน แต่เมื่อ MCS-51 ต้องการติดต่อกับหน่วยความจำในส่วนใดจะใช้ขาสัญญาณต่างกัน อย่างเช่นถ้า MCS-51 ต้องการติดต่อกับหน่วยความจำภายนอก ไซเคิลการทำงานจะเป็นดังนี้

- ถ้าอ่านข้อมูลจากหน่วยความจำโปรแกรมภายนอกขา PSEN จะเป็น Low
- ถ้าอ่านข้อมูลจากหน่วยความจำข้อมูลภายนอกขา RD เป็น Low
- ถ้าเขียนข้อมูลลงหน่วยความจำข้อมูลภายนอกขา WR เป็น Low

หน่วยความจำโปรแกรม

ใน MCS-51 จะมีหน่วยความจำโปรแกรมได้สูงสุด 64k byte หลังจากตัวมันถูกรีเซตจะเริ่มต้นทำงานที่แอดเดรส 0000 ของหน่วยความจำโปรแกรม การอ่านข้อมูลจากหน่วยความจำโปรแกรมนี้จะต้องใช้คำสั่ง MOVC นอกจากนี้ในหน่วยความจำโปรแกรมยังแบ่งแอดเดรสบางส่วนสำหรับเก็บโปรแกรมตอบสนองอินเทอร์รัปต์ เช่นแอดเดรส 0003H เป็นตำแหน่งของโปรแกรมตอบสนองการอินเทอร์รัปต์จากภายนอก (INT0) เมื่อมีสัญญาณอินเทอร์รัปต์เข้ามาทางขานี้ โปรแกรมจะกระโดดไปทำงานยังตำแหน่งนี้ทันที



รูปที่ 1.3 การจัดแอดเดรสหน่วยความจำ

หน่วยความจำข้อมูลภายนอก

หน่วยความจำข้อมูลภายนอกของ MCS-51 สามารถมีได้ 64k Byte เมื่อ MCS-51 ต้องการติดต่อกับหน่วยความจำส่วนนี้จะต้องใช้คำสั่ง MOVX และรีจิสเตอร์ DPTR, R0 หรือ R1 ในการอ้างตำแหน่งหน่วยความจำ ในการขยายพอร์ตเพิ่มให้กับ MCS-51 จะต้องใช้ตำแหน่งในส่วนนี้เป็นตำแหน่งของพอร์ตด้วย

หน่วยความจำข้อมูลภายใน

หน่วยความจำข้อมูลภายในของ MCS-51 จะมีจำนวน 256 ไบต์ โดย 128 ไบต์แรกที่มีแอดเดรสอยู่ในช่วง 00H-7FH จะเป็นส่วนของ RAM ที่ใช้งานทั่งไป รีจิสเตอร์แบงค์จำนวน 4 แบงค์ หน่วยความจำที่สามารถเข้าถึงข้อมูลระดับบิตได้ ส่วนอีก 128 ไบต์หลังเริ่มต้นตั้งแต่แอดเดรส 80H เป็นต้นไปจะเป็นส่วนของรีจิสเตอร์ฟังก์ชันพิเศษ หรือ SFR (Special Function Register) และรีจิสเตอร์ที่ใช้งานทั่วไปเช่น รีจิสเตอร์ ACC, B, PSW, SP และ DPTR เป็นต้น การอ่านหรือเขียนข้อมูลกับหน่วยความจำในส่วนนี้จะใช้คำสั่ง MOV

1.3 ความเร็วในการทำงานของ MCS-51

การให้ไมโครคอนโทรลเลอร์ทำงานนั้นจะต้องโปรแกรมให้กับตัวมันก่อน การวัดความเร็วในการทำคำสั่งของโปรแกรมจะดูจากรอบสัญญาณนาฬิกา หรือที่เรียกว่าแมชชีนไซเคิล ซึ่งในตารางคำสั่งไมโครคอนโทรลเลอร์แต่ละตัวจะมีข้อมูลบอกไว้ว่าการทำคำสั่งจะใช้สัญญาณนาฬิกากี่แมชชีนไซเคิล สำหรับไมโครคอนโทรลเลอร์ MCS-51 เบอร์ที่เป็นมาตรฐานนั้น 1 แมชชีนไซเคิลจะใช้สัญญาณนาฬิกา 12 ลูก ดังนั้นถ้า MCS-51 ทำงานที่สัญญาณนาฬิกา 12 Hz แล้วการทำงาน 1 ไซเคิลจะมีค่าเท่ากับ 1 ไมโครวินาทีหรือมีความเร็วในการทำงาน 1 MHz ถ้าหากต้องการให้ MCS-51 ตัวนั้นทำงานได้เร็วขึ้นจะต้องเพิ่มสัญญาณนาฬิกาให้กับมัน สำหรับ MCS-51 บางเบอร์จะมีความเร็วมากขึ้นกว่าปกติ 2 เท่า เช่น เบอร์ P89C51RD2 เนื่องจากหนึ่งแมชชีนไซเคิลของมันจะใช้สัญญาณนาฬิกาเพียง 6 ลูก

1.4 การพัฒนางานไมโครคอนโทรลเลอร์

การเรียนรู้ไมโครคอนโทรลเลอร์นั้นส่วนใหญ่จะอยู่ในรูปของเบอร์ฝังตัว (Embedded board) จากนั้นก็พัฒนาโปรแกรมสำหรับใช้งานลงไป ดดยบอร์ดที่ใช้ศึกษาอาจอยู่ในรูปของซิงเกิลบอร์ด (Single board) ซึ่งมีความสะดวกในการพัฒนาโปรแกรมและแก้ไขโปรแกรมรวมทั้งมีโปรแกรมย่อยในรูปของบอร์ด System Call ให้เรียกใช้ด้วย บอร์ดประเภทนี้จะมีอุปกรณ์แสดงผล และคีย์บอร์ดต่ออยู่ด้วย ซึ่งเหมาะสำหรับผู้ที่เริ่มต้นศึกษาทางด้านไมโครคอนโทรลเลอร์ การโปรแกรมสามารถทำได้โดยป้อนออปโค้ดของโปรแกรมในรูปของเลขฐานสิบหกลงไป และยังสามารถเขียนโปรแกรมบนคอมพิวเตอร์ PC จากนั้นใช้โปรแกรมแปลภาษาให้เป็นออปโค้ดที่เรียกว่าการทำแอสเซมเบลอร์ (assembler) และโหลดโปรแกรมที่ได้ลงบนบอร์ดเพื่อใช้งานต่อไป

สำหรับการพัฒนาด้วยอีพรอมอีมูเลเตอร์ (EPROM emulator) ก็เป็นอีกวิธีหนึ่งที่ใช้งานกันมานาน โดยที่บอร์ดไมโครคอนโทรลเลอร์จะอยู่ในรูปของ embedded board ซึ่งราคาถูกกว่าแบบซิงเกิลบอร์ด โดยจะเขียนโปรแกรมบนคอมพิวเตอร์ PC และนำรหัสโปรแกรมที่ได้โปรแกรมลงบนอีพรอมอีมูเลเตอร์ แทนที่จะโปรแกรมลงหน่วยความจำ ROM จริง ๆ วิธีนี้จะช่วยให้พัฒนาโปรแกรมได้ง่าย เนื่องจากการแก้ไข โปรแกรมไม่ต้องใช้ตัวล้างหน่วยความจำ ถ้าหากโปรแกรมที่พัฒนาเป็นโปรแกรมที่ใช้ได้จริงไม่มีการแก้ไขส่วนใดอีกแล้วก็จะนำรหัสโปรแกรมที่ได้โปรแกรมลงหน่วยความจำ ROM จริง ๆ เพื่อใช้งานต่อไป

ในปัจจุบันได้มีการพัฒนาระบบไมโครคอนโทรลเลอร์ด้วยเครื่องโปรแกรมแบบ Single chip โดยใช้ไมโครคอนโทรลเลอร์ที่มีหน่วยความจำภายในเป็นแบบแฟลช (flash memory) และโปรแกรมข้อมูลลงไปภายในตัวชิปเลย โดยชิปประเภทนี้สามารถโปรแกรมและลบโปรแกรมได้มากกว่า 1,000 ครั้ง นอกจากนี้ยังได้มีการพัฒนาไมโครคอนโทรลเลอร์ในรูปแบบของ ISP (In-System Programming) โดยไม่ต้องใช้เครื่องดปรแกรมลงบนชิป แต่จะโหลดโปรแกรมเข้าไปทางพอร์ตอนุกรมของชิปเลย แต่ราคาของชิปจะมีราคาสูง

ในไมโครคอนโทรลเลอร์ตระกูล MCS-51 ที่สามารถพัฒนาแบบ ISP ได้หลายเบอร์ เช่น AT89S8252 ของ Atmel, PIC16F84 ของ Microchip และตัวที่กำลังได้รับความนิยมในขณะนี้คือ P89C51RD+, 89C51RD2 ของบริษัท Philips

บทที่ 2
การเขียนโปรแกรมด้วยภาษาแอสเซมบลี

การเขียนโปรแกรมภาษาแอสเซมบลีให้กับไมโครคอนโทรลเลอร์นั้นจะทำได้โดยการนำรหัสนีโมนิก (Mnemonics) ของชุดคำสั่งมาต่อเรียงกัน ไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีชุดคำสั่งทั้งหมด 111 คำสั่ง โดยแบ่งออกได้ดังนี้

1. ชุดคำสั่งทางคณิตศาสตร์ 24 คำสั่ง โดยมีรหัสนีโมนิกเป็น ADD, ADDC, SUBB, INC, DEC, MUL, DIV, DA

2. ชุดคำสั่งทางลอจิก 25 คำสั่ง โดยมีรหัสนีโมนิกเป็น ANL, ORL, XRL, CLR, CPL, RL, RR, RRC, SWAP

3.ชุดคำสั่งการโอนย้ายข้อมูล 28 คำสั่ง โดยมีรหัสนีโมนิกเป็น MOV, MOVX, MOVC, PUSH, POP, XCH, XCHD

4. ชุดคำสั่งดำเนินการระดับบิต 17 คำสั่ง โดยมีรหัสนีโมนิกเป็น CLR, SETB, CPL, ANL, ORL, MOV, JC, JNC, JB, JBC

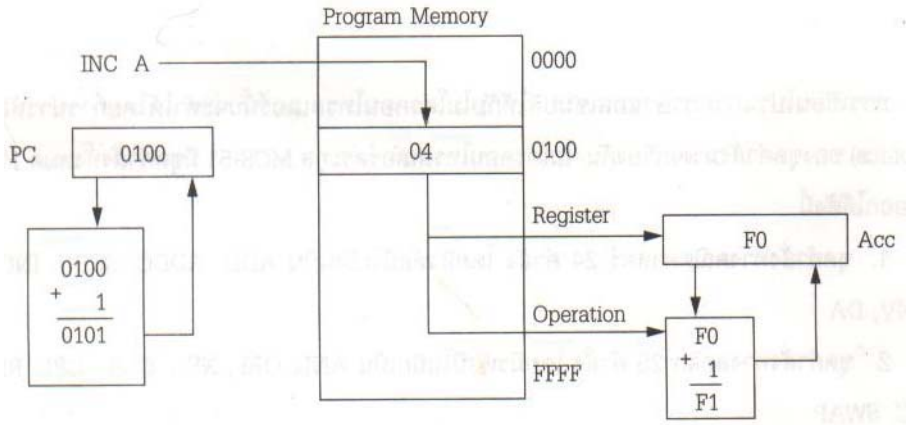
5. ชุดคำสั่งควบคุมการทำงานของโปรแกรม 17 คำสั่ง โดยมีรหัสนีโมนิกเป็น ACALL, LCALL, RET, RETI, AJMP, LJMP, SJMP, JMP, JZ, JNZ, CJNE, DJNZ, NOP

2.1 Addressing Modes

การที่ CPU จะนำข้อมูลมาประมวลผลนั้นสามารถทำได้หลายวิธี วิธีการนำข้อมูลที่อยู่ในตำแหน่งต่าง ๆ เข้ามาเรียกว่าโหมดการอ้างแอดเดรส (addressing modes) สำหรับไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะมีโหมดการอ้างแอดเดรสอยู่ 8 โหมด ได้แก่ Register addressing mode, direct addressing mode, Immediate addressing mode, Indirect addressing mode, Based-indexed indirect addressing mode, Relative addressing mode, Extended addressing mode และ Implied addressing mode

2.1.1 Register Addressing Mode

เป็นการอ้างแอดเดรสโดยใช้รีจิสเตอร์ ข้อมูลที่นำมาประมวลผลจะเป็นข้อมูลที่ถูกเก็บอยู่ในรีจิสเตอร์โดยตรง โดย MCS-51 ได้ออกแบบมาให้ประมวลผลกับรีจิสเตอร์แอกคิวมูเลเตอร์ รีจิสเตอร์ B รีจิสเตอร์ DPTR และรีจิสเตอร์ R0 ถึง R7 ที่เก็บในแบงค์ที่กำลังทำงานอยู่ สำหรับรูปที่ 2.1 เป็นตัวอย่างการทำคำสั่ง INC A ซึ่งจะใช้การอ้างแอดเดรสแบบ Register addressing mode

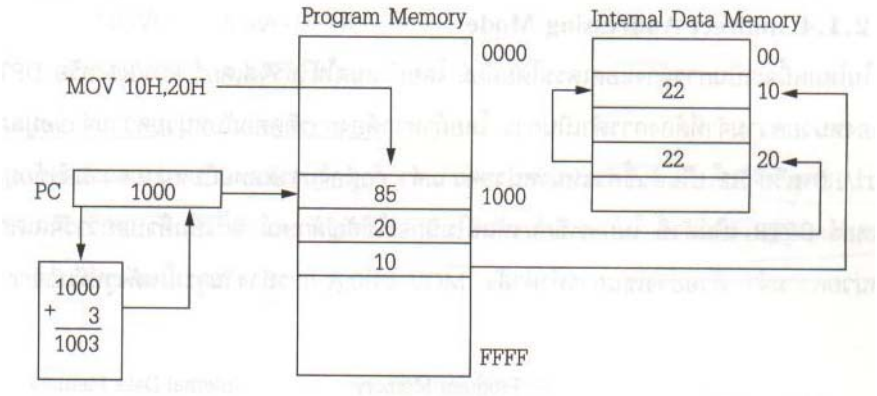


รูปที่ 2.1 การอ้างแอดเดรสแบบ Register addressing mode

ในรูปที่ 2.1 เป็นคำสั่งเพิ่มค่าในรีจิสเตอร์ A (INC A) โดยมีรหัสคำสั่งภาษาเครื่องขนาดหนึ่งไบต์เป็น 04H จากรูปจะเป็นว่าถ้าคำสั่งนี้อยู่ที่แอดเดรส 0100 และในรีจิสเตอร์ A มีค่า F0H อยู่ เมื่อ MCS-51 ทำคำสั่งนี้มันจะเพิ่มค่าใน PC ขึ้นหนึ่งค่าเพื่อชี้คำสั่งถัดไป จากนั้นจะเพิ่มค่าในรีจิสเตอร์ A ขึ้นหนึ่งค่าเป็น F1H และเก็บไว้ในรีจิสเตอร์ A ตามเดิม

2.1.2 Direct Addressing Mode

เป็นโหมดการอ้างแอดเดรสที่ติดต่อกับข้อมูลที่อยู่ในหน่วยความจำโดยตรง ตัวอย่างเช่นการทำคำสั่ง MOV 10H.20H จะเป็นการอ่านไบต์ข้อมูลที่อยู่ในตำแหน่งแอดเดรส 20H ดังรูปที่ 2.2

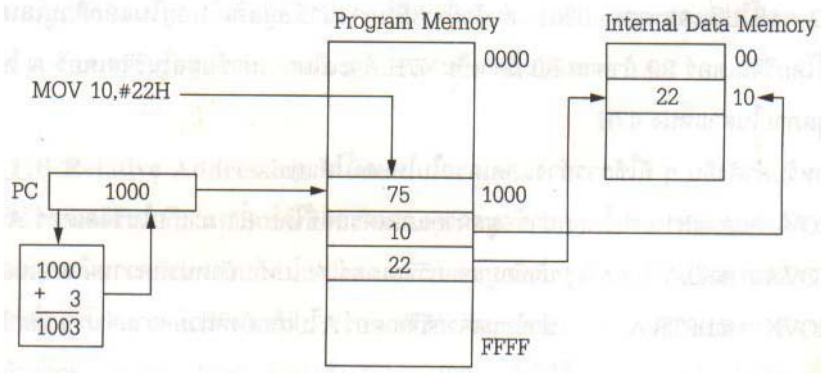


รูปที่ 2.2 การอ้างอิงแอดเดรสแบบ Direct Addressing Mode

รหัสคำสั่งของ MOV 10H.20H จะมีขนาด 3 ไบต์ โดยไบต์แรกเป็นออปโค้ด 85H อีกสองไบต์ต่อมาเป็นแอดเดรสข้อมูลต้นทางและแอดเดรสข้อมูลปลายทาง ในที่นี้คือ 20H และ10H ถ้าหากคำสั่งนี้อยู่ในตำแหน่ง 1000 เมื่อ MCS-51 ทำคำสั่งนี้ รีจีสเตอร์ PC จะไปชี้ตำแหน่ง 1003H ซึ่งเป็นแอดเดรสของคำสั่งถัดไป

2.1.3 Immediate Addressing Mode

การอ้างอิงแบบนี้ข้อมูลที่นำมาประมวลผลจะอยู่ในรหัสคำสั่งโดยตรง การเขียนรหัสนีโมนิกจะใช้เครื่องหมาย # (pound sign) นำหน้าของข้อมูล เช่น MOV 10H#22H จะเป็นการนำข้อมูล 22H มาเก็บในแอดเดรส 10H พิจารณารูปที่ 2.3

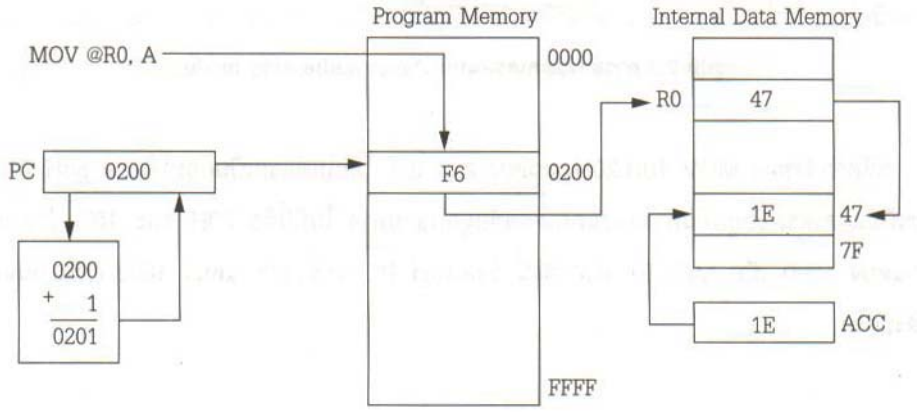


รูปที่ 2.3 การอ้างอิงแอดเดรสแบบ Immediate Addressing Mode

จากรูปที่ 2.3 จำพบว่าคำสั่ง MOV 10H.20H เป็นคำสั่งที่มีขนาด 3 ไบต์ โดยไบต์แรกเป็นออปโค้ดมีค่าเป็น 75 ไบต์ต่อมาเป็นตำแหน่งของข้อมูลปลายทาง ไบต์สุดท้ายเป็นค่าของข้อมูลที่จะนำมาดำเนินการ หลังจาก MCS-51 ทำคำสั่งนี้ PC จะชี้ที่แอดเดรส 1003 ซึ่งเป็นแอดเดรสของคำสั่งถัดไป

2.1.4 Indirect Addressing Mode

ในโหมดนี้จะเป็นการอ้างแอดเดรสโดยอ้อม โดยกำหนดให้ใช้รีจีสเตอร์ R0, R1 หรือ DPTR เก็บค่าตำแหน่งของหน่วยความจำที่ต้องการดำเนินการ โดยถ้าหากต้องการติดต่อกับหน่อยความจำข้อมูลภานใน จะใช้ รีจีสเตอร์ R0, R1 เป็นตัวชี้ตำแหน่งหน่วยความจำ ถ้าต้องการติดต่อกับหน่อยความจำข้อมูลภายนอก จะใช้รีจีสเตอร์ DPTR เป็นตัวชี้ ในการเขียนรหัสนีโมนิกจะใช้สัญลักษณ์ @ เป็นตัวบอกว่ารีจีสเตอร์นี้เก็บค่าตำแหน่งหน่วยความจำ ตัวอย่างเช่นการทำคำสั่ง MOV @R0, A การทำงานเป็นดังรูปที่ 2.4



รูปที่ 2.4 การอ้างอิงแอดเดรสแบบ Indirect Addressing Mode

จากรูปที่ 2.4 จะเห็นว่าคำสั่ง MOV @R0, A จำมีขนาด 1 ไบต์ โดยมีรหัสคำสั่งเป็น F6 หลังจากทำคำสั่ง PC จำชี้ไปที่แอดเดรส 0201 คำสั่งนี้จะเป็นการนำข้อมูลที่เก็บอยู่ในแอกคิวมูเลเตอร์ไปเก็บในแอดเดรสที่ชี้โดยรีจีสเตอร์ R0 ถ้าหาก R0 มีค่าเป็น 47H ก็จะเป็นการนำข้อมูลในรีจีสเตอร์ A ไปเก็บในหน่วยความจำข้อมูลภายในตำแหน่ง 47H

สำหรับคำสั่งอื่นๆที่ใช้การอ้างอิงแอดเดรสในโหมดนี้ได้แก่

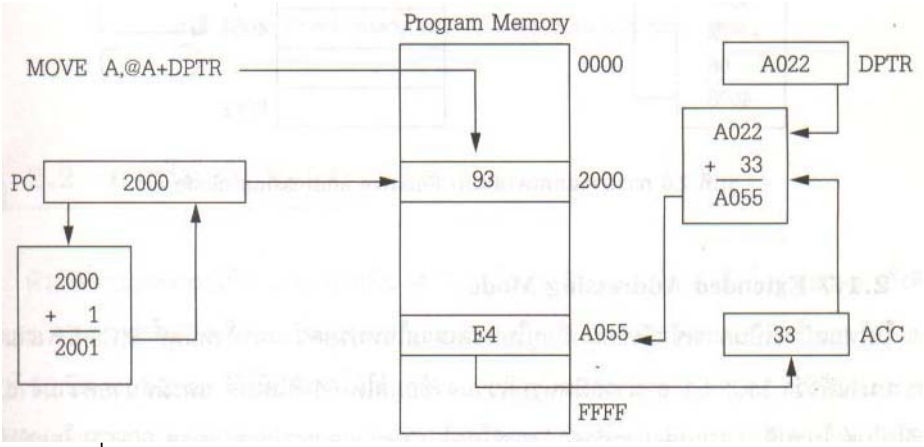
MOV	A, R1	นำข้อมูลจากแอดเดรสที่ชี้โดย R1 มาเก็บในรีจีสเตอร์ A
MOVX	@R0, A	นำข้อมูลจากรีจีสเตอร์ A ไปเก็บยังหน่วยความจำภายนอกที่ชี้โดย R0
MOVX	@DPTR, A	นำข้อมูลจากรีจีสเตอร์ A ไปเก็บยังหน่วยความจำภายนอกที่ชี้โดย DPTR

2.1.5 Based-Indexed Indirect Addressing Mode

การอ้างอิงแอดเดรสในโมดนี้ขะใช้รีจีสเตอร์สองตัวในการอ้างอิงแอดเดรส โดยจะอ่านข้อมูลออกมาในลักษณะของการเปิดตาราง(Look-up tables) ซึ่งจะมีคำสั่งอยู่สองรูปแบบคือ

```
MOVC   A,@A+DPTR
MOVC   A,@A+PC
```

การคำสั่งจะเห็นว่าเป็นการอ่านข้อมูลจากหน่วยความจำโปรแกรมมาเก็บในรีจีสเตอร์ A โดยใช้รีจีสเตอร์ DPTR หรือ PC เป็นตัวชี้ตำแหน่งเริ่มต้นของตารางจากนั้นนำค่าใน A เป็นตัวระบุตำแหน่งของข้อมูลที่อยู่ในตารางนั้น ดังแสดงในรูปที่ 2.5

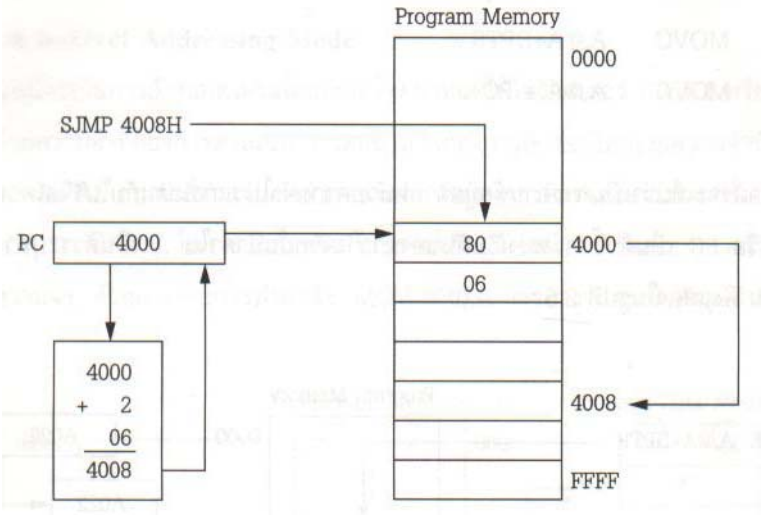


รูปที่ 2.5 การอ้างอิงแอดเดรสแบบ Based-Indexed Indirect Addressing Mode

จากรูปที่ 2.5 จะเห็นว่าการทำตำสั่ง MOVC A,@A+DPTR นั้นจะนำข้อมูลที่รีจีสเตอร์ A และ DPTR ชี้อยู่มาเก็บในรีจีสเตอร์ A โดยให้ DPTR ชี้ที่ตำแหน่งเริ่มต้นแอดเดรส A022H และถ้าใน ACC มีค่าเป็น 33H จะเป็นการนำข้อมูลจากแอดเดรส A055H มาเก็บในรีจีสเตอร์ A

2.1.6 Relative Addressing Mode

การอ้างแอดเดรสประเภทนี้จะใช้กับคำสั่งควบคุมการทำงานของโปรแกรม เช่นคำสั่งกระโดด คำสั่งเรียกโปรแกรมย่อย โดยจะเป็นการระบุตำแหน่งหน่วยความจำที่อยู่ด้านหน้า หรือด้านหลังของตำแหน่งที่รีจีสเตอร์ PC ชี้อยู่ โดยนำค่าในรีจีสเตอร์ PC บวกกับค่าตัวเลขแบบ 2's complement พิจารณาตัวอย่างในรูปที่ 2.6 ถ้าหากที่แอดเดรส 4000 เก็บคำสั่งSJMP 4008 เพื่อให้โปรแกรมกระโดดไปยังแอดเดรส 4008 โดยคำสั่งนี้จะมีรหัสคำสั่งขนาด 2 ไบต์ ไบต์แรกเป็นออปโค้ดมีค่าเป็น 80H เพื่อบอกว่าเป็นคำสั่งกระโดด ส่วนไบต์ต่อไปเป็นการอ้างแอดเดรสแบบสัมพัทธ์ (relative) มีค่าเป็น 06H เมื่อ MCS-51 ทำงานจะนำค่า 06H นี้ไปบวกกับค่าที่ PC ชี้อยู่ในขณะนั้น ซึ่งจะได้เป็น 4000H+2+06H=4008H ซึ่งจะเป็นคำสั่งต่อไปที่ MCS-51 จะกระโดดไปทำงาน



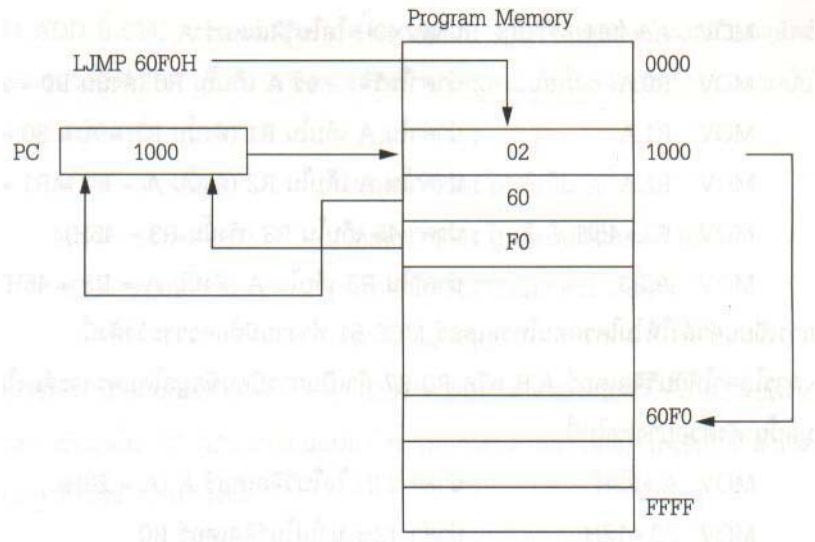
รูปที่ 2.6 การอ้างอิงแอดเดรสแบบ Relative Addressing Mode

2.1.7 Extended Addressing Mode

ในโหมดนี้จะเป็นการเข้าถึงข้อมูลที่อยู่ในแอดเดรสในหน่วยความจำทั้งหมดที่ MCS-51 สามารถมีหน่วยความจำข้อมูลได้ 64กิโลไบต์ และมีหน่วยความจำโปรแกรมได้ 64 กิโลไบต์ ในรูปที่ 2.7 แสดงการทำคำสั่งกระโดดไปยังหน่วยความจำแอดเดรส 60F0H โดยคำสั่งนี้จะมีรหัสคำสั่งจำนวน 3 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด อีกสองไบต์ต่อมาจะเป็นแอดเดรสไบต์สูงและแอดเดรสไบต์ต่ำ

2.1.8 Implied Addressing Mode

ในโหมดนี้จะไม่มีการอ้างตำแหน่งของข้อมูล หมายถึงคำสั่งที่ไม่มีโอเปอแรนด์(Operands) เช่น คำสั่ง NOP เป็นต้น



รูปที่ 2.7 การอ้างอิงแอดเดรสแบบ Extended Addressing Mode

2.2 ภายใน MCS-51

หัวข้อนี้จะแสดงการใช้รีจีสเตอร์ภานใน MCS-51 โดยจะพิจารณาจากคำสั่งพื้นฐานสองคำสั่งคือ คำสั่ง MOV และ ADD ภายใน MCS-51 รีจีสเตอร์ภายในจะมีขนาด 8 บิต ส่วนรีจีสเตอร์ที่ใช้ในการอ้างอิงตำแหน่งจะมีขนาด 16 บิต โดยรีจีสเตอร์ที่ใช้งานทั่วไปได้แก่ A (แอกคิวมูเลเตอร์) ,B,R0,R1,R2,R3,R4,R5,R6,R7,DPTR(data pointer) และ PC (program counter) โดยรีจีสเตอร์ทุกตัวจะเป็นแบบ 8 บิตยกเว้น DPTR และ PC ที่เป็นแบบ 16 บิต ตัวแอกคิวมูเลเตอร์ A จะใช้ได้กับคำสั่งทางคณิตศาสตร์และลอจิก

คำสั่ง MOV

คำสั่งนี้จำใช้สำเนาข้อมูลจากตำแหน่งต้นทางไปยังตำแหน่งปลาบทาง รูปแบบของคำสั่งเป็นดังนี้

MOV destination, source

คำสั่ง MOV นี้จะใช้ในการโอนข้อมูลภานใน CPU เท่านั้น ตัวอย่างเช่น MOV A, R0 จะเป็นการย้ายข้อมูลที่เก็บใน R0 ไปเก็บในรีจีสเตอร์ A หลังจากทำคำสั่งแล้วใน A จะเท่ากับค่าใน R0 โอเปอแรนด์ต้นทางของคำสั่งนี้สามารถเป็นตำแหน่งของหน่วยความจำภานในโดยตรงก็ได้เช่นกัน MOV A, 50H หมายความว่าให้คัดลอกค่าที่เก็บในหน่วยความจำภายในตำแหน่ง 50H มาเก็บในรีจีสเตอร์ A หรือจะใส่เป็นข้อมูลโดยตรงก็ได้ โดยเครื่องหมาย # นำหน้าข้อมูลนั้น ตัวอย่างการใช้คำสั่ง MOV มีดังนี้

```
MOV   A,#40H      ; นำค่า 40H ใส่ในรีจิสเตอร์ A
MOV   R0,A        ; นำค่าในรีจิสเตอร์ A เก็บใน R0 (ดังนั้น R0=A=4 0H)
MOV   R1,A        ; นำค่าใน A เก็บใน R1 (ดังนั้น R1=A=R0=40H)
MOV   R2,A        ; นำค่าใน A เก็บใน R2 (ดังนั้น A=R0=R1=R2=40H)
MOV   R3,#45H     ; นำค่า 45 เก็บใน R3 (ดังนั้น R3=45H)
MOV   A,R3        ; นำค่าใน R3 เก็บใน A (ดังนั้น A=R3=45H)
```

ในการเขียนคำสั่งให้ไมโครคอนโทรลเลอร์ MCS-51 ทำงานมีข้อระวังดังนี้

1. การใส่ค่าให้กับรีจิสเตอร์ A,B หรือ R0-R7 ถ้าเป็นการป้อนข้อมูลโดยตรงจะต้องใส่เครื่องหมาย # นำหน้าข้อมูลนั้น ดังตัวอย่างต่อไปนี้

```
MOV   A,#23H      ; นำค่า 23H ใส่ในรีจิสเตอร์ A (A=23H)
MOV   R0,#12H     ; นำค่า 12H มาใส่ในรีจิสเตอร์ R0
MOV   R1,#1FH     ; นำค่า 1FH มาใส่ในรีจิสเตอร์ R1
MOV   R2,#2BH     ; นำค่า 2BH มาเก็บในรีจิสเตอร์ R2
MOV   B,#3CH      ; นำค่า 3CH มาเก็บในรีจิสเตอร์ B
MOV   R7,#9DH     ; นำค่า 9DH มาเก็บในรีจิสเตอร์ R7
MOV   R5,#0F9H    ; นำค่า F9H มาเก็บในรีจิสเตอร์ R5
MOV   R6,#12      ; นำค่า 12ฐานสิบ หรือ 0CH มาเก็บในรีจิสเตอร์ R6
```

จะสังเกตเห็นว่าในคำสั่ง MOV R5,#0F9H จะต้องใส่เลข 0 ตามหลัง # เนื่องจากเลขฐานสิบหก ค่าแรกเป็น F ถ้าเขียนเป็น MOV R5,#F9H จะเกิดข้อผิดพลาด

2. ถ้าหากใส่ค่า 0 ถึง F ลงในรีจิสเตอร์ 8 บิต สามารถเขียนเป็นเลขหลักเดียว ไม่ต้องมี 0 นำหน้าก็ได้ เช่น MOV A,#5 จะทำให้ค่าใน A=05 ข้อมูลในรูปแบบของเลขฐานสองในรีจิสเตอร์ A จะเป็น 00000101

3. ถ้าหากนำค่าข้อมูลที่โตกว่าที่รีจิสเตอร์สามารถจะเก็บได้ไปเก็บในรีจิสเตอร์ จะทำให้เกิดข้อผิดพลาด ตัวอย่างเช่น

```
MOV   A,#7F2H     ; ค่า 7F2H > 8 บิต (FFH)
MOV   R2,#456     ; ค่า 456 > 255 (FFH)
```

4. ในการป้อนข้อมูลเข้าในรีจิสเตอร์ ถ้าหากไม่มีเครื่องหมาย # จะหมายถึงเป็นการนำข้อมูลจากหน่วยความจำ เช่น MOV A,17H หมายความว่านำข้อมูลที่เก็บอยู่ในหน่วยความจำตำแหน่ง 17H มาเก็บใน A ซึ่งค่าจะไม่เท่ากับ MOV A,#17H

คำสั่ง ADD

คำสั่งนี้ใช้สำหรับการบวกเลข มีรูปแบบดังนี้

```
ADD     A,source        ; บวกค่าจากโอเปอแรนด์ต้นทางกับค่าในแอกคิวมูเลเตอร์
                          และเก็บผลลัพธ์แอกคิวมูเลเตอร์
```

คำสั่ง ADD นี้ CPU จะบวกข้อมูลจากตำแหน่งต้นทางกับรีจิสเตอร์ A และเก็บผลลัพธ์ในรีจิสเตอร์ A ถ้าหากต้องการบวกค่า 25H กับ 34H จะต้องย้ายข้อมูลไปเก็บในรีจิสเตอร์ก่อน จากนั้นใช้คำสั่งบวก ซึ่งสามารถเขียนคำสั่งได้ดังนี้

```
MOV     A,#25H          ; โหลดค่า 25H ไปเก็บใน A
MOV     R2,#34H         ; โหลดค่า 35H ไปเก็บในรีจิสเตอร์ R2
ADD     A,#R2           ; บวกค่าใน R2 กับแอกคิวมูเลเตอร์
                        ; (A = A + R2)
```

จากการทำคำสั่งผลลัพธ์ที่ได้ A = 59H (25H + 34H = 59H) และค่าใน R2 จะเท่ากับ 34H โดยหลังจากการบวกค่าข้อมูลใน R2 จะไม่เปลี่ยนแปลงการบวกเลขดังกล่าวสามารถเขียนคำสั่งได้หลายแบบขึ้นอยู่กับจำนวนรีจิสเตอร์ที่จะให้ ตัวอย่างเช่น

```
MOV     R5,#25H         ; โหลดค่า 25H ใส่ใน R5 (R5 = 25H)
MOV     R7,#34H         ; โหลดค่า 34H ใส่ใน R7 (R7 = 34H)
MOV     A,#0            ; โหลดค่า 0 ลงใน A (A = 0)
ADD     A,R5            ; บวกค่าใน A ด้วยค่าใน R5
                        ; A = A + R5
ADD     A,R7            ; บวกค่าใน A ด้วยค่าใน R7
                        ; A = A + R7
```

ในการเขียนคำสั่ง ADD นี้สามารถใช้โอเปอแรนด์ที่มีการอ้างแอดเดรสแบบ immediate ได้ เช่นการเขียนคำสั่งต่อไปนี้

```
MOV     A,#25H          ; A = 25H
ADD     A,#34H          ; บวกค่าใน A กับ 34H ผลลัพธ์เก็บใน A
```

ตำแหน่งปลายทางของคำสั่ง ADD ต้องเป็นรีจิสเตอร์ A เท่านั้น ถ้าเป็นคำสั่ง ADD R2, #12H จำทำงานไม่ได้

2.3 การเขียนโปรแกรมภาษาแอสเซมบลีเบื้องต้น

ไมโครคอนโทรลเลอร์หรือ CPU สามารถทำงานได้กับคำสั่งที่อยู่ในรูปของเลขฐานสองเท่านั้น โปรแกรมที่ประกอบด้วยเลขฐานสองคือลอจิก “ 0 ” และ “ 1 ” เรียกว่าเป็นภาษาเครื่อง (machine language) แต่การเขียนโปรแกรมในลักษณะนี้จะใช้ได้ดีกับการเขียนโปรแกรมสั้นๆ ทุกวันนี้การเขียนชุดคำสั่งต่างๆ ให้กับ CPU จะใช้รหัสช่วยจำ (mnemonic) แทนเลขฐานสองแต่ละคำสั่ง เรียกว่าการเขียนโปรแกรมภาษาแอสเซมบลี เนื่องจากสามารถเข้าใจโปรแกรมได้ง่าย เมื่อจะให้โปรแกรมทำงานจะต้องแปลภาษาแอสเซมบลีให้เป็นรหัสภาษาเครื่องอีกที ขั้นตอนนี้เรียกว่า แอสเซมเบเลอร์ (assembler)

ภาษาแอสเซมบลีถือได้ว่าเป็นภาษาระดับต่ำ (Low-level language) เนื่องจากสามารถเข้าถึงโครงสร้างภายใน CPU ได้เป็นอย่างดี การเขียนโปรแกรมด้วยภาษานี้ ผู้เขียนโปรแกรมจะต้องเข้าใจส่วนประกอบภายใน CPU และรีจิสเตอร์ต่างๆ ของ CPU เป็นอย่างดี

การเขียนโปรแกรมให้กับไมโครคอนโทรลเลอร์ในปัจจุบันสามารถเขียนด้วยภาษาระดับสูงได้ เช่น ภาษา BASIC หรือภาษาซี แต่ถ้าจะให้ไมโครคอนโทรลเลอร์ทำงานจะต้องแปลภาษาเหล่านี้ให้เป็นรหัสภาษาเครื่องอีกทีหนึ่ง โดยใช้ตัวคอมไพลเลอร์ (compiler) เป็นตัวแปรภาษา สำหรับในหัวข้อนี้จะกล่าวถึงรูปแบบทั่ว ๆไปในการเขียนโปรแกรมภาษาแอสเซมบลีให้กับไมโครคอนโทรลเลอร์ตระกูล MCS-51

โครงสร้างของภาษาแอสเซมบลี

การเขียนโปรแกรมภาษาแอสเซมบลีจะทำได้โดยนำรหัสนีโมนิกของแต่ละคำสั่งมาต่อเรียงกัน รหัสนีโมนิกแต่ละตัวอาจมีโอเปอแรนด์หรือไม่มีก็ได้ และการเขียนโปรแกรมด้วยภาษาแอสเซมบลีนี้ สามารถนำคำสั่งเทียมมาใช้ได้ ซึ่งจะทำให้การเขียนโปรแกรมทำได้ง่ายขึ้น ตัวอย่างการเขียนโปรแกรมภาษาแอสเซมบลีแสดงได้ดังโปรแกรม 2.1

```
            ORG   0000H      ; เริ่มต้นที่แอดเดรส 0000
            MOV   R5,#25H    ; โหลดค่า 25H ใส่ใน R5
            MOV   R7,#34H    ; โหลดค่า 34ใส่ใน R7
            MOV   A,#0       ; โหลดค่า 0 ใส่ในรีจิสเตอร์ A
            ADD   A,R5       ; บวกค่า R5 กับ A ผลลัพธ์เก็บใน A
            ADD   A,R7       ; บวกค่า R7 กับ A ผลลัพธ์เก็บใน A
            ADD   A,#12H     ; บวกค่า A กับค่า 12H ผลลัพธ์เป็น
                             ; A = A + 12H
HERE :      SJMP  HERE       ; วนลูปซ้ำอยู่กับที่
            END
```

โปรแกรมที่ 2.1 ตัวอย่างภาษาแอสเซมบลีอย่างง่าย

จากโปรแกรมที่ 2.1 จะเป็นตัวอย่างโปรแกรมภาษาแอสเซมบลีที่ใช้คำสั่ง MOV และ ADD ส่วนคำสั่ง ORG และ END เป็นคำสั่งเทียม (pseudo-instruction) บางครั้งเรียกว่าไดเรกทีฟ (directives) ซึ่งใช้ช่วยในการเขียนโปรแกรม คำสั่ง ORG 0000H เป็นการบอกว่าให้ออปโค้ดเริ่มต้นที่หน่วยความจำตำแหน่งที่ 0 ส่วน END บอกว่าเป็นการจบโปรแกรม

ในแต่ละบรรทัดของภาษาแอสเซมบลีจะแบ่งออกเป็น 4 ส่วนดังนี้

[label:] mnemonic [operands] [;comment]

ส่วนที่อยู่ในเครื่องหมาย bracket จะมีหรือไม่ก็ได้ แต่ละส่วนของโปรแกรมจะเป็นดังนี้

1. ส่วนลาเบล (Label) จะเป็นส่วนที่ไม่ถูกแปลเป็นรหัสคำสั่ง จะใช้แทนค่าแอดเดรสหน่วยความจำที่เก็บคำสั่งนั้น โดยจะตามด้วยเครื่องหมายโคลอน

2. ส่วนที่สองอาจเป็นคำสั่งเทียม หรือเป็นคำสั่งภาษาแอสเซมบลีที่ CPU รู้จัก ประกอบด้วยคำสั่งนีโมนิก (mnemonic) และโอเปอแรนด์ (operand) เช่น

```
ADD   A,B
MOV   A,=67
```

จากตัวอย่างส่วนที่เขียนเป็น ADD และ MOV จะเป็นรหัสนีโมนิก ส่วน “A,B” และ “A,#67” จะเป็นโอเปอแรนด์ (operands)

3. ส่วนคำอธิบาย (Comment) จะใช้อธิบายต่างๆจะต้องเขียนตามหลักเครื่องหมายเซมิโคลอน “ ;” ส่วนนี้จะไม่ถูกแปลเป็นรหัสคำสั่ง

4. จากโปรแกรมที่ 2.1 จะเห็นว่าส่วนลาเบล HERE จะตามด้วยเครื่องหมายโคลอน และนีโมนิกเป็นคำสั่ง SJMP (short jump instruction) ซึ่งจะทำให้โปรแกรมมวนซ้ำกลับมาทำคำสั่งเดิม

คำถาม ?

1. คำสั่งเทียมที่ใช้กันมากได้แก่คำสั่งใด
2. ภาษาแอสเซมบลีเป็นภาษาระดับต่ำหรือภาษาระดับสูง
3. คำสั่งใดเป็นคำสั่งเทียม

   ก. ADD A,R1 ข. MOV A,#12

   ค. ORG 2000H ง. SJMP HERE
4. คำสั่งเทียมเรียกอีกชื่อหนึ่งว่าอะไร

ไฟล์ ASM และไฟล์ OBJ

การเขียนโปรแกรมภาษาแอสเซมบลีบนโปรแกรม DOS , EDIT หรือ WINDOW’S Notepad ที่เป็นไฟล์นามสกุล ASM นั้นจะเรียกว่าไฟล์ต้นฉบับ (source file) เมื่อใช้โปรแกรมแอสเซมเบลอร์แปลงให้เป็นรหัสภาษาเครื่องนั้นจะได้ไฟล์ที่มีนามสกุลเป็น obj (object) และในการแอสเซมเบลอร์โปรแกรมนั้นสามารถสร้างไฟล์นามสกุล lst (list file) ออกมาได้ด้วย ไฟล์นามสกุล lst นี้จะเป็นไฟล์ที่แสดงค่า opcodes และ address ของโปรแกรมภาษาแอสเซมบลีออกมาหลังจากการแอสเซมเบลอร์โปรแกรม ซึ่งเราสามารถดูในโปรแกรม dos edit หรือพิมพ์ออกมาทางเครื่องพิมพ์ได้ และสามารถใช้ไฟล์นี้ดูข้อผิดพลาดจากการเขียนโปรแกรมได้อีกด้วย จากตัวอย่างโปรแกรมที่2.1 ถ้าหากโปรแกรมไม่มีข้อผิดพลาดไฟล์นามสกุล lst ที่ได้ออกมาจะเป็นดังนี้

	แอดเดรส	รหัสคำสั่งภาษาเครื่อง				
1.	0000			ORG	0000H	; เริ่มต้นโปรแกรม
2.	0000	7D 25		MOV	R5,#25H	; โหลดค่า 25H ใส่ใน R5
3.	0002	7F 34		MoV	R7,#34H	; โหลดค่า 34ใส่ใน R7
4.	0004	74 00		Mov	A,#0	; โหลดค่า 0 ใส่ในรีจิสเตอร์ A
5.	0006	2D		ADD	A,R5	; บวกค่า R5 กับ A ผลลัพธ์เก็บใน A
6.	0007	2F		ADD	A,R7	; บวกค่า R7 กับ A ผลลัพธ์เก็บใน A
7.	0008	24 12		ADD	A,#12H	; บวกค่า A กับค่า 12H ผลลัพธ์เป็น
						; A = A + 12H
8.	000A	80 FE	HERE:	SJMP	HERE	; วนลูปซ้ำอยู่กับที่
9.	000C			END		

จากโปรแกรมจะเห็นว่าแอดเดรสเริ่มต้นจะเป็น 0000 ซึ่งจะเก็บออปโค้ดที่มีค่าเป็น 7D เมื่อ MCS-51 ตีความหมายจะเป็นตัวบอกว่าให้ย้ายข้อมูลมาใส่ในรีจิสเตอร์ R5 ส่วนโอเปอแรนด์จะอยู่ที่แอดเดรส 0001H

จะเก็บค่า 25H ดังนั้น คำสั่ง MOV R5,#25H จะมีรหัสภาษาเครื่องเป็น 7D25 โดย 7D เป็นออปโค้ด และ 25 เป็นโอเปอแรนด์ คำสั่งต่อมาจะมีขนาด 2 ไบต์จะมีรหัสภาษาเครื่องเป็น 7F34 โดยจะเก็บอยู่ในหน่วยความจำตำแหน่ง 0002 และ 0003 ตามลำดับ

การทำงานของโปรแกรม

จากโปรแกรมที่ 2.1 จะพบว่ารหัสโปรแกรมจะเริ่มต้นที่แอดเดรส 0000 ถ้าจะให้ MCS-51 ทำงานจะต้องนำรหัสโปรแกรมนี้ไปเก็บในหน่วยความจำ ROM ตั้งแต่แอดเดรส 0000 เป็นต้นไป ถ้าหากโปรแกรมเคาน์เตอร์ (PC) ชี้อยู่ที่แอดเดรสของหน่วยความจำ ROM ตำแหน่งใดหมายความว่า CPU จะต้องทำงานคำสั่งที่เก็บอยู่ในแอดเดรสนั้น เมื่อเริ่มจ่ายไฟให้ MCS-51 ถูกรีเซต การทำงานเป็นขั้นตอนดังต่อไปนี้

1. รีจิสเตอร์โปรแกรมเคาน์เตอร์ PC จะมีค่าเป็น 0000 ดังนั้น CPU จะเริ่มอ่านรหัสคำสั่งที่แอดเดรสนี้ และจะได้ออปโค้ดค่าแรกเป็น 7D เข้ามา จากนั้นค่า PC จะเพิ่มขึ้น 1 ค่า และ CPU จะตีความว่าจะต้องนำข้อมูลออกจากโอเปอแรนด์มาเก็บใน R5 โดยโอเปอแรนด์จะเป็นข้อมูลที่ PC ชี้อยู่ ดังนั้น จะนำค่า 25H มาเก็บใน R5 และ PC เพิ่มค่าอีกหนึ่งค่าเป็น 0002 ซึ่งจะเป็นการจบการทำคำสั่งแรก

2. MCS-51 ทำคำสั่งต่อไป โดยอ่านรหัสคำสั่งที่ PC ชี้อยู่ และจะได้ออปโค้ดค่า 7F เข้ามา และจะนำค่า 34H มาเก็บใน R7 จากนั้น PC จะไปชี้ที่แอดเดรส 0004

3. ที่หน่วยความจำ ROM ตำแหน่ง 0004 เก็บรหัสของคำสั่ง MOV A,#0 เมื่อ CPU ทำคำสั่งนี้จบ รีจิสเตอร์ PC จะชี้ไปที่ตำแหน่ง 0006

4.ที่ตำแหน่ง 0006 เก็บรหัสของคำสั่ง ADD A,R5 โดยคำสั่งนี้มีขนาด 1 ไบต์ เมื่อ CPU ทำคำสั่งนี้จบ รีจิสเตอร์ PC จะชี้ไปที่ตำแหน่ง 0007

5.ที่ตำแหน่ง 0007 รหัสคำสั่งคือ 2F ซึ่งมีขนาด 1 ไบต์เช่นกัน ดังนั้นหลังจากทำคำสั่งนี้ PC จะมีค่าเป็น 0008 จากนั้นจะทำคำสั่งต่อไปจนจบโปรแกรม

จากขั้นตอนการทำงานของโปรแกรมที่กล่าวมาจะเห็นได้ว่าถ้าหากโปรแกรมเคาน์เตอร์มีค่าใด CPU จะทำคำสั่งที่อยู่ในตำแหน่งนั้น ในไมโครโพรเซสเซอร์บางตัวจะเรียกโปแกรมเคาน์เตอร์นี้ว่า ตัวชี้คำสั่ง (instruction pointer) เช่นใน CPU ตระกูล x86

2.4 ประเภทของข้อมูลและคำสั่งเทียม

การเขียนโปรแกรมด้วยภาษาแอสเซมบลีสามารถเขียนคำสั่งเทียม (Pseudo instructions) แทรกเข้าไปในโปรแกรมได้ โดยคำสั่งประเภทนี้จะไม่ถูกโปรแกรมแอสเซมเบลอร์แปลความหมายออกเป็นคำสั่งของไมโครคอนโทรลเลอร์ แต่จะช่วยให้การเขียนโปรแกรมทำได้ง่ายขึ้น คำสั่งเทียมที่พบมากได้แก่

คำสั่งเทียม	ความหมาย
DB (define byte)	กำหนดให้ข้อมูลมีขนาด 1 ไบต์ ที่แอดเดรสของบรรทัดนั้น
ORG (origin)	กำหนดแอดเดรสเริ่มต้นของชุดคำสั่งในบรรทัดนั้น
EQU (equate)	กำหนดให้ลาเบลมีค่าเท่ากับโอเปอแรนด์ที่กำหนด
END	บอกจุดสิ้นสุดของโปรแกรม
BIT	กำหนดตำแหน่งบิตของข้อมูล

ในการเขียนโปรแกรมด้วยภาษาแอสเซมบลีนั้นจะมีข้อกำหนดบางประการเกี่ยวกับการกำหนดค่าให้กับลาเบล ตัวอักษร และสัญลักษณ์ต่าง ๆ ดังนี้

การตั้งชื่อ

การตั้งชื่อลาเบล เขียนคำสั่ง หรือประกาศตัวแปรต่าง ๆ นั้นการเขียนเป็นตัวอักษรภาษาอังกฤษเป็นตัวเขียนเล็ก หรือตัวใหญ่จะให้ผลออกมาเหมือนกัน

ค่าตัวเลข

ถ้าหากมีการกำหนดค่าโดยใส่เป็นตัวเลขตามปกติ จะหมายถึงเป็นเลขฐานสิบ (decimal number) ถ้าต้องการกำหนดเป็นเลขฐานอื่น ๆ จะต้องมีตัวอักษรกำกับดังต่อไปนี้

ตัวอักษร	ฐานตัวเลข	ตัวอย่าง
B	กำหนดเป็นเลขฐานสอง	10101110B
O	กำหนดเป็นเบลขฐานแปด	107O
D หรือ (ไม่มีตัวอักษร)	กำหนดเป็นเลขฐานสิบ	1235D หรือ 1235
H	กำหนดเป็นเลขฐานสิบหก	0E5H

ถ้าหากตัวเลขฐานสิบหกขึ้นต้นด้วยตัวอักษร จะต้องใส่เลขศูนย์นำหน้าเสมอ

การอ้างตำแหน่งของโปรแกรมเคาน์เตอร์

ถ้าหากต้องการกำหนดตำแหน่งปัจจุบันของโปรแกรมเคาน์เตอร์จะใช้ตัวอักขระ $ (Dollar sign) กำหนดในส่วนของโอเปอแรนด์คำสั่ง ตัวอย่างเช่นถ้าหากต้องการให้โปรแกรมทำลูปซ้ำ ๆ ไม่รู้จบจะเขียนคำสั่งเป็น

```
HERE:   SJMP    HERE
```

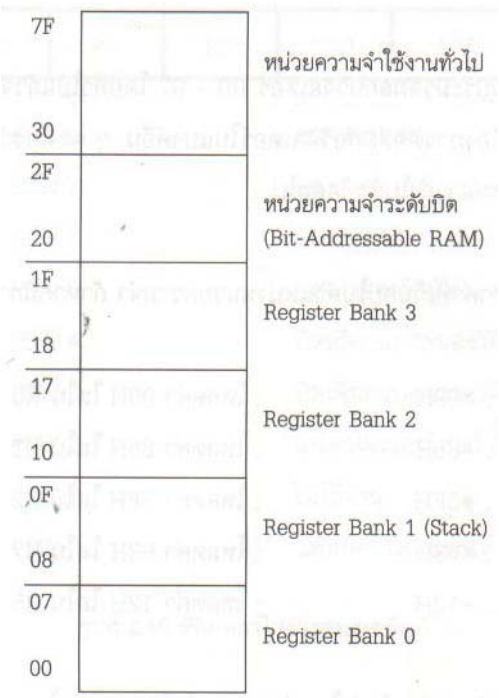
หรือสามารถเขียนอีกแบบหนึ่งได้เป็น

```
        SJMP    $
```

2.5 รีจิสเตอร์แบงค์และสแตก

หน่วยความจำ RAM ภายในของ MCS-51 ที่อยู่ในตำแหน่ง 128 ไบต์แรก คือตั้งแต่แอดเดรส 00H ถึง 7FH จะถูกแบ่งออกเป็น 3 ส่วน ดังรูปที่ 2.8 โดยแต่ละส่วนมีความหมายดังนี้

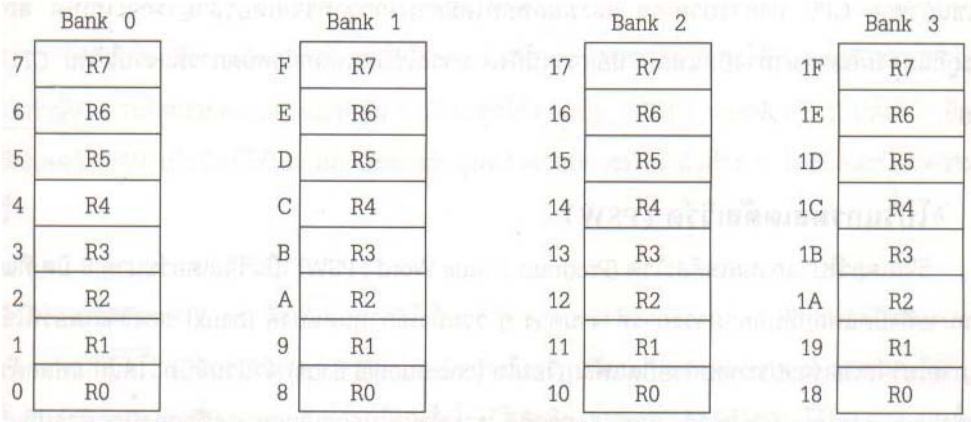
1. ใน 32 ไบต์แรก ตั้งแต่แอดเดรส 00H ถึง 1FH จะถูกใช้เป็นส่วนของรีจิสเตอร์แบงค์และสแตก
2. อีก 16 ไบต์ต่อมา ตั้งตี่แอดเดรส 20H ถึง 2FH จะเป็นหน่วยความจำที่สามารถอ่านเขียนข้อมูลแบบบิตได้ และอ้างตำแหน่งเป็นตำแหน่งบิตได้
3. อีก 80 ไบต์ ตั้งแต่ตำแหน่ง 30H ถึง 7FH จะเป็นหน่วยความจำที่สามารถใช้เก็บข้อมูลทั่วไปได้



รูปที่ 2.8 ตำแหน่งหน่วยความจำของ RAM ภายใน128 ไบต์แรก

รีจิสเตอร์แบงค์ใน MCA-51

หน่วยความจำ RAM ภายใน 32 ไบต์แรก ถูกแบ่งออกเป็น 4 แบงค์ หน่วยความจำแต่ละไบต์ในแต่ละแบงค์จะถูกตั้งชื่อเป็นรีจิสเตอร์ 8 ตัว คือ R0 –R7 ดังนั้น RAM ภายในตำแหน่งแอดเดรส 0 ถึง 7 จะเป็นตำแหน่งแบงค์แรก (Bank 0) ซึ่งจะมีชื่อเรียกเป็นรีจิสเตอร์ R0 ถึง R7 ตามลำดับ สำหรับรีจิสเตอรี R0 ถึง R7 ในแบงค์ต่อมาจะอยู่ที่ตำแหน่ง 08H ถึง 0FH โดยการเรียกชื่อริสเตอร์แบงค์ทั้ง 4 แบงค์ที่ตำแหน่งต่าง ๆ แสดงได้ดังรูปที่ 2.9



รูปที่ 2.9 ชื่อรีจิสเตอร์แบงค์และตำแหน่งหน่วยความจำที่สอดคล้องกัน

ในการเขียนโปรแกรมประมวลผลกับรีจิสเตอร์ R0 – R7 โดยทั่วไปแล้วจะเป็นการติดต่อกับรีจิสเตอร์ในแบงค์ 0 เสมอ แต่ถ้าหากต้องการติดต่อกับรีจิสเตอร์ในแบงค์อื่น ๆ จะต้องกำหนดค่าในรีจิสเตอร์ PSW (Program status word) ซึ่งจกะล่าวถึงในหัวข้อต่อไป

<u>ตัวอย่างที่ 2.1</u> จงหาค่าที่เก็บอยู่ในตำแหน่งหน่วยความจำ ถ้าหากมีการทำคำสั่งต่อไปนี้

```
MOV    R0, #99H        : โหลดค่า 99H ใส่ใน R0
MOV    R1, #85H        : โหลดค่า 85H ใส่ใน R1
MOV    R2, #3FH        : โหลดค่า 3FH ใส่ใน R2
MOV    R7, #63H        : โหลดค่า 63H ใส่ใน R7
MOV    R5, #12H        : โหลดค่า 12H ใส่ใน R5
```

คำตอบ

หลังจากทำโปรแกรมแล้วค่าในหน่วยความจำ RAM ภายในจะเป็นดังนี้

ค่าใน RAM ตำแหน่ง 00 จะเก็บค่า 99H ค่าใน RAM ตำแหน่ง 1 จะเก็บค่า 85H

ค่าใน RAM ตำแหน่ง 02 จะเก็บค่า 3FH ค่าใน RAM ตำแหน่ง 7 จะเก็บค่า 63H

ค่าใน RAM ตำแหน่ง 05 จะเก็บค่า 12H

2.6 แฟลกของ MCS-51 และรีจิสเตอร์ PSW

ข้อมูลในบิตแฟลกจะเป็นตัวแสดงถึงผลจากการทำคำสั่งของ CPU และสถานะการทำงานต่าง ๆ อย่างเช่นถ้าหาก CPU ทำการบวกเลข และผลลัพธ์ที่ได้มีค่ามากกว่าที่รีจิสเตอร์สามารถจะเก็บได้ สถานะดังนี้จะถูกแสดงผลออกมาทางบิตแฟลก นอกจากนี้ยังสามารถใช้บิตแฟลกกำหนดการทำงานให้กับ CPU ได้อีกด้วย

โปรแกรมสเตตัสเวิร์ด (PSW)

รีจิสเตอร์โปรแกรมสเตตัสเวิร์ด (Program Status Word: PSW) เป็นรีจิสเตอร์ขนาด 8 บิต ที่แต่ละบิตจะหมายถึงบิตแฟลกที่บอกสภาวะการทำงานต่าง ๆ รวมทั้งใช้กำหนดแบงค์ (bank) ของรีจิสเตอร์ที่ใช้งานด้วย ภายในรีจิสเตอร์จะประกอบด้วยบิตแฟลกเงื่อนไข (conditional flags) จำนวนสี่บิต ได้แก่ แฟลกตัวทด, แฟลกช่วยทด, แฟลกโอเวอร์โฟลว์, และแฟลกพาริตี้ รวมทั้งบิตในการเลือกแบงค์อีกสิบบิต ดังรูปที่ 2.10 รีจิสเตอร์ตัวนี้เป็นรีจิสเตอร์ที่สามารถเข้าถึงข้อมูลระดับบิตได้

CY	AC	F0	RS1	RS0	OV	-	P

บิต	ตำแหน่ง	ความหมาย
CY	PSW.7	แฟลกทด (carry flag)
AC	PSW.6	แฟลกช่วยทด (auxiliary carry flag)
F0	PSW.5	แฟลกสำหรับผู้ใช้กำหนดเองอย่างอิสระ
RS1	PSW.4	บิตเลือกแบงค์ของรีจิสเตอร์บิต 1
RS0	PSW.3	บิตเลือกแบงค์ของรีจิสเตอร์บิต 0
OV	PSW.2	แฟลกโอเวอร์โฟลว์ (overflow flag)
-	PSW.1	ไม่ใช้งาน
P	PSW.0	แฟลกพาริตี้ (parity flag)

รูปที่ 2.10 รีจิสเตอร์โปรแกรมสเตตัสเวิร์ด

รีจิสเตอร์ R0 –R7 ไม่ว่าจะเป็นรีจิสเตอร์ที่อยู่ในหน่วยความจำส่วนใดจะมีชื่อเดียวกันทั้งสิ้น แต่ถ้าหากอยู่ในหน่วยความจำแบงค์ 0 รีจิสเตอร์ R0 –R7 จะอยู่ในแอดเดรส 00H – 07H จากรูปที่ 2.10 จะเห็นว่า PSW.3 และ PSW.4 จะเป็นตำแหน่งของ RS0 และ RS1 ถ้าหากต้องการใช้งานรีจิสเตอร์ในแบงค์ใดจะต้องกำหนดค่าลงไปในสองบิตนี้ โดยการกำหนดค่าแสดงได้ดังตารางต่อไปนี้

บิต RS1	บิต RS0	รีจิสเตอร์	ตำแหน่งหน่วยความจำ
0	0	แบงค์ 0	00H – 07H
0	1	แบงค์ 1	08H – 0FH
1	0	แบงค์ 2	10H – 17H
1	1	แบงค์ 3	18H – 1FH

โดยทั่วไปแล้วการพัฒนาทางด้านไมโครคอนโทรลเลอร์ MCS-51 มักจะใช้รีจิสเตอร์ R0 – R7 ในแบงค์ที่ 0 เท่านั้น ส่วนที่หน่วยความจำแบงค์อื่น ๆ มักจะถูกใช้งานเป็น RAM ตามปกติ สำหรับบิตแฟลกอีก 4 บิตในรีจิสเตอร์ PSW เป็นบิตที่ใช้บอกสถานะของข้อมูลหลังจากมีการทำคำสั่งต่าง ๆ โดยแต่ละบิตมีความหมายดังนี้

แฟลกทด CY

เป็นบิต D7 รีจิสเตอร์ PSW โดยบิตนี้จะมีการเปลี่ยนแปลงหลังจากการบวกหรือลบข้อมูลแบบ 8 บิต แล้วมีตัวทดเกิดขึ้น นอกจากนี้สามารถเซตให้เป็นลอจิก “1” หรือลอจิก “0” ได้โดยตรง โดยใช้คำสั่ง SETB C หรือ CLR C

แฟลกช่วยทด AC

หลังจากการทำคำสั่ง ADD หรือ SUB แล้วมีการทดจาก D3 ไป D4 จะทำให้บิตนี้ถูกเซต โดยทั่วไปแล้วบิตแฟลกบิตนี้จะใช้เมื่อมีการทำคำสั่งทางคณิตศาสตร์แบบ BCD

แฟลกพาริตี้ P

แฟลกนี้จะแสดงถึงจำนวนลอจิก “1” ที่อยู่ในรีจิสเตอร์ A ถ้าค่าในรีจิสเตอร์ A มีลอจิก “1” เป็นคู่บิต (even) บิตแฟลกพาริตี้จะเป็นลอจิก “0” ถ้าค่าในรีจิสเตอร์ A มีลอจิก “1” เป็นคี่บิต (odd) บิตแฟลกพาริตี้จะเป็นลอจิก “1”

แฟลกโอเวอร์โฟลว์ (OV)

บิตแฟลกนี้จะถูกเซตถ้าผลลัพธ์จากการดำเนินการกับตัวเลขแบบคิดเครื่องหมายมีค่าเกินกว่าที่รีจิสเตอร์จะเก็บได้ โดยทั่วไปแล้วแฟลกตัวทดจะใช้ตรวจสอบความผิดพลาดจากการกระทำทางคณิตศาสตร์ของตัวเลขแบบไม่คิดเครื่องหมาย แต่แฟลกโอเวอร์โฟลว์จะใช้ตรวจสอบความผิดพลาดของการกระทำทางคณิตศาสตร์ของตัวเลขแบบคิดเครื่องหมาย

<u>ตัวอย่างที่ 2.2</u> จงแสดงค่าของบิตแฟลก CY, AC และ P หลังจากการบวกค่า 38H กับ 2FH ด้วยคำสั่งต่อไปนี้

```
MOV   A, #38H
MOV   A, #2FH          ; หลังจากการบวก A = 67H, CY = 0
```

วิธีทำ

```
  38          00111000
+ 2F          00101111
-----         --------
  67          01100111
```

CY	= 0	เพราะไม่มีการทดจากบิต D7
AC	= 1	เพราะมีการทดจากบิต D3 ไปยัง D4
P	= 1	เพราะค่าในรีจิสเตอร์ A มีค่าเป็ฯ “1” เป็นจำนวนคี่บิต

<u>ตัวอย่างที่ 2.3</u> จงแสดงค่าของบิตแฟลก CY, AC และ P หลังจากการบวกค่า 9CH กับ 64H ด้วยคำสั่งต่อไปนี้

```
MOV   A , #9CH
ADD   A , #64H                ; หลังจากการบวก A = 00 และ CY = 1
```

วิธีทำ

```
   9C          1001 1100
+  64          0110 0100
-----         ---------
  100          1 0000 0000
                ↑
               ตัวทด
```

CY	= 1	เพราะมีการทดออกจากบิต D7
AC	= 1	เพราะมีการทดจากบิต D3 ไปยังบิต D3
P	= 0	เพราะมีลอจิก “1” ในรีจิสเตอร์ A เป็นคู่บิต (เป็นศูนย์บิต)

<u>ตัวอย่างที่ 2.4</u> จงแสดงค่าของบิตแฟลก CY, Ac และ P หลังจากการบวกค่า 88H กับ 93H ด้วยคำสั่งต่อไปนี้

```
MOV   A , #88H
ADD   A , #93H                ; หลังจากการบวก A = 1BH , CY = 1
```

วิธีทำ

```
   88          10001000
+ 93           10010011
-------        --------
1 1B           00011011
```

CY	= 1	เพราะมีการทดออกจากบิต D7
AC	= 0	เพราะไม่มีการทดจากบิต D3 ไปบิต D4
P	= 0	เพราะค่าในรีจิสเตอร์ A มีลอจิก”1” เป็นจำนวนบิตคู่

<u>ตัวอย่างที่ 2.5</u> จงหาค่าที่เก็บใน RAM ตำแหน่งต่าง ๆ หลังจากทำชุดคำสั่งต่อไปนี้

```
SETB   PSW.4      ; เลือกแบงค์ 2
MOV    R0, #99H   ; โหลดค่า 99H เก็บใน R0
MOV    R1, #85H   ; โหลดค่า 85H เก็บใน R1
MOV    R2, #3FH   ; โหลดค่า 3FH เก็บใน R2
MOV    R7, #63H   ; โหลดค่า 63H เก็บใน R7
MOV    R5, #12H   ; โหลดค่า 12H เก็บใน R5
```

วิธีทำ

หลังจากรีเซต MCS-51 จะทำให้ PSW.3 = 0 และ PSW.4 = 0 จากชุดคำสั่ง ถ้าหากมีการทำคำสั่ง SETB PSW.4 จะทำให้ RS1 = 1 และ RS0 = 0 ดังนั้นจะเป็นการเลือกรีจิสเตอร์แบงค์ 2 โดย R0 – R7 จะใช้หน่วยความจำ RAM ภายในตำแหน่ง 10H – 17H ดังนั้น

RAM ตำแหน่ง 10H เก็บค่า 99H RAM ตำแหน่ง 11H เก็บค่า 85H

RAM ตำแหน่ง 12H เก็บค่า 3FH RAM ตำแหน่ง 17H เก็บค่า 63H

RAM ตำแหน่ง 15H เก็บค่า 15

คำถาม ?

1. รีจิสเตอร์แฟลกใน 8051 มีชื่อเรียกว่าอะไร
2. บิตใดของรีจิสเตอร์ PSW ที่ไม่ถูกใช้งาน
3. จงหาค่าบิต CY และ AC หลังจากทำคำสั่งต่อไปนี้

```
MOV    A,#0FFH
ADD    A,#01
```

4. จงหาค่าบิต CY และ AC หลังจากการทำคำสั่งต่อไปนี้

```
MOV    A,#0C2H
ADD    A,#3DH
```

2.7 หน่วยความจำสแตกของ MCS-51

สแตก (Stack) เป็นหน่วยความจำ RAM จำนวนหนึ่งที่ CPU จองขึ้นมาสำหรับเก็บข้อมูลและจำตำแหน่งของโปรแกรม ใน MCS-51 จะใช้รีจิสเตอร์ SP (stack pointer) เป็นตัวชี้ตำแหน่งหน่วยความจำที่เป็นสแตก ซึ่งจะเป็นตัวบอกว่าแอดเดรสของ RAM ตำแหน่งใดจะเป็นตำแหน่งที่จะนำข้อมูลเข้าไปเก็บ เมื่อมีการเก็บข้อมูลลงสแตกแล้วค่าของ SP จะเพิ่มขึ้นหนึ่งค่า เพื่อชี้ตำแหน่งถัดไป และเมื่อมีการอ่านข้อมูลออกจากสแตก ค่าของ SP จะลดลงค่าหนึ่ง

การโอนย้ายข้อมูลกับสแตกจะใช้คำสั่ง PUSH และ POP โดยคำสั่ง PUSH จะเป็นการนำข้อมูลจากโอเปอแรนด์ที่ตามมาเก็บยังสแตก หลังจากอ่านข้อมูลออกมาแล้ว SP จะเพิ่มค่าขึ้นหนึ่งค่าก่อนจึงเก็บข้อมูล สำหรับคำสั่ง POP จะเป็นการอ่านข้อมูลออกจากหน่วยความจำสแตก หลังจากอ่านข้อมูลออกมาแล้ว SP จะลดลงหนึ่งค่า รูปแบบของคำสั่ง PUSH และ POP มีลักษณะดังนี้

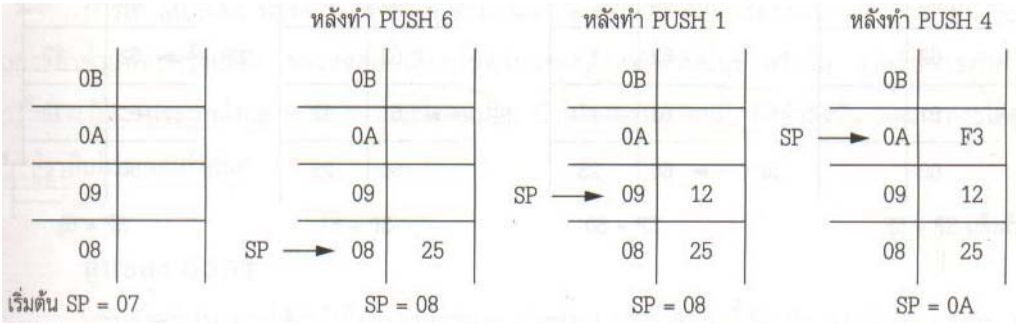
รูปแบบ	การทำงาน
PUSH direct	เพิ่มค่า SP และนำค่าในหน่วยความจำแอดเดรส direct เก็บยังหน่วยความจำที่ SP ชี้อยู่
POP direct	อ่านค่าจากตำแหน่งที่ SP ชี้อยู่ มาเก็บในหน่วยความจำแอดเดรส direct และลดค่า SP ลงหนึ่งค่า

เมื่อ MCS-51 เริ่มต้นการทำงานรีจิสเตอร์ SP จะมีค่าเป็ฯ 07H หมายความว่าหน่วยความจำสแตกจะเริ่มต้นที่แอดเดรสนี้ ซึ่งจะพบว่าในตำแหน่งนี้เป็ฯแอดเดรสของรีจิสเตร์ R0 – R7 ในแบงค์ 1 ดังนั้นถ้าหากต้องการใช้รีจิสเตอร์ในแบงค์นี้จะต้องกำหนดให้ SP ชี้ไปยังตำแหน่งอื่นเสียก่อน แต่ไม่ควยให้หน่วยความจำสแตกอยู่ในช่วงแอดเดรส 20H – 2FH เนื่องจากเป็นตำแหน่งของหน่วยความจำที่สามารถเข้าถึงข้อมูลในระดับบิตได้ โดยทั่วไปแล้วมักจะกำหนดให้สแตกอยู่ในช่วงแอดเดรส 30H – 7FH ของ RAM ภายใน โดยใช้คำสั่ง MOV SP , #XX เมื่อ XX เป็นแอดเดรสที่กำหนด

<u>ตัวอย่างที่ 2.6</u> จงแสดงสแตกและตัวชี้สแตก เมื่อมีการทำชุดคำสั่งต่อไปนี้

```
MOV   R6, #25H
MOV   R1, #12H
MOV   R4, #0F3H
PUSH  6
PUSH  1
PUSH  4
```

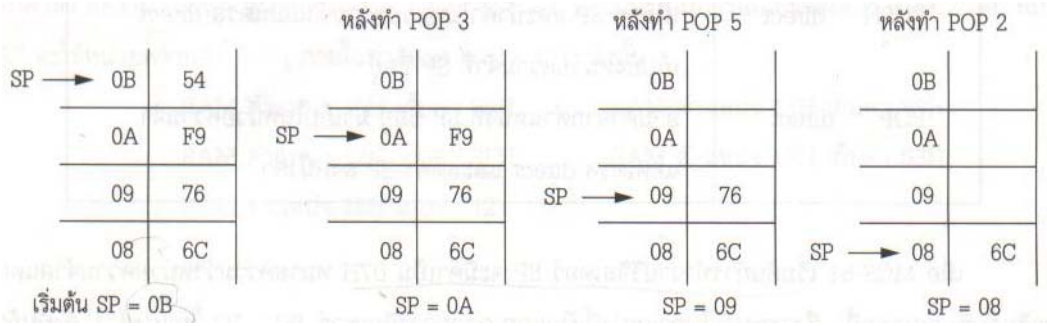
<u>วิธีทำ</u>



<u>ตัวอย่างที่ 2.7</u> จงแสดงค่าของรีจิสเตอร์ SP หลังจากการทำคำสั่งต่อไปนี้

```
POP   3     ; POP ค่าในสแตกมาเก็บใน R3
POP   5     ; POP ค่าในสแตกมาเก็บใน R5
POP   2     ; POP ค่าในสแตกมาเก็บใน R2
```

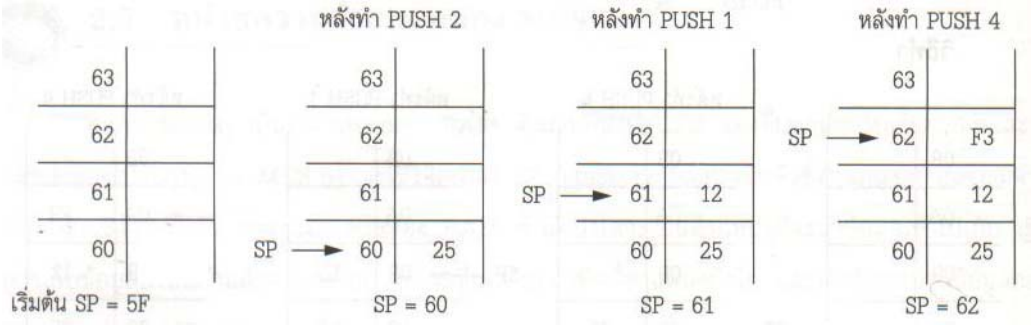
วิธีทำ



ตัวอย่างที่ 2.8 จงแสดงค่าของรีจิสเตอร์ SP และค่าในสแตก เมื่อมีการทำคำสั่งต่อไปนี้

```
MOV     SP, #5FH        ; กำหนดให้ SP ชี้ที่ RAM ตำแหน่ง 5FH
                        ; ดังนั้นสแตกแอดเดรสแรกคือตำแหน่ง 60H
MOV     R2, #25H
MOV     R4, #0F3H
PUSH    2
PUSH    1
PUSH    4
```

วิธีทำ



บทที่ 3
คำสั่งควบคุม

ในบทนี้จะกล่าวถึงชุดคำสั่งควบคุมการทำงานของโปรแกรม ซึ่งจะทำให้ MCS-51 กระโดดไปทำงานยังตำแหน่งต่าง ๆ ได้โดยจะกล่าวถึงคำสั่งทำซ้ำ ที่ทำให้ระบบทำงานซ้ำตามรอบที่กำหนด คำสั่งกระโดดซึ่งจะทำให้โปรแกรมกระโดดไปยังแอดเดรสที่ต้องการได้ และคำสั่งเรียกโปรแกรมย่อย

3.1 คำสั่งทำซ้ำและคำสั่งกระโดด

การให้ MCS-51 ทำงานใด ๆ ซ้ำ ๆ สามารถทำได้หลายวิธี แต่จะเรียกรวมกันว่าการทำลูป (loop) ในการทำซ้ำแต่ละครั้งจะมีการตรวจสอบเงื่อนไขด้วยว่าจะต้องทำงานนั้นซ้ำหรือไม่ สำหรับการทำคำสั่งกระโดดจะมีสองประเภทใหญ่ ๆ คือการกระโดดแน่ ๆ ถ้าโปรแกรมทำงานมาถึงคำสั่งนั้น และจะกระโดดถ้าเงื่อนไขเป็นไปตามที่กำหนด

ลูปของ 8051

การเขียนโปรแกรมให้กับไมโครคอนโทรลเลอร์ตระกลู MCS-51จะใช้คำสั่ง DJNZ reg, label เมื่อ MCS-51 ทำคำสั่งนี้แตละครั้งจะลดค่าในรีจิสเตอร์ reg ลงหนึ่งค่า และคำสั่งนี้จะทำให้โปรแกรมกระโดดไปยังแอดเดรส Label ถ้าค่าในรีจิสเตอร์ reg ยังไม่เป็นศูนย์ แต่ถ้าค่าในรีจีสเตอร์ reg เป็นศูนย์ โปรแกรมจะทำคำสั่งต่อจากคำสั่ง DJNZ นี้

<u>ตัวอย่างที่ 3.1</u> จงเขียนโปรแกรมให้ทำงานดังต่อไปนี้

ก. เคลียร์ค่าในแอกคิวมูเลเตอร์

ข. บวกค่า 3 เข้ากับแอกคิวมูเลเตอร์จำนวน 10 ครั้ง

<u>วิธีทำ</u>

```
        MOV   A, #0          ; A = 0, เคลียร์ค่าในรีจีสเตอร์ A
        MOV   R2, #10D       ; โหลดค่าตัวนับ R2 = 10
AGAIN:  ADD   A, #03         ; บวกค่า 3 เข้ากับ ACC
        DJNZ  R2, AGAIN      ; ทำซ้ำจนกว่า R2 = 0
        MOV   R5, A          ; เก็บผลลัพธ์ลงใน R5
```

จากตัวอย่างโปรแกรมที่ 3.1 จะใช้รีจิสเตอร์ R2 เป็นตัวนับ การนับครั้งแรกจะมีค่าเป็น 10 ต่อมาเมื่อพบกับคำสั่ง DJNZ จะทำการลดค่า R2 ลงหนึ่งค่า และทดสอบค่า ถ้าค่าใน R2 ไม่เท่ากับ 0 โปรแกรมจะกระโดดไปยังแอดเดรส AGAIN แต่ถ้าค่าใน R2 เท่ากับ 0 โปรแกรมจะเก็บค่าคำตอบในรีจิสเตอร์A ลงในรีจิสเตอร์ R5 ด้วยคำสั่ง MOV R5,A และในการทำลูปแต่ละครั้งนั้นจะทำการบวกค่าในรีจิสเตอร์ A ด้วยค่า 3

คำสั่ง DJNZ นี่ค่ารีจิสเตอร์ที่ใช้เป็นตัวนับและลดค่าจะเป็นรีจิสเตอร์ R0-R7 และยังใช้ค่าใน RAM ภายในเป็นตัวนับได้อีกด้วย

ลูปซ้อนลูป

ในการทำซ้ำแต่ละครั้งนั้นสามารถทำซ้ำได้ทั้งหมด 256 ครั้ง ซึ่งเป็นค่าที่มากที่สุดที่จะเก็บในรีจิสเตอร์ได้ถ้าหากต้องการทำซ้ำมากกว่านี้สามารถทำได้โดยการทำลูปซ้อนลูป หรือที่เรียกกันว่า nested loop ดังแสดงในตัวอย่างโปรแกรมที่ 3.2

<u>ตัวอย่างที่ 3.2</u> จงเขียนโปรแกรมโหลดค่าในรีจิสเตอร์ A ด้วย 55H และทำการคอมพลีเมนต์ค่าในรีจิสเตอร์ A จำนวน 700 ครั้ง

<u>วิธีทำ</u> เนื่องจากจะทำซ้ำเป็นจำนวน 700 ครั้ง ซึ่งมากกว่า 255 ดังนั้นจะต้องทำลูปซ้อนลูปโดยใช้รีจิสเตอร์สองตัวเป็นตัวนับ ในที่นี้จะใช้รีจิสเตอร์ R2 และ R3

```
        MOV   A, #55H      ; A = 55
        MOV   R3, #10D     ; R3 = 10 เป็นการนับจำนวนลูปนอก
NEXT:   MOV   R2, #70D     ; R2 = 70 เป็นการนับจำนวนลูปใน
AGAIN:  CPL   A            ; คอมพลีเมนต์ค่าในรีจิสเตอร์ A
        DJNZ  R2, AGAIN    ; ทำซ้ำจำนวน 70 ครั้ง
        DJNZ  R3, NEXT     ; ทำซ้ำจำนวน 10 ครั้ง
```

การกระโดดแบบมีเงื่อนไข

คำสั่งกระโดดแบบมีเงื่อนไขของ MCS-51 สรุปได้ดังตารางที่ 3.1 ซึ่งมีอยู่หลายคำสั่ง เช่น JZ คำสั่งนี้จะทำการกระโดดถ้าค่าใน A เป็น 0 และคำสั่ง JC จะกระโดดถ้าแฟลกตัวทดถูกเซต เป็นต้น คำสั่งกระโดดแบบมีเงื่อนไขมีตัวอย่างดังนี้

ตารางที่ 3.1 คำสั่งกระโดดแบบมีเงื่อนไข

คำสั่ง	ชื่อเต็มของคำสั่ง	การทำงาน
JZ	Jump If Acc is Zero	กระโดดถ้า A เท่ากับ 0
JNZ	Jump If Acc is Not Zero	กระโดดถ้า A ไม่เท่ากับ 0
DJNZ A, label	Decrement and Jump if Acc not Zero	ลดค่าและกระโดดถ้า A ไม่เท่ากับ 0
CJNE A, byte	Compare A and byte and Jump if not Equal	กระโดดถ้า A ไม่เท่ากับ byte
CJNE reg,#data	Compare reg and #data and Jump if not Equal	กระโดดถ้า reg ไม่เท่ากับ #data
JC	Jump If Carry Set	กระโดดถ้า CY เท่ากับ 1
JNC	Jump if Not Carry Set	กระโดดถ้า CY เท่ากับ 0
JB	Jump if Bit Set	กระโดดถ้าบิตที่กำหนดเท่ากับ 1
JNB	Jump if Not Bit	กระโดดถ้าบิตที่กำหนดเท่ากับ 0
JBC	Jump if Bit Set and Clear Bit	กระโดดถ้าบิตที่กำหนดเท่ากับ 1 และเคลียร์บิต

JZ (กระโดดถ้า A = 0)

คำสั่งนี้จะตรวจสอบค่าในรีจิสเตอร์ A ถ้าหากมีค่าเป็น 0 จะกระโดดไปยังแอดไปยังแอดเดรสที่กำหนด ดังตัวอย่างการเขียนคำสั่งต่อไปนี้

```
        MOV   A, R0        ; A = R0
        JZ    OVER         ; กระโดดไป OVER ถ้า A = 0
        MOV   A, R1        ; A = R1
        JZ    OVER         ; กระโดดไป OVER ถ้า A = 0
        ......................
OVER:   ......................
```

จากโปรแกรมจะเห็นว่าถ้าค่าในรีจิสเตอร์ R0 หรือ R1 เป็นศูนย์ โปรแกรมจะกระโดดไปยังแอดเดรสที่มีลาเบล OVER แต่เนื่องจากคำสั่ง JZ จะตรวจสอบค่าในรีจิสเตอร์ A เท่านั้น จึงต้องย้ายค่าจากรีจิสเตอร์ R0 และ R1 ไปเก็บในรีจิสเตอร์ A เสียก่อน ถ้าหากต้องการเขียนโปรแกรมให้ตรวจสอบค่าใน รีจิสเตอร์ A และให้กระโดดถ้าค่าใน A ไม่เป็น 0 ต้องใช้คำสั่ง JNZ ดังตัวอย่างต่อไป

<u>ตัวอย่างที่ 3.3</u> จงเขียนโปรแกรมทดสอบค่าใน R5 ถ้าหากมีค่าเป็นศูนย์ให้ใส่ค่า 55H ลงไป

<u>วิธีทำ</u>

```
        MOV   A, R5        ; ใส่ค่า R5 ลงใน A
        JNZ   NEXT         ; กระโดดถ้าค่าใน A ไม่เท่ากับศูนย์
        MOV   R5, 55H      ; ใส่ค่า 55H ลงใน A
NEXT:   ……................
```

JNC (กระโดดถ้าไม่มีตัวทด หรือ CY = 0)

คำสั่งนี้ตรวจสอบค่าบิตแฟลกตัวทดที่อยู่ในรีจิสเตอร์ PSW ถ้าไม่มีการทดจะกระโดดไปยังแอดเดรสที่กำหนด โดยมีรูปแบบคำสั่งเป็น JNC label ซึ่งจะทดสอบแฟลกตัวทดเช่นกัน แต่จะกระโดดไปยังแอดเดรส label เมื่อแฟลกตัวทดมีค่าเป็น “0”

สำหรับคำสั่ง JB (jump if bit is high) และ JNB (jump if bit is low) จะเป็นคำสั่งที่ทดสอบบิตและจะกระโดดไปยังแอดเดรสที่กำหนด ถ้าบิตนั้นเป็น “1” หรือ “0” ตามลำดับ

<u>ตัวอย่างที่ 3.4</u> จงเขียนโปรแกรมบวกค่า 79H, F5H และ E2H และใส่ค่าผลบวกไบต์ต่ำลงใน R0 และไบต์สูงลงใน R5

<u>วิธีทำ</u>

```
        MOV   A, #0        ; เคลียร์ค่าใน A
        MOV   R5, A        ; เคลียร์ค่าใน R5
        ADD   A, #79H      ; A = 0 + 79H = 79H
        JNC   N_1          ; ถ้าไม่มีตัวทดให้กระโดดไปบวกเลขต่อไป
        NC    R5           ; ถ้าตัวทดเป็น 1 ให้เพิ่มค่า R5
N_1:    ADD   A, #0F5H     ; A = 79 + F5 = 6E และแฟลกตัวทดเป็น 1
        JNC   N_2          ; กระโดดถ้าตัวทดเป็น 0
        INC   R5           ; ถ้าตัวทดเท่ากับ 1 ให้เพิ่มค่า R5 (R5=1)
```

```
N_2:   ADD   A, #0E5H     ; A = 6E + E2 = 50 และตัวทดเป็น 1
       JNC   OVER         ; กระโดดถ้าตัวทดเป็น 0
       INC   R5           ; ถ้าตัวทดเป็น 1 ให้เพิ่มค่า R5
OVER:  MOV   R0, A        ; ค่าใน R0 = 50H และ R5 = 02
```

ในการทำคำสั่งกระโดดแบบมีเงื่อนไขทุกคำสั่งจะเป็นการกระโดดแบบ short jump ซึ่งจะกระโดดถอยหลังได้ -128ไบต์ และกระโดดไปข้างหน้าได้ +127ไบต์ โดยจะนำค่าในโอเปอแรนด์ของคำสั่งไปบวกกับค่าโปรแกรมเคาน์เตอร์ (PC)

คำสั่งกระโดดแบบไม่มีเงื่อนไข

การกระโดดแบบไม่มีเงื่อนไข (Unconditional jump) จะเป็นคำสั่งที่ทำให้โปรแกรมกระโดดไปยังตำแหน่งที่กำหนดแน่ๆใน MCS-51มีคำสั่งกระโดดแบบไม่มีเงื่อนไขสองคำสั่งคือคำสั่ง LJMP (long jump) และ SJMP (short jump) โดยทั้งสองคำสั่งจะต่างกันที่ระยะทางกระโดดของโปรแกรม

LJMP (long jump)

คำสั่ง LJMP เป็นคำสั่งที่มีขนาด 3 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด อีกสองไบต์ต่อมาจะแทนตำแหน่งแอดเดรสขนาด 16 บิต ซึ่งสามารถกระโดดไปยังหน่วยความจำใด ๆ ได้ในช่วง 0000 ถึง FFFFH

แม้ว่า MCS-51 สามารถอ้างตำแหน่งหน่วยความจำภายนอกได้ 64 กิโลไบต์ หรือมีโปรแกรมเคาน์เตอร์ขนาด 16 บิต แต่ MCS-51 ที่มี ROM อยู่บนชิปก็ไม่มีทั้ง 64 กิโลไบต์อยู่ภายในทั้งหมด MCS-51 บางรุ่นมีหน่วยความจำภายในชิปสูงสุดเพียง 4 กิโลไบต์ ดังนั้นการใช้คำสั่งกระโดดที่กระโดดได้ในช่วง 0000 ถึง FFFF ก็ไม่มีความจำเป็น จึงมีคำสั่งกระโดดอีกคำสั่งหนึ่งที่กระโดดได้ในช่วงสั้น ๆ คือ SJMP (short jump) ซึ่งเป็นคำสั่งที่มีขนาด 2 ไบต์ ทำให้การกระโดดในช่วงสั้น ๆ สามารถใช้คำสั่งกระโดดที่มีขนาดของรหัสคำสั่งน้อยลงได้ ดังนั้นคำสั่งกระโดดของ MCS-51 อาจจะมองได้ว่ามีสองคำสั่งคือ คำสั่งขนาด 2 ไบต์SJMP และ คำสั่งแบบ 3 ไบต์ LJMP

SJMP (short jump)

คำสั่งนี้จะมีขนาด 2 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด ไบต์ต่อมาจะเป็นแอดเดรสที่จะกระโดดโดยอ้างแอดเดรสแบบสัมพันธ์ (Relative address) ซึ่งจะอ้างคำแหน่งได้ในช่วง 00 ถึง FFH หรือกระโดดไปข้างหลังได้ -128 ไบต์ และ กระโดดไปข้างหน้าได้ +127 ไบต์ โดยไมโครตอนโทรคอนโทรลเลอร์จะนำค่าออปโค้ดซึ่งอ้างแอดเดรสแบบสัมพันธ์นี้ไปบวกค่าในโปรแกรมเคาน์เตอร์ และ ให้เป็นค่าใหม่กับโปรแกรมเคาน์เตอร์

จากคำสั่งกระโดดทั้งหมดที่กล่าวมาจะพบว่ามีกลุ่มคำสั่งที่จะกระโดดไปยังแอดเดรสระยะสั้นๆได้แก่ คำสั่ง SJMP และคำสั่งกระโดดแบบมีเงื่อนไขทุกคำสั่งเช่น JNC, JZ และ DJNZ เป็นต้น คำสั่งเหล่านี้จะมีขนาด 2 ไบต์ และมีการอ้างตำแหน่งที่จะกระโดดไปแบบสัมพันธ์ ซึ่งถ้านำค่าแอดเดรสแบบสัมพนธ์นี้ไปบวกกับค่าใน PC ก็จะได้ตำแหน่งที่จะกระโดดไป การคำนวณค่าแอดเดรสแบบสัมพันธ์นี้จะคำนวณได้ ดังตัวอย่างที่ 3.5

ตัวอย่างที่ 3.5 จากโปรแกรมต่อไปนี้จะแสดงวิธีการคำนวณค่าแอดเดรสของคำสั่งกระโดด

Line	PC	Opcode		Mnemonic	Operand
01	0000			ORG	0000
02	0000	78 00		MOV	R0, #0
03	0002	74 55		MOV	A, #55
04	0004	60 03		JZ	NEXT
05	0006	08		INC	R0
06	0007	04	AGAIN:	INC	A
07	0008	04		INC	A
08	0009	24 77	NEXT:	ADD	A, #77H
09	000B	50 05		JNC	OVER
10	000D	E4		CLR	A
11	000E	F8		MOV	R0, A
12	000F	F9		MOV	R1, A
13	0010	FA		MOV	R2, A
14	0011	FB		MOV	R3, A
15	0012	2B	OVER:	ADD	A, R3
16	0013	50 F2		JNC	AGAIN
17	0015	80 FE	HERE:	SJMP	HERE
18	0017			END	

วิธีทำ

จากคำสั่งกระโดดในโปรแกรมจะเป็นคำสั่งกระโดดแบบมีเงื่อนไข ซึ่งจะอ้างแอดเดรสแบบสัมพัทธ์ โดยค่านี้จะบอกว่าตำแหน่งที่จะกระโดดไปนั้นอยู่ห่างจากตำแหน่งที่ PC อยู่ไปข้างหน้าหรือถอยหลังไปเป็นจำนวนกี่ไบต์ จากโปรแกรมในบรรทัดที่ 4 จะเป็นคำสั่ง JZ NEXT ซึ่งจะกระโดดไปยังแอดเดรส 0009 คำสั่งนี้เก็บอยู่ในแอดเดรส 0004 และ 0005 มีออปโค้ดเป็น 60 และ 03 เป็นค่าแอดเดรสแบบสัมพันธ์ เมื่อโปรแกรมทำงานมาถึงคำสั่งนี้ค่า PC จะอยู่ที่คำสั่งถัดไปคือคำสั่งในแอดเดรส 0006 ดังนั้นค่าโอเปอแรนด์ 03เมื่อนำมาบวกด่ากับ PC จะทำให้โปรแกรมกระโดดไปยังแอดเดรส 0009 ส่วนคำสั่ง JNC OVER ในบรรทัดที่ 9 จะอยู่ที่แอดเดรส 000B และจะเห็นว่ามีค่าโอเปอแรนด์เป็น 05 เมื่อคอมพิวเตอร์ทำคำสั่งนี้ค่า PC จะอยู่ที่แอดเดรส 000D ซึ่งอยู่ที่คำสั่ง CLR A ดังนั้นโปรแกรมจะกระโดดไปยังแอดเดรส 000D+5 คือแอดเดรส 0012H

สำหรับคำสั่ง JNC AGAIN ในแอดเดรส 0013H มีค่าออปโค้ดเป็น 50 และค่าแอดเดรสแบบสัมพัทธ์เป็น F2H เมื่อ MCS-51 ทำงานมาถึงคำสั่งนี้ PC จะชี้อยู่ยังแอดเดรสคำสั่งถัดไปคือคำสั่งในแอดเดรส0015H ดังนั้นแอดแดรสที่จะกระโดดไปคือ 15H + F2H = 07 ซึ่งเป็นตำแหน่งของคำสั่ง INC A ในลาเบลAGAIN จะเห็นว่าการกระโดดของคำสั่งนี้จะเป็นการโดดกลับไปข้างหลังจำนวน 13 ไบต์ ค่าแอดเดรสแบบสัมพัทธ์นี้จะต้องเป็น -13 ซึ่งเท่ากับ F2 (เลข 2's complement) สำหรับคำสั่ง SJMP HERE คำสั่งนี้จะมีขนาด 2 ไบต์ และจะต้องกระโดดกลับหลังเป็นระยะ 2 ไบต์มายังตำแหน่งคำสั่งเดิม ดังนั้นค่า -2 จะเป็นFEH ซึ่งเท่ากับค่าโอเปอแรนด์ในโปรแกรม ดังนั้นตำแหน่งที่จะกระโดดไปจะเป็น 17H + FEH = 15 H นั้นเอง

คำถาม ?

1.รหัสนีโมนิก DJNZ มีค่ารหัสภาษาเครื่องอะไร
2.จริงหรือไม่ที่คำสั่ง DJNZ R5,BACK จะลดค่าในรีจิสเตอร์ก่อนที่จะทำการกระโดด
3. คำสั่ง JZ NEXT จะตรวจสอบค่าในรีจิสเตอร์ใดว่ามีค่าเป็นศูนย์หรือไม่
4. คำสั่ง JZ NEXT มีขนาดกี่ไบต์
5. คำสั่ง LJMP มีขนาดกี่ไบต์

3.2 การเรียกโปรแกรมย่อย

การเขียนโปรแกรมประเภทโปรแกรมย่อยหรือ Subroutine สามารถถูกเรียกขึ้นมาใช้งานได้ด้วยคำสั่ง CALL ใน MCS-51 จะมีอยู่สองคำสั่งคือ LCALL (long call) และ ACALL (absolute call) โดยทั้งสองคำสั่งต่างกันตรงขนาดของคำสั่ง และระยะห่างของโปรแกรมย่อยที่จะเรียกใช้งาน

LCALL (long call)

คำสั่งนี้จะมีขนาด 3 ไบต์ โดยไบต์แรกจะเป็นออปโค้ด อีกสองไบต์จะเป็นแอดเดรสที่จะกระโดดไปทำงานโปรแกรมย่อย ซึ่งสามารถเรียกโปรแกรมย่อยได้ในระยะ 64 กิโลไบต์ เมื่อโปรแกรมย่อยถูกเรียกใช้ MCS-51 จะนำค่าแอดเดรสของคำสั่งถัดไปหรือค่า PC เก็บลงหน่วยความจำสแตกโดยอัตโนมัติและโหลดค่าแอดเดรสของโปรแกรมย่อยให้กับรีจิสเตอร์ PC และเมื่อทำโปรแกรมย่อไปจนพบคำสั่ง RET (return) MCS-51 จะกลับไปทำคำสั่งของโปรแกรมที่อยู่ต่อจากคำสั่งเรียกโปรแกรมแกรมย่อย โดยคืนค่าที่เก็บอยู่ในสแตกให้กับรีจิสเตอร์ PC ตามเดิม

ตัวอย่างที่ 3.6 จงเขียนโปรแกรมส่งข้อมูลออกทางพอร์ต 1 ให้ทุกบิตมีค่ากลับไปมา โดยส่งคำ 55H และ AAH สลับกัน โดยใช้คำสั่ง LCALL เรียกโปรแกรมหน่วงเวลาระหว่างการข้อมูลแต่ละค่า

วิธีทำ

```
        ORG    0000H
BACK:   MOV    A, #55H        ; โหลด A ด้วยค่า 55H
        MOV    P1, A          ; ส่งค่า 55H ออกไปทางพอร์ต P1
        LCALL  DELAY          ; เรียกโปรแกรมย่อยหน่วงเวลา
        MOV    A, #0AAH       ; โหลด A ด้วยค่า 0AAH
        MOV    P1, A          ; ส่งค่า AAH ออกไปทางพอร์ต P1
        LCALL  DELAY
        SJMP   BACK           ; วนลูปทำซ้ำ

;******* เริ่มต้นโปรแกรมย่อยหน่วงเวลา **************
        ORG    0300H          ; โปรแกรมย่อยเริ่มต้นที่แอดเดรส 300H
DELAY:  MOV    R5, #0FFH      ; ให้ R5 เท่ากับ 255 หรือ 0FFH
AGAIN:  DJNZ   R5, AGAIN      ; ทำซ้ำจน R5 เท่ากับ 0
        RET                   ; จบโปรแกรมย่อย

        END
```

จากตัวอย่างโปรแกรมที่ 3.6 จะเห็นว่ามีการสร้างโปรแกรมย่อยคือ DELAY โดยท้ายของโปรแกรมย่อยจะจบด้วยคำสั่ง RET เมื่อ MCS-51 ทำคำสั่งเรียกโปรแกรมย่อย LCALL DELAY ตัวMCS-51จะเก็บค่าของโปรแกรมเคาน์เตอร์ซึ่งเป็นค่าแอดเดรสของคำสั่ง MOV A,#0AAH ลงหน่วยความจำสแตก และโปรแกรมจะกระโดดไปทำคำสั่งโปรแกรมย่อยที่แอดเดรส 300H ในโปรแกรมย่อยเริ่มแรกจะกำหนดค่าให้รีจิสเตอร์ R5 เท่ากับ 255 (R5 = 0FFH)และจะวนลูปจำนวน 256 ครั้ง เมื่อ R5 เป็นศูนย์จะทำคำสั่ง RET ซึ่งจะอ่านค่าในหน่วยความจำสแตกคืนให้โปรแกรมเคาน์เตอร์เพื่อทำคำสั่งที่ต่อจากคำสั่ง CALL ในตัวอย่างที่ 3.6 จะเห็นว่ามีการเรียกใช้โปรแกรมย่อย ไมโครคอนโทรลเลอร์จะต้องมีการใช้งานหน่วยความจำสแตก ต่อไปจะแสดงการทำงานของหน่วยความจำสแตก จากตัวอย่างที่ 3.6 ถ้าหากแสดงค่าแอดเดรสและรหัสคำสั่งจะแสดงได้ดังนี้

```
001   0000                        ORG    0
002   0000   74 55        BACK:   MOV    A, #55H
003   0002   F5 90                MOV    P1, A
004   0004   12 03 00             LCALL  DELAY
005   0007   74 AA                MOV    A, #0AAH
006   0009   F5 90                MOV    P1, A
007   000B   12 03 00             LCALL  DELAY
008   000E   80 F0                SJMP   BACK
009   0010
010   0010   ****** เริ่มต้นโปรแกรมย่อยหน่วงเวลา *************
011   0300                        ORG    300H
012   0300                DELAY:
013   0300   7D FF                MOV    R5, #0FFH
014   0302   DD FE        AGAIN:  DJNZ   R5, AGAIN
015   0304   22                   RET
016   0305                        END
```

0A	
09	00
08	07
07	

SP = 09

ถ้าหากตัวชี้สแตก (stack pointer) อยู่ที่แอดเดรส 07 เมื่อ MCS-51 ทำคำสั่ง LCALL แอดเดรสของคำสั่งถัดไปคือคำสั่ง MOV A,#0AAH ซึ่งมีค่าเท่ากับ 0007H จะถูกเก็บลงหน่วยความจำสแตกโดยเก็บไบต์ต่ำลงไปก่อนแล้วตามด้วยไบต์สูง โดยตัวชี้สแตกจะเพิ่มตัวเองก่อนแล้วจึงเขียนข้อมูลลงไป ซึ่งจะทำให้แอดเดรส 08 มีค่าเป็น 07 และแอดเดรส 09 มีค่าเป็น 00 หลังจากทำการเรียกโปรแกรมย่อยแล้วจะทำให้ตัวชี้สแตกอยู่ที่แอดเดรส 09 และเมื่อ MCS-51 ทำโปรแกรมย่อยจนพบคำสั่ง RET ไมโครคอนโทรลเลอร์จะอ่านค่าจากหน่วยวความจำสแตกคืนให้กับโปรแกรมเคาน์เตอร์ โดยจะอ่านไบต์สูงออกมาก่อน และลดค่าตัวชี้สแตกลงและอ่านไบต์ต่ำออกมา ทำให้ MCS-51 ทำคำสั่งต่อจากคำสั่ง LCALL ได้ และหลังจากจบการทำโปรแกรมย่อยแล้วตัวชี้สแตกจะอยู่ที่แอดเดรส 07 ตามเดิม

ในหน่วยความจำสแตกเราสามารถนำมาประยุกต์ใช้เก็บข้อมูลชั่วคราวได้ การเขียนข้อมูลลงไปจะใช้คำสั่ง PUSH ส่วนการอ่านข้อมูลออกมาจะใช้คำสั่ง POP ดังตัวอย่าง ที่ 3.7 แต่การใช้งานนั้นข้อมูลที่ถูกเขียนลงไปก่อนจะถูกอ่านออกมาทีหลัง

<u>ตัวอย่างที่ 3.7</u> จงวิเคราะห์การทำงานหน่วยความจำสแตก ของการทำโปรแกรมต่อไปนี้

```
0000                ORG   0
0000  7455   BACK:  MOV   A,#55H      ; โหลดรีจิสเตอร์ A ด้วยค่า 55H
0002  F590          MOV   P1,A        ; ส่งค่า 55H ออกไปทางพอร์ต P1
0004  7C99          MOV   R4,#99H
0006  7D67          MOV   R5,#67H
0008  120300        LCALL DELAY       ; เรียกโปรแกรมย่อยหน่วงเวลา
000B  74AA          MOV   A,#0AAH     ; โหลดรีจิสเตอร์ A ด้วยค่า AAH
000D  F590          MOV   P1,A        ; ส่งค่า AA ออกไปทางพอร์ต P1
000F  120300        LCALL DELAY       ; เรียกโปรแกรมย่อยหน่วงเวลา
0012  80EC          SJMP  BACK        ; ทำซ้ำ
0014                ;****** โปรแกรมย่อยหน่วงเวลา ***************
0300                ORG   300H
0300  C004   DELAY: PUSH  4           ; PUSH R4
0302  C005          PUSH  5           ; PUSH R5
0304  7CFF          MOV   R4,#0FFH
0306  7DFF   NEXT:  MOV   R5,#0FFH
0308  DDFE   AGAIN: DJNZ  R5, AGAIN
030A  DCFA          DJNZ  R4, NEXT
030C  D005          POP   5           ; POP R5
030E  D004          POP   4           ; POP R4
0310  22            RET               ; กลับสู่โปรแกรมหลัก
0311                END               ; จบโปรแกรม
```

<u>วิธีทำ</u>

จากโปรแกรมถ้าหากตัวชี้สแตกอยู่ที่แอดเดรส 07H เมื่อมีการทำคำสั่ง LCALL ในบรรทัดที่ 06 ตัว MCS-51 จะเก็บแอดเดรสของคำสั่ง MOV A,#0AAH ลงสแตกโดยจะเพิ่มค่าตัวเองมาอยู่ที่แอดเดรส 08H แล้วเก็บไบต์ต่ำลงไป จากนั้นเพิ่มค่าตัวเองอีกหนึ่งแล้วเก็บไบต์สูง เมื่อ MCS-51 ไปทำโปรแกรมย่อยจะมีการนำค่าใน R4 และ R5 เก็บลงสแตกด้วยคำสั่ง PUSH และก่อนจบโปรแกรมย่อยจะอ่านในสแตกออกมาด้วยคำสั่ง POP จากโปรแกรมจะเห็นว่าในการเขียน โปรแกรมย่อยจะต้องเก็บค่าของรีจิสเตอร์ต่าง ๆ ที่จะใช้ในโปรแกรมย่อยลงในสแตกเสียก่อน เพื่อให้รีจิสเตอร์ต่าง ๆ ที่ใช้ในโปรแกรมหลักไม่เปลี่ยนแปลงเมื่อทำโปรแกรมย่อย

หลังทำ LCALL

0B		
0A		
09	00	PCH
08	0B	PCL

หลังทำ PUSH 4

0B		
0A	99	R4
09	00	PCH
08	0B	PCL

หลังทำ PUSH 5

0B	67	R5
0A	99	R4
09	00	PCH
08	0B	PCL

โดยทั่วไปแล้วการเขียนโปรแกรมในลักษณะของโปรแกรมย่อยจะมีโครงสร้างของโปรแกรมดังนี้

```
        ORG     0000H
MAIN:   LCALL   SUBR_1          ; เรียกใช้โปรแกรมย่อยชื่อ SUBR_1
        LCALL   SUBR_2          ; เรียกใช้โปรแกรมย่อยชื่อ SUBR_2
        LCALL   SUBR_3          ; เรียกใช้โปรแกรมย่อยชื่อ SUBR_3
        ..............
HERE:   SJMP    HERE
;* * * *  จบโปรแกรมหลัก  * * * *

SUBR_1: ....................
         ..............
        RET                     ; คำสั่ง RET จบโปรแกรมย่อย
;* * * *  จบโปรแกรมย่อย 1 * * * *

SUBR_2: ....................
         ..............
        RET                     ; คำสั่ง RET จบโปรแกรมย่อย
;* * * *  จบโปรแกรมย่อย 2 * * * *

SUBR_3: ....................
         ..............
        RET                     ; คำสั่ง RET จบโปรแกรมย่อย
;* * * *  จบโปรแกรมย่อย 3 * * * *

        END                     ; จบโปรแกรม
```

จากตัวอย่างโปรแกรมที่ผ่านมาจะเป็นการสร้างโปรแกรมย่อยขึ้นมาสามโปรแกรมย่อย นอกจากนี้ในโปรแกรมย่อยยังสามารถเรียกโปรแกรมย่อยได้อีกด้วย

ACALL (Absolute Call)

คำสั่งเรียกโปรแกรมย่อย ACALL เป็นคำสั่งทีมีขนาด 2 ไบต์ โดยใช้ข้อมูล 11 บิตในคำสั่งเป็นค่าแอดเดรส ทำให้สามารถเรียกโปรแกรมย่อยได้ในช่วง 2 กิโลไบต์ การทำงานกับสแตกของคำสั่ง ACALL และ LCALL นี้จะไม่แตกต่างกัน แต่การใช้งานจะต่างกันที่คำสั่ง LCALL เรียกโปรแกรมย่อยได้ในช่วง 64 กิโลไบต์ ส่วนคำสั่ง ACALL เรียกโปรแกรมย่อยได้ในช่วง 2 กิโลไบต์

คำถาม ?

1. รหัสนีโมนิก LCALL และ ACALL
2. จริงหรือไม่การใช้คำสั่ง LCALL สามารถเรียกโปรแกรมย่อยได้ในช่วง 64k ตลอดหน่วยความจำ
3. จะเกิดอะไรขึ้นหลังจากการทำคำสั่ง RET
4. คำสั่ง ACALL เป็นคำสั่งที่มีรหัสกี่ไบต์

3.3 การเขียนโปรแกรมหน่วงเวลา

จากตัวอย่างที่ผ่านมาได้ทดลองใช้โปรแกรมหน่วงเวลาโดยเขียนเป็นโปรแกรมย่อยชื่อ DELAY มาแล้ว โดยโปรแกรมย่อยในลักษณะนี้สามารถใช้หน่วงเวลาการทำงานของโปรแกรมหลักให้ช้าลงได้ ในหัวข้อนี้จะแสดงการคำนวณเวลาที่เสียไปในการทำโปรแกรมหน่วงเวลาต่างๆ

ไซเคิลคำสั่ง

ในการทำคำสั่งของ CPU จะต้องใช้สัญญาณนาฬิกามาคบคุมการทำงาน จำนวนของสัญญาณนาฬิกาที่ใช้จะถูกอ้างเป็นไซเคิลคำสั่งหรือแมชชีนไซเคิล (machine cycle) คำสั่งบางคำสั่งมีจำนวนแมชชีนไซเคิลไม่เท่ากัน ซึ่งจะทำให้เวลาการทำงานคำสั่งบางคำสั่งไม่เท่ากันด้วย โดยทั่วไปแล้วคำสั่งแต่ละคำสั่งจะบอกเวลาเป็นจำนนแมชชีนไซเคิล ไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะทำงานที่ความถี่สัญญาณนาฬิกาตั้งแต่ 4 MHz และหนึ่งแมชชีนไซเคิลจะใช้สัญญาณนาฬิกาจำนวน 12 ลูก

<u>ตัวอย่างที่ 3.8</u> ถ้าหากระบบ MCS-51 ทำงานที่ความถี่สัญญาณนาฬิกาแตกต่างกันดังต่อไปนี้ จงหาค่าเวลาในแต่ละแมชชีนไซเคิล

ก. 11.0592 MHz ข. 16 MHz ค. 20 MHz

<u>วิธีทำ</u>

ก. 11.0592 / 12 = 921.6 MHz ; แมชชีนไซเคิลคือ 1 / 921.6 kHz = 1.085 us

ข. 16 MHz / 12 = 1.333 MHz ; แมชชีนไซเคิลคือ 1 / 1.333 MHz = 0.75 us

ค. 20 MHz / 12 = 1.66 MHz ; แมชชีนไซเคิลคือ 1 / 1.66 MHz = 0.60 us

<u>ตัวอย่างที่ 3.9</u> ระบบ MCS-51 ทำงานที่ความถี่ 11.0592 MHz อยากทราบว่าการทำคำสั่งต่อไปนี้ใช้เวลาเท่าใด

ก. MOV R3,#55 ข. DEC R3 ค. DJNZ R2,target

ง. LJMP จ. SJMP ฉ. NOP

ช. MUL AB

<u>วิธีทำ</u>

หนึ่งแมชชีนไซเคิลที่ความถี่ 11.0592 MHz จะเท่ากับ 1.085 ไมโครวินาที จากตารางคำสั่งของ MCS-51 จะคำนวณเวลาของแต่ละคำสั่งได้ดังนี้

คำสั่ง	แมชชีนไซเคิล	เวลา
MOV R3,#55	1	1 × 1.085 us = 1.085 us
DEC R3	1	1 × 1.085 us = 1.085 us
DJNZ R2,target	2	2 × 1.085 us = 2.17 us
LJMP	2	2 × 1.085 us = 2.17 us
SJMP	2	2 × 1.085 us = 2.17 us
NOP	1	1 × 1.085 us = 1.085 us
MUL AB	4	4 × 1.085 us = 4.34 us

การคำนวณเวลาหน่วง

การเขียนโปรแกรมหน่วงเวลาเพื่อให้ไมโครคอนโทรลเลอร์เสียเวลาไปจะทำได้สองวิธีหลัก ๆ ดังนี้ คือ วิธีแรกจะใช้ตัวนับจับเวลา อีกวิธีหนึ่งจะใช้การวนลูปทำซ้ำดังตัวอย่างที่ 3.10

<u>ตัวอย่างที่ 3.10</u> จงคำนวณเวลาหน่วงจากการทำโปรแกรมต่อไปนี้ ถ้าหากระบบทำงานที่ความถี่ 11.0592 MHz

```
        MOV    A, #55
AGAIN:  MOV    P1, A
        ACALL  DELAY
        CPL    A
        SJMP   AGAIN
;……. Time delay
DELAY:  MOV    R3, #200
HERE:   DJNZ   R3, HERE
        RET
```

<u>วิธีทำ</u>

ในโปรแกรมหน่วงเวลาถ้าหากพิจารณาจำนวนแมชชีนไซเคิลจะคำนวณได้ดังนี้

			แมชชีนไซเคิล
DELAY:	MOV	R3,#200	1
HERE:	DJNZ	R3,HERE	2
	RET		1

ดังนั้นค่าเวลาหน่วงจะเท่ากับ [(200 × 2) + 1 + 1] × 1.085 us = 436.17 us

การเขียนโปรแกรมหน่วงเวลาโดยใช้การวนลูปนี้ค่าเวลานอกจากจะขึ้นกับจำนวนครั้งของการวนลูปแล้วยังขึ้นกับจำนวนคำสั่งของการวนลูปด้วย จากตัวอย่างที่ผ่านมาจะเห็นว่าค่าในรีจีสเตอร์ R3 จะมีค่าสูงสุดได้ 255 ทำให้ทำซ้ำได้ 255 ครั้ง และภายในเราสามารถให้ CPU เสียเวลาไป 1 แมชชีนไซเคิลได้โดยใช้คำสั่ง NOP ดังตัวอย่างต่อไปนี้

<u>ตัวอย่างที่ 3.11</u> จงหาเวลาหน่วงจากการทำโปรแกรมย่อยต่อไปนี้ ถ้าหากระบบทำงานที่ความถี่ 11.0592 MHz

			แมชชีนไซเคิล
DELAY:	MOV	R3,#250	1
HERE:	NOP		1
	NOP		1
	NOP		1
	DJNZ	R3, HERE	2
	RET		1

<u>วิธีทำ</u>

จากโปรแกรมหน่วงเวลาในลูป HERE จะใช้เวลาทั้งหมดเท่ากับ [250 (1 + 1 + 1 + 1 + 2)] × 1.085 us = 1500 × 1.085 us = 1627.5 us ถ้ารวมกับคำสั่งภายนอกลูปอีกสองคำสั่งจะใช้เวลาทั้งหมด 1627.5 us + 2 × 1.085 us = 1629.67 us

การหน่วงเวลาแบบทำลูปสามารถทำโปรแกรมแบบลูปซ้อนลูปได้ ที่เรียกว่า nested loop ดังตัวอย่างโปรแกรมที่ 3.12

<u>ตัวอย่างที่ 3.12</u> ถ้าหากเวลาหนึ่งแมชชีนไซเคิลเท่ากับ 1.085 us จงหาเวลาหน่วงจากโปรแกรมต่อไปนี้

```
DELAY:                          แมชชีนไซเคิล
        MOV     R2,#200         1
AGAIN:  MOV     R3,#250         1
HERE:   NOP                     1
        DJNZ    R3,#HERE        2
        DJNZ    R2,AGAIN        2
        RET                     1
```

จากโปรแกรมจะพบว่ามีลูป HERE เป็นลูปในซึ่งจะใช้รีจีสเตอร์ R3 เป็นตัวนับจำนวนลูป และลูปนอกคือลูป AGAIN จะใช้รีจีสเตอร์ R2 เป็นตัวนับ สำหรับลูป HERE จะใช้เวลาทั้งหมด (4 × 250) × 1.085 us = 1085 us ส่วนของลูป AGAIN จำนวน 200 ครั้ง ดั้งนั้นเวลาที่เสียไปทั้งหมดจะเท่ากับ 200 × 1085 us = 21700 us และในลูป AGAIN ยังต้องทำอีกสองคำสั่งคือ MOV R3,#250 และ DJNZ R2,AGAIN ซึ่งต้องเสียเวลาไปอีก 3 × 200 × 1.085 us = 615 us ดังนั้นเวลาที่เสียไปทั้งหมดจะเท่ากับ 217000 + 651 = 217651 = 217.651 มิลลิวินาที

คำถาม ?

1. จริงหรือไม่ไมโครคอนโทรลเลอร์เบอร์ 8051 นั้นหนึ่งแมชชีนไซเคิลใช้สัญญาณนาฬิกา 12 ลูป
2. คำสั่งของ MCS-51 ที่ใช้แมชชีนไซเคิลน้อยที่สุดจะใช้กี่แมชชีนไซเคิล
3. ถ้าหาก MCS-51 ที่ทำงานที่ความถี่ 12 MHz อยากทราบว่าโปรแกรมย่อย DELAY ต่อไปนี้จะหน่วยเวลาได้เท่าใด

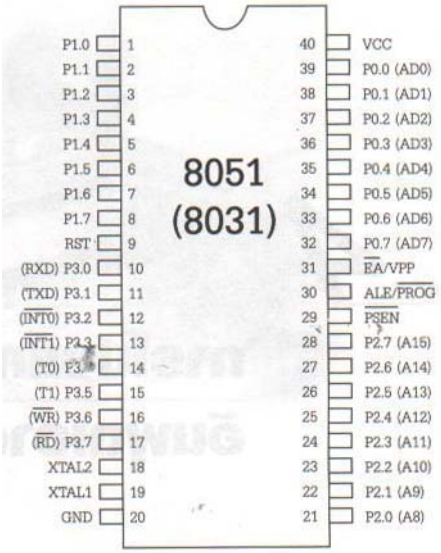
```
DELAY: MOV     R3,#100
HERE:  NOP
       NOP
       NOP
       DJNZ    R3,HERE
       RET
```

บทที่ 4
การโปรแกรมพอร์ตอินพุตเอาต์พุต

การใช้งานไมโครคอนโทรลเลอร์มักจะใช้ในการควบคุมอุปกรณ์หรือวงจรอิเล็กทรอนิกส์ภายนอกการที่ไมโครคอนโทรลเลอร์จะติดต่อกับอุปกรณ์ภายนอกได้จะต้องติดต่อผ่านทางพอร์ต ในบทนี้จะแสดงขาต่าง ๆ ของ MCS-51 เบอร์มี่เป็นมาตรฐานในการใช้งานทั่วไป และตัวอย่างการเขียนโปรแกรมติดต่อกับพอร์ตขนานที่อยู่ในตัวมัน

4.1 การจัดขาของ MCS-51

ไอซีไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีลักษณะตัวถังอยู่หลายรูปแบบ อย่างเช่นถ้าเป็นเบอร์ 8051, 8031, 8751, 89C51, 40 ขา, และเป็นตัวถังแบบ DIP (Dual In-line Package) โดยจะประกอบด้วยขาแอดเดรส ขาข้อมูล ขาอินเตอร์รัปต์ และขาควบคุมต่าง ๆ เช่น I/O, RD, WR, เป็นต้น



รูปที่ 4.1 การจัดขาของ 8051 (8031)

ในรูปที่ 4.1 จะเป็นตัวถัง 40 ขา โดยเป็นขาของพอร์ตขนาน 4 พอร์ตคือ P0, P1, P2, และ P3, จำนวน 32 ขา ส่วนอีก 8 ขา เป็น Vcc , GND , XTAL1 , XTAL2 , RST , EA , PSEN , และ ALE โดยแต่ละตัวมีหน้าที่ดังนี้

Vcc

เป็นขาที่ 40 ใช้ต่อกับแหล่งกำเนิดแรงดัน + 5V

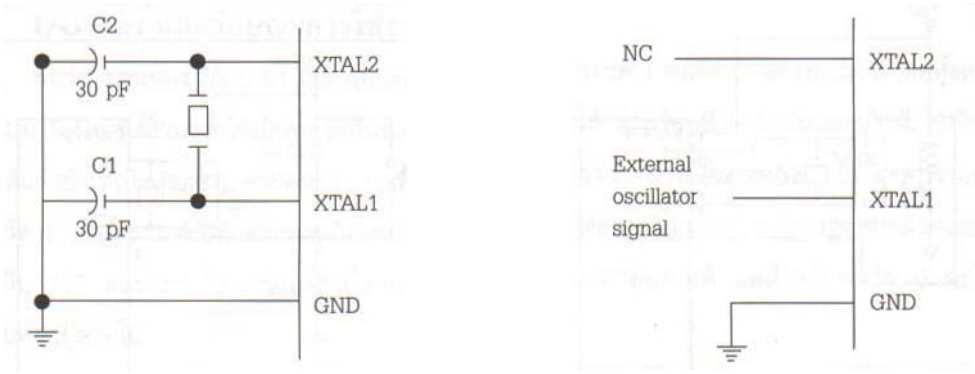
GND

เป็นขาที่ 20 ใช้ต่อกับกราวด์

XTAL1 และ XTAL2

แม้ว่าตัว MCS-51 จะมีวงจรกำเนิดสัญญาณนาฬิกาอยู่ภายในชิป แต่ก็ต้องต่อคริสตอล (crystal) ภายนอกเพื่อควบคุมสัญญาณนาฬิกา โดยต่อเข้าทาง XTAL1 และ XTAL2 ดังรูปที่ 4.2 (ก) โดยมีตัวเก็บ

ประจุ 30 pF ต่ออยู่ด้วย แต่ถ้า MCS-51 ต้องการสัญญาณนาฬิกาจากภายนอกก็สามารถทำได้ โดยต่อเข้าทาง XTAL1 ดังรูป 4.2 (ข)



(ก) การต่อ XTAL1 กับ 8051 (ข) การต่อวงจรกำเนิดสัญญาณโดยตรง

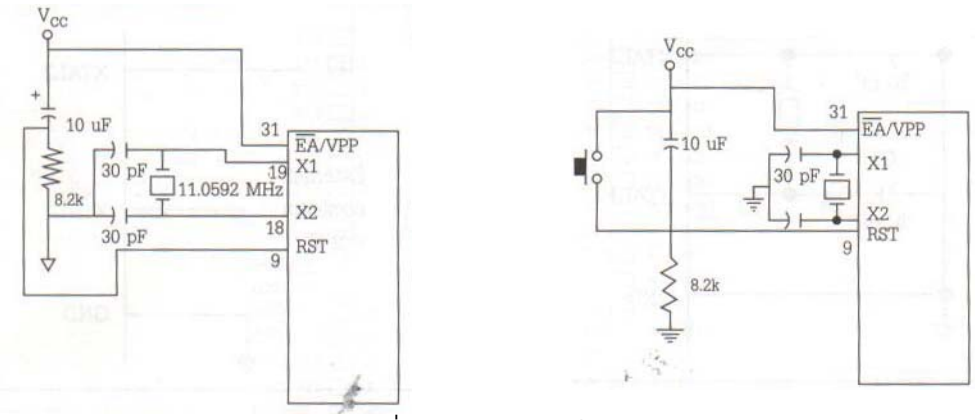
รูปที่ 4.2

RST

เป็นขารีเซต (reset) เมื่อต้องการรีเซต MCS-51 จะต้องทำให้ขานี้เปลี่ยนจากลอจิก “0” เป็นลอจิก “1” หลังจาก MCS-51 ถูกรีเซต ค่าในรีจีสเตอร์ต่าง ๆ จะเป็นดังนี้

รีจีสเตอร์	ค่าเริ่มต้น
PC	0000
Acc	00
B	00
PSW	00
SP	07
DPTR	0000

เมื่อ PC มีค่าเป็น 0 จะทำให้ CPU เริ่มทำคำสั่งแรกที่อยู่ในหน่วยความจำ ROM ที่แอดเดรส 0000 และทำชุดคำสั่งต่าง ๆ ไปตามโปรแกรม การรีเซต MCS-51 จะมีสองแบบคือ การรีเซตเมื่อเริ่มต้นจ่ายไฟให้กับวงจร เรียกว่า power-on reset อีกวิธีหนึ่งเป็นการรีเซตเมื่อมีการจ่ายไฟให้กับวงจรอยู่แล้วแต่จะกดปุ่มรีเซตวงจร การต่อวงจรรีเซตทั้งสองประเภททำได้ดังรูป 4.3 สำหรับ MCS-51 เบอร์ที่เป็นมาตรฐานเช่น 8051 ตัวมันจะถูกรีเซตได้นั้นจะต้องให้ขา RST เป็นลอจิก “1” อย่างน้อย 2 แมชชีนไซเคิล การหาช่วงเวลาในและแมชชีนไซเคิลทำได้ดังตัวอย่างที่ 4.1



รูปที่ 4.3 การต่อวงจรรีเซต

<u>ตัวอย่างที่ 4.1</u> จงหาเวลาแมชชีนไซเคิลถ้าหาก XTAL ที่ต่ออยู่มีความถี่ดังนี้

ก. XTAL = 11.0592 MHz ข. XTAL = 16 MHz

<u>วิธีทำ</u>

ก. 11.0592 MHz / 12 = 921.6 kHz

แมชชีนไซเคิล = 1 / 921.6 kHz = 1.085 ไมโครวินาที

ข. 16 MHz / 12 = 1.333 MHz

แมชชีนไซเคิล = 1 / 1.333 kHz = 0.75 ไมโครวินาที

EA (External Access)

ตามที่ทราบมาแล้วว่า MCS-51 บางรุ่นมีหน่วยความจำโปรแกรมภายในชิปเช่น 8751, 89C51 เป็นต้น บางรุ่นต้องต่อหน่วยความจำโปรแกรมภายนอกชิป เช่น 8031, 8032 เป็นต้น ถ้าหากขา EA นี้ต่อกับลอจิก “1” หมายความว่าให้อ่านโปรแกรมจากหน่วยความจำภายในชิป แต่ถ้าเป็น MCS-51 ที่มีหน่วยความจำ ROM อยู่ภายนอก ขา EA นี้ต้องต่อลง GND ให้เป็นลอจิก “0” เสมอ เพื่อให้โปรแกรมที่อยู่ใน ROM ภายนอกชิป

PSEN (Program Store Enable)

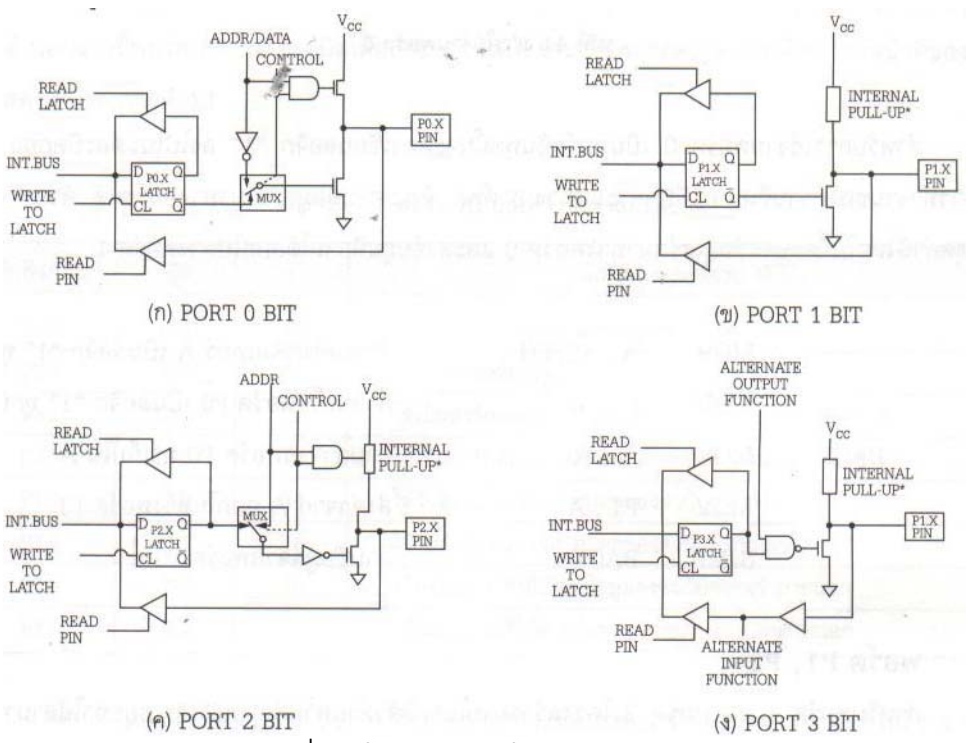
ขานี้จะแอกทีฟเป็นลอจิก “0” เมื่อ MCS-51 ต้องการอ่านโปรแกรมจากหน่วยความจำ ROM ภายนอกโดยทั่วไปแล้วมักจะนำขานี้ต่อกับขา OE ของหน่วยความจำ ROM

ALE (Address Latch Enable)

เป็นขาเอาต์พุตที่แอกทีฟลอจิก “1” โดย CPU จะส่งสัญญาณนี้ออกมาเอง เนื่องจากพอร์ต P0 นี้สามารถทำงานได้สองหน้าที่คือเป็นขาแอดเดรส และขาข้อมูล ตัว MCS-51 จะใช้สัญญาณทางขา ALE นี้มัลติเพล็กซ์พอร์ต P0 ว่าขณะนั้นจะใช้งานเป็นแอดเดรส หรือใช้งานเป็นขาข้อมูล โดยทั่วไปแล้วในวงจรระบบไมโครคอนโทรลเลอร์ MCS-51 จะต่อขานี้กับขา G ของชิป 74LS373

I/O Port และลักษณะการใช้งาน

พอร์ตขนานของ MCS-51 มี 4 พอร์ตคือ พอร์ต 0 พอร์ต 1 พอร์ต 2 และพอร์ต 3 โดยทั้งหมดเป็นพอร์ตแบบ 8 บิต ซึ่งสามารถใช้เป็นพอร์ตอินพุตหรือเอาต์พุตได้ สำหรับพอร์ต 3 นั้นนอกจากจะใช้เป็นพอร์ตตามปกติแล้วใช้เป็นเป็นสัญญาณควบคุมต่างๆอีกด้วย จากโครงสร้างภายในของ MCS-51 จะพบว่าต่ละบิตของพอร์ต 1,2 และ 3 จะมีตัวต้านทานที่มีค่าประมาณ 50k ต่อ pull-up อยู่ ทำให้สภาวะปกติแต่ละบิตจะมีลอจิกเป็น “1”และจะทำให้พอร์ตเหล่านี้สามารถขับอุปกรณ์ภายนอกได้ แต่สำหรับพอร์ต 0 จะไม่มีตัวต้านทานต่ออยู่ภายใน

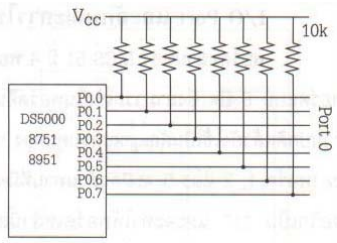


รูปที่ 4.4 โครงสร้างภายในของพอร์ตขนาน

พอร์ต 0

พอร์ตนี้อยู่ที่ขา 32 ถึง 39 นอกจากจะใช้งานเป็นพอร์ตอินพุตเอาต์พุตแล้ว ยังถูกใช้เป็นขาแอดเดรสและขาข้อมูลด้วย โดยแต่ละบิตจะมีชื่อเป็น AD0-AD7 สำหรับการมัลติเพล็กซ์ระหว่างขาแอดเดรสกับขาข้อมูลจะต้องทำงานร่วมกับวงจรภายนอก และสัญญาณจากขา ALE จากโครงสร้างภายในของมันจะพบว่าแต่ละบิตของพอร์ตนี้จะเป็นวงจรแบบ open drain ถ้าชิปถูกสร้างแบบ TTL โดยจะไม่มีตัวต้านทานต่อ pull-up อยู่ภายใน โดยทั่วไปแล้วถ้าหากต้องการใช้พอร์ตนี้เป็นพอร์ตอินพุตเอาต์พุตจะต้องนำตัวต้านทานขนาด 10k มาต่อ pull-up ภายนอกดังรูป 4.5 และถ้าหากต้องการเขียนชุดคำสั่งเพื่อส่งข้อมูลออกทางพอร์ต0 ให้มีค่าลอจิกกลับไปกลับมา สามารถเขียนชุดคำสั่งได้ดังต่อไปนี้

```
        MOV     A,#55H
BLACK:  MOV     P0,A
        ACALL   DELAY
        CPL     A
        SJMP    BACK
```



รูปที่ 4.5 การใช้งานพอร์ต 0

สำหรับการใช้งานพอร์ต 0 เป็นพอร์ตอินพุตนั้นจะต้องเขียนลอจิก “1” ลงไปในแต่ละบิตก่อน เพื่อหยุดการทำงานของทรานซิสเตอร์ที่ขับกระแสทางเอาต์พุต จึงจะอ่านข้อมูลเข้ามาทางอินพุตได้ ตัวอย่างการเขียนชุดคำสั่งต่อไปนี้จะอ่านข้อมูลเข้ามาทางพอร์ต 0 และส่งข้อมูลที่อ่านได้ออกไปทางพอร์ต 1

```
        MOV     A,#0FFH     ; กำหนดให้รีจิสเตอร์ A เป็นลอจิก”1”ทุกบิต
        MOV     P0,A        ; กำหนดให้พอร์ตP0 เป็นลอจิก”1”ทุกบิต
BLACK:  MOV     A,P0        ; อ่านค่าจากพอร์ต P0 มาเก็บไว้ใน A
        MOV     P1,A        ; ส่งค่าจากA ออกไปทางพอร์ต P1
        SJMP    BLACK       ; อ่านข้อมูลจากพอร์ตP0 ซ้ำ
```

พอร์ต P1, P2

สำหรับพอร์ต 1 และพอร์ต 2 โครงสร้างภายในจะมีตัวต้านทานต่อ Pull-up อยู่ ทำให้สามารถใช้งานเป็นพอร์ตอินพุตเอาต์พุตได้ทันที นอกจากนี้พอร์ต P2 ยังถูกใช้งานเป็นขาแอดเดรสไบต์สูงด้วย (A8-A15) ถ้าต้องการใช้งานพอร์ตนี้เป็นพอร์ตเอาต์พุตสามารถทำได้โดยการเขียนข้อมูลลงไปใน P1 หรือ P2 ได้ทันที แต่ถ้าต้องการใช้งานพอร์ตนี้เป็นพอร์ตอินพุตจะต้องส่งลอจิก"1"ออกไปในแต่ละบิตเสียก่อน ตัวอย่างเช่นถ้าหากต้องการอ่านข้อมูลจากพอร์ต P1 มาเก็บในรีจิสเตอร์ R7,R6 และR5 สามารถเขียนชุดคำสั่งได้ดังนี้

```
MOV     A,#0FFH     ; ให้Aเป็นลอจิก"1"ทุกบิต
MOV     P1,A        ; ให้P1เป็นอินพุต  โดยกำหนดให้เป็น"1"ทุกบิต
MOV     A,P1        ; อ่านข้อจากP1มาเก็บไว้ในA
MOV     R7,A        ; เก็บข้อมูลลงR7
ACALL   DELAY       ; หน่วงเวลา
MOV     A,P1        ; อ่านข้อมูลจากP1มาเก็บไว้ในA
MOV     R6,A        ; เก็บข้อมูลลงR6
ACALL   DELAY       ; หน่วงเวลา
MOV     A,P1        ; อ่านข้อมูลจากP1มาเก็บในA
MOV     R5,A        ; เก็บข้อมูลลงR5
```

พอร์ต P3

พอร์ต 3 นี้เป็นพอร์ตขนาดแบบ 8 บิตเช่นกัน สามารถใช้งานเป็นพอร์ตอินพุตเอาต์พุตเช่นP1 และP2ได้ นอกจากนี้พอร์ตP3 ในแต่ละบิตได้ออกแบบไว้ให้ใช้งานเป็นพอร์ตควบคุมอีกด้วย โดยหน้าที่ของแต่ละบิตแสดงได้ดังตาราง 4.1

ตาราง 4.1พอร์ต P3 และการทำงานแต่ละบิต

ชื่อฟังก์ชัน	บิต	ขา	ลักษณะการใช้งาน
P3.0	RxD	10	ใช้เป็นขารับข้อมูลเข้าแบบอนุกรม
P3.1	TxD	11	ใช้เป็นขาส่งข้อมูลออกแบบอนุกรม
P3.2	INT0	12	ใช้เป็นขารับสัญญาณอินเทอร์รัปต์จากภายนอกหมายเลข0
P3.3	INT1	13	ใช้เป็นขารับสัญญาณอินเทอร์รัปต์จากภายนอกหมายเลข1
P3.4	TO	14	ใช้รับสัญญาณจากภายนอกของตัวนับหมายเลข0
P3.5	T1	15	ใช้รับสัญญาณจากภายนอกของตัวนับหมายเลข1
P3.6	WR	16	เป็นสัญญาณที่ใช้เขียนข้อมูลลงหน่วยความจำภายนอก
P3.7	RD	17	เป็นสัญญาณที่ใช้อ่านข้อมูลลงหน่วยความจำภายนอก

การศึกษาเรื่องพอร์ตขนานของMCS-51 มาทั้งหมดจะพบว่ามีพอร์ตขนานให้เลือกใช้งานถึง 4 พอร์ตแม้ว่าพอร์ต P1,P2 และ P3 จะมีตัวต้านทานต่อพูลอัพอยู่ภายใน แต่เนื่องจากตัวตานทานนี้มีค่ามาก (ประมาณ 50 k) ทำให้แต่ละบิตขับกระแสภายนอกได้ไม่มากนักในการใช้งานทั่วไปแล้วถ้าต้องการนำพอร์ตเหล่านี้ขับขับกระแสให้กับวงจรภายนอกมักจะต้องต่อไอซีที่เป็นบัฟเฟอร์เพิ่มเข้าไป หรือนำตัวต้านทานขนาด 1k ต่อขนาดอยู่ภายนอกเพื่อให้สามารถขับกระแสได้มากขึ้น ถ้าหากต้องการใช้งานพอร์ตขนานเพียงพอร์ตเดียวควรจะเลือกใช้พอร์ต P1 มากที่สุด เนื่องจากพอร์ตตัวนี้จะไม่ถูกใช้ในหน้าที่อื่นๆอีก

บทที่ 5
คำสั่งทางคณิตศาสตร์และการโปรแกรม

ในบทนี้จะกล่าวถึงคำสั่งทางคณิตศาสตร์ทั้งหมดของ MCS-51 ซึ่งมีทั้งแบบที่ทำกับตัวเลขแบบคิดเครื่องหมายและไม่คิดเครื่องหมาย โดยหัวข้อที่ 5.1 จะกล่าวถึงคำสั่งการบวกลบเลขแบบไม่คิดเครื่องหมาย รวมทั้งตัวเลขข้อมูลแบบ BCD (Binary Coded Decimal) ส่วนคำสั่งคูณและหารจะกล่าวถึงในหัวข้อที่ 5.2 ส่วนหัวข้อที่ 5.3 จะกล่าวถึงตัวเลขแบบคิดเครื่องหมาย

5.1การบวกลบเลขแบบไม่คิดเครื่องหมาย

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 จะประมวลผลกับข้อมูลขนาด 8 บิตมาเก็บตัวเลขแบบไม่คิดเครื่องหมายจะเก็บได้ตั้งแต่ 00 ถึง FF (0 ถึง 255)

การบวกแบบไม่คิดเครื่องหมาย

การบวกเลขของMCS-51 จะกระทำกับรีจิสเตอร์แอกคิวมูเลเตอร์หรือรีจิสเตอร์ A โดยจะใช้คำสั่ง ADD ซึ่งมีรูปแบบดังนี้

```
ADD     A, source                ; A=A + Source
```

คำสั่ง ADD จะมีโอเปอแรนต์สองตัว โดยโอเปอแรนต์ปลายทางต้องเป็นรีจิสเตอร์ A เท่านั้น ส่วนโอเปอแรนต์ต้นทางจะเป็นรีจิสเตอร์ ข้อมูล หรือตำแหน่งหน่วยความจำก็ได้ เมื่อมีการทำคำสั่งแล้วจะมีผลต่อบิตแฟลก AC, CY หรือ P ในรีจิสเตอร์แฟลก ตัวอย่างการใช้คำสั่งแสดงได้ดังตัวอย่างที่5.1

<u>ตัวอย่างที่ 5.1</u> จงแสดงการบวกเมื่อมีการทำคำสั่งต่อไปนี้

```
MOV     A,#0F5H                  ; A=F5H
ADD     A,#0BH                   ; A=F5+0B=00
```

<u>วิธีทำ</u>

```
  F5H          1111  0101
+ 0BH        + 0000  1011
-----        -----------
100H           0000  0000
```

หลังจากการบวกค่าในรีจิสเตอร์A จะเป็น 00 ส่วนค่าของบิตแฟลกจะเป็นดังนี้

CY = 1 มีการทดจากบิต D7

P = 1 เพราะเป็น 0 ทุกบิต

AC = 1 มีการทดจากบิตD3 ไปยัง D4

การบวกหลายไบต์

เราสามารถใช้คำสั่ง ADD บวกข้อมูลหลายๆไบต์ได้ แต่ผลรวมจะต้องไม่เกินค่าที่รีจิสเตอร์ 8 บิต จะเก็บได้ ซึ่งก็คือค่า FFH ในตัวอย่างที่5.2 เป็นตัวอย่างการข้อมูลจำนวน 5ไบต์ โดยใช้รีจิสเตอร์ R7 และแอกคิวมูเลเตอร์ A เก็บผลลัพธ์

<u>ตัวอย่างที่5.2</u> ถ้าหากมีข้อมูลเลขฐานสิบหกอยู่ใน RAM ตำแหน่ง 40 ถึง 44 จงเขียนโปรแกรมบวกค่าของข้อมูลดังกล่าวและให้ผลลัพธ์ไบต์ต่ำเก็บในรีจิสเตอร์ A ไบต์สูงเก็บในรีจิสเตอร์ R7

40H = [7D]

41H = [EB]

42H = [C5]

43H = [5B]

44H = [30]

<u>วิธีทำ</u>

```
        MOV   R0,#40H      ; ใส่ค่าให้พอยน์เตอร์
        MOV   R2,#5        ; ใส่ค่าให้คัวนับ
        CLR   A            ; A=0
        MOV   R7,A         ; เคลียร์ค่า R7
AGAIN:  ADD   A,@RO        ; บวกค่าในAกับค่าในตำแหน่งR0
        JNC   NEXT         ; ถ้าCY=0 หมายความว่าไม่มีการทดจากA
        INC   R7           ; เพิ่มค่าR7
NEXT:   INC   R0           ; เพิ่มค่าพอยน์เตอร์
        DJNZ  R2, AGAIN    ; ทำซ้ำจนกว่าR2จะเป็นศูนย์
```

จากตัวอย่างที่5.2 จะเห็นว่าจะใช้การวนลูปในการบวกเลข โดยมีลำดับการทำงานดังนี้

1. ลูปแรกจะบอกค่า 7DH กับรีจิสเตอร์A จะได้ค่าCY = 0 และ R7 = 00 ส่วนตัวนับR2 = 04
2. ลูปที่สองจะบวกค่า EBH กับรีจิสเตอร์A ผลลัพธ์จะได้ A = 68H และ CY = 1 ซึ่งจะทำการเพิ่มค่าในรีจิสเตอร์ R7 ส่วนตับนับ R2 = 03
3. ต่อไปบวกค่า C5H กับรีจิสเตอร์ A จะได้ A = 2DH และมีการทดอีกครั้ง ทำให้ค่าในR7 เพิ่มอีกครั้งส่วนตัวนับ R2 = 02

เมื่อการทำงานไปถึงลูปสุดท้าย ผลลัพธ์จะเก็บในรีจิสเตอร์ A และ R7 โดยรีจิสเตอร์ A เก็บไบต์ต่ำ และ R7 เก็บไบต์สูง

คำสั่ง ADDC กับการบวกเลข 16 บิต

เนื่องจากคำสั่งการบวกของ MCS-51 จะบวกเลขแบบ 8 บิต ถ้าหากเขียนคำสั่งสำหรับบวกข้อมูลขนาด 16 บิต หรือสองไบต์จะต้องบวกข้อมูลทีละไบต์ โดยบวกไบต์ต่ำก่อนแล้วจึงบวกไบต์สูง โดยการบวกไบต์สูงจะต้องนำตัวทดจากการบวกไบต์ต่ำมาร่วมบวกด้วย เมื่อไบต์ต่ำบวกกันแล้วมีตัวทดเกิดขึ้น ตัวทดนั้นจะถูกเก็บในบิตแฟลกตัวทด สำหรับคำสั่งที่บวกโดยการนำตัวทดมาบวกด้วยคือคำสั่ง ADDC (Add with carry)ตัวอย่างการบวกข้อมูลแบบ 16 บิต เช่นการบวกค่าของ 3CE7H กับ 3B8DH สามารถทำได้ดังนี้

```
   1          ตัวทดจากไบต์ต่ำ
  3C E7
+ 3B 8D
  78 74
```

เมื่อบวกไบต์แรก(E7 + 8D + 74, CY = 1) จะมีตัวทดเกิดขึ้น และต้องนำไปบวกในไบต์ถัดไป ดังนั้นการบวกไบต์สูงจะเป็น 3C + 3B + 1 = 78

<u>ตัวอย่าง 5.3</u> จงเขียนโปรแกรมบวกข้อมูลขนาด 16 บิต ระหว่าง 3CE7H กับ 3BBDH โดยเก็บคำตอบในรีจิสเตอร์ R7 และ R6 โดย R6 เก็บไบต์สูง

<u>วิธีทำ</u>

```
CLR   C          ; ให้ค่าCY = 0
MOV   A,#0E7H    ; โหลดค่าไบต์ต่ำ A = E7H
ADD   A,#8DH     ; บวกค่าไบต์ต่ำได้ผลลัพธ์เป็น A = 74H และ CY = 1
MOV   R6,A       ; เก็บค่าไบต์ต่ำลงในรีจิสเตอร์ R6
MOV   A,#3CH     ; โหลดค่าไบต์สูง
ADDC  A,3BH      ; บวกค่าไบต์สูงโดยบวกบิตตัวทดด้วย
                 ; 3B+3C+1=78
MOV   R7,A       ; เก็บค่าไบต์สูงของผลบวก
```

ระบบเลข BCD

เลข BCD (Binary Coded Decimal) เป็นการนำเลขฐานสองขนาด 4 บิต มาแทนตัวเลข 0 ถึง 9 ของเลขฐานสิบแต่ละหลัก ทำให้เราสามารถนำเลขฐานสิบมาเก็บในรูปแบบรหัสเลขฐานสองได้ง่าย โดยถ้ามีเลขฐานสิบค่าหนึ่งก็ให้แทนค่าแต่ละหลักด้วยเลขฐานสองขนาด 4 บิต ดังรูปที่ 5.1 โดยเลข BCD จะมีสองรูปแบบ คือ 1.unpacked BCD และ 2.packed BCD

เลขฐานสิบ	เลข BCD	เลขฐานสิบ	เลข BCD
1	0001	15	0001 0101
2	0010	123	0001 0010 0011
8	1000	256	0010 0101 0110
10	0001 0000	702	0111 0000 0010
12	0001 0010	90	1001 0000

รูปที่ 5.1

Unpacked BCD

ตัวเลขแบบ unpacked BCD จะใช้เลขฐานสองจำนวน 8 บิต หรือหนึ่งไบต์แทนเลขฐานสิบแต่ละหลัก แต่จะให้ 4 บิตบนเป็น 0 ทั้งหมด และ 4บิตล่างแทนเลข 0 ถึง 9 ตัวอย่างเช่นค่า 0000 1001 และ 0000 0101 จะเป็นเลข unpacked BCD ที่แทนค่า 9 และ 5 ตามลำดับ

Packed BCD

ตัวเลขแบบ packed BCD นี้เลขฐานสองหนึ่งไบต์จะแทนเลข BCD ได้สองหลัก คือ 4 บิตล่างหนึ่งหลัก และ 4 บิตบนอีกหนึ่งหลัก ตัวอย่างเช่น 0101 1001 จะเป็นเลข packed BCD ที่แทนค่า 59H ดังนั้นทำให้ข้อมูลขนาด 1 ไบต์สามารถเก็บข้อมูลในรหัส BCD ได้สองหลัก

การใช้คำสั่งในบวกเลขตามระบบ BCD นั้นจะต้องมีการปรับค่าหลังการบวกเพื่อให้คำตอบที่ได้อยู่ในรูปแบบของเลข BCD พิจารณาการบวกต่อไปนี้

เลขฐานสิบ	เลข BCD
17	0001 0111
+ 28	+ 0010 1000
45	0011 1111

ถ้าเขียนเป็นคำสั่งจะได้เป็น

```
MOV     A,#17H
ADD     A,#28H
```

จากตัวอย่างจะเห็นว่าถ้ามีการบวกเลขฐานสิบคำตอบที่ได้จะเป็น 45 แต่ถ้าเราแทนตัวเลขคำตอบนี้ด้วยรหัส BCD และให้ไมโครคอนโทรลเลอร์บวกกันผลลัพธ์ที่ได้จะเป็น 0011 1111B(3FH) ซึ่งจะไม่ใช่เลข BCD เพราะเลข BCD ต้องมีค่าอยู่ระหว่าง 0000 ถึง 1001 (หรือ 0 ถึง 9) ถ้าเป็นเลข BCD ที่ถูกต้องผลลัพธ์จะต้องเป็น 0100 0101 หรือ 45H ดังนั้นจะต้องเขียนโปรแกรมแก้ไขผลลัพธ์ที่ได้ โดยนำ 6(0110) เข้าไปบวกกับเลข 4 บิตต่ำจะเป็น 3FH + 06H = 45H ทำให้ผลลัพธ์ที่ออกมาเป็นเลข BCD ที่ถูกต้องต่อไปพิจารณาการบวกเลข BCD ระหว่าง 52H กับ 87H

```
  52          52H
 +87        + 87H
 ---        -----
 139          D9
```

จะเห็นว่าผลลัพธ์จากการบวกเลข BCD จะได้ค่า D9 ซึ่งไม่เป็นเลข BCD จะต้องแก้ไขโดยการบวกค่า 6 เข้าไปในหลักสูง จะได้เป็น D9H + 60H = 139H ซึ่งจะทำให้ผลลัพธ์ที่ได้เป็นเลข BCD ที่ถูกต้อง เราจะเห็นว่าในการบวกเลข BCD ถ้าหากผลลัพธ์ที่ออกมาไม่เป็นเลข BCD เราจะต้องแก้ไขข้อมูลโดยนำค่า 6 ไปบวกกับเลขฐานสองในหลักที่ไม่เป็นเลข BCD ใน MCS-51 มีคำสั่งที่ใช้รับค่าหลังจากการบวกให้เป็นเลข BCD คือคำสั่ง DA A

คำสั่ง DA

MCS-51 มีคำสั่ง DA (decimal adjust for addition) เพื่อใช้ในคำสั่งบวกเลขแบบ BCD โดยจะใช้ค่าหลังจากการบวกให้ผลลัพธ์เป็นเลข BCD คำสั่งนี้จะมีเอเปอแรนต์ตัวเดียว คือ รีจิสเตอร์ A โดยคำสั่ง DA จะทำการบวกค่า 6 เข้าไปในสี่บิตต่ำถ้าหากค่าในสี่บิตต่ำมีค่ามากกว่า 9 ตัวอย่างการใช้คำสั่ง DA แสดงได้ดังนี้

```
MOV     A,#47H      ; ให้ A = 47 เป็นเลข BCD ค่าแรก
MOV     B,#25H      ; ให้ A = 25 เป็นเลข BCD ค่าสอง
ADD     A,B         ; บวกแบบไบนารี ได้ผลลัพธ์เป็น A= 72H
DA      A           ; ปรับค่าหลังการบวกให้เป็นเลข BCD คือ A=72H
```

จากตัวอย่างโปรแกรมผลลัพธ์จากการบวกจะเท่ากับ 72H (47 + 25 = 72) โดยใช้คำสั่ง DA จะใช้หลังคำสั่งบวก และจะทำให้ผลลัพธ์ที่ได้ไม่มีหลักใดอยู่ระหว่าง A ถึง F โดยทั่วไปคำสั่ง DA นี้จะใช้หลังคำสั่ง ADD หรือ ADDC จะไม่ใช้หลังคำสั่ง INC โดยมีขั้นตอนการดำเนินการของคำสั่งจะเป็นดังนี้

1.ถ้าสี่บิตต่ำมีค่ามากกว่า 9 หรือถ้า AC = 1 จะทำการบวก 4 บิตต่ำด้วย 0110

2.ถ้าสี่บิตบนมีค่ามากกว่า 9 หรือถ้า CY = 1 จะทำการบวก 4 บิตบนด้วย 0110

ตัวอย่างเช่นถ้าหากบวกเลข BCD ระหว่าง 29H กับ 18H ผลลัพธ์จะเป็น 41H โดยมีขั้นตอนดังนี้

HEX	BCD	
29	0010 1001	
+ 18	+ 0001 1000	
41	0100 0001	AC = 1
+ 6	+ 0110	
47	0100 0111	

จะเห็นว่าถ้า AC = 1 คำสั่ง DA A จะบวก 6 เข้ากับ 4 บิตต่ำ

<u>ตัวอย่างที่ 5.4</u> ถ้าหากมีเลข BCD จำนวน 5 ตัวอยู่ในหน่วยความจำ RA< ตั้งแต่ตำแหน่ง 40H เป็นต้นไป จงเขียนโปรแกรมบวกเลขแบบ BCD ทั้งหมด และเก็บผลลัพธ์เป็นเลข BCD

	44	37
	43	59
	42	65
	41	11
R0→	40	71

<u>วิธีทำ</u>

```
        MOV    R0,#40H        ; โหลดตัวชี้ไปยังแอดเดรส 40
        MOV    R2,#5          ; โหลดตัวนับจำนวน 5 ตัว
        MOV    A              ; เคลียร์ A = 0
        MOV    R7,A           ; เคลียร์ค่า R7
AGAIN:  ADD    A,#R0          ; บวกค่าใน A กับค่าที่ชี้โดยรีจิสเตอร์ R0
        DA     A              ; ปรับค่าเป็นเลข BCD
        JNC    NEXT           ; กระโดดไป NEXT ถ้ารีจิสเตอร์ A ไม่มีการทด
        JNC    R7             ; เก็บตัวทด
NEXT:   INC    R0             ; เพิ่มค่าแอดเดรสตัวชี้
        DJNZ   R2, AGAIN      ; ทำซ้ำจน R2 มีค่าเป็นศูนย์
```

คำตอบที่ได้จะเก็บผลลัพธ์ในรีจิสเตอร์ A และถ้ามีการทดจะเก็บตัวทดใน R7

การลบแบบไม่คิดเครื่องหมาย

โดยทั่วไปแล้วไมโครโพรเซสเซอร์จะมีคำสั่งลบเลขอยู่สองคำสั่งคือ SUB และ SUBB (subtract with borrow) แต่ใน MCS-51 มีคำสั่งลบคำสั่งเดียวคือ SUBB ซึ่งเป็นการลลบที่นำแฟลกตัวทดมาร่วมลบด้วย แต่สามารถพิจารณาได้สองกรณีคือ กรณีที่ CY = 0 และกรณี CY = 1 โดยคำสั่ง SUBB มีรูปแบบดังนี้

```
SUBB   A,source              ; มีค่าเป็น A = A – source – CY
```

คำสั่ง SUBB เมื่อ CY = 0

การลบในไมโครโปรเซสเซอร์ยุคใหม่ ๆ จะใช้วิธีการทำทูคอมพลีเมนต์ (2's complement) เมื่อ MCS-51 ทำคำสั่ง SUBB กับตัวเลขแบบไม่คิดเครื่องหมาย เมื่อ CY = 0 จะมีการทำงานดังนี้

1. ทำเลข 2's complement กับตัวเลขที่นำมาลบ (source operand)
2. บวกค่าที่ได้เข้ากับค่าในรีจิสเตอร์ A
3. กลับค่าบิตแฟลกตัวทด

โดยทั้งสามขั้นตอนนี้จะถูกทำโดย CPU ที่อยู่ภายใน MCS-51

<u>ตัวอย่างที่ 5.5</u> จงแสดงขั้นตอนการทำคำสั่งต่อไปนี้

```
        CLR   C          ; เคลียร์ค่า CY = 0
        MOV   A,#3FH     ; โหลดค่า 3FH ใส่ใน A (A = 3FH)
        MOV   R3,#23H    ; โหลดค่า 23H ใส่ใน R3 (R3 = 23H)
        SUBB  A,R3       ; ทำ A - R3 ผลลัพธ์เก็บใน A
```

<u>วิธีทำ</u>

```
A  = 3F     0011 1111          0011 1111
R3 = 23     0010 0011        + 1101 1101    (2's complement)
     1C                      1 0001 1100
                             0    CF = 0    (ขั้นตอนที่ 3)
```

ผลลัพธ์ที่ได้จะพบว่ามีค่าเท่ากับ 1CH โดย CY = 0 , AC = 0 ซึ่งแฟลกตัวทดนี้จะเป็นตัวบอกว่าผลลัพธ์เป็นเลขบวกหรือลบ

ถ้าหากหลังทำคำสั่ง SUBB ทำให้ CY = 0 หมายความว่าเป็นเลขบวก ถ้า CY = 1 หมายความว่าเป็นเลขลบ ถ้าหากคำตอบที่ได้เป็นเลขลบ และอยากทราบว่าเป็นลลบเท่าใดให้ทำ 2's complement กลับซึ่งอาจทำได้โดยการใช้คำสั่ง CPL ซึ่งเป็นการกลับค่าบิตทุกบิต จากนั้นตามด้วยคำสั่ง INC ก็จะได้เลข 2's complement ออกมา ดังตัวอย่างที่ 5.6

<u>ตัวอย่างที่ 5.6</u> จงวิเคราะห์การทำโปรแกรมต่อไปนี้

```
        CLR   C
        MOV   A,#4CH     ; ใส่ค่า 4CH ลงในรีจิสเตอร์ A
        SUBB  A,#6EH     ; ลบค่าใน A ด้วย 6E
        JNC   NEXT       ; ถ้า CY = 0 ให้กระโดดไป NEXT
        CPL   A          ; ถ้า CY = 1 ทำ 1's complement
        INC   A          ; บวกด้วย 1 จะได้เลข 2's complement
NEXT:   MOV   R1,A       ; เก็บค่าคำตอบ A ลงใน R1
```

<u>วิธีทำ</u>

การทำคำสั่ง SUBB A,6EH จะมีขั้นตอนดังนี้

```
  4C    0100 1100                          0100 1100
- 6E    0110 1110     2's complement =     0001 0010
- 22                  กลับค่า              0 1101 1110
```

จะเห็นว่า CY = 1 หมายความว่าเป็นเลขลบ ให้ทำ 2's complement กลับ จะได้เป็น 0010 0010 ซึ่งก็คือ -22H นั่นเอง

คำสั่ง SUBB เมื่อ CY = 1

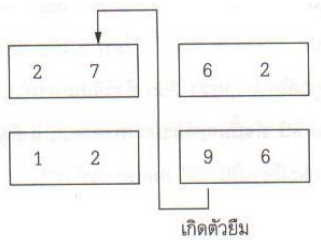
เป็นการลบเลขที่นำค่าของการยืมมาดำเนินการด้วย ถ้าค่าของตัวลบมีค่ามากกว่าตัวตั้งจะต้องทำการยืมจากบิตสูงถัดไป ดังตัวอย่างที่ 5.7

ตัวอย่างที่ 5.7 จงวิเคราะห์โปรแกรมต่อไปนี้

```
CLR     C           ; CY = 0
MOV     A,#62H      ; A = 62H
SUBB    A,#96H      ; 62H – 96H = CCH โดย CY = 1
MOV     R7,A        ; เก็บค่าผลลัพธ์
MOV     a,#27H      ; A = 27H
SUBB    A,#12H      ; 27H - 12H – 1 = 14H
MOV     R6,A        ; เก็บค่าผลลัพธ์
```

วิธีทำ

จากโปรแกรมจะพบว่าหลังจากทำคำสั่ง SUBB คำสั่งแรกจะได้ผลลัพธ์เป็น A = 62H – 96H = CCH และ CY จะเป็น 1 เนื่องจากมีการยืมเกิดขึ้น เมื่อทำคำสั่ง SUBB ครั้งที่สองจะได้ A = 27H – 12H – 1 = 14H ดังนั้นโปรแกรมนี้จะทำงานเป็น 2762H – 1296H = 12CCH



คำถาม ?

1. คำสั่ง ADD A,source ผลลัพธ์จะเก็บอยู่ที่ใด
2. คำสั่ง ADD R1,R2 จะทำงานได้หรือไม่
3. จงหาค่า A และแฟลก CY จากการทำคำสั่งต่อไปนี้

ก. MOV	A,#4FH	ข. MOV	A,#9CH
ADD	A,#0B1H	ADD	A,#63H

4. จงแสดงการลบค่า 05H ออกจาก 43H

5.2 การคูณและการหารแบบไม่คิดเครื่องหมาย

คำสั่งคูณและคำสั่งหารของ MCS-51 จะใช้รีจิสเตอร์ A และ B เป็นโอเปอแรนด์เท่านั้น ไม่สามารถกระทำกับตัวเลขที่อยู่ในตำแหน่งอื่นๆ ได้

การคูณ

ไมโครคอนโทรลเลอร์ MCS-51 จะทำการคูณแบบไบต์กับไบต์โดยใช้คำสั่งเป็น

```
MUL     AB          ; A x B ได้ผลลัพธ์เป็นข้อมูล 16 บิต
```

โดยการคูณนี้จะนำค่าในรีจิสเตอร์ A และรีจีสเตอร์ B มาคูณกัน ผลลัพธ์ที่ได้จะมีขนาด 16 บิต และจะเก็บในรีจิสเตอร์ A และ B โดยไบต์ต่ำเก็บใน A และไบต์สูงเก็บใน B ตัวอย่างเช่นถ้าหากคูณ 25H ด้วย 65H และเก็บผลลัพธ์ในรีจิสเตรอ์ A และ B สามารถเขียนคำสั่งได้เป็น

```
MOV     A,#25H          ; โหลดค่า  25H  เก็บใน A
MOV     B,#65H          ; โหลดค่า  65H  เก็บใน  B
MUL     AB              ; 25H * 65H = 0E99H
                        ; โดย B เก็บ 0EH และ A เก็บ 99H
```

ถ้าหากผลลัพธ์จากการคูณมีค่ามากกว่า 255 ซึ่งรีจิสเตอร์ตัวเดียวไม่สามารถเก็บค่าผลลัพธ์นี้ได้ จะทำให้แฟลกโอเวอร์โฟลว์เกิดการทดขึ้น ดังนั้นจะต้องอ่านค่าคำตอบไบต์สูงจากรีจิสเตอร์ B ออกมาด้วย และในการทำคำสั่งคูณนี้ ค่าแฟลกตัวทดจะมีค่าเป็น"0" ตลอดการทำงาน

การหาร

ไมโครคอสโทรลเลอร์ MCS-51 จะทำการหารแบบไบต์กับไบต์ โดยใช้คำสั่งที่มีรูปแบบดังนี้

```
DIV     AB              ; หารค่าใน A ด้วยค่าใน B
```

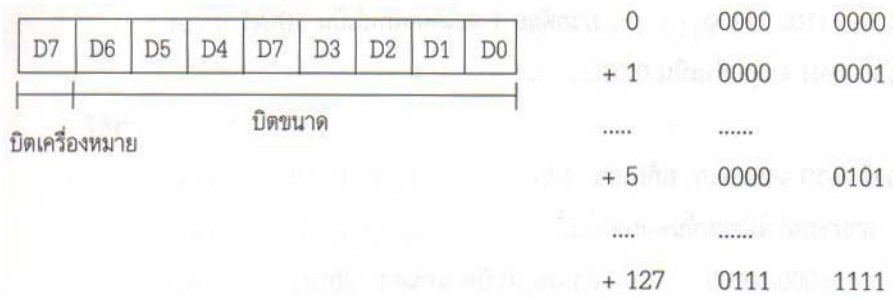
การหารจะใช้รีจิสเตอร์ A เป็นตัวตั้ง และใช้รีจิสเตอร์ B เป็นตัวหาร หลังจากการหารผลลัพธ์จะเก็บในรีจิสเตอร์ A และเศษจะเก็บใน B พิจารณาตัวอย่างต่อไปนี้

```
MOV     A,#95           ; โหลด  A  ด้วย  95
MOV     B,#10           ; โหลด  B ด้วย  10
DIV     AB              ; A = 9 (เก็บผลลัพธ์)
                        ; B = 5 (เก็บเศษ)
```

ในการใช้คำสั่งหารนั้นจะต้องตรวจสอบก่อนว่าค่าในรีจิสเตอร์ B เป็น 0 หรือไม่ ถ้าหาก B เป็นค่า 0 จะทำให้ตัวหารเป็นศูนย์ ซึ่งจะทำให้แฟลกโอเวอร์โฟลว์ (OV) มีค่าเป็น "1"

5.3 ตัวเลขแบบคิดเครื่องหมาย

การเก็บเลขจำนวนเต็มในรูปแบบเลขฐานสองแบบคิดเครื่องหมายนั้นจะใช้ค่าบิตนัยสำคัญสูงสุด (MSB) เป็นบิตเครื่องหมาย เพื่อบอกว่าเลขจำนวนเต็มนั้นเป็นเลขบวกหรือเลขลบ ส่วนบิตนัยสำคัญต่ำลงมาจะเป็นบิตที่บอกขนาดของข้อมูล เช่นถ้าหากเก็บข้อมูลขนาดหนึ่งไบต์ จะใช้บิต 7 เป็นบิตเครื่องหมาย ส่วนบิต 0-6 เป็นบิตขนาดข้อมูล



รูปที่ 5.1 แสดงการเก็บข้อมูลแบบ 8 บิต

จากรูปที่ 5.1 จะพบว่าถ้าหากใช้รีจิสเตอร์ขนาด 8 บิต เก็บเลขจำนวนเต็ม จะได้เลขบวกที่มีค่าตั้งแต่ 0 ถึง +127

สำหรับการเก็บเลขจำนวนเต็มลบจะต้องให้บิตสูงสุมีลอจิกเป็น “1” การแทนตัวเลขจำนวนเต็มแบบคิดเครื่องหมายนี้โดยทั่วไปจะใช้วิธีการทูคอมพลีเมนต์ (2’s complement) โดยการเปลี่ยนเลขลบให้เป็นเลขแบบทูคอมพลีเมนต์จะมีวิธีการดังนี้

1. เขียนขนาดของตัวเลขแบบ 8 บิต (ไม่คิดเครื่องหมาย)
2. กลับค่าบิตทุกบิต
3. บวกค่าที่ได้ด้วย 1

ตัวอย่างที่ 5.8 จงแสดงการเก็บเลข 5

วิธีทำ สามารถทำได้ตามขั้นตอนต่อไปนี้

1. 0000 0100 ตัวเลข 8 บิต แทนค่า 5
2. 1111 1010 กลับค่าทุกบิต
3. 1111 1011 บวกด้วย 1 จะได้ผลลัพธ์เป็น 0FBH

ดังนั้นค่า -5 จะถูกเก็บเป็น 0FBH

ตัวอย่างที่ 5.9 จงแสดงการเก็บเลข -34H

วิธีทำ สามารถทำได้ตามขั้นตอนต่อไปนี้

1. 0011 0100 ตัวเลข 8 บิต แทนค่าใน 34H
2. 1100 1011 กลับค่าทุกบิต
3. 1100 1100 บวกด้วย 1 จะได้ผลลัพธ์เป็น 0CCH

ดังนั้น -34H จะถูกเก็บเป็น 0CCH

ตัวอย่างที่ 5.10 จงแสดงการเก็บเลข -128

วิธีทำ สามารถทำได้ตามขั้นตอนต่อไปนี้

1. 1000 0000 ตัวเลข 8 บิต แทนค่า 128
2. 0111 1111 กลับค่าทุกบิต
3. 1000 0000 บวกด้วย 1 จะได้เป็น 80H

ดังนั้น -128 จะถูกเก็บเป็น 80H

จากตัวอย่างที่ 5.10 จะพบว่ารีจิสเตอร์ขนาด 8 บิตนั้น สามารถเก็บเลขจำนวนเต็มลบได้ในช่วง -1 ถึง -128 การเก็บตัวเลขแบบคิดเครื่องหมายโดยใช้รีจิสเตอร์ขนาด 8 บิต สามารถสรุปได้ดังนี้

เลขฐานสิบ	เลขฐานสอง	เลขฐานสิบหก
-128	1000 0000	80
-127	1000 0001	81
-126	1000 0010	82
.....	.....	....
-2	1111 1110	FE
-1	1111 1111	FF
0	0000 0000	00
+1	0000 0001	01
+2	0000 0010	02
.....	.....	….
+127	0111 1111	7F

ปัญหาการโอเวอร์โฟลว์ในตัวแบบแบบคิดเครื่องหมาย

การประมวลผลแบบคิดเครื่องหมายบางครั้งคำตอบที่ MCS-51 คิดมาได้จะไม่ตรงกับคำตอบที่มนุษย์คิด ตัวอย่างเช่น ถ้าให้บวกตัวเลข +96 กับ +70 คำตอบที่ MCS-51 คิดได้คือ -90 โดยแฟลก OV จะเป็นลอจิก “1” แต่คำตอบที่มนุษย์คิดจะได้เป็น 166 ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 5.11 จงวิเคราะห์ผลลัพธ์จากการทำชุดคำสั่งต่อไปนี้

```
MOV   A, #+96      ; A  = 0110 0000 (A = 60H)
MOV   R1, #+70     ; R1 = 0100 0110 (R1 = 46H)
ADD   A, R1        ; A  = 1010 0110
                   ; A  = A6H   = -90
```

วิธีทำ

```
   +96        0110 0000
 + +70        0100 0110
 ------       ---------
  +166        1010 0110      และ OV=1
```

ตัวอย่างที่ 5.12 ตัวอย่างนี้จะให้แฟลก OV เป็นลอจิก “1”

```
MOV   A, #-128     ; A =  1000 0000 (A = 80H)
MOV   R4, #-2      ; R4 = 1111 1110 (R4 = FEH)
ADD   A, R4        ; A =  0111 1110 (A = 7EH = +126 ไม่ถูกต้อง)
```

วิธีทำ

```
 -128        1111 1110
 +  -2       1111 1110
 -----       ---------
 -130        0111 1110      และ OV = 1
```

ใน CPU จะไม่เก็บค่า +126 แต่จะพบว่า OV = 1

ตัวอย่างที่ 5.13 ตัวอย่างนี้ แฟลก OV จะเป็นลอจิก “0”

```
MOV   A, # -2      ; A = 1111 1110(A = FEH)
MOV   R1, # -5     ; R1 = 1111 1011(R1 = FBH)
ADD   A, R1        ; A = 1111 1001(A = F9H = -7 ถูกต้อง OV = 0)
```

วิธีทำ

```
   -2        1111 1110
 +  -5       1111 1011
 -----       ---------
   -7        1111 1001      และ OV = 0
```

ใน CPU เก็บค่า -7 ซึ่งเป็นค่าที่ถูกต้อง ขณะที่ OV = 0

ตัวอย่างที่ 5.14 ตัวอย่างนี้ แฟลก OV จะ เป็นลอจิก “0”

```
MOV   A, # +7      ; A = 0000 0111(A = 07H)
MOV   R1, # +18    ; R1 = 0001 0010(R1 = 12H)
ADD   A, R1        ; A = 0001 1001(A = 19H = +25, ถูกต้อง OV = 0 )
```

วิธีทำ

```
    7        0000 0111
 + 18        0001 0010
 ----        ---------
   25        0001 1001      และ OV = 0
```

ใน CPU เก็บค่า +25 ซึ่งเป็นคำตอบที่ถูกต้อง

บทที่ 6
คำสั่งทางลอจิกและการโปรแกรม

ในบทนี้จะกล่าวถึงชุดคำสั่งทางลอจิกได้แก่คำสั่ง AND, OR และ XOR รวมทั้งคำสั่งกลับค่าข้อมูลด้วย ในหัวข้อที่ 6.2 จะกล่าวถึงคำสั่งการหมุนข้อมูล (rotate) และสลับข้อมูล (swap) ในหัวข้อที่ 6.3 จะแสดงถึงการเขียนโปรแกรมเปลี่ยนรหัส BCD เป็นรหัส ASCII

6.1 คำสั่งทางลอจิกและคำสั่งเปรียบเทียบ

ในไมโครคอนโทรลเลอร์สามารถประมวลผลแบบบูลีนบิตต่อบิตได้ เช่น การดำเนินการแอนด์ (AND) การดำเนินการออร์ (OR) และการทำเอ็กคลูซีฟออร์ (XOR) โดยลักษณะของคำสั่งต่าง ๆ จะเป็นดังนี้

การ AND

คำสั่งนี้ใช้ในการแอนด์ข้อมูลแบบบิตต่อบิตมีรูปแบบดังนี้

```
ANL destination,Source          ; dest = dest AND Source
```

คำสั่งนี้จะนำค่าโอเปอแรนด์สองตัวมา AND กัน โดยผลลัพธ์ที่ได้จะถูกเก็บในตำแหน่งปลายทาง (destination) สำหรับตำแหน่งต้นทาง (source) จะเป็นข้อมูลในรีจิสเตอร์ หน่วยความจำ หรือข้อมูลแบบอินมีเดียตก็ได้ คำสั่งนี้หลังการทำคำสั่งจะไม่มีผลต่อบิตแฟลกใด ๆ การแอนด์แบบบิตต่อบิต ข้อมูลที่ได้จะเป็นไปตามตารางความเป็นจริงต่อไปนี้

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

ตัวอย่างเช่น

```
        MOV   A, #39 H          ; A = 39H
        ANL   A, #09 H          ; A = 39H AND 09
39      0011  1001
09      0000  1001
-------------------
09      0000  1001
```

การใช้คำสั่ง ANL จะมีการอ้างแอดเดรสแบบต่าง ๆ ได้ 6 โหมด โดยจะมี 4 โหมดใช้รีจิสเตอร์ A เป็นตำแหน่งปลายทางดังนี้

```
1. Immediate:          ANL   A, #data      ; ANL   A, #25H
2. Register:           ANL   A, #Rn        ; ANL   A, R3
3. Direct:             ANL   A, direct     ; ANL   A, 30H
4. Register – indirect: ANL  A, @R0        ; ANL   A, @R0
```

สำหรับการอ้างแอดเดรสอีกสองโหมด ตำแหน่งปลายทางจะเป็นหน่วยความจำภายในหรือรีจิสเตอร์พิเศษตัวใดตัวหนึ่ง ส่วนตำแหน่งต้นทางจะเป็นรีจิสเตอร์ A หรือข้อมูลแบบอิมมีเดียต

```
5.                     ANL   direct, #data
6.                     ANL direct, A
```

ตัวอย่างที่ 6.1 จงแสดงผลลัพธ์จากการทำคำสั่งต่อไปนี้

```
MOV   A,#35H      ; A = 35H
ANL   A,#0FH      ; A = A AND 0FH
```

วิธีทำ

```
35H   00110101
0FH   00001111
-------------
05H   00000101
```

คำตอบที่ได้จะทำให้ค่าใน A เป็น 05

การ OR มีรูปแบบคำสั่งดังนี้

```
ORL   destination,source      ;dest = dest OR source
```

คำสั่งนี้จะนำค่าโอเปอแรนด์ต้นทางและปลายทางมารวมกันแบบบิตต่อบิต โดยโอเปอแรนด์ต้นทางอาจจะเป็จค่าในรีจิสเตอร์ หน่วยความจำ หรือข้อมูลแบบอิมมีเดียตก็ได้ คำสั่งนี้หวังการทำงานจะไม่มีผลต่อบิตแฟลกใดๆ สำหรับตารางความจริงของการ OR แสดงได้ดังนี้

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

คำสั่งนี้จะมีการอับแอดเดรสได้ 6 โหมด โดยมี 4 โหมดใช้รีจิสเตอร์ A เป็นตำแหน่งปลายทางดังนี้

```
1. Immediate        ORL   A, #data      ; ORL   A, #25H
2. Register         ORL   A, #Rx        ; ORL   A, R3
3. Direct           ORL   A, Direct     ; ORL   A, 30H
4. Register-Indirect ORL  A,@Rn         ; ORL   A,R0
```

สำหรับอีกสองโหมด จะใช้หน่วยความจำภายในหรือรีจิสเตอร์ฟังก์ชันพิเศษเป็นตำแหน่งปลายทาง สำหรับตำแหน่งต้นทางจะใช้รีจิสเตอร์ A หรือข้อมูลแบบอิมมีเดียตดังนี้

```
5.                  ORL   direct, # data
6.                  ORL   direct, A
```

ตัวอย่างที่ 6.2 จงหาค่าข้อมูลจากการทำคำสั่งต่อไปนี้

```
MOV   A, #39H     ; A = 39H
MOV   A, #09H     ; A = 39H OR 09H
```

วิธีทำ

```
39H   0011 1001
09H   0000 1001
---------------
39    0011 1001
```

คำตอบที่ได้จะทำให้ค่าใน A เท่ากับ 39H

ตัวอย่างที่ 6.3 จงหาค่าข้อมูลจากการทำคำสั่งต่อไปนี้

```
MOV   A, #32H      ; A = 32H
MOV   R4, #50H     ; R4 = 50H
ORL   A,R4         ; A = 72H
```

วิธีทำ

32H	0011 0010
50H	0101 0000
72	0111 0010

คำตอบที่ได้จะทำให้ค่าใน A เท่ากับ 72H

การ XOR มีรูปแบบคำสั่งดังนี้

XRL destination, source ; dest = dest XOR source

คำสั่งนี้จะนำโอเปอร์แลนด์ทั้งสองตัวมาทำการเอกคลูซิฟออร์กันแบบบิตต่อบิต โดยตารางความจริงของการทำเอกคลูซิฟออร์แสดงได้ดังนี้

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

คำสั่ง XRL จะมีการอ้างแอดเดรสได้คล้ายกับคำสั่งที่ผ่านมาดังนี้

1. Immediate	XRL	A, #data	; XRL	A, #25H
2. Register	XRL	A,Rn	; XRL	A,R3
3. Direct	XRL	A,direct	; XRL	A,30H
4. Register-Indirect	XRL	A,@Rn	; XRL	A,R0
5.	XRL	direct,#data	; XRL	32H,#44H
6.	XRL	direct,A	; XRL	0F0H,A

ตัวอย่างที่ 6.4 จงหาค่าจากการกรำทำคำสั่งต่อไปนี้

```
MOV   A,#32H       ;A =32H
MOV   R4,#50H      ;R4=50H
XRL   A,R4
```

วิธีทำ

32H	0011 0010
50H	0101 0000
62H	0110 0010

หลังจากคำสั่งค่าใน A จะเท่ากับ 62H

ตัวอย่างที่ 6.5 จงหาค่าจากการทำคำสั่งต่อไปนี้

```
MOV   32H, #67H
XRL   32H, #44H
MOV   A,32H
```

วิธีทำ

```
44H   0100 0100
67H   0110 0111
-------------------
23H   0010 0011
```

ตัวอย่างที่ 6.6 จงหาค่าในรีจิสเตอร์ B หลังจากทำคำสั่งต่อไปนี้

```
MOV   B, #44H
MOV   A, #67H
XRL   0F0H,A
```

วิธีทำ

เนื่องจากรีจิสเตอร์ B จะอยู่ในแอดเดรส F0H ของหน่วยความจำ RAM ภายใน ดังนั้นหลังจากทำคำสั่งจะทำให้ค่าใน B เท่ากับ 23H

คำสั่ง XRL นี้จะมีโอเปอแรนด์เป็นรีจิสเตอร์บางตัวได้ เช่น XRL A, R1 จะเป็นการกระทำการแอกคลูซิฟออร์ระหว่างรีจิสเตอร์ A กับรีจิสเตอร์ R1 ถ้าหากผลลัพธ์ใน A มีค่าเป็น 0 เราสามารถใช้คำสั่ง JZ ตามหลังให้โปรแกรมกระโดดได้

ตัวอย่างที่ 6.7 ตัวอย่างนี้จะอ่านค่าจากพอร์ต P1 และทดสอบว่าค่าเท่ากับ 45H หรือไม่ ถ้าเท่าจะส่งค่า 99H ออกไปหาพอร์ต P2

วิธีทำ

```
        MOV   P2, #00        ; เคลียร์ค่า P2
        MOV   P1, #0FFH      ; ทำให้ P1 เป็นพอร์ตอินพุต
        MOV   R3, #45H       ; R3 = 45H
        MOV   A,P1           ; อ่านค่าจากพอร์ต P1
        XRL   A,R3           ; ถ้าหาก A เท่ากับ 45 ผลลัพธ์ที่ได้จะเท่ากับ 00H
        JNZ   EXIT           ; กระโดดถ้าค่าใน A ไม่เท่ากับศูนย์
        MOV   P2, #99H
EXIT:   NOP
```

เราสามารถนำคำสั่ง XRL นี้มาประยุกต์ในการกลับค่าบิตข้อมูลไปมา (toggle bit) ของโอเปอแรนด์ได้ตัวอย่างเช่นถ้าต้องการกลับค่าบิต2ของรีจิสเตอร์Aไปมาเราสามารถเขียนคำสั่งได้ดังนี้

```
XRL   A, #04H        ;EX-OR A กับค่า 0000 0101
```

คำสั่ง CPL A

คำสั่งนี้จะใช้ทำคอมพลีเมนต์(Complement) ค่าในรีจิสเตอร์ Aโดยกลับค่าบิตที่เป็น0ให้เป็น1บิตที่เป็น1ให้เป็น0หรือที่เรียกอีกอย่างหนึ่งว่าการทำคอมพลีเมนต์ (1's complement)

```
MOV     A, #55H
CPL     A
```

หลังจากทำคำสั่งใน A จะเท่ากับ AAH เนื่องจากค่า 55H (0101 0101) เมื่อถูกทำคอมพลีเมนต์แล้วจะได้เป็น AAH (1010 1010)

สำหรับการทำเลขทูคอมพลีเมนท์ (2's complement) เราสามารถทำได้โดยการบวก 1 เข้ากับตัวเลขแบบวันคอมพลีเมนท์ ดังเช่นตัวอย่างที่ 6.8

<u>ตัวอย่างที่ 6.8</u> แสดงการทำทูคอมพลีเมนต์ของค่า 85H

```
MOV     A, #85H         ; 85H = 1000 0110
CPL     A               ; 1's = 0111 1010
ADD     A,#1            ; 2's =    +1
                                  0111 1011 = 7BH
```

หลังจากทำคำสั่งใน A จะเท่ากับ 7BH ซึ่งเป็นเลขทูคอมพลีเมนท์ของ 85H

คำสั่งเปรียบเทียบ

ใน MCS-51 มีคำสั่งที่ใช้ในการเปรียบเทียบข้อมูลโดยมีรูปแบบคำสั่งดังนี้

```
CJNE    destination, source, relative address
```

คำสั่ง CJNE นี้ย่อมาจาก Compare and jump if not equal คำสั่งนี้จะมีโอเปอแรนด์สองตัว เมื่อ MCS-51 ทำงานคำสั่งนี้จะนำโอเปอแรนด์ทั้งสองมาเปรียบเทียบกัน ถ้าค่าไม่เท่ากันจะกระโดดไปยังแอดเดรสแบบสัมพัทธ์ (relative address) โดยการเปรียบเทียบจะทำโดยนำค่าโอเปอแรนด์ ต้นทางไปลบออกจากโอเปอแรนด์ปลายทาง และหลังทำคำสั่งค่าบิตแฟลก CY จะมีการเปลี่ยนแปลง ตัวอย่างเช่น CJNE,A#67H,NEXT ถ้าค่าในรีจิสเตอร์ A ไม่เท่ากับ 67H ตัว MCS-51 จะกระโดดไปยังแอดเดรส NEXT

ในคำสั่ง CJNE ค่าโอเปอแรนด์ต้นทางจะต้องเป็นรีจิสเตอร์ A หรือ Rn เท่านั้น ส่วนโอเปอแรนด์ต้นทางจะเป็นรีจิสเตอร์ ค่าในหน่วยความจำ หรือการอ้างแอดเดรสแบบอิมมีเดียตก็ได้หลังจากทำคำสั่งค่าในบิตแฟลก CY จะเปลี่ยนไปดังตารางต่อไปนี้ ส่วนค่าโอเปอแรนด์จะไม่มีการเปลี่ยนแปลง

Compare	Carry Flag
destination > source	CY = 0
destination < source	CY = 1

พิจารณาการทำคำสั่งต่อไปนี้

```
            CJNE    R5, #80,NOT_EQUAL   ; ตรวจสอบว่า R5 เท่ากับ 80 หรือไม่
            ................            ; R5 = 80
NOT_EQUAL:  JNC     NEXT                ; กระโดดถ้า R5>80
            ................
NEXT:       ................            ; R5<80
```

จากชุดคำสั่งด้านบนจะเปรียบเทียบว่าค่าใน R5 เท่ากับ 80 หรือไม่ ถ้าเท่าจะทำคำสั่งต่อไป ถ้าไม่เท่าจะกระโดดไปยังแอดเดรส NOT_EQUAL และในแอดเดรส NOT_EQUAL จะใช้คำสั่ง JNZ ซึ่งถ้าบิตแฟลก CY เป็น 0 หมายความว่า R5 มากกว่า 80 ถึงกระโดดไปยังแอดเดรส NEXT ถ้า R5 น้อยกว่า 80 จะทำค่าสั่งต่อจาก JNZ นี้

ตัวอย่างที่ 6.9 จากชุดคำสั่งที่กำหนดให้จงตอบคำถามต่อไปนี้

ก. โปรแกรมจะกระโดดไปยัง NEXT หรือไม่

ข. หลังจากการทำคำสั่ง CJNE ในรีจิสเตอร์ A มีค่าอะไร

```
        MOV     A, #55H
        CJNE    A, #99H,NEXT
        ............
NEXT:   ............
```

วิธีทำ

ก. จะกระโดดไปยังแอดเดรส NEXT เนื่องจากค่า 55H และ 99H ไม่เท่ากัน

ข. ค่าใน A เท่ากับ 55H เพราะหลังทำคำสั่งค่าในโอเปอแรนด์ไม่เปลี่ยนแปลง

ตัวอย่างที่ 6.10 จงเขียนคำสั่งตรวจสอบค่าใน A เท่ากับ99Hหรือไม่ ถ้าเท่าให้ R1 = FFH ถ้าไม่เท่าให้ R1 = 0

วิธีทำ

```
        MOV     R1, #0          ; เคลียร์ค่า R1
        CJNE    A, #99H,NEXT    ; ถ้า A ไม่เท่ากับ 99 จะกระโดดไปยัง NEXT
        MOV  R1, #0FFH          ; ถ้า A เท่ากับ 99 ใส่ค่า FF ใน A
NEXT:   ......
OVER:   ......
```

ตัวอย่างที่ 6.11 ถ้าหากพอร์ต P1 ถูกใช้เป็นพอร์อินพุต และต่ออยู่กับตัววัดอุณหภูมิ จงเขียนโปรแกรมอ่านค่าอุณหภูมิ และทดสอบว่าเท่ากับ 75 หรือไม่ ถ้าน้อยกว่า 75 ให้เก็บค่าอุณหภูมิใน R1 แต่ถ้ามากกว่าให้เก็บใน R2 ตามเงื่อนไขดังนี้

T = 75 ; A = 75

T < 75 ; R1 = T

T > 75 ; R2 = T

วิธีทำ

```
        MOV     P1, #0FFH       ; โปรแกรมให้ P1 เป็นพอร์ตอินพุต
        MOV     A,P1            ; อ่านค่าอุณหภูมิจาก P1
        CJNE    A, #75,OVER     ; เปรียบเทียบค่า A เท่ากับ 75 หรือไม่
        SJMP    EXIT            ; กระโดดถ้า A เท่ากับ 75
OVER:   JNC     NEXT            ; ถ้า CY = 0 หมายความว่า A>75
        MOV     R1,A            ; A <75 เก็บค่าใน R1
        SJMP    EXIT            ; จบโปรแกรม
NEXT:   MOV     R2,A            ; A>75 เก็บค่าใน R2
EXIT:   .........
```

ตัวอย่างที่ 6.12 จงเขียนโปรแกรมให้ MCS-51 คอยตรวจสอบค่าของพอร์ต P1 ถ้าหากค่าของ P1 เท่ากับ 63 ให้ทำคำสั่งต่อไป

วิธีทำ

```
        MOV   P1, #0FFH       ; โปรแกรมให้ P1 เป็นพอร์ตอินพุต
HERE:   MOV   A,P1            ; อ่านค่าจาก P1 มาเก็บใน A
        CJNE  A, #63H,HERE    ; ทำซ้ำถ้า A ไม่เท่ากับ 63H
```

ตัวอย่างที่ 6.13 ถ้าหากหน่วยความจำ RAM ภายในตั้งแต่ตำแหน่ง 40H ถึง 44H มีข้อมูลต่างๆ เก็บอยู่จงเขียนโปรแกรมค้นหาว่าข้อมูลในตำแหน่งใดมีค่าเท่ากับ 65 โดยนำค่าตำแหน่งนั้นเก็บในรีจิสเตอร์ R4

วิธีทำ

สมมุติว่าข้อมูลในหน่วยความจำมีค่าดังนี้

[40H] = 76 [41H] = 79 [42H] = 69 [43H] = 65 [44H] = 62

สามารถเขียนโปรแกรมได้ดังนี้

```
        MOV   R4, #0          ; R4 = 0
        MOV   R0, #40H        ; โหลดค่าให้ตัวชี้
        MOV   R2, #05         ; โหลดค่าตัวรับ
        MOV   A, #65          ; ให้ A = 65H เป็นข้อมูลที่จะค้นหา
BACK:   CJNE  A,@R0,NEXT      ; ถ้าค่าใน RAM ไม่เท่ากับ 65 ให้กระโดดNEXT
        MOV   R4,R0           ; ค่าใน RAM เป็น 65 เก็บค่าแอดเดรส
        SJMP  EXIT            ; จบโปรแกรม
NEXT:   INC   R0              ; เพิ่มค่าพอร์ตเตอร์
        DJNZ  R2,BACK         ; ทำซ้ำจนกว่าตัวนับเท่ากับ 0
EXIT:   ............
```

คำถาม?

1. จงหาค่าในรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

```
ก.      MOV   A, #37H
        ANL   A, #0CAH
ข.      MOV   A, #37H
        ORL   A, #0CAH
ค.      MOV   A, #37H
        XAL   A,#0CAH
```

2. ถ้าหากต้องการให้บิต 3 บิต 4 ของแอกคิวมูเลเตอร์มีค่าเป็น 0 จะทำอย่างไรโดยใช้คำสั่ง ANL
3. ถ้านำคำสั่ง XRL กระทำกับข้อมูลในรีจิสเตอร์ของตัวมันเองจะได้ค่าใด
4. จากคำสั่งต่อไปนี้ค่าใน R4 ควรเป็นอย่างไรโปรแกรมจึงไม่กระโดด

```
        CJNE  R4, #53,OVER
```

5. จงหาค่าในรีจิสเตอร์หลังจากทำคำสั่งต่อไปนี้

```
        CLR   A
        ORL   A, #99H
        CPL   A
```

6.2 คำสั่งหมุนและสลับข้อมูล

ในไมโครคอนโทรลเลอร์ MCS-51 การหมุนบิตต่างๆ ในไบต์ของข้อมูลจำทำได้โดยการใช้คำสั่งต่อไปนี้ RL, RR, RLC และ RRC โดยสามารถหมุนข้อมูลในแอกคิวมูเลเตอร์ไปทางซ้ายหรือทางขวาได้ สำหรับลักษณะของการหมุนจะมีสองประเภท คือ การหมุนบิตในรีจิสเตอร์ A และการหมุนบิตผ่านตัวทด

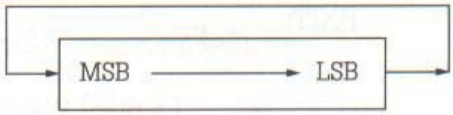
การหมุนบิตไปทางซ้ายและขวา

การหมุนบิตข้อมูลแบบนี้จะใช้รีจิสเตอร์ A เป็นโอเปอแรนด์ ถ้าเป็นการหมุนไปทางขวาจะใช้คำสั่งดังนี้

```
RR      A               ; หมุนค่าใน A ไปทางขวา
```

ข้อมูลจากบิตสูงจะถูกส่งไปยังบิตต่ำข้อมูลจากบิตต่ำสุด(บิต D0) จะถูกดันออกมาและกลับไปยังบิตสูงสุด ตัวอย่างของการหมุนเป็นดังนี้

```
MOV     A, #36H         ; A = 0011 0110
RR      A               ; A = 0001 1011
RR      A               ; A = 1000 1101
RR      A               ; A = 1100 0110
RR      A               ; A = 0110 0011
```

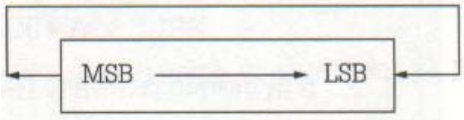


สำหรับการหมุนไปทางซ้ายคำสั่งจะเป็นรูปแบบดังนี้

```
RL      A               ; หมุนค่าใน A ไปทางซ้าย
```

ข้อมูล 8 บิต ในรีจิสเตอร์ A จะถูกหมุนโดยบิตต่ำจะถูกเลื่อนไปยังบิตสูงและบิตสูงสุดจะถูกส่งกลับมายังบิตต่ำสุด ดังตัวอย่างต่อไปนี้

```
MOV     A, #72H         ; A = 0111 0010
RL      A               ; A = 1110 0100
RL      A               ; A = 1100 1001
```



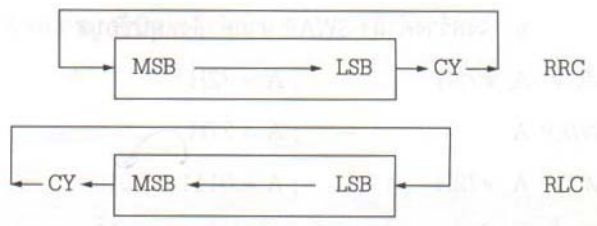
คำสั่งหมุนค่า RR และ RL หลังจากทำคำสั่งจะไม่มีผลต่อบิตแฟลกใด ๆ

การหมุนผ่านตัวทด

คำสั่งหมุนอีกประเภทหนึ่งของ MCS-51 จะนำข้อมูลบิตตัวทดมาช่วยหมุนด้วยเรียกว่าการหมุนผ่านบิตตัวทด หรือเรียกอีกอย่างหนึ่งว่าการหมุนแบบ 9 บิต ซึ่งมีทั้งหมุนทางซ้ายและหมุนทางขวาแต่ละคำสั่งมีรูปแบบดังนี้

```
RRC     A               ; หมุนค่าใน A ไปทางขวาผ่านบิตตัวทด
RLC     A               ; หมุนค่าใน A ไปทางซ้ายผ่านบิตตัวทด
```

การหมุนไปทางขวาผ่านบิตตัวทดนั้น ค่าในบิตต่ำสุดจะถูกส่งเข้าไปยังบิตแฟลกตัวทดส่วนค่าในบิตแฟลกตัวทดเดิมจะถูกส่งไปยังบิตสูงสุดดังรูป



ลักษณะของการหมุนแสดงได้ดังชุดคำสั่งต่อไปนี้

```
CLR   C          ; ให้ CY = 0
MOV   A, #26H    ; A = 0010 0110
RRC   A          ; A = 0001 0011      CY = 0
RRC   A          ; A = 0000 1001      CY = 1
RRC   A          ; A = 1000 0100      CY = 1
```

สำหรับการหมุนไปทางซ้ายผ่านบิตตัวทด จะใช้คำสั่งรูปแบบดังนี้

```
RLC   A          ; หมุนค่าใน A ไปทางซ้ายผ่านบิตตัวทด
```

เมื่อ MCS-51 ทำคำสั่งนี้จะเลื่อนบิตต่ำไปยังบิตสูงและนำบิตสูงสุดเก็บในแฟลกตัวทดส่วนค่าในแฟลกตัวทดเดิมจะถูกส่งกลับมายังบิตต่ำสุด ตัวอย่างดังชุดคำสั่งต่อไปนี้

```
SET   C          ; เซตแฟลกตัวทดให้เป็น 1
MOV   A, #15H    ; A = 0001 0101
RLC   A          ; A = 0010 1011      CY = 0
RLC   A          ; A = 0101 0110      CY = 0
RLC   A          ; A = 1010 1100      CY = 0
RLC   A          ; A = 0101 1000      CY = 1
```

คำสั่ง SWAP A

คำสั่งนี้จะกระทำกับแอกคิวมูเลเตอร์เท่านั้น โดยจะสลับค่าระหว่าง4 บิตบน กับ 4บิตล่างการทำงานของคำสั่งแสดงได้ดังต่อไปนี้

ก่อนทำคำสั่ง	D7-D4	D3-D0	ก่อนทำคำสั่ง	0111	0010
หลังทำคำสั่ง	D3-D0	D7-D4	หลังทำคำสั่ง	0010	0111

<u>ตัวอย่างที่ 6.14</u> ก. จงยกตัวอย่างหาค่าของข้อมูลหลังจากทำคำสั่ง SWAP

ข. จงสร้างคำสั่ง SWAP จากคำสั่งหมุนข้อมูล

<u>วิธีทำ</u> ก.

```
MOV   A, #72H    ; A = 72H
SWAP  A          ; A = 27H
```

ข.

```
MOV   A, #72H    ; A = 0111 0010
RL    A          ; A = 1110 0100
RL    A          ; A = 1100 1001
RL    A          ; A = 1001 0011
RL    A          ; A = 0010 0111
```

<u>ตัวอย่างที่ 6.15</u> จงเขียนโปรแกรมหาจำนวนบิตที่เป็น “1”ในไบต์ข้อมูล

<u>วิธีทำ</u> โปรแกรมลักษณะนี้สามารถทำได้โดยเลื่อนข้อมูลในไบต์ข้อมูลมายังบิตแฟลก แล้วทดสอบว่าบิตแฟลกเป็น “1”หรือไม่ ถ้าเป็นให้เพิ่มค่าตัวนับขึ้นหนึ่งค่า ดังโปรแกรมต่อไปนี้

```
        MOV     R1, #0          ; R1 เป็นตัวนับจำนวนบิตที่เป็น"1"
        MOV     R7, #8          ; ตัวนับจำนวนบิต
        MOV     A, #97H         ; ข้อมูลตัวอย่างที่จะหาบิตที่เป็น “1”
AGAIN:  RLC     A               ; หมุนค่าใน A ผ่านตัวทด
        JNC     NEXT            ; ตรวจสอบค่าในแฟลกตัวทด
        INC     R1              ; ถ้าแฟลกตัวทดเป็น “1”ให้บวก 1 เข้ากับตัวนับ
NEXT:   DJNZ    R7,AGAIN        ; ทำจนครบ 8 ครั้ง
```

คำสั่งการหมุนข้อมูลนี้ สามารถนำมาประยุกต์ใช้ในการเปลี่ยนข้อมูลแบบอนุกรมให้เป็นข้อมูลแบบขนานได้ ตัวอย่างเช่น ถ้าหากมีข้อมูลส่งเข้ามาทางบิต P1.3 เราสามารถนำค่าบิตนั้นมาเก็บในแฟลกตัวทดก่อนแล้วหมุนข้อมูลเข้ามาในรีจิสเตอร์ A ถ้าทำแบบนี้ 8 ครั้ง จะทำให้ข้อมูลที่เข้ามาแบบอนุกรมทางพอร์ต P1.3 ถูกเก็บเป็นข้อมูล 8 บิตในรีจิสเตอร์ A ได้ ดังการเขียนชุดคำสั่งต่อไปนี้

```
        MOV     A, #00H
        MOV     C, P1.3         ; อ่านข้อมูลบิตแรกที่เข้าทาง P1.3 ให้กับแฟลกตัวทด
        RLC     A               ; เลื่อนข้อมูลบิตแรกเข้ามาใน A
        MOV     C, P1.3         ; อ่านข้อมูลบิตต่อไป
        RLC     A               ; เลื่อนข้อมูลเข้ามาใน A
        < ทำจนครบ 8 บิต >
```

คำถาม ?

1. หลังจากไมโครคอนโทรลเลอร์ทำคำสั่งต่อไปนี้ ค่าใน A จะมีค่าเท่าใด

```
MOV     A, #25H
RR      A
RR      A
RR      A
RR      A
```

2. หลังจากไมโครคอนโทรลเลอร์ทำคำสั่งต่อไปนี้ ค่าใน A จะมีค่าเท่าใด

```
MOV     A, #A2H
RL      A
RL      A
RL      A
RL      A
```

3. หลังจากไมโครคอนโทรลเลอร์ทำคำสั่งต่อไปนี้ ค่าใน A จะมีค่าเท่าใด

```
CLR     A
SETB    C
RRC     A
SETB    C
RRC     A
```

4. ถ้าเขียนคำสั่งเป็น RLC R1 ระบบจะทำงานอย่างไร
5. จงหาค่าใน A หลังจากทำคำสั่งต่อไปนี้

```
MOV   A, #85H
SWAP  A
ANL   A, #0F0H
```

6.3 การเขียนโปรแกรมประยุกต์ BCD และ ASCII

ความสัมพันธ์ระหว่างตัวเลข รหัส ASII และเลข BCD แสดงได้ดังนี้

Key	ASCII	Binary	BCD(Unpacked)
0	30	0011 0000	0000 0000
1	31	0011 0001	0000 0001
2	32	0011 0010	0000 0010
3	33	0011 0011	0000 0011
4	34	0011 0100	0000 0100
5	35	0011 0101	0000 0101
6	36	0011 0110	0000 0110
7	37	0011 0111	0000 0111
8	38	0011 1000	0000 1000
9	39	0011 1001	0000 1001

รหัส ACSII

โดยทั่วไปแล้วการติดต่อกับจอภาพ เครื่องพิมพ์ หรือแป้นพิมพ์ มักจะใช้รหัส ASCII ในการติดต่อ เช่น ถ้าหากเรากดคีย์บนแป้นพิมพ์ เราจะได้ค่าออกมาเป็นรหัส ACSII เช่น ถ้ากดคีย์ “0” ค่าที่ได้ออกมาคืน “001 0000” (30 H) ถ้ากดคีย์ “1”ค่าที่ได้ออกมาจะเป็น “001 0001” (32H) ดังตาราง ถ้าหากระบบคอมพิวเตอร์ จะนำค่าจากคีย์ไปประมวลผลจะต้องแปลงค่ารหัส ASCII ที่ได้ให้เป็นเลข BCD จึงจะประมวลผลได้ง่าย

ถ้าหาระบบคอมพิวเตอร์ประมวลผลได้ข้อมูลออกมาเป็นเลข BCD และต้องการแสดงผลทางอุปกรณ์ต่างๆ จะต้องเปลี่ยนเลข BCD นั้นให้เป็นรหัส ASCII เช่นกัน

การแปลง Packed BCD เป็นรหัส ASCII

ในไมโครคอนโทรลเลอร์เบอร์ DS5000 T จะมีวงจรเวลาจริงอยู่ภายในที่เรียกกันว่า RTC (realtime clock) ในวงจร RTC นี้สามารถบอกเวลาบางวันได้โดยให้ค่าชั่วโมง นาที วินาที ออกมา และบอกวันออกมาเป็น ปี เดือน และวันของสัปดาห์ ค่าที่ให้ออกมาจะอยู่ในรูปของรหัส BCB และถ้าหากเราต้องการนำไปแสดงผลทาง LCB หรือ LED 7 ส่วน เราจะต้องแปลงค่ารหัส BCD นี้เป็นรหัส ASCII เสียก่อน

<u>ตัวอย่างที่ 6.14</u> ถ้าหากมีข้อมูลรหัส packed BCD เก็บอยู่ในรีจิสเตอร์ A จงเขียนโปรแกรมเปลี่ยนเลข BCD ให้เป็นรหัส ASCII เก็บนรีจิสเตอร์ R2 และ R6

<u>วิธีทำ</u>

```
MOV   A, #29H      ; A= 29H, packed BCD
MOV   R2,A         ; ก๊อปปี้ตัวเลข BCD ลงใน R2
```

```
ANL   A, #0FH      ; เคลียร์ค่า 4 บิตบนจะได้ A = 09
ORL   A, #30H      ; เปลี่ยนเป็นรหัส ASCII A = 39H ('9')
MOV   R6, A        ; เก็บข้อมูล (R6 = 39H)
MOV   A, R2        ; A = 29H ข้อมูลเดิม
ANL   A, #0F0H     ; เคลียร์ค่าบิตล่าง (A = 20)
RR    A            ; หมุนไปทางขาว
RR    A            ; หมุนไปทางขวา
RR    A            ; หมุนไปทางขวา
RR    A            ; หมุนไปทางขวา
ORL   A, #30H      ; A = 32 H รหัส ASCII ของ 2
MOV   R2, A        ; เก็บค่ารหัส ASCII ลงใน R2
```

จากตัวอย่างเราสามารถแทนคำสั่ง RR สี่คำสั่งด้วยคำสั่ง SAWP ได้

คำถาม?

1. จงเปลี่ยนตัวเลขฐานสิบต่อไปนี้ให้เป็น packed BCB และ unpacked BCD
   ก. 15 ข. 99
2. จงนำค่า 76 เขียนเป็นเลขขนานเลขฐานสิบหก และเลข BCD
3. ตัวเลข BCD ที่มีค่าเป็น 67H ถ้าเปลี่ยนเป็นรหัส ASCII จะได้เป็น...H และ ....H
4. การเขียนคำสั่งต่อไปนี้สามารถเปลี่ยนเลข unpacked BCD ในรีจิสเตอร์ A ให้เป็นรหัส ASCII ได้หรือไม่

สรุป ที่ผ่านมาได้กล่าวถึงคำสั่งทางลอจิกของ MCS-51 ได้แก่การ AND, OR, XOR และ Complement ส่วนคำสั่งหมุนและสลับค่า สามารถนำมาประยุกต์ใช้ในการเปลี่ยนรหัสข้อมูลระหว่าง BCD และ ASCII ได้

คำถามท้ายบท

1. ถ้าหากมีข้อมูลในรีจิสเตอร์เป็นดังนี้ A = F0, B = 56 และ R1 = 90 จงหาผลลัพธ์จากการกระทำคำสั่งต่อไปนี้

ก.	ANL	A, #45H	ข.	RRL	A, B
ค.	XRL	A, #76H	ง.	ANL	A, B
จ.	XRL	A, R1	ฉ.	ORL	A, R1
ช.	ANL	A, #0FFH	ช.	ORL	A, #99H
ฉ.	XRL	A, #0EEH	จ.	XRL	A, #OAAH

2. จงหาค่าในรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

ก.	MOV	A, #65H	ข.	MOV	A, #70H
	ANL	A, #76H		ORL	A, #6BM
ค.	MOV	A, #95H	ง.	MOV	A, #5DH
	XRL	A, #0AAH		MOV	R3, #78H
จ.	MOV	A, #005H	ฉ.	MOV	A, 6AH
	MOV	R6, #12H		MOV	R4, #6EH

```
    ORL   A, R6                    XRL   A, R0
```

3. จริงหรือไม่ คำสั่ง CJNE จะทดสอบค่าในแอกคิวมูเลเตอร์เท่านั้น
4. จงหาค่าบางบิตแฟลก CY หลังจากทำคำสั่ง CJNE ดังต่อไปนี้

```
ก.  MOV   A, #25H          ข.  MOV   A, #0FFH
    CJNE  A, #44H, OVER        CJNE  A, #6FH,NEXT
ค.  MOV   A, #34           ง.  MOV   R1, #0
    CJNE  A, #34, NEXT         CJNE  R1, #0,NEXT
จ.  MOV   R5,  #54H        ฉ.  MOV   R3, #0AAH
    CJNE  R5,  #0FFH, NEXT     ANL   R3, #55H
                               CJNE  R3, #00H,NEXT
```

5. จงหาค่าบางรีจิสเตอร์ A หลังจากทำคำสั่งต่อไปนี้

```
ก.  MOV   A, #56H          ข.  MOV   A, #39H
    SWAP  A                    CLR   C
    RR    A                    RL    A
    RR    A                    RL    A
ค.  CLR   C                ง.  SETB  C
    MOV   A, #4DH              MOV   A, #7AH
    SWAP  A                    SWAP  A
    RRC   A                    RLC   A
    RRC   A                    RLC   A
    RRC   A
```

6. จงเขียนโปรแกรมหาจำนวนบิตที่เป็นศูนย์ในข้อมูล 8 บิต
7. ถ้าหากมีข้อมูล packed BCD อยู่ใน ROM ตำแหน่ง 300H จะเป็นโปรแกรมอ่านค่าออกมา และเปลี่ยนเป็นรหัส ASCII เก็บใน RAM ตั้งแต่ตำแหน่ง 40H เป็นต้นไป ถ้าหากข้อมูลใน ROM เป็นดังนี้

```
            ORG     300H
MYDATA:     DB      76H, 87H, 98H, 43H
```

8. จงเขียนโปรแกรมเปลี่ยนรหัส ASCII ให้เป็นข้อมูล packed BCD ถ้าหากข้อมูลรหัส ASCII อยู่ใน ROM ตั้งแต่ตำแหน่ง 300H และให้นำค่าที่เปลี่ยนได้ไปเก็บใน RAM ตั้งแต่ตำแหน่ง 60H

```
            ORG     300H
MYDATA:     DB      '87675649'
```

9. จงเขียนโปรแกรมรับข้อมูล 8 บิต ทางพอร์ต P1 และเปลี่ยนเป็นรหัส ASCII เก็บใน RAM ตำแหน่ง 40H, 41H และ 92H และถ้าหากค่าใน P1 เป็น 1000 1101 ค่าใน RAM จะมีค่าเป็นเท่าใด
10. จงแสดงการสร้างคำสั่ง SWAP จากคำสั่งต่อไปนี้
    - หมุนไปทางซ้าย
    - หมุนไปทางขวา

บทที่ 7
คำสั่งกระทำต่อบิตและการโปรแกรม

ไมโครคอนโทรลเลอร์ MCS-51 สามารถดำเนินการครั้งละบิตได้ เช่น เซตบิต เคลียร์บิตย้ายบิตข้อมูลและกลับค่าบิตข้อมูลโดยการดำเนินการดังกล่าวสามารถกระทำกับหน่วยความจำพอร์ต หรือรีจิสเตอร์ก็ได้ในบทนี้จะกล่าวถึงตัวอย่างการเขียนโปรแกรมและการประยุกต์การกระทำระดับบิต

7.1 การโปรแกรมแบบหนึ่งบิต

เมื่อไมโครคอนโทรลเลอร์ต้องการโอนย้ายข้อมูลระหว่างตัวมันกับหน่วยความจำ การทำงานประเภทนี้ไมโครคอนโทรลเลอร์ MCS-51 สามารถทำงานได้เป็นอย่างดีโดยจะโอนย้ายข้อมูลครั้งละเวิร์ด (word) หรือไบต์ (byte) แต่ในงานบางประเภทข้อมูลที่รับเข้ามาหรือส่งออกไปยังพอร์ตจะไม่เต็มไบต์ อาจเป็นเพียงหนึ่งบิตหรือสองบิต เช่นการเขียนโปรแกรมไปปิดหรือเปิดอุปกรณ์ภายนอก หรือ การเขียนโปแกรมไปแก้ไขข้อมูลบางบิตในหน่วยความจำ คำสั่งในการทำงานระดับบิตอยู่หลายคำสั่งดังต่อไปนี้

คำสั่งแบบหนึ่งบิต

คำสั่งที่ใช้ดำเนินการแบบหนึ่งบิตแสดงได้ดังตารางที่ 7.1 สำหรับสัญลักษณ์ที่เขียนว่า bit หมายถึงตำแหน่งหน่วยความจำที่สามารถเข้าถึงข้อมูลระดับบิตได้

ตารางที่ 7.1 คำสั่งระดับบิต

คำสั่ง	การทำงาน
SET bit	เซตบิต (bit = 1)
CLR bit	เคลียร์บิต (bit = 0)
CPL bit	กลับค่าบิต (bit = Not bit)
JB bit, target	กระโดดไปยัง target ถ้า bit = 1 (jump if bit)
JNB bit, target	กระโดดไปยัง target ถ้าbit = 0 (jump if no bit)
JBC bit, target	กระโดดไปยัง target ถ้า bit = 1 และเคลียร์บิต

พอร์ต I/O และตำแหน่งข้อมูลบิ

ไมโครคอนโทรลเลอร์ MCS-51 มีพอร์ตอินพุตเอาต์พุตแบบ 8 บิต จำนวน 4 พอร์ต คือ P0, P1, P2 และ P3 โดยพอร์ตทั้ง 4 พอร์ต เราสามารถเข้าถึงข้อมูลทีละบิตได้ ตัวอย่างเช่นถ้าหากต้องการเซตบิตของพอร์ตอาจเขียนรหัสคำสั่งได้ในรูปแบบดังนี้ “SETB X.Y” โดยที่ X เป็นพอร์ตหมายเลข 0, 1, 2 หรือ 3 ส่วน Y จะเป็นตำแหน่งบิตซึ่งมีค่าเท่ากับ 0 ถึง 7 ตัวอย่างเช่น ถ้าหากต้องการเซตบิตที่ 5 ของพอร์ต P1 จะเขียนคำสั่งได้ดังนี้ “SETB P1.5”

<u>ตัวอย่างที่ 7.1</u> จงเขียนโปรแกรมให้ทำงานดังนี้

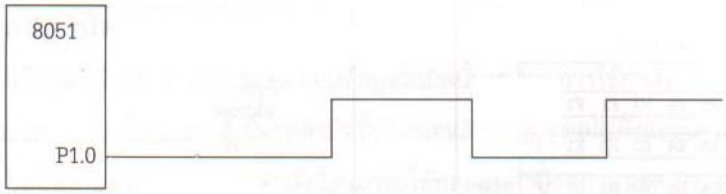
1. สร้างคำสั่งรูปสี่เหลี่ยมที่มีค่า duty cycle เท่ากับ 50% ออกมาทางบิต 0 ของพอร์ต 1
2. สร้างคำสั่งรูปสี่เหลี่ยมที่มีค่า duty cycle เท่ากับ 68% ออกมาทางบิต 3 ของพอร์ต 2

<u>วิธีทำ</u> 1. ค่า duty cycle เท่ากับ 50% หมายความว่าระยะเวลาที่เป็นลอจิก “1” และ “0” จะเท่ากัน ดังนั้นเราจะเขียนโปรแกรมโดยกลับค่า (toggle) ทางบิต P1.0 ได้ ดังนี้

```
HERE:  SETB   P1.0     ; เซตบิต 0 ของพอร์ต 1
       LCALL  DELAY    ; เรียกโปรแกรมย่อยหน่วยเวลา
       CLR    P1.0     ; เคลียร์พอร์ต P1.0 = 0
       LCALL  DELAY
       SJMP   HERE     ; ทำซ้ำ
```

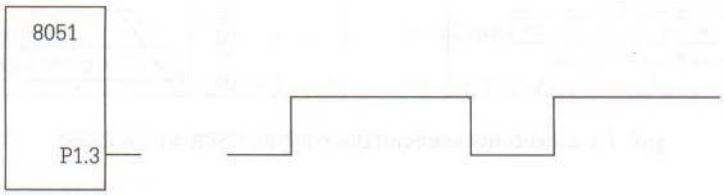
หรือเขียนโปรแกรมอีกแบบหนึ่งได้ดังนี้

```
HERE:  CPL    P1.0     ; กลับค่าบิต 0 ของพอร์ต 1
       LCALL  DELAY    ; เรียกโปรแกรมย่อยหน่วยเวลา
       SJMP   HEEE     ; ทำซ้ำ
```

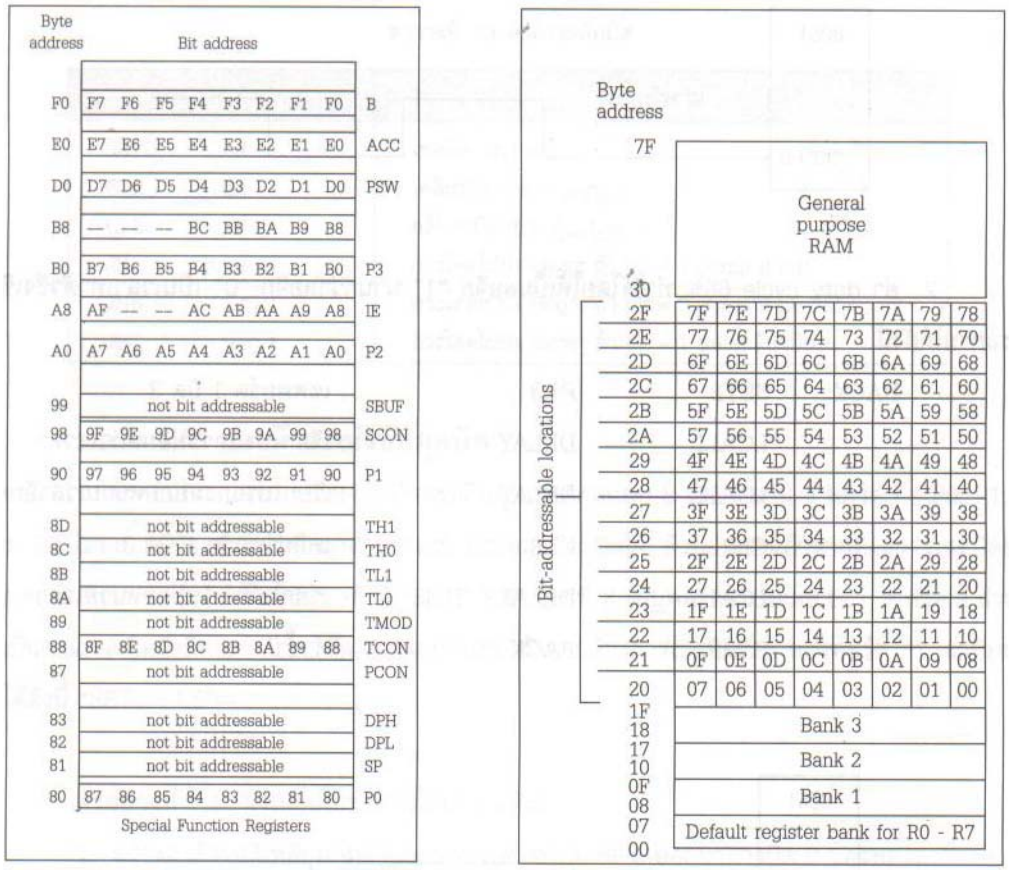


2. ค่า duty cycle 66% ทำได้โดยให้เป็นลอจิก “1” นานกว่าลอจิก “0” เป็นเวลาเท่าตัวซึ่งเขียนโปรแกรมได้ดังนี้

```
BACK;  SETB   P1.3      ; เซตพอร์ต 1 บิต 3
       LCALL  DELAY     ; เรียกโปรแกรมย่อยหน่วยเวลา
       LCALL  DELAY     ; เรียกโปรแกรมย่อยหน่วยเวลาอีกครั้ง
       CLR    P1.3      ; เคลียร์พอร์ต 1 บิต 3
       LCALL  DELAY     ; เรียกโปรแกรมย่อยหน่วยเวลา
       SJMP   BACK      ; ทำซ้ำ
```



แม้ว่าการเซตบิตจะเขียนคำสั่งเป็น “SETB P1.0” แต่เมื่อโปรแกรมทำการแปลเป็นรหัสภาษาเครื่องจะมีค่าเท่ากับคำสั่ง “SETB 90 H” เนื่องจาก P1.0 มีค่าตรงกับ RAM ในตำแหน่ง 90 H จากรูปที่ 7.1 จะแสดงค่าแอดเดรสของหน่วยความจำภายในที่เป็นส่วนของรีจีสเตอร์ฟังก์ชันพิเศษ (SFR) โดยจะแสดงเป็นตำแหน่งแบบไบต์ และตำแหน่งแบบบิตของรีจิสเตอร์ต่างๆ ที่สามรถอ้างแอดเดรสระดับบิตได้ซึ่งจะเห็นว่า P0 แต่ละบิตจะเริ่มที่ตำแหน่งบิต 80H ถึง 87H ส่วน P1 จะเริ่มที่ตำแหน่งบิต 90H ถึง 97H



รูปที่ 7.1 แสดงแอดเดรสของหน่วยความจำ SFR ของ MCS-51

<u>ตัวอย่างที่ 7.2</u> จากการเขียนคำสั่งต่อไปนี้จงตอบว่ามีผลต่อบิตใดในรีจิสเตอร์ตัวใดโดยดูรูปที่ 7.1 ประกอบ

ก. SETB 86H	ข. CLR 87H	ค. SETB	29H
ง. SETB 0A7H	จ. CLR 0F2H	ฉ. SETB	0E7H

<u>วิธีทำ</u>

ก. SETB	86H	จะเป็นการเซตบิต P0.6
ข. CLR	87H	จะเป็นการเคลียร์บิต P0.1
ค. SETB	92H	จะเป็นการเซตบิต P1.2
ง. SETB	0A7H	จะเป็นการเซตบิต P2.7
จ. CLR	0F2H	จะเป็นการเคลียร์บิต D2 ในรีจิสเตอร์ B
ฉ. SETB	0E7H	จะเป็นการเซตบิตที่ 7 ในรีจิสเตอร์ A

การทดสอบบิต

ถ้าหากมีข้อมูลหนึ่งบิตเราสามารถทดสอบข้อมูลในบิตนั้นได้ โดยการใช้คำสั่ง JNB (jump if no nit) และ JB (jump if bit = 1) ทั้งสองคำสั่งนี้จะดูค่าบิตที่กำหนดและจะกระโดดไปยังแอดเดรสที่กำหนดในคำสั่งขึ้นกับว่าข้อมูลในบิตนั้นเป็น “0” หรือ “1” คำสั่งนี้สามารถใช้ทดสอบค่าบิตจากพอร์ต P1, P2 และ P3 ได้

ตัวอย่างที่ 7.3 ถ้าหากมีข้อมูลเข้ามาทางบิต P3.2 จงเขียนโปรแกรมตรวจสอบค่าทางบิตนี้ถ้าเป็น “1” ให้ส่งสัญญาณที่มีการเปลี่ยนแปลงจาก “1” เป็น “0” ออกมาทางพอร์ต P1.5 เพื่อเปิด buzzer ให้ส่งเสียงออกมา

วิธีทำ

สมมติว่า บิต P1.5 ต่อกับ buzzer สำหรับส่งเสียง

```
HERE:  JNB   P2.3, HERE     ; monitoring จนค่าเป็น high
       SETB  P1.5           ; เซตบิต P1.5 = 1
       CLR   P1.5           ; ทำให้เปลี่ยนจากลอจิก “1” ไปเป็นลอจิก “0”
```

ตัวอย่างที่ 7.4 จงเขียนโปรแกรมทดสอบว่าค่าในรีจิสเตอร์ A เป็นเลขคู่หรือไม่ ถ้าเป็นให้นำค่านั้นมาหารด้วยสองถ้าไม่เป็นให้ทำเป็นเลขคู่และนำค่ามาหารด้วยสอง

วิธีทำ

การทดสอบว่าเป็นเลขคี่หรือคู่ทำได้โดยการดูว่าบิตต่ำสุด ถ้าเป็น 0 หมายความว่าเป็นเลขคู่ ถ้าเป็น 1 หมายความว่าเป็นเลขคี่ซึ่งสามารถเขียนเป็นโปรแกรมได้ดังนี้

```
       MOV   B,#2           ; B = 2
       JNB   Acc.0, YES     ; ถ้าบิต D0 ใน A เป็น 0 จะเป็นเลขคู่ให้กระโดดไป YES
       INC   A              ; ทำให้เป็นเลขคู่โดยการบวกด้วย 1
YES:   DIV   AB             ; A/B
```

ตัวอย่างที่ 7.5 จงเขียนโปรดแกรมทดสอบบิต 0 และบิต 5 ของรีจิสเตอร์ B ว่าเป็น “1” หรือไม่ ถ้ายังไม่เป็นให้ทำเป็นลอจิก “1” และเก็บค่าลงในรีจิสเตอร์ R0

วิธีทำ

```
       JNB   0F0H, Next_1      ; กระโดดถ้า B.0 เป็น “0”
       SETB  0F0H              ; เซตบิต B.0 ให้เป็น “1”
Next_1: JB   0F5H, Next_2      ; กระโดดถ้า B.5 เป็น “0”
       SETB  0F5H              ; เซตบิต B.5 ให้เป็น “1”
Next_2: MOV  R0,B              ; นำค่าในรีจิสเตอร์ B เก็บใน R0
```

ตัวอย่างที่ 7.6 จงเขียนโปรแกรมเก็บค่ารีจิสเตอร์แอกคิวมูเลเตอร์ลงในรีจีสเตอร์ R7 แบงค์ที่ 2

วิธีทำ

```
       CLR   PSW.3
       SETB  PSW.4
       MOV   R7,A
```

ในแต่ละบิตของรีจิเตอร์ PSW (Program Status Word) สามารถอ้างแอดแดรสในระดับบิตได้เช่นกัน ถ้าหาก MCS-51 ถูกรีเซตจะทำให้บิตสำหรับเลือกแบงค์รีจีสเตอร์ RS1 และ RS0 มีค่าเป็น “0” ทั้งสองบิตซึ่งจะทำให้ MCS-51 ติดต่อกับรีจิเตอร์ R0 ถึง R7 ในแบงค์ที่ 0 สำหรับรีจิเตอร์ PSW และการเลือกแบงค์ของรีจิสเตอร์สามารถแสดงได้ดังรูปที่ 7.2

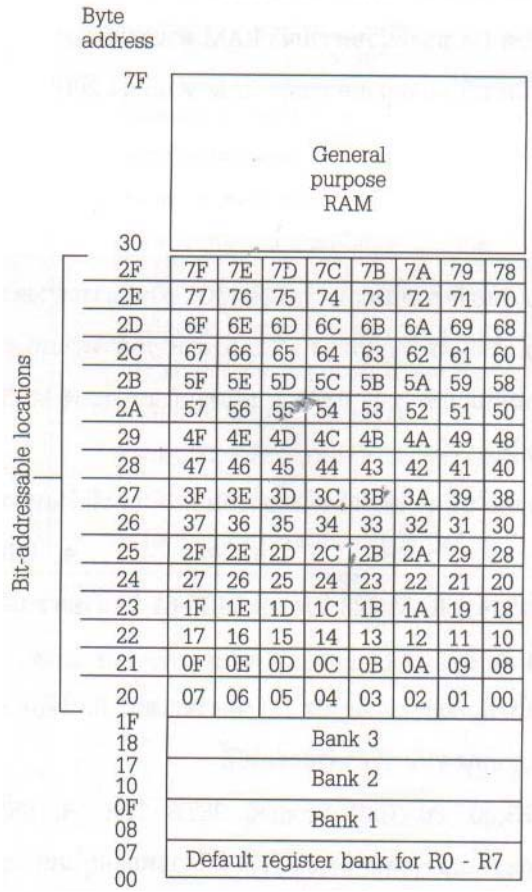
CY	AC	-	RS1	RS0	OV	-	P

RS1	RS0	Register Bank	Address
0	0	0	00H – 07H
0	1	1	08H – 0FH
1	0	2	10H – 17H
1	1	3	18H – 1FH

รูปที่ 7.2 บิตของรีจิสเตอร์ PSW

Bit – addressable RAM

ไมโครคอนโทรลเลอร์ตระกูล MCS – 51 ตัวที่เป็นพื้นฐานสำหรับศึกษากันทั่วไปคือเบอร์ 8031 หรือ 8051 ที่มีหน่วยความจำ RAM ภายใน 128 ไบต์นั้นจะมีอยู่ 16 ไบต์ ที่สามารถเข้าถึงข้อมูลในระดับบิตได้ (bit - addressable) โดย 16 ไบต์ดังกล่าวอยู่ที่ตำแหน่งแอดเดรสไบต์ 20H ถึง 2FH โดยจะอ้างตำแหน่งแบบบิตได้ 16 × 8 = 128 บิต ซึ่งจะเริ่มต้นที่แอดเดรสบิต 00H ถึง 7FH ดังรูปที่ 7.3 สำหรับบิต 0 ถึงบิต 7 จะเป็นการอ้างตำแหน่งบิตของไบต์แรกของหน่วยความจำส่วนนี้ (ไบต์แอดเดรส 20H) สำหรับบิต 8 ถึง 0FH จะเป็นตำแหน่งของไบต์ถัดไป จากรูปที่ 7.1 และรูปที่ 7.3 จะเห็นว่าตำแหน่งบิต 00 – 7F จะอยู่ใน RAM ไบต์ 20 – 2FH สำหรับตำแหน่งบิต 80 – 7FH จะอยู่ในรีจิเตอร์ฟังก์ชันพิเศษ



รูปที่ 7.3 หน่วยความจำภายใน 128 ไบต์

<u>ตัวอย่างที่ 7.7</u> จากชุดคำสั่งต่อไปนี้จงแสดงว่าคำสั่งใดมีผลต่อบิตใดในหน่วยความจำตำแหน่งใด

ก. SETB	42H	; เซตบิตที่ 42H
ข. CLR	67H	; เคลียร์บิตที่ 67H
ค. CLR	0FH	; เคลียร์บิตที่ 0FH
ง. SETB	28H	; เซตบิตที่ 28H
จ. CLR	12	; เคลียร์บิตที่ 12 (เลขฐานสิบ)
ฉ. SETB	05	; เซตบิตที่ 5

<u>วิธีทำ</u>

ก. บิต D2 ของหน่วยความจำ RAM ตำแหน่ง 28H

ข. บิต D7 ของหน่วยความจำ RAM ตำแหน่ง 2CH

ค. บิต D7 ของหน่วยความจำ RAM ตำแหน่ง 21H

ง. บิต D0 ของหน่วยความจำ RAM ตำแหน่ง 25H

จ. บิต D4 ของหน่วยความจำ RAM ตำแหน่ง 21H

ฉ. บิต D5 ของหน่วยความจำ RAM ตำแหน่ง 20H

คำถาม ?

1. จริงหรือไม่พอร์ตภายในทุกพอร์ตของ MCS–51 สามารถอ้างตำแหน่งระดับบิตได้
2. จริงหรือไม่รีจิสเตอร์ทุกตัวของ MCS–51 สามารถอ้างตำแหน่งระดับบิตได้
3. จริงหรือไม่หน่วยความจำ RAM ภายในทุกตำแหน่งของ MCS–51 สามารถอ้างตำแหน่งระดับบิตได้
4. รีจิสเตอร์ต่อไปนี้ ตัวใดเข้าถึงข้อมูลระดับบิตได้และมีตำแหน่งบิตในช่วงแอดเดรสใด

   ก. A ข. B ค. R4 ง. PSW จ. R7
5. หน่วยความจำ RAM 128 ไบต์ของ MCS–51 มีหน่วยความจำที่สามารถอ้างตำแหน่งระดับบิตได้กี่ไบต์
6. จะเขียนโปรแกรมตรวจสอบบิต D0 ของรีจิสเตอร์ R3 ได้อย่างไรว่าเป็นลอจิก “0” หรือ “1”
7. จงบอกเอาต์พุตจากการทำคำสั่งต่อไปนี้

   ก. SETB 20 ข. CLR 32 ค. SETB 12H ง. SETB 95H จ.SETB 0E6H
8. บิตแอดเดรสในช่วงตำแหน่ง 00–7FH มีกี่ตำแหน่งอยู่ในหน่วยความจำช่วงใด และ บิตแอดเดรสในช่วงตำแหน่ง 80-F7H อยู่ในหน่วยความจำช่วงใด
9. จริงหรือไม่ พอร์ต P0, P1, P2 และ P3 อยู่ในช่วงรีจิสเตอร์ฟังก์ชันพิเศษ
10. จริงหรือไม่ รีจิสเตอร์ TCON สามารถอ้างแอดเดรสระดับบิตได้

7.2 การกระทำกับบิตตัวทด

การเขียนโปรแกรมดำเนินการทางคณิตศาสตร์และลอจิกกับบิตนั้นจะมีบิตพิเศษบิตหนึ่งที่คำสั่งมากมายมีผลกระทำต่อบิตนี้ และคำสั่งหลายคำสั่งจะดำเนินการทดสอบบิตนี้ โดยอาจเรียกบิตนี้ว่าเป็นบิตแอกคิวมูเลเตอร์ได้ บิตนี้ก็คือบิตแฟลกตัวทด (Carry flag: CY)

ในตารางที่ 7.2 จะแสดงคำสั่งต่าง ๆ ที่กระทำกับบิตแฟลกตัวทด โดยมีทั้งคำสั่งย้ายข้อมูลและคำสั่งการกระทำทางลอจิก

ตารางที่ 7.1 คำสั่งการกระทำกับบิตตัวทด

คำสั่ง		การทำงาน
SET	C	ให้บิตแฟลกตัวทดเป็น 1
CLR	C	เคลียร์บิตแฟลกตัวทด
CPL	C	กลับค่าบิตแฟลกตัวทด
MOV	bit,C	ย้ายค่าจากบิตแฟลกตัวทดให้กับตำแหน่งบิต
MOV	C,bit	ย้ายค่าจากตำแหน่งบิตให้กับแฟลกตัวทด
JNC	target	กระโดดไปยังแอดเดรส target ถ้าแฟลกตัวทดเป็น 0
JC	target	กระโดดไปยังแอดเดรส target ถ้าแฟลกตัวทดเป็น 1
ANL	C, bit	AND ค่าบิตและเก็บไว้ในบิตตัวทด
ORL	C, bit	OR ค่าบิตและเก็บในบิตตัวทด

ตัวอย่างที่ 7.8 จงเขียนโปรแกรมอ่านค่าจากบิต P1.2 และ P1.3 จากนั้นนำไปเก็บไว้ใน RAM ตำแหน่งบิตที่ 6 และ 7

วิธีทำ

```
        MOV     C,P1.2          ; อ่านค่าจาก P1.2 เก็บในบิตตัวทด
        MOV     06,C            ; อ่านค่าจากบิตตัวทดเก็บในตำแหน่ง 6
        MOV     C,P1.3          ; อ่านค่าจาก P1.3 เก็บในบิตตัวทด
        MOV     07,C            ; อ่านค่าจากบิตตัวทดเก็บในตำแหน่ง 7
```

ตัวอย่างที่ 7.9 ระบบฝากข้อความทางโทรศัพท์ระบบหนึ่งใช้ MCS–51 เป็นตัวควบคุม โดยถ้าข้อมูลในตำแหน่งบิตที่ 12H มีค่าเป็น “1” แสดงว่ามีข้อความฝากไว้และให้แสดงคำว่า “New Messages” ทางจอ LCD แต่ถ้าในตำแหน่งนั้นเป็น “0” ให้แสดงคำว่า “No New Messages” จงเขียนโปรแกรมให้ทำงานดังกล่าว

วิธีทำ

```
        MOV     C,12H           ; อ่านค่าจากบิตตำแหน่ง 12H มาเก็บในแฟลก
        JNC     NO              ; ตรวจสอบว่าเป็น “1” หรือไม่
        MOV     DPTR            ; ถ้าเป็น “1” ให้โหลดค่าแอดเดรส Messages
        LCALL   DISPLAY         ; แสดงผลทาง LCD
        SJMP    EXIT            ; จบโปรแกรม
NO:     MOV     DPTR,#429H      ; โหลดค่าแอดเดรส No Messages
        LCALL   DISPLAY         ; แสดงผลทาง LCD
EXIT:                           ; จบโปรแกรม

        ORG     400H
Yes-Mg: DB      'New Messages'
        ORG     420H
No-Mg:  DB      'No New Messages'
```

ตัวอย่างที่ 7.10 ถ้าหากบิต P2.2 ถูกใช้ในการควบคุมการเปิดไฟนอกอาคาร และบิต P2.5 ถูกใช้ควบคุมการเปิดไฟภายในอาคาร จงเขียนโปรแกรมเปิดไฟนอกอาคาร และปิดไฟในอาคาร

วิธีทำ

```
SETB   C          ; CY = 1
ORL    C,P2.2     ; CY = P2.2 OR กับบิต CY
MOV    P2.2,C     ; เปิดไฟ ถ้าเดิมยังไม่ได้เปิด
CLR    C          ; CY = 0
ANL    C,P2.5     ; CY = P2.5 AND กับบิต CY
MOV    P2.5,C     ; ปิดไฟ ถ้าเดิมยังไม่ได้ปิด
```

คำถาม ?

1. จงหาค่าบิตแฟลก CY ถ้ามีการทำคำสั่งต่อไปนี้

```
        CLR    A
        ADD    A,#0FFH
        JNC    OVER
        CPL    C
OVER:   ...................
```

2. จงหาค่าบิตแฟลก CY ถ้ามีการทำคำสั่งต่อไปนี้

```
        CLR    B
        JNC    OVER
        SETB   C
OVER:   ...................
```

3. จงหาค่าบิตแฟลก CY ถ้ามีการทำคำสั่งต่อไปนี้

```
        CLR    C
        JNC    OVER
        CPL    C
OVER:   ...................
```

4. จงเขียนคำสั่งอ่านค่าจากบิต P2.7 และเก็บใน RAM ตำแหน่งที่ 31
5. จงเขียนคำสั่งนำค่าใน RAM ตำแหน่งที่ 09 ส่งออกไปยัง P1.4

บทที่ 8
การโปรแกรมตัวนับจับเวลา

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 เบอร์ที่นิยมใช้กันทั่วไปจะมีตัวนับจับเวลาอยู่สองตัว โดยสามารถเขียนโปรแกรมให้มันนับสัญญาณนาฬิกาภายในหรือใช้เป็นตัวจับเวลาได้ถ้าหากเขียนให้นับสัญญาณจากภายนอกจะเป็นการใช้งานเป็นตัวนับ ในหัวข้อที่ 8.1 จะกล่าวถึงการใช้งานเป็นตัวจับเวลา ส่วนหัวข้อที่ 8.2 จะกล่าวถึงการใช้งานเป็นตัวนับ

8.1 การโปรแกรมเป็นตัวจับเวลา

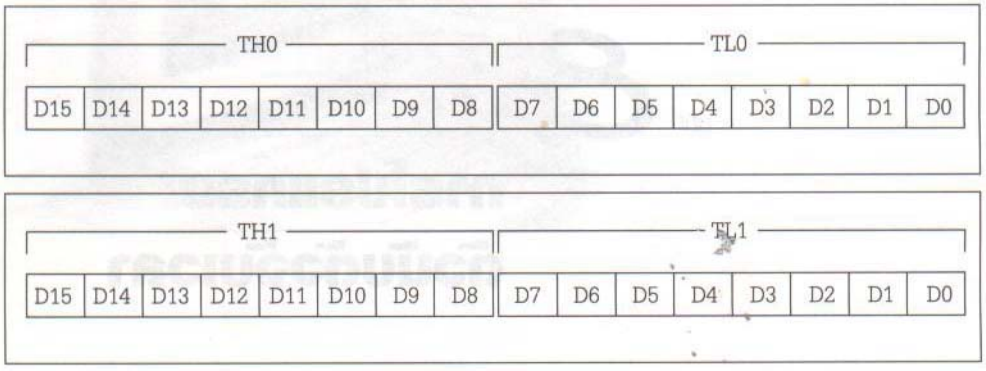
MCS-51 มีรีจิสเตอร์ตัวจับเวลาอยู่สองตัวเรียกชื่อว่า Timer 0 และ Timer 1 ทั้งสองตัวจะเป็นรีจิสเตอร์แบบ 16 บิต และแต่ละตัวสามารถแยกออกเป็นไบต์ต่ำและไบต์สูง เราสามารถเขียนโปรแกรมให้ทำงานเป็นตัวจับเวลาได้ โดยรีจิสเตอร์ที่เกี่ยวข้องได้แก่

รีจิสเตอร์ Timer 0

รีจิสเตอร์ตัวนี้จะมีขนาด 16 บิต เกิดจากรีจิสเตอร์สองตัวประกอบกันคือรีจิสเตอร์ไบต์ต่ำชื่อว่า TL0 (Timer 0 low byte) และรีจิสเตอร์ไบต์สูงชื่อว่า TH 0 (Timer 0 high byte) การใช้งานรีจิสเตอร์ตัวนี้จะคล้ายกับรีจิสเตอร์อื่น ๆ ทั่วไป เช่น เขียนคำสั่งว่า “MOV TL0,#4FH” จะเป็นการกำหนดค่า 4FH ใส่ในรีจิสเตอร์ไทเมอร์ 0 ไบต์ต่ำ หรือถ้าเขียนคำสั่งเป็น MOV R5,TH0 จะเป็นการนำค่าในรีจิสเตอร์ไทเมอร์ 0 ไบต์สูงมาใส่ในรีจิสเตอร์ R5

รีจิสเตอร์ Timer 1

เป็นรีจิสเตอร์ขนาด 16 บิต คล้ายกับรีจิสเตอร์ไทเมอร์ 0 โดยประกอบจากรีจิสเตอร์ 8 บิตสองตัวคือไบต์ต่ำ TL1 และ ไบต์สูง TH 1



รูปที่ 8.1 รีจิสเตอร์ Timer 0 และ Timer 1

รีจิสเตอร์ TMOD

การใช้งานไทเมอร์ 0 และไทเมอร์ 1 เราสามารถกำหนดโหมดการทำงานได้สี่รูปแบบ โดยการกำหนดค่าลงในรีจิสเตอร์ควบคุมที่ชื่อว่า TMOD (Timer mode) ซึ่งเป็นรีจิสเตอร์ขนาด 8 บิตโดย 4 บิตล่างใช้กำหนดการทำงานรีจิสเตอร์ไทเมอร์ 0 และ 4 บิตบนใช้กำหนดการทำงานไทเมอร์ 1

MSB							LSB
Gate	C/T	M1	Mo	Gate	C/T	M1	M0
Timer1				Timer0			

GATE เป็นบิตเซตวงจรให้ทำงาน

C/T ใช้เลือกเป็นตัวนับ หรือตัวจับเวลา

M1 โหมดบิต 1

M0 โหมดบิต 0

M1M0	Mode	การทำงาน
00	0	เป็น Timer แบบ 13 บิต โดยใช้ TH × 8 บิต และ TH × 5 บิต
01	1	เป็น Timer แบบ 16 บิต
10	2	เป็น Timer แบบ 8 บิต auto reload
11	3	แยกการทำงานออกเป็น Timer 8 บิต และ Counter 8 บิต

รูปที่ 8.2 รีจิสเตอร์ TMOD

บิต M1,M0

สองบิตนี้จะใช้เลือกโหมดการทำงานดังแสดงในรูปที่ 8.2 โดยโหมด 0 จะเป็นการทำงานแบบ 13 บิต โหมด1 จะทำงานแบบ 16 บิต โหมด 2 ทำงานแบบ 8 บิตที่สามารถโหลดค่ากลับได้เอง ส่วนโหมด 3 จะใช้งานเป็นเมอร์ 8 บิต สองตัว และ16 บิต หนึ่งตัว

C/T (Clock/Timer)

ในรีจิสเตอร์ TMOD จะมีบิตที่ใช้เลือกว่าจะให้ไทเมอร์ทำงานเป็นตัวนับหรือตัวจับเวลาบิตนี้คือบิต C/T ถ้าหากมีค่าลอจิกเป็น"0" หมายความว่าใช้งานเป็นตัวจับเวลาโดยจะขึ้นกับค่าความถี่ของสัญญาณนาฬิกาของระบบ ถ้าเป็น "1" หมายความว่าใช้งานเป็นตัวนับสัญญาณจากภายนอก

<u>ตัวอย่างที่ 8.1</u> จงบอกโหมดการทำงานของไมเมอร์ถ้าเขียนคำสั่งต่อไปนี้

ก. MOV TMOD,#01H ข. MOV TMOD,#20H ค. MOV TMOD,#12H

<u>วิธีทำ</u>

จากเลขฐานสิบหกให้ตีความหมายของแต่ละบิตใน TMOD ออกมาโดยดูรูปที่ 8.2 ประกอบ

ก. TMOD = 00000001 ไทเมอร์ 0 ทำงานในโหมด 1

ข. TMOD = 00100000 ไทเมอร์ 1 ทำงานในโหมด 2

ค. TMOD = 00010010 ไทเมอร์ 0 ทำงานในโหมด 2 และ ไทเมอร์ 1 ทำงานในโหมด 1

สัญญาณนาฬิกาของไทเมอร์

ตามที่ทราบกันมาแล้วว่าตัวไทเมอร์นี้จะทำการนับสัญญาณนาฬิกาที่ถูกส่งเข้ามา ถ้า C/T = 0 จะนับสัญญาณนาฬิกาในระบบที่ได้จากความถี่ของ Crystal ที่ต่อกับ MCS-51 หารด้วย 12 ถ้าหาก MSC-51 ทำงานด้วยความถี่สัญญาณนาฬิกา 12 MHz และให้ไทเมอร์ทำงานเป็นตัวจับเวลาจะทำให้สัญญาณที่เข้าสู่ไทเมอร์มีความถี่เท่ากับ 1 MHz โดยทั่วไปแล้วระบบที่ใช้ MCS-51 ในปัจจุบันจะใช้ XTAL ที่มีความถี่ตั้งแต่ 10 MHz ถึง 40 MHz แต่บางระบบจะต้องใช้ความถี่ 11.0592 MHz เพื่อให้สามารถสื่อสารข้อมูลแบบอนุกรมกับอุปกรณ์อื่นๆได้อย่างถูกต้องซึ่งจะได้ศึกษารายละเอียดในบทต่อไป

<u>ตัวอย่างที่ 8.2</u> จงหาค่าความถี่ของสัญญาณนาฬิกาที่เข้าสู่ไทเมอร์ ถ้าหาก MCS-51 ใช้ XTAL ที่มีความถี่ดังนี้

ก. 12 MHz ข.16 MHz ค. 11.0592 MHz

<u>วิธีทำ</u> ก. 1/12 x 12 MHz = 1 MHz และ T = 1/1 MHz = 1 us

ข. 1/12 x 16 MHz = 1.333 MHz และ T = 1/1.333 MHz = 0.75 us

ค. 1/12 x 11.0592 MHz = 921.6 MHz และ T = 1/921.6 MHz = 1.085 us

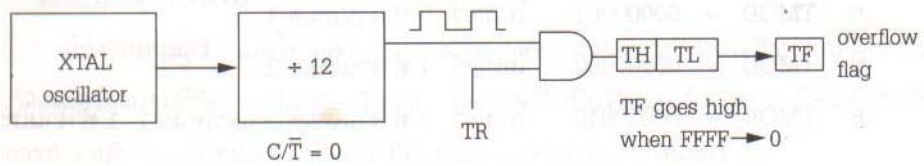
การโปรแกรมในโหมด 1

ลักษณะการทำงานของไทเมอร์ในโหมด 1 จะเป็นดังนี้

1. ทำงานเป็นรีจิสเตอร์แบบ 16 บิต ซึ่งจะเก็บค่าได้ตั้งแต่ 0000 ถึง FFFF โดยจะกำหนดค่าได้โดยโปรแกรมข้อมูลลงไปในรีจิสเตอร์ TL และ TH

2. หลังจากโหลดข้อมูลขนาด 16 บิตลงไปแล้ว ตัวไทเมอร์จะเริ่มทำงาน เมื่อได้รับคำสั่ง SETB TR0 และสำหรับไทเมอร์ 0 และ SETB TR1 สำหรับไมเมอร์ 1

3. เมื่อให้ไทเมอร์เริ่มทำงานแล้วในตัวไทเมอร์จะเกิดการนับขึ้นตั้งแต่ค่าเริ่มต้นที่ป้อนเข้าไปจนถึงค่า FFFFH เป็น0000H จะมีการเซ็ตบิต TF (timer flag) ให้เป็นลอจิก “1” ดังนั้นการใช้งานเราจะต้องมองว่าบิต TF นี้เป็นลอจิก “1” หรือยัง ถ้าเป็นแสดงว่าได้นับมาถึงค่าที่กำหนดแล้ว จากนั้นจะต้องหยุดไทเมอร์โดยการใช้คำสั่ง CLR TR0 หรือ CLR TR1 สำหรับไทเมอร์ 0 และไทเมอร์ 1 ตามลำดับ และจะต้องเคลียร์บิตแฟลก TF ด้วย เนื่องจากบิตนี้จะเซตโดยฮาร์ดแวร์เมื่อไทเมอร์มีการเปลี่ยนจาก FFFFH เป็น 0000H แต่ต้องเคลียร์โดยซอฟต์แวร์



4. หลังจากไทเมอร์ทำงานมาตามกำหนดแล้วจะต้องโหลดค่าให้กับรีจิสเตอร์ TH และ TL อีกครั้งเพื่อใช้งานครั้งต่อไป

ขั้นตอนการโปรแกรมในโหมด 1

ถ้าหากต้องการเขียนโปรแกรมจับเวลาโดยใช้ไทเมอร์โหมด 1 จะต้องเขียนโปรแกรมตามลำดับดังนี้

1. โหลดค่าลงรีจิสเตอร์ TMOD เพื่อกำหนดโหมดการทำงานของไทเมอร์ตัวที่ต้องการใช้งาน
2. โหลดค่าลงรีจิสเตอร์ TL และ TH เพื่อกำหนดค่าเริ่มต้นของการนับ
3. ให้ไทเมอร์เริ่มทำงาน
4. ตรวจสอบแฟลกไทเมอร์ (TF) ว่าเป็น “1” หรือยัง ถ้ายังให้วนลูปตรวจสอบจนกว่าจะเป็น “1”
5. หยุดไทเมอร์
6. เคลียร์บิตแฟลก TF เพื่อใช้งานครั้งต่อไป
7. กลับไปยังขั้นตอนที่ 2 เพื่อโหลดค่าสำหรับการทำงานครั้งต่อไป

จากขั้นตอนที่กล่าวมาสามารถเขียนชุดคำสั่งได้ดังตัวอย่างที่ 8.3 สำหรับการคำนวนค่าเริ่มต้นให้กับไทเมอร์เพื่อใช้ในการจับเวลาจะขึ้นอยู่กับความถี่ของ XTAL ด้วยโดยจะศึกษาได้ดังตัวอย่างที่ 8.4 ซึ่งจะสร้างคลื่นสี่เหลี่ยม (square wave) ออกมาทางขา P1.5 และตัวอย่างที่ 8.5 จะเป็นการคำนวนค่าความถี่ของสัญญาณที่ได้จากโปรแกรมในตัวอย่างที่ 8.4

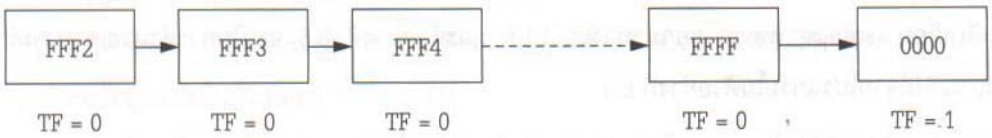
<u>ตัวอย่างที่ 8.3</u> ต่อไปจะแสดงการวิเคราะห์โปรแกรมการสร้างคำสั่งรูปสี่เหลี่ยมที่มีค่า duty cycle เท่ากับ 50% (เวลาเป็น “1” เท่ากับเวลาเป็”0”) โดยการใช้ไทเมอร์ 0 เป็นตัวจับเวลา

```
        MOV    TMOD = 01        ; ไทเมอร์ 0 โหมด 1 (16 บิต)
HERE:   MOV    TL0, #0F2H       ; ไบต์ต่ำ TL0 = F2H
        MOV    TH0,#0FFH        ; ไบต์สูง TH0 =FFH
        CPL    P1.5             ; toggle P1.5
        ACALL  DELAY
        SJMP   HERE             ; โหลดค่า TL และ TH อีกครั้ง
DELAY:  SETB   TR0              ; ให้ไทเมอร์เริ่มทำงาน
AGAIN:  JNB    TF0, AGAIN       ; เฝ้ามองบิตแฟลกจนเป็น “1”
        CLR    TR0              ; หยุดไทเมอร์
        CLR    TF0              ; เคลียร์บิตแฟลกให้ป็น “0”
        RET
```

จากโปรแกรมวิเคราะห์การทำงานดังนี้

1. โหลดข้อมูลลง TMOD เพื่อเลือกโหมดการทำงาน
2. โหลดค่า FFF2H ลงใน TH0-TL0
3. ทำให้บิต P1.5 toggle จาก “1” เป็น “0” จาก “0” เป็น “1”
4. เรียกโปรแกรมหน่วงเวลา DELAY เพื่อให้เป็น “0” หรือ “1” ชั่วขณะ
5. ในโปรแกรหน่วงเวลา DELAY จะใช้ timer 0 โดยเริ่มจับเวลาด้วยคำสั่ง SETB TR0
6. ไทเมอร์ 0 เริ่มนับ 1 ซึ่งมีสัญญาณเข้ามาจาก FFF3, FFF4, FFF5, FFF6, FFF7, FFF8 ไปจนถึง FFFFH เมื่อมีสัญญาณเข้ามาอีกลูกจะกลับเป็น 0 และบิตแฟลกไทเมอร์จะเซต (TF0=1) โดยใช้คำสั่ง JNB คอยดูบิตแฟลก
7. หยุดไทเมอร์ 0 ด้วยคำสั่ง CLR TR0 จะจบโปรแกรมย่อยหน่วงเวลา

การเปลี่ยนแปลงของรีจิสเตอร์ไทเมอร์และบิตแฟลก TF เป็นดังรูปต่อไปนี้



<u>ตัวอย่างที่ 8.4</u> จากตัวอย่างที่ 8.3 จงคำนวนหาเวลาหน่วงในโปรแกรมย่อย DELAY ถ้าหาก XTAL มีความถี่ 11.0594 MHz

<u>วิธีทำ</u>

สัญญาณที่จะเข้าสู่ตัวไทเมอร์จะเท่ากับ 1/12 ของความถี่ XTAL ดังนั้นสัญญาณที่เข้าสู่ไทเมอร์จะมีความถี่เท่ากับ 11.0592 MHz/12 = 921.6 KHz และคาบเวลาจะเท่ากับ T = 1/921.6 KHz = 1.085 us ดังนั้นถ้าหาค่าเวลาที่ใช้ในการนับจะต้องนำค่าคำนวนสัญญาณที่ทำให้ไทเมอร์เกิดโอเวอร์โฟลว์มาคูณกับค่าเวลา 1.085 us นี้

จำนวนของการนับที่ทำให้เกิดการโอเวอร์โฟลว์จะเท่ากับ FFFFH – FFF2H = 0DH หรือ 13 D และรวมกับสัญญาณที่ทำให้ค่าในไทเมอร์ FFFF กลับเป็น 0 อีกลูกหนึ่ง รวมเป็น 14 ลูก ดังนั้นเวลาหน่วงจะเท่ากับ 14 x 1.085 us = 15.19 us สำหรับสัญญาณพัลส์ครึ่งลูก ดังนั้นคาบเวลาจะเท่ากับ T = 2 x 15.19 us = 30.38 us

ตัวอย่างที่ 8.5 จากโปรแกรมในตัวอย่างที่ผ่านมาจงหาความถี่ของคลื่นทางบิต P1.5

วิธีทำ

จะต้องรวมเวลาทั้งหมดของการทำโปรแกรมและรวมเวลาหน่วยในโปรแกรมย่อย DELAY ด้วย โดยรวมไซเคิลของคำสั่งทั้งหมด โดยดูจากตารางเวลาของคำสั่งจะได้เป็น

			ไซเคิล
HERE:	MOV	TL0, #0F2H	2
	MOV	TH0,#0FFH	2
	CPL	P1.5	1
	ACALL	DELAY	2
	SJMP	HERE	1
DELAY:	SETB	TR0	1
AGAIN:	JNB	TF0, AGAIN	14
	CLR	TR0	1
	CLR	TF0	1
	RET		1
		รวม	27

ดังนั้นคาบเวลาจะได้เป็น T = 2 x 27 x 1.085 us = 58.59 us และ F = 17067.75 Hz

ตัวอย่าที่ 8.6 จากชุดคำสั่งต่อไปนี้จงหาเวลาหน่วงจากไทเมอร์ 0 โดยไม่รวมเวลาที่เสียไปในโปรแกรม

```
        CLR   P2.3        ; เคลียร์ P2.3
        MOV   TMOD,#01    ; ใช้ไทเมอร์ 0 โหมด 1
HERE:   MOV   TL0,#3EH    ; TL0 = 3EH โปรแกรมค่าไบต์ต่ำ
        MOV   TH0,#0B8H   ; TH0 = B8H ใส่ค่าไบต์สูง
        SETB  P2.3        ; ให้บิต P2.3 เป็น “1”
        SETB  TR0         ; ให้ไทเมอร์ 0 เริ่มทำงาน
AGAIN:  JNB   TF0,AGAIN   ; ตรวจสอบบิตแฟลก
        CLR   TR0         ; หยุดไทเมอร์ 0
        CLR   TH0         ; เคลียร์บิตแฟลก
        CLR   P2.3        ; ให้บิต P2.3 เป็น “0”
```

วิธีทำ

หาจำนวนสัญญาณของการนับ

(FFFF – B83E+1) = 47C2H = 18370 จะเป็นเวลาเท่ากับ 18370 x 1.085 us = 19.93145 ms

ตัวอย่าที่ 8.7 จากตัวอย่างที่ 8.6 จะแก้ไขในรีจิสเตอร์ TH และ TL ให้หน่วงเวลาได้มากที่สุด

วิธีทำ

จะต้องให้รีจิสเตอร์ TH และ TL มีค่าเป็น 0 เพื่อให้การนับเริ่มจาก 0000 ไปจนถึง FFFFH จะได้โปรแกรมดังนี้

```
        CLR     P2.3            ; เคลียร์บิต P2.3
        MOV     TMOD,#01        ; ไทเมอร์ 0 โหมด 1 (16-bit mode)
HERE:   MOV     TL0,#0          ; ใส่ค่าไบต์ต่ำ
        MOV     TH0,#0          ; ใส่ค่าไบต์สูง
        SETB    P2.3            ; เซตให้บิต P2.3 เป็น “1”
        SETB    TR0             ; ให้ไทเมอร์ 0 เริ่มทำงาน
AGAIN:  JNB     TF0,AGAIN       ; เฝ้ามองบิตแฟลก 0
        CLR     TR0             ; หยุดไทเมอร์ 0
        CLR     TF0             ; เคลียร์บิตแฟลกไทเมอร์ 0
        CLR     P2.3
```

การนับตั้งแต่ 0000 ไปจนถึง FFFF และให้เกิดการทดที่แฟลก TF จะใช้สัญญาณทั้งหมด 65536 ลูก ดังนั้นเวลาที่เสียไปจะเท่ากับ (65536-0) x 1.085 us = 71.1065 us

จากตัวอย่าที่ 8.6 และ 8.7 ที่ผ่านมาจะเห็นว่าโหลดค่าเข้ารีจิสเตอร์ TH และ TL เพียงครั้งเดียว ถ้า MSC-51 ทำงานตามนี้จะให้พัลส์ออกมาเพียงลูกเดียวในตัวอย่างที่ 8.8 จะให้โหลดค่าเข้ารีจิสเตอร์ไทเมอร์อีกครั้ง และให้โปรแกรมทำซ้ำก็จะได้พัลส์ที่ต่อเนื่องออกมา

<u>ตัวอย่างที่ 8.8</u> โปรแกรมต่อไปนี้จะให้คลื่นรูปสี่เหลี่ยมออกมาทางบิต P1.5 โดยใช้ไทเมอร์ 1 เป็นตัวหน่วงเวลา จงหาค่าความถี่ของคลื่นถ้า XTAL ที่ใช้มีความถี่ 11.0592 MHz โดยไม่ต้องคำนวนเวลาที่เสียไปในคำสั่งอื่นๆในลูป

```
        MOV     TMOD,#10H       ; ไทเมอร์ 1 โหมด 1
AGAIN:  MOV     TL1,#34H        ; TL1 = 34H ไบต์ต่ำ
        MOV     TH1,#76H        ; TH1 = 76H ไบต์สูง
                                ; ไทเมอร์ = 7634H
        SETB    TR1             ; เริ่มทำงานไทเมอร์1
BACK:   JNB     TF1,BACK        ; รอจนตัวจับเวลาเกิดโอเวอร์โฟลว์
        CLR     TR1             ; หยุดไทเมอร์1
        CPL     P1.5            ; กลับค่าลอจิกบิต P1.5
        CLR     TF1             ; เคลียร์แฟลกไทเมอร์
        SJMP    AGAIN           ; โหลดค่าและเริ่ทำงานใหม่
```

<u>วิธีทำ</u>

จากโปรแกรมจะทำงานในโหมด 1 และใช้ทำซ้ำโหลดค่าเข้าไทเมอร์ใหม่ด้วยคำสั่ง SJMP จำนวนสัญญาณนาฬิกาที่ทำให้แฟลกไทเมอณ์เซตเท่ากับ FFFFH – 7634H = 89CBH + 1 = 89CCH หรือสัญญาณนาฬิกาจำนวน 35276 ลูก ซึ่งเป็นเวลาเท่ากับ 35276 x 1.085 us = 38274 us ทำให้ได้ความถี่ 26.127 MHz ออกมา

การคำนวนค่าสำหรับโหลดเข้าไทเมอร์

การที่จะให้ไทเมอร์ทำการหน่วงเวลาตามที่เราต้องการนั้นจะต้องโหลดค่าให้กับรีจิสเตอร์ TH และ TL ก่อน โดยค่าในรีจิสเตอร์จะขึ้นกับความถี่สัญญาณนาฬิกาที่ใช้ในระบบด้วย ถ้าหาก 8051 ทำงานที่ความถี่ 11.0592 MHz การคำนวนค่าในรีจิสเตอร์สามารถทำได้ดังนี้

1. หารค่าเวลาที่ต้องการจับเวลาด้วยค่า 1.085 us
2. คำนวนค่า 65536 – N โดย N เป็นค่าที่ได้จากขั้นตอนที่ 1
3. แปลงค่าที่ได้จากขั้นตอนที่ 2 เป็นเลขฐานสิบหกในรูปของ yyxx ซึ่งเป็นค่าที่จะโหลดให้กับรีจิสเตอร์ไทเมอร์
4. กำหนดค่า TL = xx และ TH = yy

<u>ตัวอย่างที่ 8.9</u> ถ้าหาก 8051 ทำงานกับ XTAL ที่มีความถี่ 11.0592 MHz อยากทราบว่าต้องโหลดค่าใดให้กับไทเมอร์และจะต้องโปรแกรมอย่างไร ถ้าต้องการสร้างพัลส์ขนาด 5 ms ออกมาทางขวา P2.3

<u>วิธีทำ</u>

จาก XTAL ที่กำหนดจะทำให้ตัวไทเมอร์เกิดการนับเมื่อเวลาผ่านไป 1.085 us ดังนั้นถ้าต้องการสร้างพัลส์ขนาด 5 ms จะต้องใช้สัญญาณนาฬิกาจำนวน 5 ms / 1.085 us = 4608 ลูก ค่าที่ใส่ให้รีจิสเตอร์ไทเมอร์จะเท่ากับ 65536 – 4608 = 60928 = EE00H โดยให้ TH เก็บ EE และ TL เดก็บ 00 และ เขียนโปรแกรมได้ดังนี้

```
        CLR   P2.3          ; เคลียร์ P2.3
        MOV   TMOD, #01     ; ไทเมอร์ 0 โหมด 1
MERE:   MOV   TL0, #0       ; ไบต์ต่ำ TL0 = 0
        MOV   TH0, #0EEH    ; ไบต์สูง TH0 = EE
        SETB  P2.3          ; ให้ P2.3 เป็นลอจิก “1”
        SETB  TR0           ; ให้ไทเมอร์ 0 เริ่มทำงาน
AGAIN:  JNB   TF0, AGAIN    ; ตรวจสอบบิตแฟลกไทเมอร์ 0
        CLR   TR0           ; หยุดไทเมอร์
        CLR   TF0           ; เคลียร์บิตแฟลกไทเมอร์ 0
```

<u>ตัวอย่างที่ 8.10</u> ถ้าความถี่ XTAL เท่ากับ 11.0592 MHz จงเขียนโปรแกรมสร้างคลื่นสี่เหลี่ยมความถี่ 2 kHz ออกมาทางขา P1.5

<u>วิธีทำ</u>

ตัวอย่างนี้จะคล้ายกับตัวอย่างที่ผ่านมา แต่จะทำให้บิต P1.5 เป็นค่าลอจิกกลับไปกลับมาโดยมีขั้นตอนดังนี้

1. คาบเวลาของคลื่น T = 1 / f = 1 / 2 kHz = 500 us
2. ให้เป็นลอจิก “High” และ “Low” เป็นเวลา 250 us
3. 250 us / 1.085 us = 230 และ 65536 – 230 = 65306 ซึ่งเท่ากับ FF1AH
4. ใส่ค่า TL = 1A และ TH = FF และเขียนโปรแกรมได้ดังนี้

```
        MOV   TMOD, #10H    ; ไทเมอร์ 1 โหมด 1
AGAIN:  MOV   TL1, #1AH     ; TL1 = 1A ไบต์ต่ำ
        MOV   TH1,#0FFH     ; TH1 = FF ไบต์สูง
        SETB  TR1           ; ไทเมอร์ 1 เริ่มทำงาน
```

```
BACK:  JNB   TF1, BACK      ; การตรวจสอบบิตแฟลกไทเมอร์
       CLR   TR1            ; หยุดไทเมอร์
       CPL   P1.5           ; กลับค่าบิต P1.5
       CLR   TF1            ; เคลียร์บิตแฟลก
       SJMP  AGAIN          ; โหลดค่าไทเมอร์ และทำซ้ำ
```

<u>ตัวอย่างที่ 8.11</u> ถ้าหาก XTAL มีความถี่ 11.0592 MHz จงเขียนโปรแกรมสร้างคลื่นสี่เหลี่ยมความถี่ 50 Hz ออกมาทางขวา P2.3

<u>วิธีทำ</u>

สามารถทำได้ตามขั้นตอนดังนี้

1. คาบเวลาของคลื่นสี่เหลี่ยม T = 1/50 Hz = 20 ms
2. จะต้องเป็นลอจิก “High” และ “Low” เป็นเวลา 10 ms
3. 10 ms/1.085 us = 9216 และ 65536–9216 = 56320หรือ DC00H และโหลดเข้ารีจิสเตอร์ไทเมอร์
4. TL = 00H และ TH = DCH

```
       MOV   TMOD, #10H     ; ไทเมอร์ 1 โหมด 1
AGAIN: MOV   TL1, #00       ; โหลดไบต์ต่ำ TL1 = 00
       MOV   TH1, #0DCH     ; โหลดไบต์สูง TH1 = DC
       SETB  TR1            ; ให้ไทเมอร์ 1 เริ่มทำงาน
BACK:  JNB   TF1, BACK      ; ตรวจสอบบิตแฟลกไทเมอร์
       CLR   TR1            ; หยุดไทเมอร์
       CPL   P2.3           ; กลับค่าลอจิกบิต P2.3
       CLR   TF1            ; เคลียร์บิตแฟลก
       SJMP  AGAIN          ; โหลดค่าเข้าสู่ไทเมอร์และทำซ้ำ
```

ในการใช้ไทเมอร์เป็นตัวจับเวลาจะพบว่าไม่สามารถใช้ไทเมอร์นับสัญญาณนาฬิกามากกว่า 65536 ลูกได้ โดยการจับเวลานานที่สุดทำได้โดยโหลดค่า 0000 เข้ารีจิสเตอร์ไทเมอร์ แต่ถ้าหากต้องการจับเวลาที่นานขึ้นเราสามารถทำได้สองวิธีคือ เปลี่ยนความถี่สัญญาณนาฬิกาให้ต่ำลง อีกวิธีหนึ่งจะใช้ไทเมอร์แบบ 16 บิต และให้ทำลูบซ้ำหลายๆครั้ง ดังตัวอย่างโปรแกรมที่ 8.12

<u>ตัวอย่างที่ 8.12</u> จากโปรแกรมต่อไปนี้จงคำนวณหาค่าเวลาหน่วง โดยไม่ต้องคำนวณเวลาย่อยๆที่เสียไปในคำสั่งลูป

```
       MOV   TMOD, #10H     ; ไทเมอร์ 1 โหมด 1
       MOV   R3 , #200      ; ลูปของการจับเวลา
AGAIN: MOV   TL1, #08       ; โหลดไบต์ต่ำ TL1 = 08
       MOV   TH1, #01       ; โหลดไบต์สูง TH1 = 01
       SETB  TR1            ; เริ่มต้นไทเมอร์ 1
BACK:  JNB   TF1, BACK      ; ตรวจสอบบิต TF
       CLR   TR1            ; หยุดไทเมอร์
       CLR   TF1            ; เคลียร์บิตแฟลกไทเมอร์ 1
       DJNZ  R3, AGAIN      ; ทำซ้ำจน R3 เป็นศูนย์
```

<u>วิธีทำ</u>

จากโปรแกรม จะพบว่าค่าในไทเมอร์เป็น 1085 H หรือ 264 จำนวนสัญญาณของไทเมอร์จะเท่ากับ 65536 – 264 = 65272 ซึ่งจะเสียเวลาไป 65272 x 1.085 us หรือ 70.820 ms และโปรแกรมทำซ้ำ 200 ครั้ง ดังนั้นจะเสียเวลาไป 200 x 70.820 ms เท่ากับ 14.164024 วินาที

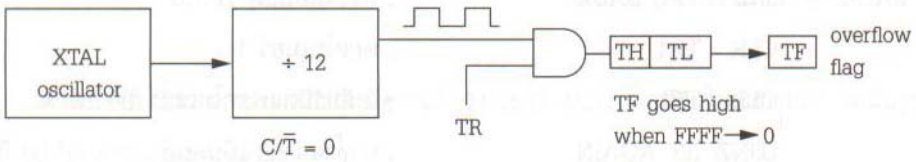
การเขียนโปรแกรมในโหมด 0

การทำงานโหมด 0 จะคล้ายกับโหมด 1 แต่จะเป็นตัวนับตัวจับเวลาแบบ 13 บิต ทำให้ค่าในตัวนับ (TH - TL) อยู่ระหว่าง 0000 ถึง 1FFF และการนับจะทำให้เกดการโอเวอร์โฟลว์โดยทดทางแฟลก TF เมื่อมีการเปลี่ยนแปลงจาก 1FFF เป็น 0000

การเขียนโปรแกรมในโหมด 2

ตัวนับตัวจับเวลาในโหมด 2 เป็นโหมดที่สามารถโหลดค่าการนับเข้าไปในรีจิสเตอร์ได้ (auto – reload) โดยจะมีการดำเนินการดังนี้

1. เป็นไทเมอร์แบบ 8 บิต โดยการนับจะอยู่ในช่วง 00 ถึง FFHโดยโหลดค่าการนับเข้าในรีจิสเตอร์ TH
2. หลังจากโหลดค่า 8 บิต เข้าในรีจิสเตอร์ TH แล้ว ตัว MCS-51 จะก๊อปปี้ข้อมูลเข้าสู่รีจิสเตอร์ TL และจะเริ่มนับเมื่อมีการทำคำสั่ง “ SETB TRO”สำหรับไทเมอร์ 0 และ “ SETB TR1 ” สำหรับไทเมอร์ 1 โดยมีการทำงานคล้ายกับโหมด 1
3. หลังจากไทเมอร์เริ่มทำงานจะมีการนับค่าในรีจิสเตอร์ TL ขึ้นไปจนถึง FFH เมื่อมีการเปลี่ยน FFH ไปเป็น 00H จะมีการเซตบิตแฟลก TF ให้เป็น “1”
4. เมื่อ TL มีค่าเปลี่ยนจาก FFH เป็น 00H และค่าแฟลก TF เป็น “ 1 ” รีจิสเตอร์ TL จะโหลดค่าจากรีจิสเตอร์ TH ลงไปใหม่เพื่อให้สามารถใช้ไทเมอร์อีกครั้งได้ แต่ถ้าเป็นโหมด 1 จะต้องเขียนคำสั่งโหลดข้อมูลเข้าไทเมอร์อีกครั้งถ้าจะใช้งานไทเมอร์ต่อไป



รูป 8.3 การทำงานของไทเมอร์ในโหมด 2

ขั้นตอนการเขียนโปรแกรมในโหมด 2

การใช้งานไทเมอร์โหมด 2 มีขั้นตอนการเขียนคำสั่งดังนี้

1. โหลดค่ารูปแบบการทำงานเข้าสู่รีจิสเตอร์ TMOD โดยโปรแกรมให้เป็นโหมด 2
2. โหลดค่าเริ่มต้นของการนับเข้าสู่ริสเตอร์ TH
3. ให้ไทเมอร์เริ่มทำงาน
4. ตรวจสอบบิตแฟลก (TF) ด้วยคำสั่ง “JNB TFx,target” และให้ออกนอกลูปเมื่อบิตแฟลก TF เป็นลอจิก “1”
5. เคลียร์บิตแฟลก TF
6. กลับเข้าสู่ขั้นตอนที่ 4

ตัวอย่างที่ 8.13 จากตัวอย่างโปรแกรมถ้าหาก MCS – 51 ใช้ XTAL ที่มีความถี่ 11.0592 MHz จงหาค่า

ก. ความถี่ของคลื่นรูปสี่เหลี่ยมที่ออกมาทางขา P1.0

ข. คำนวนหาค่าความถี่ต่ำสุดที่สามารถทำได้โดยการเปลี่ยนค่าใน TH

```
        MOV   TMOD, #20H      ; ไทเมอร์ 1 โหมด 2 8-bit auto-reload
        MOV   TH1, #5         ; TH1 = 5
        SETB  TR1             ; เริ่มต้นให้ไทเมอร์ 1 ทำงาน
BACK:   JNB   TF1, BACK       ; ตรวจสอบบิตแฟลก TF
        CPL   P1.0            ; กลับค่าบิต P1.0
        CLR   TF1             ; เคลียร์บิตแฟลก TF
        SJMP  BACK            ; ทำงานโหมด 2 ในแบบ auto-reload
```

วิธีทำ

ก. จะเห็นว่าโปรแกรมการทำงานซ้ำๆ ด้วยคำสั่ง SJMP และเนื่องจากไทเมอร์ทำงานในโหมด 2 ตัวมันจะโหลดค่ารีจิสเตอร์ TH ให้กลับรีจิสเตอน์ TL เอง ส่วนเวลาที่เสียไปในไทเมอร์จะเท่ากับ (250 – 05) x 1.085 us = 251 x 1.085 us = 272.33 us จากนั้นโปรแกรมจะกลับค่าบิต P1.0 ทำให้ สัญญาณคลื่นสี่เหลี่ยมที่ออกมามีค่า duty cycle เท่ากับ 50% คาบเวลาที่ได้จะเท่ากับ T= 2 x 272.33 us = 544.67 และความถี่เท่ากับ 1.83597 kHz

ข. ความถี่ต่ำสุดจะเกิดจากการโหลดค่า 00 ให้กับรีจิสเตอร์ TH ดังนั้น คาบเวลาจะเท่ากับ T = 2 x 256 x 1.085 us = 555.52 us และความถี่เท่ากับ 1.8 kHz

ตัวอย่างที่ 8.14 จงหาค่าความถี่ของคลื่นสี่เหลี่ยมที่เกิดจากขา P1.0 ถ้าทำโปรแกรมดังนี้

```
        MOV   TMOD ,#2H       ; ไทเมอร์ 0 ทำงานโหมด 2
        MOV   TH0 ,#0         ; TH0 =0
AGAIN:  MOV   R5, #250        ; จำนวนการทำลูป
        ACALL DELAY
        CPL   P1.0
        SJMP  AGAIN
DELAY:  SETB  TR0             ; ให้ไทเมอร์ 0 เริ่มทำงาน
BACK:   JNB   TF0,BACK        ; ตรวจสอบบิตแฟลก
        CLR   TR0             ; หยุดไทเมอร์ 0
        CLR   TF0             ; เคลียร์บิตแฟลก
        DJNZ  R5, DELAY
        RET
```

ดังนั้นคาบเวลาสามารถหาได้จาก T = 2(250 x 256 x 1.085 us) = 138.88 us และความถี่ f =72 Hz

ตัวอย่างที่ 8.15 ถ้าหากโปรแกรมให้ไทเมอร์ทำงานในโหมด 2 ด้วยค่าที่โหลดให้รีจิสแตอร์ TH ดังต่อไปนี้ จงหาค่าเลขฐานสิบหกที่อยู่ในรีจิสเตอร์ TH

ก. MOV TH1,#-200 ข. MOV TH0,#-60 ค. MOV TH1,#-3

ง. MOV TH1,#-12 จ. MOV TH0,#-48

วิธีทำ

จะได้ดังต่อไปนี้

เลขฐานสิบ	เลข 2' s คอมพลีเมนต์ในTH
-200	38H
-60	C4H
-3	FDH
-12	F4H
-48	D0H

ในการเขียนโปรแกรมภาษาแอสเซมบลีนั้นสามารถใส่เลขลบที่เป็นเลขฐานสิบโดยให้โปรแกรมแอสเซมเบลอร์คำนวณเป็นเลขฐานสิบหกออกมาได้ ตัวอย่างเช่น "MOV TH 1, #-100" ตัวแอสเซมเบลอร์จะคำนวณค่า -100 เป็น 9C และให้ TH1 เท่ากับ 9C

ตัวอย่างที่ 8.16 จงหาค่าความถี่จากโปรแกรมต่อไปนี้พร้อมทั้งคำนวณหาค่า duty cycle

```
        MOV     TMOD , #2H      ; ไทเมอร์ 0 โหมด 2
        MOV     TH0,# -150      ; TH0 = 6AH
AGAIN:  SETB    P1.3            ; P1.3=1
        ACALL   DELAY
        ACALL   DELAY
        CLR     P1.3            ; P1.3=0
        ACALL   DELAY
        SJMP    AGAIN
DELAY:  SETB    TR0             ; ให้ไทเมอร์ 0 เริ่มทำงาน
BACK:   JNB     TF0,BACK        ; ตรวจสอบบิตแฟลก
        CLR     TR0             ; หยุดไทเมอร์ 0
        CLR     TF0             ; เคลียร์บิตแฟลก
        RET
```

วิธีทำ

จากโปรแกรมจะเป็นการโปรแกรมให้ TH0 เกิดการโอเวอร์โฟลว์เมื่อมีสัญญาณนาฬิกาเข้าไป150ลูก ดังนั้น โปรแกรมย่อย DELAY จะหน่วงเวลาไปเท่ากับ 150 x 1.085 us =162 us และให้P1.3 เป็นลอจิก "1" นานกว่าลอจิก "0" หนึ่งเท่า เนื่องจากหลังการเซตค่าบิต P1.3 จะเรียกโปรแกรมย่อย DELAY สองครั้ง ดังนั้น ค่า duty cycle จะเท่ากับ 66 % และเวลาที่เป็นลอจิก "1" รวมกับเวลาที่เป็นลอจิก "0" จะเท่ากับ 325.5 us + 162.25 us = 488.25 us และความถี่เท่ากับ 2.048 kHz

คำถาม ?

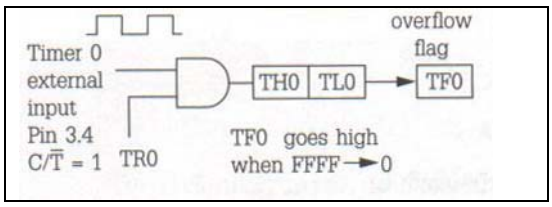
1. ไทเมอร์ของ 8051 ใช้ทำอะไร และมีอยู่กี่ตัว
2. รีจิสเตอร์ TMOD มีชื่อเต็มว่าอะไร มีขนาดกี่บิต
3. รีจิสเตอร์ TMOD สามารถอ้างแอดเดรสระดับบิตได้หรือไม่
4. คำสั่ง MOV TMOD ,#20H จะเป็นการทำงานลักษณะใด
5. ถ้าไทเมอร์ทำงานในโหมด 1 จะมีการนับจากช่วงใดถึงช่วงใด
6. ถ้าไทเมอร์ทำงานในโหมด 2 จะมีการนับจากช่วงใดถึงช่วงใด
7. ถ้าเขียนคำสั่งเป็น MOV TH1,#-200 ค่าในรีจิสเตอร์ TH จะเป็นเท่าใด
8. ถ้าต้องการหน่วงเวลา 2 us และให้ไทเมอร์ทำงานในโหมด 1 จะต้องโหลดค่าอะไรในรีจิสเตอร์ TH และ TL ถ้า XTAL มีความถี่ 11.0592 MHz
9. ถ้าต้องการหน่วงเวลา 100 us จะต้องโหลดค่าอะไรให้กับรีจิสเตอร์ TH ถ้าให้ไทเมอร์ทำงานในโหมด 2 ถ้าหาก XTAL มีความถี่ 11.0592 MHz

8.2 การโปรแกรมเป็นตัวนับ

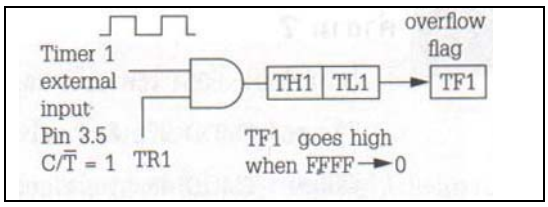
จากที่ผ่านมาเราได้ใช้ไทเมอร์เคาเตอร์ของ 8051 เป็นตัวจับเวลามาแล้วถ้าหากใช้เป็นตัวนับหรือเคาน์เตอร์ก็สามารถทำได้เช่นกัน โดยโปรแกรมให้นับสัญญาณพัลส์จากภายนอกตัว 8051 เมื่อมีสัญญานพัลส์เข้ามาจะเกิดการนับขึ้นในรีจิสเตอร์ TH และ TL เช่นเดียวกับการนับเมื่อใช้เป็นตัวจับเวลา

บิต C/T ในรีจิสเตอร์ TMOD

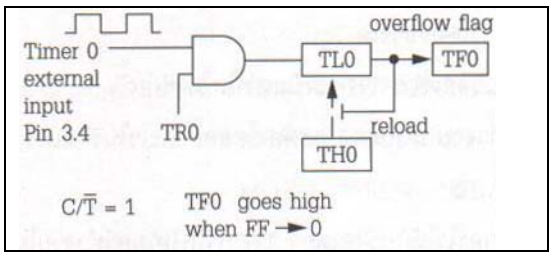
ในรีจิสเตอร์ควบคุม TMOD นั้นถ้าหากบิต C/T ถูกโปรแกรมให้เท่ากับ 0 จะเป็นการนับสัญญาณจาก crystal ถ้าเป็น 1 จะเป็นการนับสัญญาณกระตุ้นจากภายนอกซึ่งสามารถใช้เป็นตัวนับได้ ถ้าหากสัญญาณเข้ามาทางขา T0 (timer 0 input) หรือขาP3.4 จะเป็นการนับของตัวไทมอร์ 0 ถ้าหากสัญญาณเข้ามาทางขา T1 (timer 1 input) หรือขา P3.5 จะเป็นการนับของตัวไทเมอร์1 และการเริ่มนับจะควบคุมที่บิต TR



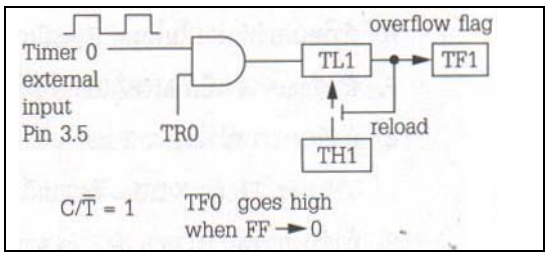
(ก) Timer 0 w/ External Input (Mode 1)



(ข) Timer 1 with External Input (Mode 1)



(ค) Timer 0 with External Input (Mode 2)



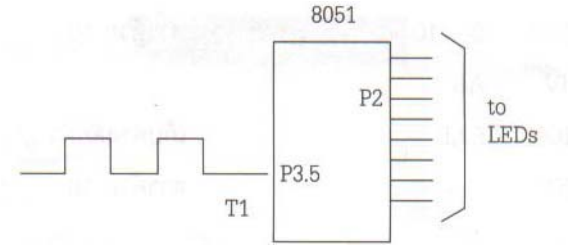
(จ) Timer 1 with External Input (Mode 2)

รูปที่ 8.4 การใช้ Timer เป็นตัวนับ

<u>ตัวอย่างที่ 8.17</u> ถ้าหากมีสัญญาณเข้ามาทางขา T1 จงเขียนโปรแกรมให้เคาน์เตอร์ 1 ทำงานในโหมด 2 นับสัญญาณเข้ามา และแสดงผลค่าของ TL1 บนพอร์ต P2

<u>วิธีทำ</u>

```
        MOV   TMOD,#01100000B     ; เคาน์เตอร์ 1 โหมด 2 C/T =1
        MOV   TH1,#0              ; นับพัลส์จากภายนอก
        SETB  P3.5                ; โปรแกรมให้ T1 เป็นอินพุต
AGAIN:  SETB  TR1                 ; เริ่มนับ
BACK:   MOV   A,TL1               ; อ่านค่าตัวนับ TL1
        MOV   P2,A                ; แสดงผลทางพอร์ต P2
        JNB   TF1,BACK            ; ตรวจสอบบิต และกระโดดถ้า TF=0
        CLR   TR1                 ; หยุดนับ
        CLR   TF1                 ; เคลียร์บิตแฟลก
        SJMP  AGAIN               ; นับอีกครั้ง
```



จากโปรแกรมเห็นว่ามีการใช้คำสั่ง SETB P3.5 เพราะว่าในการจ่ายไฟครั้งแรกพอร์ตของ 8051 จะเป็นพอร์ตของ 8051 จะเป็นพอร์ตเอาต์พุต การใช้งาน P3.5 ให้เป็นพอร์ตอินพุตเพื่อรับสัญญาณจากภายนอกจะต้องส่งลอจิก”1” ออกไปก่อน

จากตัวอย่างที่8.17 จะเป็นการใช้ไทเมอร์ 1 ทำงานเป็นตัวนับ เมื่อสัญญาณเข้ามาทางขา P3.5 ค่าที่ได้ในรีจิสเตอร์ TL1 จะเป็นจำนวนพัลส์จากภายนอกที่เข้ามาซึ่งเราสามารถนำค่านี้ไปแสดงผลทางอุปกรณ์แสดงผลได้ ในตัวอย่างที่8.18 จะเปลี่ยนข้อมูลในรีจิสเตอร์ TL ให้เป็นรหัส ACSII และแสดงผลทาง LCD

<u>ตัวอย่างที่8.18</u> ถ้าหากมีความถี่ 1Hz เข้ามาทางอินพุตขา P3.4 จงใช้ตัวนับจาก timer 0 ทำการนับและแสดงผลทาง LCD โดยกำหนดค่าเริ่มต้นให้ TH0 เป็น -60

<u>วิธีทำ</u>

การแสดงผลการนับใน TL บน LCD นั้นทำได้โดยเปลี่ยนข้อมูล 8 บิต ให้เป็นรหัส ASCII เสียก่อน แล้วส่งให้โปรแกรมแสดงผลทาง LCD

```
        ACALL  LCD_SET_UP          ; กำหนดค่าเริ่มต้นให้ LCD
        MOV    TMOD,#00000110B     ; เคาน์เตอร์ 0 โหมด 2, C/T=1
        MOV    TH0,#-60            ; ให้นับพัลส์ 60 ลูก
        SETB   P3.4                ; ให้ T0 เป็นอินพุต
AGAIN:  SETB   TR0                 ; ให้เคาน์เตอร์เริ่มทำงาน
BACK:   MOV    A,TL0               ; อ่านค่าจาก TL0ให้กับ A
        ACALL  CONV                ; เปลี่ยนเป็นรหัส BCD เก็บใน R2,R3,R4
        ACALL  DISPLAY             ; แสดงผลทาง LCD
        JNB    TF0,BACK            ; ทำซ้ำถ้า TF0=0
        CLR    TR0                 ; ให้ TF0=0
        CLR    TF0
        SJMP   AGAIN               ; ทำซ้ำ
; แปลงข้อมูล 8 บิต เป็นรหัส ACSII
; คืนค่ารหัส ASCII ใน R4,R3,R2(R2เป็นหลักต่ำ)
CONV:   MOV    B,#10               ; หารด้วย10
        DIV    AB
        MOV    R2,B                ; เก็บค่าหลักต่ำ
        MOV    B,#10               ; หารด้วย 10
        DIV    AB
        ORL    A,#30H              ; ทำเป็นรหัส ASCII
        MOV    R4,A                ; เก็บหลักสูงสุด
        MOV    A,B
        ORL    A,#30H              ; ทำเป็นรหัส ASCII หลักที่สอง
        MOV    R3,A                ; เก็บค่า
        MOV    A,R2
        ORL    A,#30H              ; ทำเป็นรหัส ASCII หลักต่ำสุด
        MOV    R2,A                ; เก็บค่ารหัส ASCII
        RET
```

จากตัวอย่างที่ผ่านมาเราสามารถนำตัวนับมาประยุกต์ใช้งานเป็นนาฬิกาได้ ถ้าป้อนความถี่ 1 Hz เข้าไปจะทำให้เกิดการนับทุกๆ วินาที เราสามารถเขียนโปรแกรมให้แสดงผลเวลาเป็นชั่วโมง นาที และวินาทีได้

คำถามท้ายบท

1. ใน 8051 มีไทเมอร์กี่ตัว
2. ไทเมอร์ใน 8051 เป็นแบบกี่บิต
3. รีจิสเตอร์ TMOD เป็นรีจิสเตอร์แบบกี่บิต
4. รีจิสเตอร์ TMOD ใช้ทำอะไร
5. รีจิสเตอร์ TMOD สามารถเข้าถึงข้อมูลระดับบิตได้หรือไม่
6. จงหาค่าความถี่และคาบเวลาในการใช้งานเป็นไทเมอร์ เมื่อใช้ XTAL ที่มีความถี่ดังนี้

   ก. 11.0592 MHz ข. 20 MHz

   ค. 29 MHz ง. 30 MHz
7. การใช้งานไทเมอร์โหมดใดจับเวลาได้นานที่สุด
8. ถ้าหาก XTAL มีความถี่ 11.0592 MHz บิต TF0 จะเกิดการเซตเมื่อใดถ้าทำโปรแกรมต่อไปนี้

```
MOV     TMOD,#01
MOV     TL0,#12H
MOV     TH0,#1CH
SETB    TR0
```

9. ถ้าหาก XTAL มีความถี่ 16 MHz บิต TF0 จะเซตเมื่อใดถ้าโปรแกรมทำคำสั่งต่อไปนี้

```
MOV     TMOD,#01
MOV     TL0,#12H
MOV     TH0,#1CH
SETB    TR0
```

10. ถ้าหาก XTAL มีความถี่ 11.0592 MHz บิต TF0 จะเซตเมื่อใดถ้าโปรแกรมทำคำสั่งต่อไปนี้

```
MOV     TMOD,#01
MOV     TL0,#10H
MOV     TH0,#0F2H
SETB    TR0
```

11. ถ้าหาก XTAL มีความถี่ 11.0592 MHz จงหาค่า TH1 และ TL1 ถ้าต้องการใช้งานไทเมอร์ 1 โหมด 1 จับเวลา 2 ms

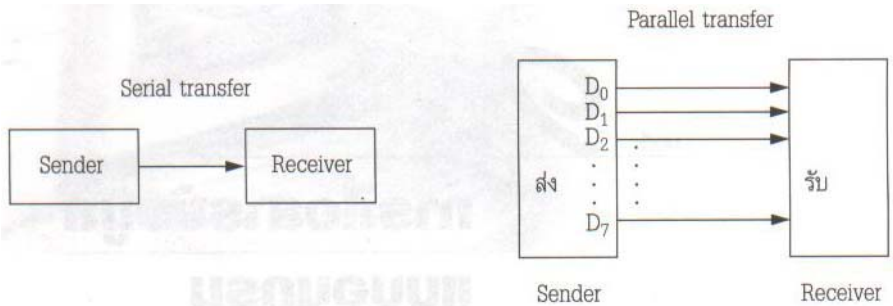
บทที่ 9
การสื่อสารข้อมูลแบบอนุกรม

ระบบคอมพิวเตอร์สามารถส่งข้อมูลได้สองรูปแบบคือแบบขนาน (Parallel) ซึ่งจะส่งข้อมูลทุกบิตออกปพร้อมกัน และแบบอนุกรม (serial) ซึ่งจะส่งข้อมูลออกไปครั้งละบิต การส่งข้อมูลแบบขนานนี้จะต้องใช้สายในการส่งข้อมูลจำนวนมากซึ่งไม่เหมาะกับการส่งระยะไกล โดยมากแล้วจะใช้ในการส่งระยะใกล้ เช่น ระหว่างไมโครโพรเซสเซอร์กับฮาร์ดดิสก์ เครื่องพิมพ์ เป็นต้น ส่วนการส่งข้อมูลระยะไกลๆ ควรใช้การส่งข้อมูลแบบอนุกรมโดยจะส่งข้อมูลออกไปครั้งละบิต การส่งข้อมูลแบบอนุกรมนั้นจะส่งได้ช้ากว่าการส่งแบบขนานแต่ค่าใช้จ่ายจะต่ำกว่า

ในไมโครโพรเซสเซอร์ 8051 จะมีพอร์ตอนุกรมอยู่ภายในซึ่งจะรับส่งข้อมูลได้สองทิศทางในพอร์ตเดียวกันสำหรับในบทนี้จะกล่าวถึงการสื่อสารข้อมูลของ 8051 โดยหัวข้อ 9.1 จะกล่าวถึงการสื่อสารทั่วไป ในหัวข้อ 9.2 จะกล่าวถึงการสื่อสารแบบ RS-232 โดยจะนำไอซี MAX232 มาร่วมใช้งานด้วย

9.1 พื้นฐานการสื่อสารอนุกรม

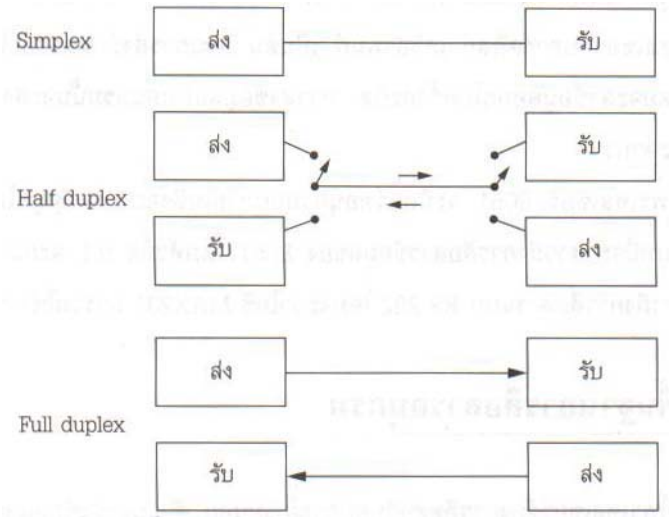
เมื่อไมโครโพรเซสเซอร์ต้องการติดต่อกับอุปกรณ์ภายนอก ตัวมันจะส่งข้อมูลออกมามีขนาดเป็นไบต์หรือ 8 บิต แต่เนื่องจากไมโครโพรเซสเซอร์มีบัสข้อมูลขนาด 8 บิต การโอนถ่ายข้อมูลต่างๆ จะทำแบบขนาน ถ้าต้องการส่งข้อมูลออกไปแบบอนุกรมจะต้องเปลี่ยนข้อมูลแบบขนานนี้ให้เป็นข้อมูลอนุกรมเสียก่อนแล้วจึงส่งอกไป ส่วนการรับข้อมูลแบบอนุกรมนี้จะรับข้อมูลเข้ามาครั้งละบิตและเปลี่ยนข้อมูลให้เป็นข้อมูลแบบขนานเพื่อส่งให้ไมโครโพรเซสเซอร์ประมวลผลต่อไป ในระบบคอมพิวเตอร์ตัวที่เปลี่ยนข้อมูลอนุกรมเป็นขนานและเปลี่ยนข้อมูลขนานเป็นอนุกรมจะใช้อุปกรณ์ที่มีชื่อว่า UART (Universal asynchronous receiver transmitter)



รูปที่ 9.1 ลักษณะการรับส่งข้อมูลแบบอนุกรมและแบบขนาน

รูปแบบการสื่อสารข้อมูล

การสื่อสารข้อมูลระหว่างตัวรับและตัวส่งนั้นมีหลายวิธี ถ้าหากตัวส่งทำหน้าที่ส่งอย่างเดียวและตัวรับทำหน้าที่รับอย่างเดียวจะเรียกว่าการสื่อสารแบบซิมเพล็กซ์ (simplex) เช่น การส่งข้อมูลระหว่างคอมพิวเตอร์กับเครื่องพิมพ์ แต่ถ้าหากตัวรับและตัวส่งสามารถรับและส่งข้อมูลได้แต่ทำในเวลาต่างกันเรียกว่าการสื่อสารแบบ ฮาล์ฟดูเพล็กซ์ (half duplex) เช่นการสื่อสารแบบเครื่องอินเตอร์คอมที่มีสายส่งสัญญาณได้ทางเดียวกัน แต่ถ้าหากตัวรับและตัวส่งสามารถรับส่งข้อมูลได้สองทิศทางในเวลาเดียวกันจะเรียกว่าเป็นการสื่อสารแบบฟูลล์ดูเพล็กซ์ (full duplex) การสื่อสารแต่ละแบบแสดงได้ดังรูปที่ 9.2



รูปที่ 9.2 รูปแบบการส่งข้อมูลแบบ Simple, Half Duplex และ Full Duplex

สำหรับวิธีในการส่งข้อมูลแบบอนุกรมมีสองวิธีคือ การส่งแบบเข้าจังหวะเวลา (Synchronous) และการส่งแบบไม่เข้าจังหวะเวลา (Asynchronous) โดยการส่งแบบเข้าจังหวะเวลาจะต้องมีการส่งสัญญาณนาฬิการ่วมไปด้วยเพื่อควบคุมการส่งรับข้อมูล แต่การส่งรับแบบไม่เข้าจังหวะเวลาจะไม่ต้องมี แต่จะใช้การกำหนดอัตราเร็วในการรับส่งข้อมูลให้มีค่าเท่ากันที่เรียกว่า อัตราบอด หรือบอดเรต (Baud rate) ซึ่งมีหน่วยเป็นบิตต่อวินาที (Bit pet second: bps)

การส่งข้อมูลแบบซิงโครนัส

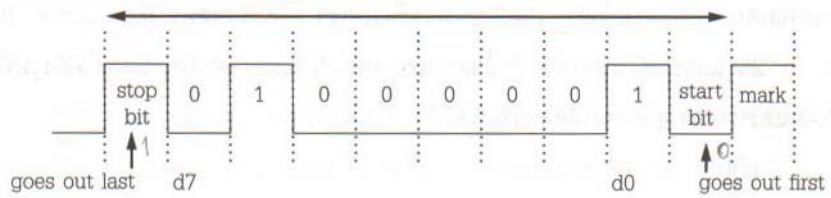
เมื่ออุปกรณ์สองตัวจะสื่อสารข้อมูลซึ่งกันและกันจะต้องกำหนดรูปแบบกฎเกณฑ์การสื่อสารซึ่งกันและกันเพื่อจะให้เข้าใจกันได้ รูปแบบการรับส่งข้อมูลนี้เรียกว่าโพรโตคอล (protocol) ซึ่งจะเป็นตัวบอกลักษณะของข้อมูล การเริ่มข้อมูล การจบข้อมูล เป็นต้น

การรับส่งข้อมูลแบบซิงโครนัสจะประกอบด้วย 4 ส่วนดังนี้

1. บิตเริ่มต้น (start bit)
2. ข้อมูลอนุกรม
3. บิตตรวจสอบความถูกต้อง
4. บิตสุดท้าย (stop bit)

บิตเริ่มต้นและบิตสุดท้าย

การส่งข้อมูลแบบอะซิงโครนัสนั้นจะส่งข้อมูลออกไปเป็นชุดเรียกว่า เฟรม (framing) ภายในเฟรมจะประกอบด้วยข้อมูลหรือรหัส ASCII ก็ได้ ในแต่ละเฟรมจะเริ่มต้นด้วยบิตเริ่มต้น (start bit) ที่จะบอกว่าสิ่งที่ตามมาคือข้อมูล และจะจบข้อมูลด้วยบิตสุดท้าย (stop bit) ที่จะบอกว่าข้อมูลในเฟรมนั้นๆ สิ้นสุดลงแล้วในการส่งข้อมูลแบบนี้ถ้าหากยังไม่มีการส่งข้อมูลระดับลอจิกที่สายส่งจะเป็นลอจิก “1” เรียกว่าสภาวะรอ(waiting stage) ถ้าหากมีการส่งข้อมูล สัญญาณจะเป็นลอจิก “0” ในช่วงเวลาหนึ่ง บิตข้อมูลบิตนี้เรียกว่าบิตเริ่มต้น จากนั้นจะตามด้วยข้อมูล ถ้าหากจะส่งรหัส ASCII ของตัว “A” จะเป็นดังรูปที่ 9.3 โดยส่งค่อข้อมูล 8 บิต (01000001) ออกไปจากนั้นจะจบด้วยบิตปิดท้ายหรือบิตหยุดที่มีภาวะเป็นลอจิก “1” โดยบิตหยุดนี้อาจเป็นลอจิก “1” ในช่วงเวลา 1 บิต, 1.5 บิต หรือ 2 บิตก็ได้ขึ้นกับการกำหนดลักษณะการส่งข้อมูล



รูปที่ 9.3 เฟรมรหัส ASCII ของตัว “A”

สำหรับข้อมูลที่รับส่งกันนั้นอาจมีขนาด 7 บิต หรือ 8 บิตก็ได้ ขึ้นอยู่กับการออกแบบระบบ และข้อมูลสามารถทำการตรวจสอบความถูกต้องได้ระดับหนึ่งเรียกว่า การตรวจสอบบิตพาริตี (parity bit) ซึ่งจะเป็นบิตข้อมูลที่จะเพิ่มเข้าไปก่อนบิตปิดท้าย โดยการตรวจสอบบิตพาริตีนี้สามารถกำหนดเป็นพาริตีคี่ (odd) หรือ พาริตีคู่ (even) ก็ได้ การกำหนดบิตพาริตีคี่หมายความว่าบิตข้อมูลทั้ง 7 หรือ 8 บิต ถ้าหากรวมกับบิตพาริตีแล้วจะต้องเป็นลอจิก “1” คี่บิต ถ้าหากเป็นพาริตีคู่จะต้องรวมแล้วได้ลอจิก “1” คู่บิต ตัวอย่างเช่น ถ้าหากส่งรหัส ASCII ของตัว “A” ออกไป ซึ่งข้อมูลจะเป็น 0100 0001 ถ้ากำหนดเป็นพาริตีคู่ บิตพารีตีต้องมีลอจิกเป็น “0” แต่ถ้ากำหนดเป็นพาริตีคี่บิตพาริตีต้องเป็นลอจิก “1” การสร้างบิตพาริตีนี้จะถูกสร้างจากภาคส่งของ UART ซึ่งเราสามารถโปรแกรมได้ว่าการส่งนั้นจะเป็นแบบพาริตีคี่หรือคู่ หรือไม่มีพาริตี

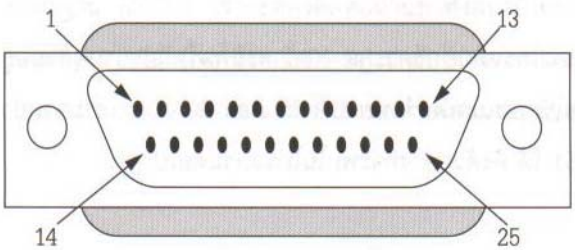
อัตราการรับส่งข้อมูล

ความเร็วในการรับส่งข้อมูลแบบอนุกรมจะบอกเป็นจำนวนบิตต่อวินาที (Bits per second: bps) ที่เรียกว่าบอดเรต (baud rates) พอร์ตอนุกรมของ MCS-51 สามารถติดต่อกับเครื่องคอมพิวเตอร์ส่วนบุคคลได้ แต่ต้องกำหนดความเร็วเครื่องได้เท่ากัน ในการสื่อสารทางพอร์ตอนุกรม RS-232 ของคอมพิวเตอร์ PC ได้กำหนดอัตราเร็วไว้หลายค่าตั้งแต่ 100 ถึง 9600 bps สำหรับคอมพิวเตอร์ที่ใช้ไมโครโพรเซสเซอร์เพนเทียมสามารถส่งข้อมูลได้ความเร็วสูงถึง 56 กิโลบิตติอวินาที

มาตรฐาน RS 232

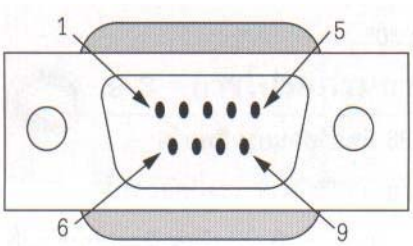
การสื่อสารแบบอนุกรมกับคอมพิวเตอร์ PC มักจะใช้รูปแบบมาตรฐาน RS-232 ซึ่งกำหนดโดย Electronics Industries Association หรือ EIA ในปี 1960 และได้มีการแก้ไขมาตรฐานในปี 1963, 1965 และ 1969 เรียกว่า RS-232A, RS-232B และ RS-232C ตามลำดับ ระดับแรงดันของลอจิกที่ใช้ในการสื่อสาร RS-232 นั้นลอจิก “1” จะแทนด้วยแรงดัน -3 ถึง -25 โวล์ต ส่วนลอจิก “0” จะแทนด้วยแรงดัน +3 ถึง +25 โวล์ต แรงดันในช่วง +3 จะไม่ถูกกำหนดให้ใช้งานซึ่งจะเห็นว่าแรงดันดังกล่าวไม่สามารถใช้กับไมโครคอนโทรลเลอร์ตระกูล MSC-51 ได้ โดยทั่วไปแล้วถ้าหากต้องการให้ไมโครคอนโทรลเลอร์ติดต่อกับคอมพิวเตอร์ PC ตามมาตรฐาน RS-232 จะต้องออกแบบวงจรอิเล็กทรอนิกส์เพิ่มเติม แต่ในปัจจุบันจะใช้ไอซี MAX232 ทำหน้าที่เปลี่ยนระดับแรงดันทางลอจิกให้อยู่ในมาตรฐาน RS-232 แต่ถ้าหากจะใช้ไมโครคอนโทรลเลอร์สองตัวสื่อสารกันจะไม่ใช้มาตรฐานนี้ก็ได้

ในคอมพิวเตอร์ PC จะมีขั้วต่อ RS-232 หรือที่เรียกว่า คอนเนกเตอร์ (connector) อยู่สองแบบคือ ขั้วต่อแบบ DB-25 และขั้วต่อแบบ DP-9 ขั้วต่อแบบ DB-25 และขาต่างๆ แสดงได้ดังรูปที่ 9.4 ส่วนขั้วต่อแบบ DB-9 และขาต่างๆ แสดงได้ดังรูปที่ 9.5



Pin	Description
1	Protective ground
2	Transmitted data (TxD)
3	Received data (RxD)
4	Request to send (RTS)
5	Clear to send (CTS)
6	Data set ready (DSR)
7	Signal ground (GND)
8	Data carrier detect (DCD)
9/10	Reserve for data testing
11	Unassigned
12	Secondary data carrier detect
13	Secondary clear to send
14	Secondary transmitted data
15	Transmit signal element timing
16	Secondary received data
17	Receive signal element timing
18	Unassigned
19	Secondary request to send
20	Data terminal ready (DTR)
21	Signal quality detector
22	Ring indicator
23	Data signal rate select
24	Transmit signal element timing
25	Unassigned

รูปที่ 9.4 ขั้วต่อแบบ DB-25 และหน้าที่ของขาต่างๆ



Pin	Description
1	Data carrier detect (DCD)
2	Received data (RxD)
3	Transmitted data (TxD)
4	Data terminal ready (DTR)
5	Signal ground (GND)
6	Data set ready (DSR)
7	Request to send (RTS)
8	Clear to send (CTS)
9	Ring indicator (RI)

รูปที่ 9.5 ขั้วต่อแบบ DB-9 และหน้าที่ของขาต่างๆ

ในระบบ RS-232 มีขาต่างๆ ที่สำคัญดังนี้

1. DTR (data terminal ready) เป็นสายสำหรับการทำแฮนด์เชคจาก DTE ไปยัง DCE
2. DSR (data set ready) เป็นสายสำหรับการทำแฮนด์เชคจาก DCE ไปยัง DTE
3. RST (request to send) เป็นอุปกรณ์ DTE ต้องการส่งข้อมูลมันจะส่งสัญญาณที่ขาเป็นนี้เป็นลอจิก “Low” ออกไป
4. CTS (clear to send) เป็นสายสัญญาณสำหรับการทำแฮนด์เชคจาก DCE ไปยัง DTE
5. DCD (carrier detect, หรือ data carrier detect)
6. RI (ring indicator) เป็นสายที่ใช้โดยโมเด็มเพื่อบอกว่าตัวมันเองได้รับสัญญาณเรียกเข้ามา

ในคอมพิวเตอร์ส่วนบุคคล IBM ที่ใช้ไมโครโพรเซสเซอร์ตระกูล X86 จะมีพอร์ตสื่อสารอนุกรมอยู่สองพอร์ตเรียกว่าพอร์ต COM1 และ COM2 โดยใช้คอนเนกเตอร์แบบ DB-25 และ DB-9 เราสามารถนำพอร์ตทั้งสองนี้มาเชื่อมต่อกับพอร์ตอนุกรมของ 8051 ได้ ดังตัวอย่างที่พบในโปรแกรมต่อไป

คำถาม ?

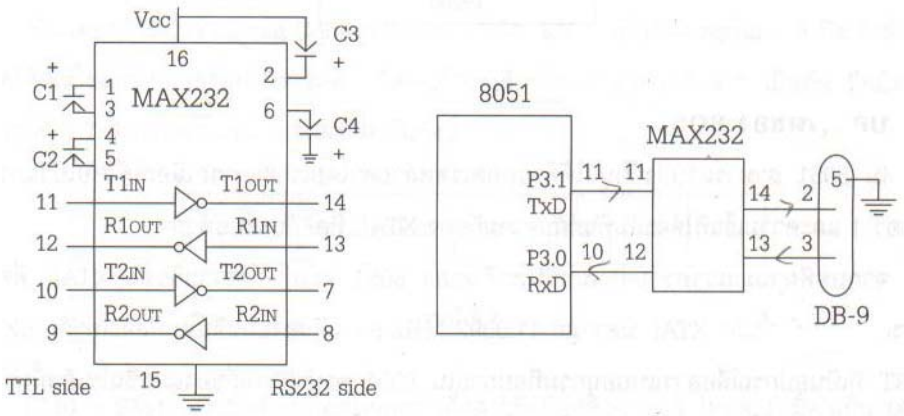
1. จงเปรียบเทียบความเร็วและราคาของการสื่อสารแบบอนุกรมและแบบขนาน
2. จริงหรือไม่ที่มีการส่งข้อมูลระหว่างคอมพิวเตอร์และเครื่องพิมพ์เป็นแบบ duplex
3. จริงหรือไม่การส่งข้อมูลแบบ full duplex จะใช้สายสองชุดคือ ชุดส่งและชุดรับ
4. การใช้บิตเริ่มต้นและบิตหยุดเป็นวิธีการส่งข้อมูลแบบซิงโครนัส หรือแบบอะซิงโครนัส
5. ถ้าหากจะส่งรหัส ASCII ของตัว “E” (01000101) แบบไม่มีพาริตีบิต และมีหนึ่งบิตหยุดจงแสดงลำดับการส่งข้อมูลแบบอนุกรมแต่ละบิต
6. ถ้าหากส่งข้อมูลแบบอนุกรมด้วยความเร็ว 9600 bps จงคำนวณเวลาการส่งข้อมูลจำนวน 10,000 ตัวอักขระ
7. สัญญาณลอจิกระดับ TTL สอดคล้องกับสัญญาณ RS-232 หรือไม่
8. ในระบบ RS-232 จะใช้แรงดันไฟฟ้าเท่าใดแทนลอจิก “0”
9. จริงหรือไม่ใน 8051 มีชิป UART อยู่ภายใน
10. ไมโครคอมพิวเตอร์ที่ใช้ไมโครโพรเซสเซอร์ตระกูล X86 มีพอร์ตอนุกรมกี่พอร์ต

9.2 การต่อ MCS-51 กับ RS-232

ในหัวข้อนี้จะกล่าวถึงการต่อพอร์ตอนุกรมของ MSC-51 กับมาตรฐาน RS232 แต่เนื่องจากระดับสัญญาณของการสื่อสาร RS-232 ไม่เป็นไปตามมาตรฐานแรงดัน TTL เราสามารถนำไปชิป MAX232 มาช่วยปรับระดับแรงดันให้กับ 8051 ได้

ขา RxD และ TxD ของ 8051

พอร์ตที่ใช้ในการรับส่งข้อมูลแบบอนุกรมของ 8051 จะรับส่งออกมาทางขา RxD และ TxD ซึ่งอยู่ในพอร์ต 3 (P3.0 และ P3.1) โดย TxD จะเป็นขาที่ 9 และ RxD จะเป็นขาที่ 10 การรับส่งข้อมูลกับมาตรฐาน RS-232 เราจะต้องติดต่อผ่านขาทั้งสองนี้โดยสามารถนำไอซี MAX232 มาเชื่อมต่อได้ ดังวงจรในรูปที่ 9.6 ไอซีตัวนี้จะใช้ไฟเลี้ยง +5V เท่ากับไฟเลี้ยงของไมโครคอลโทรเลอร์แต่สามารถยกระดับแรงดันตั้งแต่ -25 V ถึง +25 V ได้โดยมาต้องใช้แหล่งจ่ายไปแบบคู่ (dual power supplies)



รูปที่ 9.6 โครงสร้างภายใน MAX232 และการเชื่อมต่อกับ MCS-51

9.3 การโปรแกรมการสื่อสารแบบอนุกรมของ 8051

ในหัวข้อนี้จะกล่าวถึงรีจีสเตอร์ที่ใช้สื่อสารข้อมูลแบบอนุกรมของ 8051 และการเขียนโปรแกรมพื้นฐานในการรับส่งข้อมูล ถ้าหากคอมพิวเตอร์ IBM PC ต้องการติดต่อกับระบบ 8051 จะต้องสื่อสารแบบอนุกรมผ่านทาง COM พอร์ต โดยความเร็วที่ใช้ในการสื่อสารได้แสดงดังตารางที่ 9.1 โดยเราสามารถเขียนโปรแกรมเลือกอัตราเร็วได้ หรือเซตโปรแกรม WINDOWS ตั้งแต่รุ่น 3.1 ขึ้นไป

ตารางที่ 9.1 บอดเรตของคอมพิวเตอร์ PC

110
150
300
600
1200
2400
4800
9600
19200

บอดเรตของ 8051

ใน 8051 สามารถรับส่งข้อมูลได้ด้วยบอดเรตหลายค่าโดยเราสามารถเลือกได้โดยโปรแกรมเข้าไปทางไทเมอร์ 1 และความเร็วที่ได้จะสัมพันธ์กับความถี่ของ XTAL ที่ต่อกับระบบ

จากบทที่ผ่านมาเราทราบว่าแมชชีนไซเคิลของ 8051 จะเท่ากับควมาถี่ของ XTAL ที่ต่ออยู่กับระบบหารด้วย 12 ดังนั้นถ้า XTAL มีความถี่ 11.0592 MHz ความถี่ของแมชชีนไซเคิลจะเท่ากับ 921.0 kHz ตัว UART ที่เป็นอุปกรณ์สื่อสารแบบอนุกรมที่อยู่ภายใน 8051 จะนำความถี่ของแมชชีนไซเคิลนี้มาหารด้วยค่า 32 หรือมากกว่านี้ขึ้นกับค่าที่โปรแกรมให้กับไทเมทอร์ 1 ดังนั้นค่าอัตราการรับส่งข้อมูลหรือบอดเรตสูงสุดของ 8051 จะเท่ากับ 921.6 kHz หารด้วย 32 เท่ากับ 28,800 Hz ถ้าหากเราต้องการค่าอัตราบอดค่าอื่นๆทำได้โดยการโปรแกรมเข้าไปในไทเมอร์ 1 ดังตัวอย่างเช่น ถ้าให้ไทเมอร์ 1 ทำงานในโหมด 2(auto-reload) อัตราบอดเรตและค่าในรีจิสเตอร์จะเป็นไปดังตารางที่ 9.2

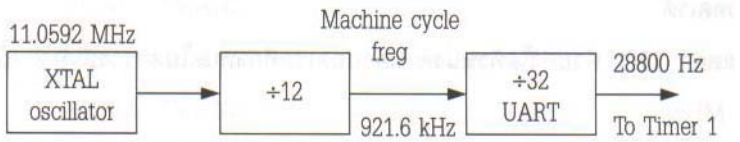
ตารางที่ 9.2 ค่ารีจิสเตอร์ TH1 ที่ใช้กำหนดบอดเรต

บอดเรต	TH1(Decimal)	TH1 (Hex)
9600	-3	FD
4800	-6	FA
2400	-12	F4
1200	-24	E8

รีจิสเตอร์ SBUF

รีจิสเตอร์ตัวนี้เป็นรีจิสเตอร์บัฟเฟอร์อนุกรม (Serial data buffer register) ในการรับส่งข้อมูลจะต้องกระทำผ่านรีจิสเตอร์ตัวนี้ การส่งข้อมูลออกทางขา TxD จะต้องเขียนข้อมูลลงในรีจิสเตอร์ SBUF จากนั้น UART จะทำหน้าที่ส่งข้อมูลออกไปทีละบิตส่วนการรับข้อมูลที่เข้ามาทางขา RxD ข้อมูลจะถูกเก็บอยู่ในรีจิสเตอร์ SBUF นี้ และจะต้องอ่านข้อมูลจากรีจิสเตอร์ SBUF ออกมาใช้งานดังตัวอย่างคำสั่งต่อไปนี้

```
MOV     SBUF, #'D'      : สำหรับ ASCII ของตัว D ไปให้ SBUF
MOV     SBUF,A          : ส่งค่าในรีจิสเตอร์ A ให้กับ SBUF
MOV     A,SBUF          : อ่านค่าจากรีจิสเตอร์ SBUF ให้รีจิสเตอร์ A
```



รีจิสเตอร์ควบคุมพอร์ตอนุกรม

รีจิสเตอร์ตัวนี้ชื่อว่า Serial control Register หรือ SCON เป็นรีจิสเตอร์แบบ 8 บิต ที่เข้าถึงข้อมูลระดับบิตได้ ใช้ในการโปรแกรมค่าบิตเริ่มต้น บิตสุดท้าย เลือกโหมดของการสื่อสาร เป็นต้น ชื่อบิตแต่ละบิตแสดงดังรูปที่ 9.7 สำหรับความหมายของแต่ละบิตเป็นดังนี้

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

รูปที่ 9.7 รีจิสเตอร์ SCON (Serial control Register)

SM0–SM1: เป็นบิตที่ใช้เลือกโหมดการสื่อสารซึ่งมีอยู่ด้วยกัน 4 โหมด ถ้ามีค่าเป็น 00 จะเป็นการสื่อสารในโหมด 0 ซึ่งจะมำงานลักษณะของชิปรีจิสเตอร์ ถ้าเป็น 01 จะเป็นโหมด 1 ทำงานรับส่งข้อมูลแบบ 8 บิต ที่สามารถเลือกอัตราบอดได้ ถ้าเป็น 10 จะทำงานโหมด 2 รับส่งข้อมูลแบบ 9 บิต มีอัตราบอดคงที่ และถ้าเป็น 11 จะทำงานโหมด 3 รับส่งข้อมูลแบบ 9 บิต ที่เลือกอัตราบอดได้

SM2: เป็นบิตที่ใช้อีนาเบิลการสื่อสารแบบมัลติโพรเซสเซอร์

REN: หรือ receive enable เป็นบิตที่ใช้อีนาเบิลการรับข้อมูลของพอร์ตอนุกรม ถ้าบิตนี้ถูกเซตเป็นลอจิก “1” หมายความว่าจะรับข้อมูลเข้ามาได้

TB8: หรือ transfer bit 8 เป็นบิตที่ใช้เก็บข้อมูลที่ 9 ที่จะส่งออกไปทางพอร์ตอนุกรมเมื่อทำงานในโหมด 2 และโหมด 3

RB8: หรือ receive bit 8 เป็นบิตที่ใช้เก็บข้อมูลที่ 9 เมื่อรับข้อมูลเข้ามาทางพอร์ตอนุกรมเมื่อทำงานในโหมด 2 และโหมด 3

TI: หรือ transmit interrupt เมื่อ 8051 ส่งข้อมูลหนึ่งไบต์ออกไปทางพอร์ตอนุกรมเรียบร้อยแล้วบิตนี้จะมีลอจิกเป็น “1” ซึ่งสามารถนำไปอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ได้โดยบิตนี้จะเซตเมื่อมีการส่งบิตหยุดของข้อมูลออกไปและต้องเคลียร์ด้วยซอฟต์แวร์

RI: หรือ receive interrupt เมื่อ 8051 รับข้อมูลเข้ามาทางพอร์ตอนุกรมเรียบร้อยแล้วบิตนี้จะถูกเซตเป็นลอจิก “1” ซึ่งสามารถนำไปอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ได้ โดยบิตนี้จะถูกเซตเมื่อบิตหยุดข้อมูลได้ส่งเข้ามาแล้ว และต้องเคลียร์ด้วยซอฟต์แวร์

การโปรแกรมให้ 8051 ส่งข้อมูล

การเขียนโปรแกรมให้ไมโครคอนโทรลเลอร์ส่งไบต์ข้อมูลออกไปสามารถทำได้ดังนี้

1. โหลดข้อมูลเข้ารีจิสเตอร์ TMOD ด้วยค่า 20H เพื่อให้ทำงานในโหมด 2 (8-bit auto-reload) และกำหนดค่าบอดเรต
2. โหลดค่า TH1 ด้วยค่าใดค่าหนึ่งตามค่าบอดเรตที่กำหนดในตารางที่ 9.2 (ถ้าหาก XTAL มีความถี่ 11.0592 MHz)
3. โหลดข้อมูลเข้ารีจิสเตอร์ SCON ด้วยค่า 50H เพื่อให้ทำงานในโหมด 1 และส่งข้อมูลแบบ 8 บิต มีบิตเริ่มต้นและบิตหยุด
4. ให้ไทเมอร์ 1 ทำงานด้วยการเซตบิต TR1
5. เคลียร์บิต T1 ด้วยคำสั่ง CLR T1
6. นำข้อมูลที่จะส่งเขียนลงรีจิสเตอร์ SBUF
7. วนลูปทดสอบแฟลก T1 ด้วยคำสั่ง JNB T1, XX ถ้าแฟลกนี้เป็น “1” หมายความว่าส่งข้อมูลออกไปแล้ว
8. ส่งข้อมูลไบต์ต่อไปโดยทำซ้ำในขั้นตอนที่ 5

<u>ตัวอย่างที่ 9.1</u> จงเขียนโปรแกรมให้ 8051 ส่งตัวอักขระตัว “A” ออกไปด้วยความเร็ว 4800 บิตต่อวินาที

<u>วิธีทำ</u>

```
        MOV   TMOD, #20H      ; ไทเมอร์ 1 โหมด 2 (auto-reload)
        MOV   TH1, #-6        ; บอดเรต 4800
        MOV   SCON, #50H      ; ส่ง 8 บิต, stop, อีนาเบิล REN
        SETB  TR1             ; ให้ไทเมอร์ 1 เริ่มทำงาน
AGAIN:  MOV   SBUF, # “A”     ; ส่งตัว A ออกไป
HERH:   JNB   T1,HERE         ; รอจนส่งออกไปทุกบิต
        CLR   T1              ; เคลียร์บิต T1 เพื่อส่งตัวต่อไป
        SJMP  AGAIN           ; วนลูป
```

<u>ตัวอย่าง 9.2</u> จงเขียนโปรแกรมให้คำสั่งว่า YES ออกไปทางพอร์ตอนุกรมแบบ 8 บิต บิตหยุด 1 บิตด้วยความเร็ว 9600 บิตต่อวินาที

<u>วิธีทำ</u>

```
        MOV    TMOD, #20H        ; ไทเมอร์ 1 โหมด 2
        MOV    TH1, #-3          ; บอดเรต 9600
        MOV    TR1               ; ส่ง 8 บิต หนึ่งบิตหยุด อีนาเบิล REN
        SETB   TR1               ; ให้ไทเมอร์ 1 เริ่มทำงาน
AGAIN:  MOV    A, # "Y"          ; ส่งตัว Y
        ACALL  TRANS
        MOV    A, # "E"          ; ส่งตัว E
        ACALL  TRANS
        MOV    A, # "S"          ; ส่งตัว S
        ACALL  TRANS
        SJMP   AGAIN             ; ทำซ้ำ
; โปรแกรมย่อยส่งข้อมูลผ่านพอร์ตอนุกรม
TRANS:  MOV    SBUF, A           ; โหลดข้อมูลลงบัฟเฟอร์
HERE:   JNB    TI, HEREE         ; รอจนส่งออกไปทุกบิต
        CLR    TI                ; เคลียร์บิตแฟลกเพื่อส่งไบต์ต่อไป
        RET
```

ความสำคัญของบิตแฟลก TI

การส่งข้อมูลแบบอนุกรมนี้เราจะพบว่าบิตแฟลก TI จะเป็นตัวบอกว่าข้อมูลได้ส่งไปเรียบร้อยแล้วหรือยัง ซึ่งขั้นตอนการส่งข้อมูลจะเป็นดังนี้

1. เขียนไบต์ข้อมูลที่จะส่งในรีจิสเตอร์ SBUF
2. ส่งบิตเริ่มต้นออกไป
3. ข้อมูล 8 บิต ถูกส่งออกไปครั้งละบิต
4. ส่งบิตหยุดตามออกไป เมื่อส่งบิตหยุดไปแล้วแฟลก TI จะเป็นลอจิก "1"
5. การส่งข้อมูลจะต้องเฝ้ามองบิตแฟลก TI นี้ ถ้าบิตนี้เป็นลอจิก "1" แสดงว่าส่งข้อมูลไปแล้ว
6. หลังจากส่งไบต์ต่อไปจะต้องเคลียร์บิตแฟลก TI ด้วยคำสั่ง "CLR TI" ให้เป็นลอจิก "0" ก่อนที่จะเขียนข้อมูลใหม่ลงในรีจิสเตอร์ SBUF

การเขียนโปรแกรมรับข้อมูล

การเขียนโปรแกรมให้ 8051 รับไบต์ข้อมูลสามารถทำได้ดังนี้

1. กำหนดให้รีจิสเตอร์ TMOD มีค่าเป็น 20H เพื่อใช้ไทเมอร์ 1 โหมด 2 (8-bit auto-reload) กำหนดค่าบอดเรต
2. โหลดรีจิสเตอร์ TH1 ด้วยค่าในตารางที่ 9.2 เพื่อกำหนดค่าบอดเรต
3. โหลดรีจิสเตอร์ SCON ด้วยค่า 50H เพื่อใช้พอร์ตอนุกรมโหมด 1
4. เซตบิต TR1 เพื่อให้ไทเมอร์ 1 เริ่มทำงาน
5. เคลียร์บิต R1 ด้วยคำสั่ง CLR RI

6. เฝ้ามองบิตแฟลก R1 ด้วยคำสั่ง JNB RI,XX ถ้าค่าเป็นลอจิก “1” แสดงว่ามีข้อมูลเข้ามาแล้ว
7. เมื่อค่า RI เป็นลอจิก “1” ให้โหลดข้อมูลจากรีจิสเตอร์ SBUF มาใช้งาน
8. กลับไปขั้นตอนที่ 5 เพื่อรับค่าไบต์ต่อไป

<u>ตัวอย่างที่ 9.3</u> จงเขียนโปรแกรมให้ 8051 รับข้อมูลเข้ามาทางพอร์ตอนุกรมและส่งออกไปทางพอร์ต P1 โดยเซตบอดเรตเท่ากับ 4800 รับข้อมูลแบบ 8 บิต และมีบิตหยุดหนึ่งบิต

<u>วิธีทำ</u>

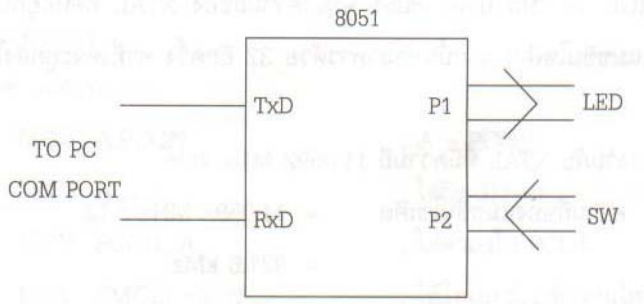
```
        MOV   TMOD, #20H      ; ไทเมอร์ 1 โหมด 2
        MOV   TH1, #-6        ; บอดเรต 4800
        MOV   SCON, #50H      ; รับแบบ 8 บิต
        SETB  TR1             ; ให้ไทเมอร์ 1 เริ่มทำงาน
HERE:   JNB   RI, HERE        ; เฝ้ามองบิต RI
        MOV   A,SBUF          ; เก็บข้อมูลที่รับเข้ามาใน A
        CLR   RI              ; เคลียร์บิตแฟลกเพื่อรับไบต์ต่อไป
        SJMP  HERE
```

<u>ตัวอย่างที่ 9.4</u> ถ้าหากพอร์ตอนุกรมของ 8051 ต่ออยู่กับ COM พอร์ตของ IBM PC และบนเครื่อง IBM ได้รันโปรแกรม terminal.exe สำหรับส่งข้อมูล สำหรับตัว 8051 นั้นพอร์ต P1 ต่ออยู่กับหลอด LED พอร์ต P2 ต่ออยู่กับสวิตซ์ จงเขียนโปรแกรมให้ 8051 ทำงานดังนี้

ก. ส่งข้อมูลคำว่า “We Are Ready” ให้กับคอมพิวเตอร์ PC
ข. รับข้อมูลจาก PC และแสดงผลทาง LED ที่ต่ออยู่ทางพอร์ต P1
ค. อ่านข้อมูลจากสวิตซ์ที่ต่ออยู่ทางพอร์ต P2 และส่งให้คอมพิวเตอร์ PC
โดยการสื่อสารทั้งหมดทำด้วยความเร็ว 4800 บิตต่อวินาที

```
        ORG   0000
        MOV   P2, #0FFH       ; โปรแกรมให้ P2 เป็นอินพุต
        MOV   TMOD, #20H      ; ไทเมอร์ 1 ทำงานโหมด 2
        MOV   TH1, #0FAH      ; บอดเรต 4800
        MOV   SCON, #50H      ; 8 บิต, stop, อีนาเบิล REN
        SETB  TR1             ; ให้ไทเมอร์ เริ่มทำงาน
H_1:    CLR   A
        MOV   A, @A + DPTR    ; อ่านตังอักขระออกมา
        JZ    B_1             ; ถ้าเป็นตัวสุดท้านให้กระโดด
        ACALL SENP            ; เรียกโปรแกรมย่อยส่งข้อมูล
        JNC   DPTR            ; ชี้ตัวต่อไป
        SJMP  H_1             ; ทำซ้ำ
B_1:    MOV   A, P2           ; อ่านค่าจากพอร์ต P2
        ACALL SEND            ; ส่งข้อมูลออกไป
        ACALL RECV            ; เรียกโปรแกรมย่อยรับข้อมูล
        MOV   P1, A           ; แสดงผลทาง LED
        SJMP  B_1             ; ทำซ้ำ
```

```
; โปรแกรมย่อยส่งข้อมูล
SEND:   MOV     SBUF, A             ; เขียนข้อมูลบัฟเฟอร์
H_2:    JNB     T1, H_2             ; เฝ้ามองบิต T1
        CLR     T1                  ; เคลียร์บิตเพื่อส่งไบต์ต่อไป
        RET                         ; จบโปรแกรมย่อย
; โปรแกรมย่อยรับข้อมูล
RECV:   JNB     R1, RECV            ; รอรับข้อมูล
        MOV     A, SBUF             ; อ่านข้อมูลที่รับเข้ามา
        CLR     R1                  ; เคลียร์แฟลกเพื่อรับข้อมูลต่อไป
        RET                         ; จบโปรแกรมย่อย
MYDATA: DB      "We Are Ready", 0
        END
```



ความสำคัญของบิตแฟลก RI

เมื่อ 8051 รับข้อมูลเข้ามาทางขา RxD การทำงานต่างๆ จะเป็นดังนี้

1. รับบิตเริ่มต้นเข้ามาเป็นบิตแรก
2. รับข้อมูลอีก 8 บิต เข้ามาครั้งละบิต เมื่อข้อมูลเข้ามาหมดแล้วจะเก็บอยู่ในรีจีสเตอร์ SBUF
3. เมื่อรับบิตหยุดเข้ามา บิตแฟลก RI จะถูกเซตเป็นลอจิก "1"
4. ตรวจสอบบิตแฟลก RI ถ้าบิตนี้เป็นลอจิก "1" หมานความว่าข้อมูลรับเข้ามาหมดแล้ว
5. หลังจากอ่านข้อมูลจารีจีสเตอร์ SBUF ไปแล้วให้เคลียร์บิตแฟลก RI ด้วยคำสั่ง CLR RI ให้เป็นลอจิก "0" เพื่อรับข้อมูลไบต์ต่อไป

การเปลี่ยนค่าบอดเรตของ 8051

การเปลี่ยนอัตราเร็วการรับส่งข้อมูลของ 8051 ทำได้สองวิธีคือ

1. เปลี่ยนความถี่ของ crystal
2. เปลี่ยนค่าบิตในรีจีสเตอร์ PCON โดยค่าในรีจีสเตอร์เป็นดังรูป

ถ้าหากเปลี่ยนความเร็วโดยการเปลี่ยนค่าความถี่ของ Crystal นั้นบางครั้งความถี่ที่ได้จะทำให้ความเร็วในการสื่อสารไม่สอดคล้องกับความเร็วในคอมพิวเตอร์ IBM PC โดยมากแล้วมักจะเปลี่ยนค่าความเร็วโดยเปลี่ยนค่าในรีจิสเตอร์ PCON (power control) ซึ่งเป็นรีจิสเตอร์แบบ 8 บิต ที่ใช้ควบคุมพลังงานของ 8051 โดยปกติแล้วบิต D7 (SMOD) จะเป็นลอจิก "0"ถ้าหากโปรแกรมให้บิต SMOD นี้เป็นลอจิก "1" อัตราบอดที่ได้จะมีความเร็วเพิ่มขึ้นหนึ่งเท่าตัวเนื่องจากรีจิสเตอร์ PCON เป็นรีจิสเตอร์ที่ไม่สามารถเข้าถึงข้อมูลระดับบิตได้แต่การเซตบิต SMOD ในรีจิสเตอร์ PCON ทำได้ดังนี้

```
MOV     A, PCON          ; อ่านค่า PCON มาเก็บใน A
SETB    ACC.7            ; ทำให้บิต D7 เป็น “1”
MOV     PCON, A          ; นำค่าใน A ใส่ใน PCON
```

อัตราเร็วเมื่อบิต SMOD = 0

เมื่อบิต SMOD มีค่าเป็น 0 ตัว 8051 จะนำความถี่ของ XTAL ที่ต่ออยู่กับระบบมาหารด้วย 12 ซึ่งจะได้ค่าความถี่ของแมชชีนไซเคิล จากนั้นนำมาหารด้วย 32 อีกครั้ง ค่าที่ได้จะถูกส่งให้ไทเมอร์ 1 คำนวณค่าอัตราบอด

ถ้า 8051 ทำงานกับ XTAL ที่มีความถี่ 11.0592 MHz จะได้

ความถี่ของแมชชีนไซเคิล = 11.0592 MHz / 12
= 921.6 kMz

และถ้า SMOD = 0 จะได้

921.6 kMz / 32 = 28,800 Hz

ค่าความถี่ 28,800 นี้จะถูกส่งให้ไทเมอร์ 1 ทำการเซตอัตราบอดโดยค่าที่ส่งให้รีจีสเตอร์ TH1 สำหรับอัตราบอดค่าต่างๆ จะเป็นดังตารางที่ 9.3

อัตราเร็วเมื่อบิต SMOD = 1

ถ้าหากระบบใช้ XTAL ที่มีความถี่ค่าเดิมและบิต SMOD ถูกเซตให้เป็นลอจิก “1” ความเร็วหรืออัตราบอดมี่ได้จะเพิ่มขึ้นเท่าตัว เนื่องจากระบบจะนำค่าความถี่ของแมชชีนไซเคิลมาหารด้วย 16 และนำค่านี้ให้ไทเมอร์ 1 ทำการกำหนดค่าบอดเรต ตัวอย่างเช่น ถ้าระบบใช้ XTAL ที่มีความถี่ 11.0592 MHz จะได้

ความถี่แมชชีนเคิล = 11.0592 MHz /12
= 921.6 kMz

และถ้า SMOD = 1 จะได้

921.6 kMz = 57,600 Hz

ถ้าหากใช้ไทเมอร์ 1 กำหนดค่าบอดเรตจะเป็นไปตามตารางที่ 9.3 ซึ่งจะแสดงค่าที่โหลดให้กับ TH1 สำหรับบอดเรตค่าต่างๆ และเปรียบเทียบระหว่างบิต SMOD เป็น “0” และ “1”

ตารางที่ 9.3 เปรียบเทียบบอดเรตเมื่อ SMOD = 0 และ SMOD = 1

TH1 (Decimal)	Hex	SMOD = 0	SMOD = 1
-3	FD	9,600	19,200
-6	FA	4,800	9,600
-12	F4	2,400	4,800
-24	E8	1,200	2,400

<u>ตัวอย่างที่ 9.5</u> ถ้า 8051 ต่อกับ XTAL มีค่าความถี่ 11.0592 MHz จากโปรแกรมตัวอย่างจงตอบคำถามต่อไปนี้

ก. โปรแกรมทำอะไร

ข. คำนวณค่าบอดเรต

```
        MOV   A, PCON        ; A = PCON
        SETB  ACC.7          ; ให้บิต D7 = 1
        MOV   PCON, A        ; ใส่ค่าลงใน PCON
        MOV   TMOD, #20H     ; ให้ไทเมอร์ 1 ทำงานโหมด 2
        MOV   TH1, -3        ; กำหนดค่าบอดเรตเป็น 19200
                             ; (เกิดจาก 57,600/3 =19200)
        MOV   SCON, #50H     ; ส่ง 8 บิต, บิตหยุด อีนาเบิล R1
        SETB  TR1            ; ให้ไทเมอร์ 1 เริ่มทำงาน
        MOV   A, # 'B'       ; ส่งตัว B ออกไป
A_1:    CLR   TI             ; เคลียร์บิต T1
        MOV   SBUF, A        ; ส่งข้อมูล
H_1:    JNB   TI, H_1
        SJMP  A_1
```

<u>วิธีทำ</u>

ก. โปรแกรมจะส่งรหัส ASCII ของตัว B (01000010) ออกไปอย่างต่อเนื่อง

ข. XTAL มีความถี่ 11.0592 MHz และ SMOD = 1 จะได้

11.0592 / 12	= 921.6 kHz	เป็นความถี่ของแมชชีนไซเคิล
921.6 / 16	= 57600 Hz	ความถี่ที่จะส่งให้ไทเมอร์ 1 กำหนดบอดเรต
57,600 / 3	= 19200	ค่าบอดเรตที่ได้

<u>ตัวอย่างที่ 9.6</u> จงหาค่าในรีจิสเตอร์ TH1 ในรูปแบบของเลขฐานสิบหก ถ้ามีกานกำหนดบอดเรตดังนี้

ก. 9600 ข. 4800 ถ้าหาก SMOD = 1 และ XTAL มีความถี่ 11.0592MHz

<u>วิธีทำ</u>

ถ้า XTAL = 11.0592 และ SMOD = 1 จะได้ความถี่ของไทเมอร์ 1 เท่ากับ 51,600Hz

ก. 57600 / 9600 = 6 ดังนั้น TH1 = -6 หรือ TH1 = FAH

ข. 57600 / 4800 = 12 ดังนั้น TH1 = -12 หรือ TH1 = F4H

คำถาม ?

1. ไทเมอร์ของ 8051 ใช้เซตบอดเรตได้อย่างไร
2. ถ้า XTAL มีความถี่ 11.0592 MHz ความถี่ของไทเมอร์ที่ใช้กำหนดบอดเรตจะเป็นเท่าใด
3. เราใช้ไทเมอร์ 1 ในโหมดใดสำหรับกำหนดค่าบอดเรต
4. บิต T1 จะถูกเซตเมื่อใด
5. เมื่อจ่ายไฟให้ 8051 (Power Up) ค่าบิต SMOD จะเป็นเท่าใด

คำถามท้ายบท

1. การส่งข้อมูลแบบขนานและแบบอนุกรมแบบใดมีราคาแพงกว่ากัน
2. ถ้าหากต้องการส่งอักขระตัว “z” ออกไปแบบอนุกรม ไม่มีพาริตีบิตและมี 1 บิตหยุดข้อมูลแต่ละบิตที่ออกไปจะเป็นอย่างไร
3. ไอซี MAX232 ใช้ทำอะไร เหตุใดต้องใช้
4. รีจิสเตอร์ SBUF คืออะไรมีกี่บิต
5. ถ้ากำหนดบอดเรตด้วยคำสั่ง MOV TH1, #-1 จะได้บอดเรตเท่าใด
6. บิต REN ในรีจิสเตอร์ SCON ใช้ทำอะไร
7. ถ้าหาก XTAL มีความถี่ 11.0592 MHz จงหาค่าใน TH1 ในรูปแบบเลขฐานสิบ และฐานสิบหกในการกำหนดบอดเรตดังงต่อไปนี้

   ก. 9600 ข. 4800 ค. 1200 ง. 300 จ. 150
8. บิตแฟลก T1 และ R1 มีไว้ทำอะไร
9. จงเขียนโปรแกรมให้ 8051 ส่งตัวอักขระ “Z” ออกไปด้วยบอดเรต 9600 bps
10. จงเขียนโปรแกรมให้ 8051 ส่งคำว่า “My name of computer” ออกไปทางพอร์ตอนุกรมด้วยความเร็ว 19200 bps เมื่อระบบใช้ XTAL 18.432MHz

บทที่ 10
การโปรแกรมอินเทอร์รัปต์

การที่อุปกรณ์ภายนอกที่ต่ออยู่ทางพอร์ตอินพุตเอาท์พุตจะติดต่อกับไมโครคอนโทรลเลอร์นั้นจะทำได้สองวิธีคือการทำโพลลิ่ง(Polling) ซึ่งไมโครคอนโทรลเลอร์จะต้องทำการตรวจเช็กพอร์ตต่างๆ อีกวิธีหนึ่งจะให้อุปกรณ์อินพุตเอาต์พุตบอกกับไมโครคอนโทรลเลอร์เองว่าต้องการจะติดต่อด้วย วิธีนี้ที่เรียกว่าการอินเทอร์รัปต์ (interrupt)

การทำงานแบบโพลลิ่งหรือการหยั่งสัญญาณนั้นไมโครโพรเซสเซอร์จะต้องมาคอยตรวจสอบว่าอุปกรณ์ภายภายนอกต้องการติดต่อกับไมโครโพรเซสเซอร์หรือยัง การทำแบบนี้ไมโครโพรเซสเซอร์เองว่าให้ไมโครโพรเซสเซอร์มาทำโปรแกรมสำหรับติดต่อกับตัวมัน การทำงานแบบนี้ไมโครโพรเซสเซอร์ไม่ต้องมาเสียเวลาตรวจสอบอุปกรณ์ภายนอก

10.1การอินเทอร์รัปต์ของ MCS-51

การอินเทอร์รัปต์ (interrupt) อาจจะเรียกอีกอย่างหนึ่งว่าเป็นการขัดจังหวะการทำงานปกติของไมโครโพรเซสเซอร์ ถ้าหากไมโครโพรเซสเซอร์ทำงานปกติอยู่หรือทำงานโปรแกรมหลักอยู่ เมื่อถูกอินเทอร์รัปต์ไมโครโปรเซสเซอร์จะต้องมาทำโปรแกรมตอบสนองการอินเทอร์รัปต์ (interrupt service routine : ISR) เมื่อทำจบแล้วไมโครโปรเซสเซอร์ก็จะมาทำโปรแกรมหลักในคำสั่งต่อจากขณะที่ถูกอินเตอร์รัปต์ต่อไป สำหรับตำแหน่งที่เก็บโปรแกรมตอบสนองการอินเทอร์รัปต์นั้นเรียกว่า อินเทอร์รัปต์เวกเตอร์ (interrupt vector) ซึ่งเป็นตำแหน่งที่มีค่าแอดเดรสแน่นอนในหน่อวยความจำโปรแกรม

ขั้นตอนการอินเทอร์รัปต์

เมื่อไมโครคอนโทรลเลอร์ถูกอินเทอร์รัปต์การทำงานต่างๆ จะเป็นไปต่มขั้รตอนดังนี้

1. ทำคำสั่งที่กำลังทำอยู่ให้จบและเก็บค่าแอดเดรสของคำสั่งถัดไป (ค่าของ PC) ลงหน่วยความจำสแตกเองโดยอัตโนมัติ
2. เก็บค่าสถานะต่างๆ ของไมโครคอนลเลอร์
3. กระโดดไปยังตำแหน่งหน่วยคสามจำที่เป็นค่าอินเทอร์รัปต์เวกเตอร์
4. ทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่อยู่ในอินเทอร์รัปต์เวกเตอร์จนพบคำสั่ง RETI (return from interrupt)
5. เมื่อไมโครคอนโทรลเลอร์พบคำสั่ง RETI จะกลับมาทำโปรแกรม ตำแหน่งต่อจากที่ถูกอินเทอร์รัปต์โดยจะอ่านค่าจากหน่วยความจำสแตกสองไบต์มาให้โปรแกรมเคาน์เตอร์ และทำแอดดรสตำแหน่งนั่นเองโดยอัตโนมัติ

ในการเขียนโปรแกรมตอบสนองการอินเทอร์รัปต์นั้นควรจะเก็บค่าของรีจิสเตอร์และแฟลกต่างๆที่ใช้งานโปรแกรมทางอินเทอร์รัปต์ของหน่วยความจำสแตกด้วยคำสั่ง PUSH และเมื่อจบโปรแกรมตอบสนองการอินเทอร์รัปต์ให้อ่านค่าที่เก็บเอาไว้ด้วยคำสั่ง POP ก่องทำคำสั่ง RETI การทำแบบนี้เพื่อป้องกันการผิดพลาดของรีจสเตอร์ต่างๆที่อาจเกิดขึ้นได้ขณะทำโปรแกรมตอบสนอง การอินเทอร์รัปต์

แหล่งกำเนิดสัญญาณอินเทอร์รัปต์

สัญญาณอินเทอร์รัปต์ของ 8051 จะมาจากสองระบบใหญ่ๆคือ การอินเทอร์รัปต์ด้วยสัญญาณจากภายนอกซึ่งมีอยู่สามแหล่งและการอินเทอร์รัปต์ด้วยสัญญาณจากภายในซึ่งมีอยู่สามแหล่งโดยสัญญาณต่าง ๆ เป็นดังนี้

1. สัญญาณรีเซต (reset) เป็นสัญญาณภายนอเมื่อ MCS-51 ถูกรีเซตจะกระโดดไปทำงานยังแอดเดรส 0000 เข้าสู่ขั้นตอนการทำ power-up

2. สัญญาณอินเทอร์รัปต์จากภายในซึ่งเกิดจากไทเมอร์สองแหล่งคือไทเมอร์0 และไทเมอร์1 โดยจะกระโดดไปทำงานโปรแกรมที่อยู่ในแอดเดรส 000BH และ 001BH ของไทเมอร์0และไทเมอร์1 ตามลำดับโดยสัญญาณที่อินเทอร์รัปต์คือบิตแฟลกของไทเมอร์ (TF0 และ TF1)

3. สัญญาณอินเทอร์รัปต์จากภายนอกสองแหล่งที่เกิดจากสัญญาณที่เข้ามาทางขา 12 (P3.2) และสัญญาณที่เข้ามาขา 13 (P3.3) ของพอร์ต 3 สัญญาณนี้มีชื่อทางฮาร์ดแวร์ว่า INT0 และ INT1 ตามลำดับ เมื่อ 8051 ถูกอินเทอร์รัปต์นี้จะกระโดดไปทำโปรแกรมที่อยู่ในแอดเดรส 0003H และ 0013H สำหรับ INT0 และ INT1 ตามลำดับ

4. สัญญาณอินเทอร์รัปต์จากพอร์ตอนุกรม ถ้าหากมีการรับข้อมูลแบบอนุกรมเข้ามา (RI) หรือหลังจากที่จะส่งข้อมูลแบบอนุกรมออกไป (TI) จะมีสัญญาณอินเทอร์รัปต์ 8051 และจะกระโดดไปทำงานโปรแกรมที่อยู่ตำแหน่ง 0023H

ในตารางที่ 10.1 จะแสดงตำแหน่งของอินเทอร์รัปต์เวกเตอร์ของการอินเตอร์รัปต์จากสัญญาณต่างๆ จะเห็นว่าแอดเดรสของอแนเตอร์รัปต์เวกเตอร์แต่ละตัวจะห่างกัน 8 ไบต์ ดังนั้นถ้าเขียนโปรแกรมตอบสนองการอินเทอร์รัปต์ของสัญญาณบางสัญญาณอาจทำให้โปรแกรมไปทับตำแหน่งอินเทอร์รัปต์ของไทเมอร์ 0 ไปวางไว้ที่แอดเดรส 000BH ถ้าหากโปรแกรมนี้ยาวเกิน 8 ไบต์ก็จะไปทับตำแหน่งอินเทอร์รัปต์เวกเตอร์ INT1 (ตำแหน่ง 0013)

ตารางที่ 10.1 อินเทอร์รัปต์เวกเตอร์ของ MCS-51

อินเตอร์รัปต์	ตำแหน่งโปรแกรมใน ROM	ขา
รีเซต	0000	9
อินเทอร์รัปต์ภายนอก INT0	0003	P3.2(12)
อินเทอร์รัปต์ไทเมอร์ TF0	000B	
อินเทอร์รัปต์ภายนอก INT1	0013	P3.3(13
อินเทอร์รัปต์ไทเมอร์ TF1	001B	
อินเทอร์รัปต์พอร์ตอนุกรม RI และ TI	0023	

เมื่อ MCS-51 เริ่มทำงานจะต้องทำงานโปรแกรมหลักที่อยู่ในแอดเดรส 0000H และจะเห็นว่าโปรแกรมหลักถ้ามีความยาวเกิน 3 ไบต์ก็จะทับตารางอินเทอร์รัปต์เวกเตอร์แล้ว ดังนั้นในการเขียนดปรแกรมจะต้องใช้คำสั่ง LJMP ในตำแหน่ง 0000 เพื่อให้โปรแกรมหลักไปอยู่ในแอดเดรสต่อจากตารางอินเทอร์รัปต์เวกเตอร์และถ้าโปรแกรมตอบสนองอินเทอร์รัปต์มีความยาวเกิน 8 ไบต์ก็จะใช้คำสั่งกระโดดไปวางไว้ที่ตำแหน่งอินเทอร์รัปต์เวกเตอร์และให้ไปทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่อยู่ตำแหน่งอื่น ๆ แทน

```
ORG     0000        : ตำแหน่งเริ่มต้นของโปรแกรมใน ROM
LJMP    MAIN        : กระโดดข้ามอินเทอร์รัปต์เวกเตอร์ไปโปรแกรมหลัก
.......................
```

```
; พื้นที่เขียนโปรแกรมหลัก
        ORG     0030H
MAIN:   ……….
        ……….
        END
```

รูปที่ 10.1 ลักษณะการเขียนโปรแกรมให้กระโดดข้ามอินเทอร์รัปต์เวกเตอร์

การอีนาเบิลอินเทอร์รัปต์

เมื่อ MCS-51 ถูกรีเซตการอินเทอร์รัปต์ต่าง ๆ จะอยู่ในภาวะดิสเอเบิล (disable) คือไม่สามารถถูกอินเทอร์รัปต์ได้ เราสามารถอีนาเบิลการอินเทอร์รัปต์ได้ด้วยการโปรแกรมไปในรีจิสเตอร์ IE (interrupt enable) ซึ่งเป็นรีสเตอร์ที่สามารถเข้าถึงระดับบิตได้โดยขั้นตอนเป็นดังนี้

1 ให้บิด D7 เป็น “1” หมายความว่าอินเทอร์รัปต์ได้

2 เลือกบิดของการอินเทอร์รัปต์แต่ละประเภท ถ้าต้องการอินเทอร์รัปต์ประเภทใดให้บิดนั้นเป็น 1

ความหมาของบิดต่าง ๆ ในรีจิสเตอร์ IE แสดงได้ดังรูป 10.2

D7							D0
EA	-	ET2	ES	ET1	EX1	ET0	EX0

EA	IE7	ถ้า EA เป็น “0” หมายความว่าไม่สามารถอินเทอร์รัปต์ได้ ถ้าเป็น 1หมายความว่ายอมให้อินเทอร์รัปต์จากแหล่งต่างๆได้ โดยกำหนดแหล่งกำเนิดอินเทอร์รัปต์ในบิดที่ตามมา
....	IE.6	ไม่ใช้งาน
ET2	IE.5	อีนาเบิลหรือดิสเอเบิลของอินเทอร์ไทเมอร์2 (เฉพาะเบอร์ที่มีเช่น 8952)
ES	IE.4	อีนาเบิล หรือ ดิสเอเบิล ของอินเทอร์รัปต์ของอนุกรม
ET1	IE.3	อินนาเบิลหรือดีสเอเบิลของอินเทอร์รัปต์ไทเมอร์ 1
EX1	IE.2	อีนาเบิลหรือดีสเอเบิลของอินเทอร์รัปต์ภายนอกหมายเลข 1
ET0	IE.1	อินนาเบิลหรือดีสเอเบิล ของอินเทอร์รัปต์ไทเมอร์ 0
EX0	IE.0	อีนาเบิลหรือดีสเอเบิล ของอินเทอร์รัปต์ภายนอกหมายเลข 0

รูปที่ 10.2 ความหมายของแต่ละบิดในรีจิสเตอร์ IE

<u>ตัวอย่างที่ 10.1</u> จงแสดงการเขียนคำสั่งต่อไปนี้

ก. อีนาเบิลอินเทอร์รัปต์พอร์ตอนุกรมไทเมอร์ 0 และอินเทอร์รัปต์จากภายนอก

ข. ใอีนาเบิลอินเทอร์รัปต์ไทเมอร์ 0

ค. อีนาเบิลอินเทอร์รัปต์ทุกชนิด

<u>วิธีทำ</u>

ก. MOV IE, # 10010110B

ข. CLR IE.1 : อีนาเบิลอินเทอร์รัปต์ไทเมอร์ 0 เท่านั้น

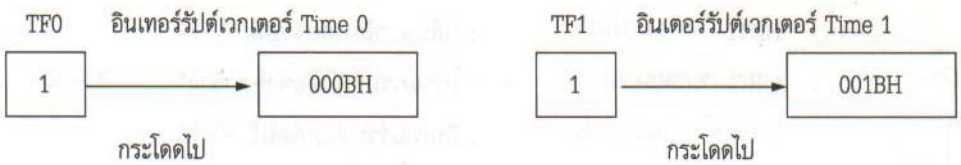
ค. CLR IE.7 : อินนาเบิลอินเทอร์รัปต์ทุกชนิด

การเขียนคำสั่งในข้อ ก. นั้นอาจใช้คำสั่งกระทำทีละบิตก็ได้ดังนี้

```
SETB    IE.7    ; อีนาเบิลอินเทอร์รัปต์
SETB    IE.4    ; อีนาเบิลอินเทอรีรัปต์พอร์ตอนุกรม
SETB    IE.1    ; อีนาเบิลอินเทอร์รัปต์ไทเมอร์ 0
SETB    IE.2    ; อีนาเบิลอินเตอร์รัปต์ภายนอก
```

10.2 การโปรแกรมไทเมอร์อินเทอร์รัปต์

ในบทที่ 8 ได้ศึกษาการใช้งานไทเมอร์มาแล้วซึ่งการตรวจสอบว่าไทเมอร์ได้นับมาถึงเวลาที่กำหนดแล้วจะทำได้โดยการตรวจสอบบิตแฟลก TF ด้วยคำสั่ง JNB TF,TEROUT โดยการทำแบบนี้ไมโครคอนโทรลเลอร์จะไม่สามารถไปทำงานอื่นได้สะดวกนักเราสามารถตรวจสอบการนับของไทเมอร์ว่านับมาถึงค่าที่กำหนดหรือยังโดยใช้การอินเทอร์รัปต์ได้ โดยการอินเทอร์รัปต์เวกเตอรีของไทเมอร์ 0 และไทเมอร์ 1 แสดงไว้ดังรูปที่ 10.3



รูปที่ 10.3 อินเทอร์รัปต์TF

การใช้อินเทอร์รัปต์ไทเมอร์ของไมโครคอนโทรลเลอร์ทำได้โดยเซตรีจิสเตอร์ IE ให้สามารถอินเตอร์รัปต์ไทเมอร์ได้เมื่อตัวไทเมอร์เกิดการโอเวอร์โฟลว์ค่าในบิต TF จะเป็น “1” และจะอินเทอร์รัปต์ไมโครคอนโทรลเลอร์ทำให้ไมโครคอนโทรลเลอร์กระโดดไปทำงานยังตำแหน่งอินเทอร์รัปต์เวกเตอร์ของไทเมอร์ได้ ดังตัวอย่างโปรแกรมที่ 10.2 ซึ่งมีขั้นตอนการทำงานดังนี้

1. นำโปรแกรมหลักไปเก็บยังแอดเดรสเริ่มต้นที่อยู่หลังตารางอินเทอร์รัปต์เวกเตอร์ จากโปรแกรมจะให้กระไปทำโปรแกรมหลักที่อยู่ในแอดเดรส 30H เมื่อระบบเริ่มทำงานจะทำคำสั่ง LJMP ก่อนแล้วกระโดดไปยังหน่วยความจำที่อยู่หลังตารางอินเทอร์รัปต์เวกเตอร์

2. โปรแกรมตอบสนองการอินเทอร์รัปต์ของไทเมอร์ 0 ให้เริ่มต้นที่แอดเดรส 000BH ซึ่งเป็นอินเทอร์รัปต์เวกเตอร์ของไทเมอร์ 0 และให้จบโปรแกรมด้วยคำสั่ง RETI

3. อีนาเบิลอินเทอร์รัปต์ 0 ด้วยคำสั่ง MOV IE#10000010B หรือ MOV IE#82H ในโปรแกรมหลัก (MAIN)

4. ในโปรแกรมหลักขียนคำสั่งอ่านข้อมูลจากพอร์ต P0 และส่งไปยังพอร์ต P1 และให้ทำซ้ำถ้าหากบิต TF0 ถูกเซตเมื่อใดจะเป็นการอินเทอร์รัปต์ไมโครคอนโทรลเลอร์และกระโดดไปทำคำสั่งที่ตำแหน่ง 000BH

5. ในโปรแกรมตอบสนองการอินเทอร์รัปต์ของไทเมอร์ 0 โดยไม่ต้องใช้คำสั่ง CLR TF0 ก่อนคำสั่ง RETI

<u>ตัวอย่างที่ 10.2</u> จงเขียนโปรแกรมให้ 8051 อ่านข้อมูลจากพอร์ต 0 และส่งข้อมูลที่อ่านได้ออกทางพอร์ต 1 รวมทั้งสร้างสัญญาณคลื่นสี่เหลี่ยมที่มีคาบเวลา 200 us ออกทางขา P2.1 โดยใช้ไทเมอร์ 0 ถ้า XTAL ที่ใช้มีความถี่ 11.0592 MHz

<u>วิธีทำ</u>

ให้ไทเมอร์ 0 ทำงานในโหมด 2 (auto-reload) โดยมี

TH0 = 100/1.085 us = 92

```
        ORG     0000H
        JMP     MAIN            ; By-Pass interrupt vector

        ORG     000BH           ; อินเทอร์รัปต์ไทเมอร์ 0
        CPL     P2.1            ; toggle P2.1
        RETI                    ; กลับจากอินเทอร์รัปต์

        ORG     0030H           ; โปรแกรมหลักต่อจากตารางอินเทอร์รัปต์เวกเตอร์
MAIN:   MOV     TMOD,#02H       ; ไทเมอร์ 0 โหมด 2
        MOV     P0,#0FFH        ; ให้ P0 เป็นพอร์ตที่อินพุต
        MOV     TH0,#-92        ; TH0 = A4H
        MOV     IE,#82H         ; อีนาเบิลไทเมอร์ 0
        SETB    TR0             ; ให้ไทเมอร์ 0 เริ่มทำงาน
BACK:   MOV     A,P0            ; อ่านข้อมูลจาก P0
        MOV     P1,A            ; ส่งข้อมูลออกทาง Py
        JMP     BACK            ; ทำซ้ำ
        END
```

<u>ตัวอย่างที่ 10.3</u> จงนำโปรแกรมในตัวอย่าง 10.2 มาเขียนใหม่ ให้สร้างคลื่นสี่เหลี่ยมที่เป็น High 1085 us และเป็น Low 15 us

<u>วิธีทำ</u>

เวลา 1085 us เกิดจาก 1000 × 1.085 us โดยใช้ไทเมอร์ 1 โหมด 1

```
        ORG     0000H
        LJMP    MAIN
; Interrupt Vector ของไทเมอร์1
        ORG     001BH           ; อินเทอร์รัปต์เวกเตอร์ไทเมอร์ 1
        LJMP    ISR_T1          ; กระโดดไป ISR
; Main Program
        ORG     0030H           ; กระโดดไปหลังอินเทอร์รัปต์เวกเตอร์
MAIN:   MOV     TMOD,#10H       ; ไทเมอร์ 1 โหมด 1
        MOV     P0,#0FFH        ; ให้ P0 เป็นอินพุต
        MOV     TL1,#018H       ; TL1 = 18H (ไบต์ต่ำของ -1000)
        MOV     TH1,#0FCH       ; TH1 = 0FCH(ไบต์สูงของ -1000)
        MOV     IE,#88H         ; IE = 10001000 อีนาเบิลไทเมอร์ 1
        SETB    TR1             ; เริ่มไทเมอร์1
BACK:   MOV     A,P0            ; อ่านข้อมูลจาก P0
        MOV     P1,A            ; ส่งข้อมูลออกไป P1
        SLMP    BACK
; Interrupt Service Routine ของ Timer1
IS_TR1: CLR     TR1             ; หยุดไทเมอร์ 1
        CLR     P2.1            ; P2.1 = 0 ให้เป็น Low
        MOV     R2,#4           ; 2MC
HERE:   DJNZ    R2,HERE         ; 4×2 แมชชีนไซเคิล      8 MC
        MOV     TL1,#18H        ; โหลดไบต์ต่ำ           2 MC
        MOV     TH1,#0FCH       ; โหลดไบต์สูง           2 MC
        SETB    TR1             ; เริ่มไทเมอร์1         1MC
        SETB    P2.1            ; P2.1 กลับเป็น high    1MC
        RETI                    ; กลับโปรแกรมหลัก
        END
```

เวลาในช่วงที่เป็น Low จะเท่ากับ 14 แมชชีนไซเคิล โดย MC = 1.085 us และ 14×1.085 uSec = 15.19 uSec

<u>ตัวอย่างที่ 10.4</u> จงเขียนโปรแกรมสร้างคลื่นรูปสี่เหลี่ยมความถี่ 50 Hz ออกทางขา P1.2 โดยการใช้อินเทอร์รัปต์ของไทเมอร์ 0 ถ้า XTAL มีความถี่ 11.0592 MHz

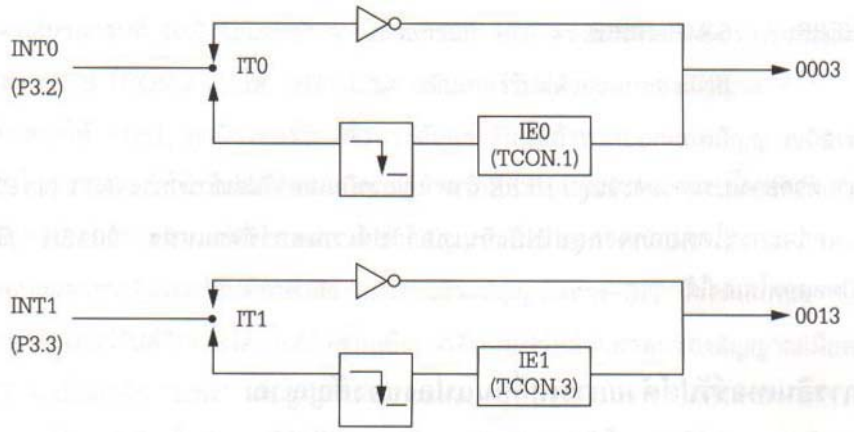
<u>วิธีทำ</u>

```
        ORG   0000
        LJMP  MAIN
        ORG   000BH              ; ISR ของไทเมอร์ 0
        CPL   P1.2
        MOV   TL0,#00
        MOV   TH0,#0DCH
        RETI
        ORG   0030H              ; เริ่มต้นโปรแกรมหลัก
MAIN:   MOV   TMOD,#00000001B    ; ไทเมอร์ 0 โหมด 1
        MOV   TL0,#00
        MOV   TH0,#0DCH
        MOV   IE,#82H            ; อีนาเบิลอินเทอร์รัปต์ไทเมอร์ 0
        SETB  TR0
HERE:   SJMP  HERE
        END
```

10.3 การโปรแกรมอินเทอร์รัปต์ภายนอก

ไมโครคอนโทรลเลอร์ MCS-51 สามารถรับอินเทอร์รัปต์จากภายนอกได้สองทางคือ INT0 และ INT1 ทางขา P3.2 และขา P3.3 ตามลำดับ เมื่อตัวมันถูกอินเทอร์รัปต์ก็จะกระโดดไปทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่ถูกเก็บอยู่ในอินเทอร์รัปต์เวกเตอร์ที่กำหนด การรับสัญญาณอินเทอร์รัปต์จากภายนอกแสดงได้ดังรูปที่ 10.4 ซึ่งจะเห็นว่า MCS-51 สามารถรับสัญญาณอินเทอร์รัปต์จากภายนอกได้ 2 รูปแบบคือ

1. รับด้วยระดับแรงดัน (level triggered)
2. รับด้วยขอบการเปลี่ยนแปลงของสัญญาณ (edge triggered)



รูปที่ 10.4 การอินเทอร์รัปต์ทาง INT0 และ INT1

การอินเทอร์รัปต์ด้วยระดับแรงดันไฟฟ้า

ในโหมดการอินเทอร์รัปต์ด้วยระดับแรงดันไฟฟ้านั้น ตัว MCS-51 ในภาวะปกติขา INT0 และ INT1 จะเป็นลอจิก High ถ้าหากขาใดขาหนึ่งเปลี่ยนแปลงเป็นระดับแรงดัน Low จะมีการอินเทอร์รัปต์เกิดขึ้น และไมโครคอนโทรลเลอร์จะกระโดดไปทำโปรแกรมตอบสนองการอินเทอร์รัปต์ที่อยู่ในตำแหน่งอินเทอร์รัปต์เวกเตอร์ที่กำหนด การทำงานลักษณะนี้เรียกว่า level-triggered หรือ level-activated หลังจาก MCS-51 ถูกอินเทอร์รัปต์สัญญาณทางขา INT จะต้องกลับเป็นลอจิก High ก่อนที่โปรแกรมจะตอบสนองการอินเทอร์รัปต์ จะทำคำสั่ง RETI เพื่อไม่ให้เกิดการอินเทอร์รัปต์ซ้ำอีก

<u>ตัวอย่างที่ 10.5</u> ถ้าหากขา INT1 ต่ออยู่กับสวิตซ์ที่ในภาวะปกติเป็นลอจิก “High” จงเขียนโปรแกรมถ้าหากกดสวิตซ์นี้จะทำให้หลอด LED ที่ต่ออยู่ทาง P1.3 สว่าง

<u>วิธีทำ</u>

```
        ORG     0000H
        LJMP    MAIN                ; กระโดดไปโปรแกรมหลัก
; โปรแกรม ISR ของ INT1
        ORG     0013H
        SETB    P1.3                ; เปิดหลอด LED
        MOV     R3,#255
BACK:   DJNZ    R3,BACK             ; หน่วงเวลา
        CLR     P1.3                ; ปิดหลอด LED
        RETI                        ; กลับสู่โปรแกรมหลัก
        ORG     0030H
MAIN:   MOV     IE,#10000100B       ; อีนาเบิล INT1
HERE:   SJMP    HERE
        END
```

จากตัวอย่างโปรแกรมจะวนลุป HERE ถ้าหากมีการอินเทอร์รัปต์เข้ามาทาง INT1 (ขา P3.3) ไมโครคอนโทรลเลอร์จะกระโดดออกจากลูปไปยังอินเทอร์รัปต์เวกเตอร์ที่ตำแหน่ง 0013H ซึ่งจะเขียนโปรแกรมเปิดหลอดไฟเองได้

การอินเทอร์รัปต์ด้วยการเปลี่ยนแปลงของสัญญาณ

จากที่ผ่านมาเราได้ทดลองให้ 8051 รับสัญญาณอินเทอร์รัปต์ด้วยระดับแรงดันจากภายนอกมาแล้ว เราสามารถให้ 8051 รับอินเทอร์รัปต์จากภายนอกทาง INT0 และ INT1 เมื่อมีการเปลี่ยนแปลงจากลอจิก High ไปเป็นลอจิก Low ได้อีกด้วย การอินเทอร์รัปต์ด้วยสัญญาณแบบนี้เรียกว่าการอินเทอร์รัปต์ด้วยขอบของสัญญาณ (Edge-triggered interrupt) การที่จะให้ 8051 ถูกอินเทอร์รัปต์ด้วยสัญญาณแบบนี้ เราจะต้องเซตบิต IT0 และ IT1 ในรีจิสเตอร์ TCON เสียก่อน โดย IT0 เป็นบิต D0 ในรีจิสเตอร์ TCON ส่วน IT1 เป็นบิต D2 ในรีจิสเตอร์ TCON ดังนั้นถ้าต้องการให้ 8051 รับอินเทอร์รัปต์ภายนอกแบบการอินเทอร์รัปต์ด้วยขอบ จะต้องใช้คำสั่ง SETB TCON.0 หรือ SETB IT0 สำหรับ INT0 และ SETB TCON.2 หรือ SETB IT1 สำหรับ INT1

<u>ตัวอย่างที่ 10.6</u> ถ้าหากขา INT1 ของ 8051 ต่ออยู่กับตัวกำเนิดสัญญาณพลัส จงเขียนโปรแกรมให้ 8051 ตรวจสอบว่าถ้ามีพลัสเข้ามาให้บิต P1.3 เป็นลอจิก “High” ซึ่งมี LED ต่ออยู่ก็ให้ LED หลอดนั้นสว่าง

<u>วิธีทำ</u>

```
        ORG   0000H
        LJMP  MAIN          ; กระโดดไปโปรแกรมหลัก
        ORG   0013H         ; ISR ของ INT1
        SETB  P1.3          ; ให้ไฟติด
        MOV   R3,#255
BACK:   DJNZ  R3,BACK       ; ทำซ้ำจน R3 เป็นศูนย์
        CLR   P1.3          ; ให้ไฟดับ
        RETI                ; กลับสู่โปรแกรมหลัก
        ORG   0030H         ; ตำแหน่งเริ่มต้นโปรแกรมหลัก
MAIN:   SETB  IT1           ; ให้อินเทอร์รัปต์ด้วยขอบสัญญาณ
        MOV   IE,#10000100B ; อีนาเบิล INT1
HERE:   SJMP,HERE           ; ทำซ้ำ
        END
```

จากโปรแกรมที่ 10.6 จะแตกต่างจากโปรแกรมที่ 10.5 ตรงบรรทัดแรกของโปรแกรมหลัก MAIN จะใช้คำสั่ง SETB TCON.2 เพื่อให้ INT1 เป็นการอินเทอร์รัปต์ด้วยขอบของสัญญาณ

ถ้าหากให้ 8051 ถูกอินเทอร์รัปต์ด้วยระดับแรงดันนั้นถ้าหากออกแบบสัญญาณอินเทอร์รัปต์ไม่เหมาะสมกับโปรแกรมจะทำให้ระบบทำงานผิดพลาดได้ จากที่ทราบมาแล้วว่าการอินเทอร์รัปต์ด้วยระดับแรงดันระบบจะถูกอินเทอร์รัปต์ด้วยลอจิก “Low” ถ้าหากสัญญาณที่เข้ามาทางขา INT เป็น Low นานจนกระทั่งโปรแกรมตอบสนองการอินเทอร์รัปต์ทำคำสั่ง RETI แล้ว สัญญาณทาง INT ยังไม่กลับเป็น “High” ก็จะทำให้ระบบถูกอินเทอร์รัปต์อีกครั้งได้ แต่ถ้าหากเป็นการอินเทอร์รัปต์ด้วยขอบของสัญญาณเมื่อค่าสัญญาณทางขา INT จะเป็นลอจิก “Low” นานกว่าที่โปรแกรมตอบสนองการอินเทอร์รัปต์ทำคำสั่ง RETI ระบบก็จะถือว่าถูกอินเทอร์รัปต์เพียงครั้งเดียว

10.4การโปรแกรมการอินเทอร์รัปต์พอร์ตอนุกรม

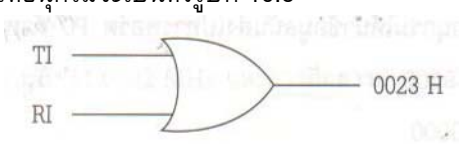
ในบทที่9 ได้กล่าวถึงการรับส่งข้อมูลแบบอนุกรมของ MCS-51 มาแล้ว โดยทุกตัวอย่างในบทที่ 9 จะใช้วิธีพูลลิ่ง (polling method) ในการติดต่อกับพอร์ตอนุกรมในบทนี้จะศึกษาเกี่ยวกับการอินเทอร์รัปต์ของพอร์ตอนุกรม โดยถ้ามีข้อมูบส่งเข้ามาจะมีสัญญาณอินเทอร์รัปต์ MCS-51 เพื่อให้ MCS-51 ทำการอ่านข้อมูลที่ถูกส่งเข้าไป

บิตแฟลก RI และ TI

เมื่อพอร์ตอนุกรมของ MCS-51 ได้รับไบต์ข้อมูลเข้ามาจะใช้บิตแฟลก RI(Received Interrupt) เป็นบิตอินเทอร์รัปต์ ถ้าหากพอร์ตอนุกรมทำงานในโหมด 0 บิตนี้จะเซตเมื่อรับข้อมูลบิตที่ 7 หรือบิตที่ 8 เข้ามาเสร็จสมบูรณ์ แต่ถ้าพอร์ตอนุกรมทำงานในโหมดอื่น บิตนี้จะเซตเมื่อรับข้อมูลบบิตปิดท้ายเข้ามา สำหรับการเคลียร์บิตนี้จะต้องทำโดยการเขียนคำสั่ง สำหรับการรับข้อมูลเข้ามาทางพอร์ตอนุกรมนั้นจะใช้บิตแฟลก

TI(Transmit Interrupt) เป็นบิตอินเทอร์รัปต์ เมื่อพอร์ตอนุกรมทำงานในโหมด 0 บิตนี้จะเซตเมื่อมีการส่งข้อมูลบิตที่ 7 หรือบิตที่ 8 ออกไป แต่ถ้าหากทำงานในโหมดอื่น บิตนี้จะเซตเมื่อมีการส่งบิตหยุดหรือบิตปิดท้ายออกไป สำหรับการเคลียร์บิตนี้จะต้องทำโดยการเขียนคำสั่ง

ดังนั้นถ้าหากจะต้องใช้การอินเทอร์รัปต์พอร์ตอนุกรมจะต้องนำโปรแกรมตอบสนองการอินเทอร์รัปต์ไปเก็บยังตำแหน่งอินเทอร์รัปต์เวกเตอร์ของอินเทอร์รัปต์พอร์ตอนุกรมคือตำแหน่ง 0023H โดยลักษณะการทำงานของบิตอินเทอร์รัปต์พอร์ตอนุกรมจะเป็นดังรูปที่ 10.5



รูปที่ 10.5 สัญญาณอินเทอร์รัปต์จาก TI และ RI

<u>ตัวอย่างที่ 10.7</u> จงเขียนโปรแกรมให้ 8051 อ่านข้อมูลจากพอร์ต P1 และส่งออกไปทางพอร์ตอนุกรมและพอร์ต P2 ถ้าหาก XTAL มีความถี่ 11.0592 และค่า baud rate เท่ากับ 9600

<u>วิธีทำ</u>

```
        ORG   0000H
        LJMP  MAIN          ; กระโดดไปโปรแกรมหลัก
        ORG   0023H
        LJMP  SERIAL        ; กระโดดไปโปรแกรมตอบสนองการอินเทอร์รัปต์
        ORG   0030H
MAIN:   MOV   P1, #OFFH     ; ให้พอร์ต P1 ให้เป็นพอร์ตอินพุต
        MOV   TMOD, #20H    ; ใช้งานไทเมอร์ 1 โหมด 2
        MOV   TH1, # 0FDH   ; baud rate 9600
        MOV   SCON, #50H    ; ส่งข้อมูล 8 บิต 1 บิตหยุด
        MOV   IE, #10010000B ; อีนาเบิลอินเทอร์รัปต์พอร์ตอนุกรม
        SETB  TR1           ; ให้ไทเมอร์ 1 เริ่มทำงาน
BACK:   MOV   A,P1          ; อ่านค่าจาก P1 มาเก็บใน A
        MOV   SBUF,A        ; นำค่า A ส่งทางพอร์ตอนุกรม
        MOV   P2,A          ; นำค่า A ส่งออกทางพอร์ต P2
        SJMP  BACK          ; ทำลูปซ้ำ
; โปรแกรม ISR พอร์ตอนุกรม
        ORG   0100H
SERIAL: JB    TI, TRANS     ; กระโดดถ้าบิต TI เป็น high
        MOV   A, SBUF       ; อ่านข้อมูลที่รับเข้ามา
        CLR   RI            ; เคลียร์บิต RI
        RETI                ; จบโปรแกรม ISR
TRANS:  CLR   TI            ; เคลียร์บิต TI
        RETI                ; จบโปรแกรม ISR
        END
```

ตัวอย่างที่ 10.8 จงเขียนโปรแกรมให้ 8051 อ่านข้อมูลจากพอร์ต P1 และส่งไปยังพอร์ต P2 ถ้าหากมีข้อมูลส่งเข้ามาทางพอร์ตอนุกรมให้นำข้อมูลนั้นส่งไปทางพอร์ต P0 ถ้าหาก XTAL มีความถี่ 11.0592 MHz และ baud rate เท่ากับ 9600

วิธีทำ

```
        ORG   0000
        LJMP  MAIN
        ORG   0023H
        LJMP  SERIAL          ; กระโดดไป ISR พอร์ตอนุกรม
        ORG   0030H
MAIN:   MOV   P1, #0FFH       ; ให้พอร์ต P1 ให้เป็นพอร์ตอินพุต
        MOV   TMOD, #20H      ; ใช้งานไทเมอร์ 1 ทำงานโหมด 2
        MOV   TH1, # 0FDH     ; baud rate 9600
        MOV   SCON, #50H      ; รับ 8 บิต 1 stop , อีนาเบิล REN
        MOV   IE, #10010000B  ; อีนาเบิลอินเทอร์รัปต์พอร์ตอนุกรม
        SETB  TR1             ; ให้ไทเมอร์ 1 เริ่มทำงาน
BACK:   MOV   A,P1            ; อ่านค่าจากพอร์ต P1
        MOV   P2,A            ; ส่งข้อมูลออกไปพอร์ต P2
        SJMP  BACK            ; ทำซ้ำ
; โปรแกรม ISR พอร์ตอนุกรม
        ORG   100H
SERIAL: JB    TI, Trans       ; กระโดดถ้าแฟลก TI เป็น high
        MOV   A, SBUF         ; อ่านค่าที่รับเข้ามา
        MOV   P0, A           ; ส่งค่าออกไปทางพอร์ต P0
        CLR   R               ; เคลียร์บิต RI
        RETI                  ; กลับโปรแกรมหลัก
Trans:  CLR   TI              ; เคลียร์บิต TI
        RETI                  ; กลับโปรแกรมหลัก
        END
```

ตัวอย่างที่ 10.9 จงเขียนโปรแกรมโดยใช้อินเทอร์รัปต์ให้ทำงานต่อไปนี้

ก. รับข้อมูลจากพอร์ตอนุกรม

ข. อ่านข้อมูลจากพอร์ต P1 และส่งออกไปทาง P2 และพอร์ตอนุกรม

ค. ให้ไทเมอร์ 0 สร้างความถี่ 5 kHz ทาง P0.1

ถ้าหาก XTAL มีความถี่ 11.0592 MHz และการสื่อสารอนุกรมมี baud rate 4800

วิธีทำ

```
        ORG   0000
        LJMP  MAIN            ; กระโดดไปโปรแกรมหลัก
        ORG   000BH           ; ISR ของไทเมอร์ 0
        CPL   P0.1            ; กลับค่า P0.1
        RETI                  ; กลับโปรแกรมหลัก
```

```
        ORG     0023H
        LJMP    SERIAL          ; กระโดดไปโปรแกรม ISR พอร์ตอนุกรม
        ORG     0030H
MAIN:   MOV     P1, #0FFH       ; ให้พอร์ต P1 ให้เป็นพอร์ตอินพุต
        MOV     TMOD, #22H      ; ใช้งานไทเมอร์ 0 ทำงานโหมด 2
        MOV     TH1, # 0F6H     ; baud rate 4800
        MOV     SCON, #50H      ; 8 บิต 1 stop
        MOV     TH0, # - 92     ; ความถี่ 5 kHz
        MOV     IE, #10010010B  ; อีนาเบิลอินเทอร์รัปต์
        SETB    TR1             ; เริ่มไทเมอร์ 1
        SETB    TR0             ; เริ่มไทเมอร์ 0
BACK:   MOV     A,P1            ; อ่านค่าจากพอร์ต P1
        MOV     SBUF, A         ; ส่งค่าออกพอร์ตอนุกรม 2
        MOV     P2,A            ; ส่งค่าออกพอร์ต P2
        SJMP    BACK            ; ทำซ้ำ
        ORG     100H
SERIAL: JB      TI, TRANS
        MOV     A, SBUF
        MOV     P0, A
        CLR     RI
        RETI
TRANS:  CLR     TI
        RETI
        END
```

ก่อนที่จะจบหัวข้อนี้จะขอสรุปบิตแฟลกของการอินเทอร์รัปต์จากแหล่งต่างๆ ซึ่งจะทำหน้าที่อินเทอร์รัปต์ไมโครคอนโทรลเลอร์โดยบิตต่างๆจะอยู่ในรีจิสเตอร์ TCON ดังตารางที่ 10.2

ตารางที่ 10.2 บิตแฟลกอินเทอร์รัปต์

อินเทอร์รัปต์	แฟลก	บิตในรีจิสเตอร์
External 0	IE0	TCON.1
External 1	IE1	TCON.3
Timer 0	TF0	TCON.5
Timer 1	TF1	TCON.7
Serial port	TI + RI	TCON.1
Timer 2	TF2	T2CON.7 (AT89C52)

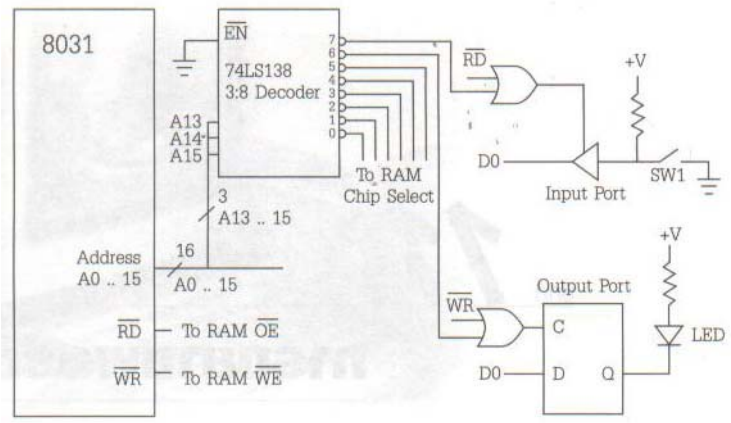
บทที่ 11
การขยายพอร์ต

ไมโครคอนโทรลเลอร์มักจะถูกใช้งานการควบคุมอุปกรณ์ภายนอก โดยอุปกรณ์แต่ละชิ้นจะต่ออยู่กับพอร์ตของไมโครคอนโทรลเลอร์ สำหรับ MCS-51 จะมีขนานอยู่ 4 พอร์ต แต่จะพบว่าบางพอร์ตจะต้องใช้งานในหน้าที่อื่นด้วย ถ้าหากต้องการขยายพอร์ตเพิ่มก็สามารถทำได้โดย MCS-51 จะใช้แอดเดรสของพอร์ตเป็นแอดเดรสเดียวกับหน่วยความจำ RAM โดยชุดคำสั่งและสัญญาณที่ใช้กับพอร์ตจะเป็นเหมือนกับกานติดต่อกับหน่วยความจำ RAM

จากโครงสร้างของ MCS-51 จะพบว่าต้องการขยายหน่วยความจำเพิ่มเติมภายนอกจะต้องนำพอร์ต 0 และพอร์ต2 มาใช้บัสแอดเดรสและบัสข้อมูลด้วย สำหรับพอร์ต0 นั้นจะทำงานสองหน้าที่คือเป็นแอดเดรสไบต์ต่ำ (A0-A7) และเป็นบัสข้อมูลและจะต้องใช้อุปกรณ์ภายนอกมาช่วยในการแลตซ์ส่วนที่เป็นแอดเดรสให้ค้างไว้ โดยมีสัญญาณ ALE เป็นสัญญาณควบคุมดังนั้นถ้าหากต้องการขยายพอร์ตเพิ่มเติมจะต้องนำสัญญาณดังกล่าวมาถอดรหัสเป็นตำแหน่งของพอร์ตด้วย จากนั้นจึงต่ออุปกรณ์เพิ่มเติมสำหรับพอร์ตลงไป เช่น การใช้งานเป็นพอร์ตอินพุตจะต้องใช้ไอซีบัฟเฟอร์สามสถานะ(tri-state buffers) มาต่อเป็นพอร์ตถ้าใช้งานเป็นพอร์ตเอาต์พุตจะต้องนำเอา ฟลิปฟลอปที่สามารถแลตซ์ (latches) ข้อมูลได้มาต่อเป็นอุปกรณ์เอาต์พุต

11.1 การถอดรหัสตำแหน่งพอร์ต

สำหรับการถอดรหัสตำแหน่งพอร์ตนั้นจะต้องไม่ให้มีตำแหน่งตรงกับตำแหน่งของหน่วยความจำ RAM ส่วนวงจรถอดรหัสสามารถทำได้หลายลักษณะ แต่โดยทั่วไปแล้วมักจะใช้ไอซีถอดรหัสมาเป็นตัวถอดรหัสตำแหน่งพอร์ต ดังตัวอย่าง 11.1



รูปที่11.1 การต่อ 8031 กับพอร์ตภายนอกเพิ่มเติม

เนื่องจาก MCS-51 สามารถต่อหน่วยความความจำ RAM ภายนอกได้ 64 K ตำแหน่ง คือตั้งแต่แอดเดรส 0000H ไปจนถึงFFFFH จากวงจรในรูปที่ 11.1 จะนำไอซีถอดรหัสเบอร์ 74IS138 มาต่อโดยจะรับสัญณาณอินพุตเข้าไป 3 ค่า และให้เอาท์พุตออกมา 8 ค่า โดยค่าทางเอาท์พุตจะเป็นลอจิก “0” ทีละบิด ขึ้นกับสัญญาณที่เข้าไปทางอินพุต จากวงจรดังรูป ที่ 11.1

ถ้าหากให้เอาท์พุต 0 ถึงเอาท์พุต 5 ของไอซี 74LS138 ต่ออยู่กับสัญญาณเลือก RAM จำนวน 6 ตัว และให้เอาท์พุต และ ให้เอาต์พุต 7 เป็นตัวถอดรหัส โดยแอดเดรสของตำแหน่งต่างๆ รวมทั้งสัญญาณอินพุตและเอาท์พุตของตัวถอดรหัสแสดงได้ดังตารางที่ 11.1

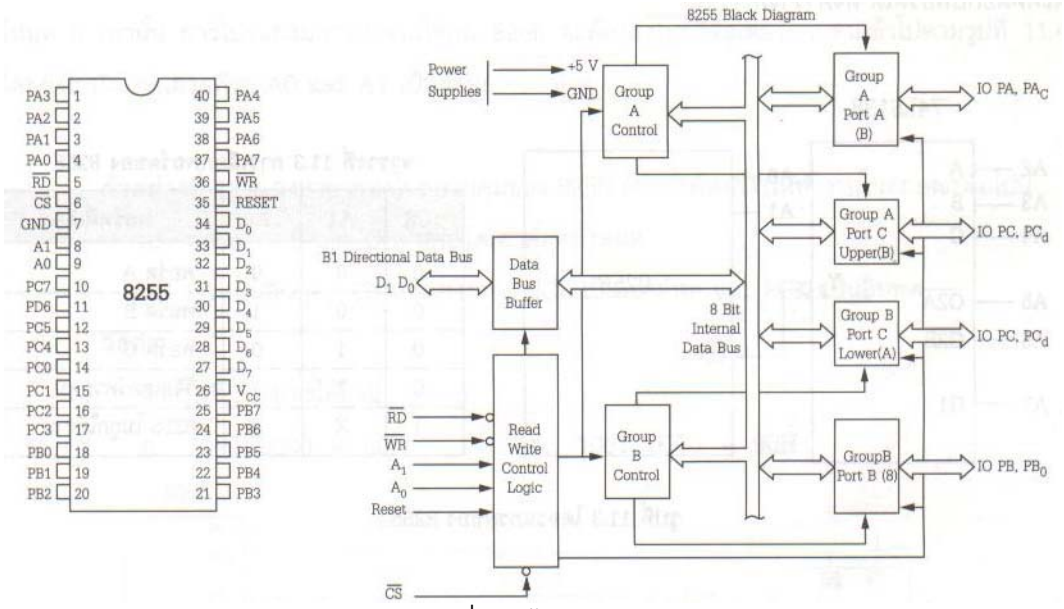
ตารางที่ 11.1 แอดเดรสหน่วยความจำและพอร์ตของวงจรในรูปที่ 11.1

ช่วงแอดเดรส (HEX)	บิดแอเดรส A15 A14 A13	เอาต์พุตตัวถอดรหัส 7 6 5 4 3 2 1 0	อุปกรณ์ที่ถูกเลือก
0000-1FFF	0 0 0	1 1 1 1 1 1 1 0	RAM 0
2000-3FFF	0 0 1	1 1 1 1 1 1 0 1	RAM 1
4000-5FFF	0 1 0	1 1 1 1 1 0 1 1	RAM 2
6000-7FFF	0 1 1	1 1 1 1 0 1 1 1	RAM 3
8000-9FFF	1 0 0	1 1 1 0 1 1 1 1	RAM 4
A000-BFFF	1 0 1	1 1 0 1 1 1 1 1	RAM 5
C000-DFFF	1 1 0	1 0 1 1 1 1 1 1	Output Port
E000-FFFF	1 1 1	0 1 1 1 1 1 1 1	Input Port

ถ้าหาก MCS-51 ทำคำสั่งที่ใช้ไปยังข้อมูลไปยังแอดเดรสภายนอกจะทำให้สัญญาณ WR เป็นลอจิก “0” ดังนั้นจะต้องใช้สัญญาณนี้ในกรถอดรหัสพอร์ตเอาท์พุตด้วย และถ้าหาก MCS -51 ทำคาสั่งอ่านข้อมูลจากหน่วยความจำภายนอกจะทำให้สัญญาณ Rd เป็นลอจิก “0” ดังนั้นจะต้องใช้สัญญาณนี้ในการถอดรหัสพอร์ตอินพุตด้วย (สำหรับไดอะแกรมเวลาการทำคำสั่งศึกษาได้จากเครื่องมือของไมโครคอนโทรลเลอร์ MCS-51) การถอดรหัสแอดแดรสของพอร์ตประเภทนี้เรียกว่า memory mapped I/O จากวงจรจะเห็นว่าถ้าส่งข้อมูลไปยังแอดแดรสในช่วง C000H ถึง DFFFH จะเป็นการอ่านข้อมูลเข้ามาทางอินพุต

11.2 ไอซี 8255 และการโปรแกรม

จากที่กล่าวมาแล้วว่าอุปกรณ์ที่ใช้เป็นพอร์ตอินพุตจะต้องใช้บัฟเฟอร์แบบ 3 สถานะ และอุปกรณ์ ที่ใช้เป็นพอร์ตเอาต์พุตจะต้องพลิปฟลิปที่สามารถแลตซ์ข้อมูลได้นอกจากนี้งานประเภทนี้บางประเภทจะต้องมีการใช้พอร์ตขนานหลายพอร์ต ในหัวข้อนี้จะกล่าวถึงไอซีพอร์ตขนานที่ภายในตัวมันมีพอร์ตที่ใช้งานได้ถึงสามพอร์ต โดยแต่ละพอร์ตสามารถโปรแกรมให้เป็นพอร์ตอินพุตหรือพอร์ตเอาต์พุตได้ตามต้องการ ความสามารถนี้เรียกว่า programmable ลักษณะตัวถังและโครงสร้างภายในของไอซี 8255 แสดงได้ดังรูป 11.2



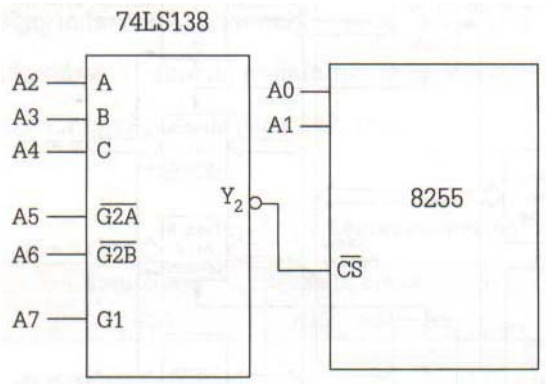
รูปที่11.2 ไอซี 8255

ไอซี 8052 จะมีขนาดอยู่ 3 พอร์ต คือพอร์ต A พอร์ต B และพอร์ต C โดยแต่ละพอร์ตจะเป็นแบบ 8 บิต พอร์ตเหล่านี้สามารถแบ่งออกได้สองกลุ่มคือ กลุ่ม A กลุ่ม B โดยกลุ่ม A ประกอบด้วยพอร์ต A 8 บิต และพอร์ต C 4 บิต (4 บิตบน) กลุ่ม B ประกอบด้วยพอร์ต B 8 บิต และพอร์ต C 4 บิต (4 บิตล่าง) สำหรับขาต่างๆ แสดงได้ดังตารางที่ 11.2

ตารางที่ 11.2 หน้าที่การทำงานของขาไอซี B255

ช่องสัญญาณ	การทำงาน
D0 – D7	กลุ่มสัญญาณที่ใช้อ่านข้อมูลให้กับ 8255
PA0 - PA7	กลุ่มสัญญาณพอร์ต A ขนาด B บิด
PB0 - PC7	กลุ่มสัญญาณพอร์ต B ขนาด 8บิด
PC0 - PC7	กลุ่มสัญญาณพอร์ต C ขนาด 8 บิด
CS\	สัญญาณเลือกชิป 8255 ให้สามารถอ่านเขียนข้อมูลได้ จะแอกทีฟลอจิกต่ำ
RD\	สัญญาณการอ่านข้อมูลออกจาก 8255
WR\	สัญญาณการเขียนข้อมูลให้ 8255
A0Al	สัญญาณเลือกพอร์ตของ 8255
RESET	สัญญาณรีเซตการทำงานของ 8255

การเชื่อมต่อ 8255 กับ MCs-51 นั้นจะสามารถจะใช้สัญญาณดังนี้ D0-D7, RD, WR, CS, A0 และ A1 ดังรูปที่ 11.3 เมื่อขาCs เป็นลอจิก 0 จะสามารถใช้งาน 8255 ได้โดยใช้สัญญาณ A1 กับ A0 เป็นตัวเลือกจะติดต่อกับพอร์ตใดดังตารางที่ 11.3

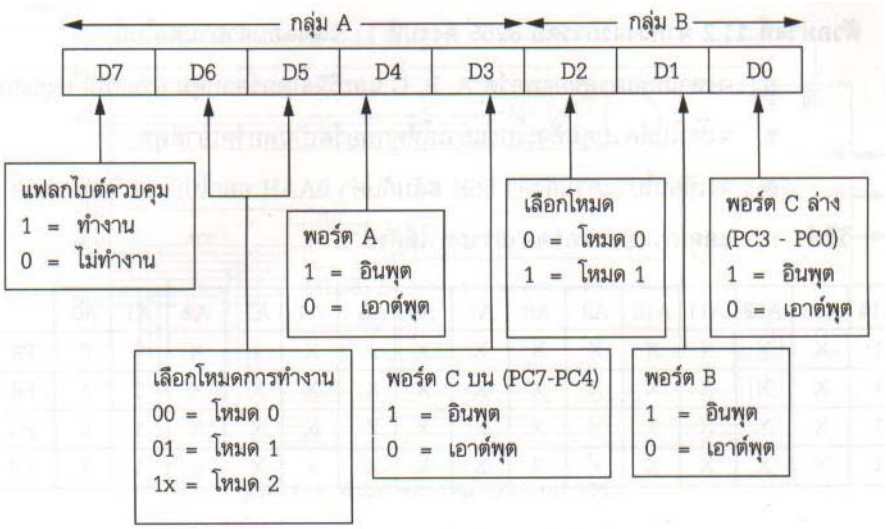


รูปที่11.3 ไดอะแกรมของ 8255

ตารางที่ 11.3 การเลือกพอร์ตของ 8255

CS	A1	A0	พอร์ตที่เลือก
0	0	0	พอร์ต A
0	0	1	พอร์ต B
0	1	0	พอร์ต C
0	1	1	รีจิสเตอร์ควบคุม
1	X	X	8255ไม่ถูกใช้งาน

การใช้งาน 8255 เป็นพอร์ตขนาดได้นั้นจะต้องโปนแกรมการทำงานให้กับรีจิสเตอร์ควบคุมเสียก่อนว่าต้องการใช้พอร์ตใดเป็นอินพุตพอร์ตใดเป็นเอาต์พุต สำหรับรีจิสเตอร์ควบคุมแสดงได้ดังรูปที่ 11.4



รูปที่11.4 ความหมายแต่ละบิตของรีจิสเตอร์ควบคุม

การศึกษาการใช้งาน 8255 ในขณะนี้กล่าวถึงการทำงานในโหมดอินพุตเอาต์พุตอย่างง่ายหรือโหมด 0 เท่านั้น การโปรแกรมการทำงานให้กับ 8255 จะต้องส่งไบต์จิสเตอร์ควบคุมเข้าไปตามรูปที่ 11.4 โดยส่งไปยังแอดเดรสที่ขา A0 และ A1 เป็นลอจิก 1 ทั้งคู่

<u>ตัวอย่างที่ 11.1</u> จงหาค่าของไบต์ควบคุมของ8255 ถ้าหากต้องการให้การทำงานในลักษณะต่อไปนี้

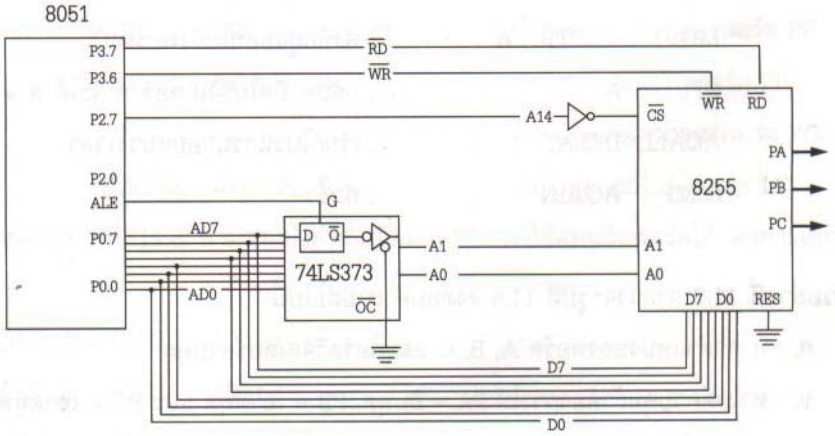
ก. พอร์ต A, B และ C เป็นพอร์ตเอาต์พุตทั้งหมด

ข. PA เป็นอินพุต, PB เป็นเอาต์พุต, PCL เป็นเอาต์พุต และ PCH เป็นอินพุต

<u>วิธีทำ</u>

จากรูปที่ 11.4 จะได้ดังนี้

ก. 1000 0000 = 80H ข. 1001 0000 = 90H



รูปที่11.5 การต่อ 8255 เข้ากับ 8051

ตัวอย่างที่ 11.2 จากวงจรการต่อ 8255 ดังรูปที่ 11.5 จงตอบคำถามต่อไปนี้

ก. จงหาแอดเดรสของพอร์ต A, B, C และรีจิสเตอร์ควบคุม

ข. จงหาไบต์ควบคุมที่จะโปรแกรมให้ทุกพอร์ตเป็นพอร์ตเป็นเอาต์พุต

ค. จงเขียนโปรแกรมส่งค่า55H สลับกับค่า 0AAH ออกไปยังพอร์ต

วิธีทำ

ก. แอดเดรสของพอร์ตสามารถหาได้ดังนี้

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
X	1	X	X	X	X	X	X	X	X	X	X	X	X	0	0	PA = 4000H
X	1	X	X	X	X	X	X	X	X	X	X	X	X	0	1	PB = 4001H
X	1	X	X	X	X	X	X	X	X	X	X	X	X	1	0	PC = 4002H
x	1	x	X	x	X	x	X	x	X	x	X	x	X	1	1	CR = 4003H

ข. ไบต์ควบคุมที่จะให้ทุกพอร์ตเป็นเอาต์พุตคือ 8OH ตามตัวอย่างที่ 11.1

ค. สามารถเขียนโปรแกรมได้ดังนี้

```
MOV     A, #80H          ; กำหนดไบต์ควบคุมให้ทุกพอร์ตเป็นพอร์ตเอาต์พุต
MOV     DPTR, #4003H     ; โหลดตำแหน่งพอร์ตของไบต์ควบคุม
MOVX    @DPTR, A         ; ส่งไบต์ออกไป
MOV     A,#55H           ; ให้ A=55
MOV     DPTR, #400H      ; โหลดแอดเดรสของพอร์ต A
MOVX    @DPTR, A         ; ส่งข้อมูลออกไปยังพอร์ต A
INC     DPTR             ; กำหนดแอดเดรสของพอร์ต B
MOVX    @DPTR ,A         ; ส่งข้อมูลไปยังพอร์ต C
CPL     A                ; กลับค่าบิตในรีจิสเตอร์ A จะได้ A=CAAH
ACALL   DELAY            ; เรียกโปรแกรมย่อยหน่วงเวลา
SJMP    AGAIN            ; ทำซ้ำ
```

ตัวอย่างที่ 11.3 จากรูปวงจรรูปที่ 11.6 จงตอบคำถามต่อไปนี้

ก. หาค่าตำแหน่งของพอร์ตA, B, C และพอร์ตรีจิสเตอร์ควบคม

ข. หาไบต์ควบคุมถ้าต้องการให้PA อินพุต PB= เอาต์พุตและ PC= เอาต์พุต

ค. จงเขียนโปนแกรมอ่านข้อมูลจากพอร์ตA และส่งข้อมูลออกไปยังพอร์ตจุด B และพอร์ต C

วิธีทำ

ก. สมมุติว่าบิตแอดเดรสที่ไม่ใช้งานเป็น 0 ทุกบิด จะได้ว่าแอดเดรสเริ่มต้นของ 8255 คือ 1000H

ดังนั้น PA เป็นแอดเดรส 1000H

PB เป็นแอดเดรส 1001H

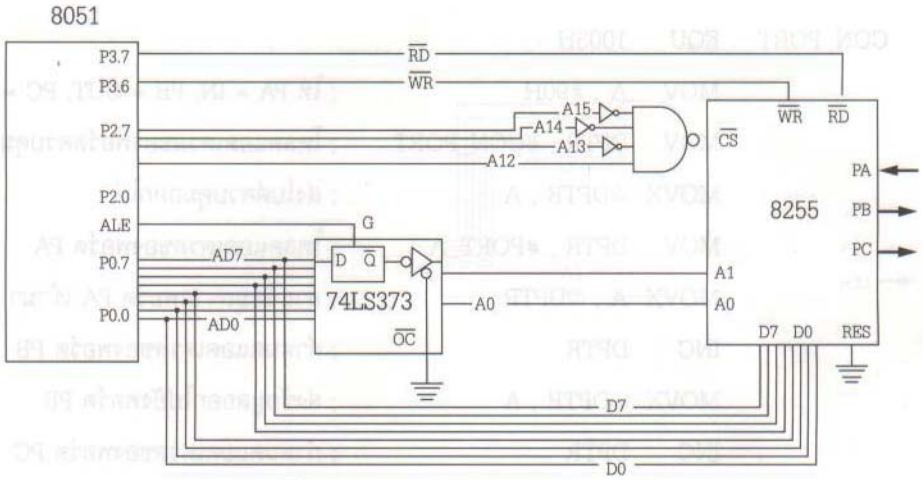
PC เป็นแอดเดรส 1002H

พอร์ตควบคุม เป็นแอดเดรส 1003H

ข. ไบต์ควบคุมคือ 10010000 หรือ 90H

ค .สามารถเขียนเป็นโปรแกรมได้ดังนี้

```
MOV   A, #90H        : ให้PA=IN.PB=OUT, PC=OUT
MOV   DPTR, #1003H   : โหลดแอดเดรสของพอร์ตของพอร์ตควบคุม
MOVX  @DPTR.A        : ส่งไบต์ควบคุมออกไป
MOV   DPTR, # 1000H  : โหลดแอดเดรสของพอร์ตPA
MOVX  A, @ DPTR      : อ่านข้อมูลจากพอร์ตPA เข้ามา
INC   DPTR           : กำหนดแอดเดรสของพอร์ตPB
MOVX  @DPTR.A        : ส่งข้อมูลออกไปยังพอร์ต PB
INC   DPTR           : กำหนดแอดเดรสของพอร์ต PC
MOVX  @DPTR.A        : ส่งข้อมูลออกไปยังพอร์ต PC
```



รูป11.6 ตัวอย่างการต่อ 8255 กับ 8051

จากตัวอย่างที่ 11.3 สามารถนำคำสั่งเทียม EQU มาแทนชื่อพอร์ตได้ ดังการเขียนโปรแกรมต่อไปนี้

```
PORT_A     EQU   1000H
PORT_B     EQU   1001H
POTR_C     EQU   1002H
CON_PORT   EQU   1003H
           MOV   A, #90H          : ให้PA=IN, PB=OUT, PC-OUT
           MOV   DPTR, #CON_PORT  : โหลดแอดเดรสของพอร์ตควบคุม
           MOVX  @DPTR, A         : โหลดแอดเดรสของพอร์ต
           MOV   DPTR, #PORT_A    : โหลดแอดเดรสของพอร์ตPA
           MOVX  A,@DPTR          : อ่านข้อมูลจาดกพอร์ต PA เข้ามา
           INC   DPTR             : กำหนดแอดเดรสของพอร์ตPB
           MOVX  @DPTR, A         : ส่งข้อมูลออกไปยังพอร์ตPB
           INC   DPTR             : กำหนดแอดเดรสของพอร์ตPC
           MOVX  @DPTR, A         : ส่งข้อมูลออกไปยังพอร์ตPC
```

จากตัวอย่างโปรแกรมจะพบว่าในการอ้างตำแหน่งของพอร์ตที่ 8255 ต่ออยู่ จะต้องใช้รีจิสเตอร์ DPTR เป็นตัวชี้เนื่องจากจากแอดเดรสที่ถอดรหัสเอาไว้เป็นแอดเดรสแบบ 16 บิต แต่ถ้าหากต้องการอ้างตำแหน่งพอร์ตในช่วงแอดเดรส 8 บิต สามารถใช้รีจิสเตอร์ R0 หรือ R1 เป็นตัวชี้ได้ เช่นการเขียนคำสั่ง MOVX A, @RO หรือ MOVX @R0, A,

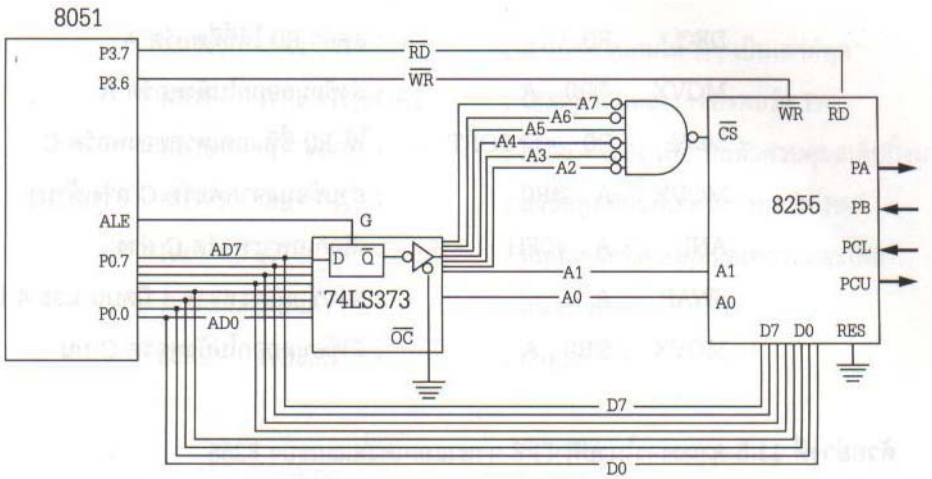
จากนตัวอย่างที่ 11.2 11.3 จะเห็นได้ว่าแอดเดรสพอร์ตที่ถอดรหัสออกมาจะอยู่ในช่วงที่กว้าง ถ้าพิจารณาตัวอย่างที่ 11.2 ในตารางตัว X จะเป็นตำแหน่งที่ไม่ได้ถอดรหัสซึ่งบิตนี้จะมีค่าลอจิกเป็นค่าใดก็ได้ ถ้า X เป็นลอจิก 0 ทุกบิต พอร์ต PA จะมีแอดเดรสเป็น 4000H แต่ถ้า อ้างตำแหน่ง 4000H ก็ยังเป็นพอร์ต PA เช่นกัลป์ เนื่องจากเราไม่สนใจค่า X สำหรับตัวอย่างที่ 11.4 และ 11.5 จะเป็นการนำAO –A7 มาถอดรหัสด้วยซึ่งจะทำให้แอดเดรสอยู่ในช่วงที่แคบมา

<u>ตัวอย่างที่ 11.4</u> จากวงจรในรูปที่ 11.7 จงตอบคำถามต่อไปนี้

ก. จงหาแอดเดรสของพอร์ต A, B, C และพอร์ตควบคุม

ข. จงหาไบต์ควบคุมที่ทำให้ PA=OUT, PB=IN, PC0-PC3=INและ PC7=OUT

ค. จงเขียนโปรแกรมอ่านข้อมูลจากพอร์ต PB และส่งข้อให้พอร์ต PA รวมทั้งอ่านข้อมูลจากพอร์ต PC ล่างและส่งข้อมูลออกทางพอร์ต PC บน



รูปที่11.7 ตัวอย่างการต่อ 8255 กับ 8051

<u>วิธีทำ</u>

ก. แอดเดรสของพอร์ตจะเป็นดังนี้

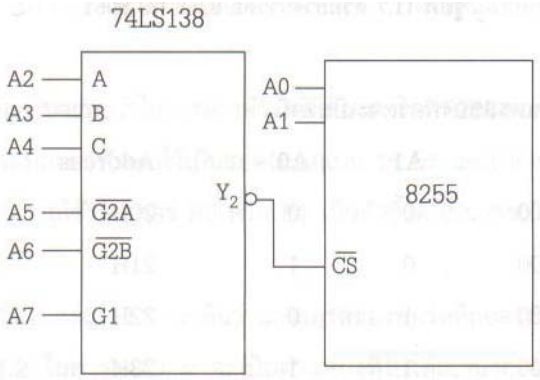
CS	A1	A0	Address	Port
0010 00	0	0	20H	พอร์ต A
0010 00	0	0	21H	พอร์ต B
0010 00	0	0	22H	พอร์ต C
0010 00	0	0	23H	พอร์ตควบคุม

ข. ไบต์ควบคุมคือ 1000 0011 หรือ 83H

ค. เขียนเป็นโปรแกรมได้ ดังนี้

```
CONTRBYT    EQU   83E            ; กำหนดให้ A=OUT, PB=IN, PCL=IN, PC=OUT
A_PORT      EUQ   20H            ; พอร์ต PA
B_PORT      EUQ   21H            ; พอร์ต PB
C_PORT      EUQ   22H            ; พอร์ต PC
CNT_PORT    EUQ   23H            ; พอร์ตควบคุม
            MOV   A, #CONTRBYT   ; PA=OUT, PB=IN, PCL=IN, PCU=OUT
            MOV   R0, #CNTPORT   ; โหลดแอดเดรสของพอร์ตควบคุมให้รีจิสเตอร์ R0
            MOVX  @R0, A         ; ส่งไบต์ควบคุมไปยังตำแหน่งที่ R0 ชี้อยู่
            MOV   R0, #B_PORT    ; ให้ R0 เก็บตำแหน่งของพอร์ต
            MOVX  A,@R0          ; อ่านข้อมูลจากพอร์ต B เข้ามาในรีจิสเตอร์ A
            DEC   R0             ; ลดค่า R0 ให้ชี้ที่พอร์ต A
            MOVX  @R0, A         ; ส่งข้อมูลออกไปยังพอร์ต A
            MOV   R0, #C_PORT    ; ให้ R0 ชี้ที่แอดเดรสของพอร์ต C
            MOVX  A, @R0         ; อ่านข้อมูลจากพอร์ต C ล่างเข้ามา
            ANL   A, #0FH        ; เลือกเฉพาะพอร์ต C ล่าง
            SWAP  A              ; สลับข้อมูลระหว่าง 4 บิตบน และ 4 บิตล่าง
            MOVX  @R0,A          ; ส่งข้อมูลออกไปยังพอร์ต C บน
```

<u>ตัวอย่างที่ 11.5</u> จากวงจรในรูปที่ 11.8 จงหาแอดเดรสแรกของ 8255



รูป11.8 การถอดรหัสแอดเดรส 8255 โดยใช้ 74LS138

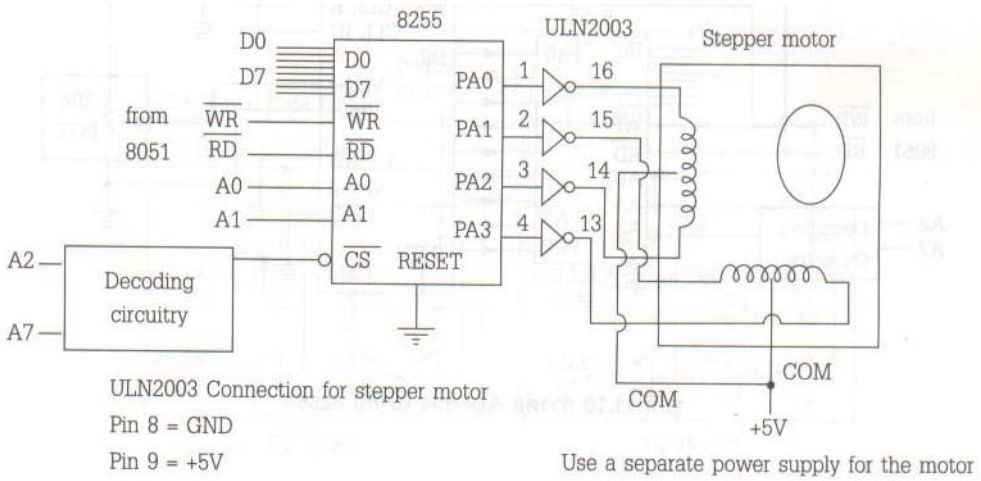
<u>วิธีทำ</u>

G1	G2B	G2A	C	B	A			แอดเดรส
A7	A6	A5	A4	A3	A2	A1	A0	
1	0	0	0	1	0	0	0	88H

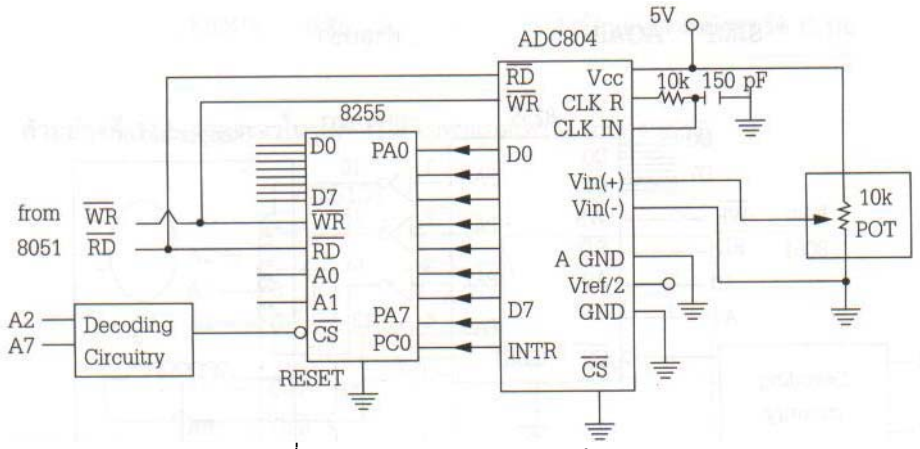
ดังนั้นแอดเดรสของพอร์ต A ของ 8255 คือตำแหน่ง 88H

ในระบบไมโครคอนโทรลเลอร์ถ้าหากต้องการใช้พอร์ตเป็นจำนวนมาก สามารถนำ 8255 หลาย ๆตัว มาต่อกันได้ โดยออกแบบวงจรถอดรหัสพอร์ตให้กับ 8255 แต่ละตัว จากนั้นพอร์ตของ 8255 จะถูกนำไปใช้ในการควบคุมอุปกรณ์อื่น ๆ ต่อไป ดังตัวอย่างรูปที่ 11.9 จะเป็นการนำ 8255 ไปควบคุม stepper motor และสามารถเขียนชุดคำสั่งควบคุมได้ดังต่อไปนี้

```
        MOV     A, #80H         ; กำหนดไบต์ควบคุมให้ PA เป็นเอาต์พุต
        MOV     R1, CRPORT      ; ให้ R1 ชี้ไปที่แอดเดรสพอร์ตควบคุม
        MOVX    @R1, A          ; ส่งไบต์ควบคุมให้ PA เป็นเอาต์พุต
        MOV     R1, #A_PORT     ; โหลดค่าแอดเดรสของพอร์ต PA
        MOV     A, #66H         ; ให้ A = 66H เพื่อเป็นตัวควบคุมสเต็ปปิ้งมอเตอร์
AGAIN:  MOVX    @R1, A          ; ส่งข้อมูลขับมอร์เตอร์ทางพอร์ต PA
        RR      A               ; หมุนลอจิกที่ควบคุมมอเตอร์แต่ละเฟส
        ACALL   DELAY           ; หน่วงเวลา
        SJMP    AGAIN           ; ทำลูปซ้ำ
```



รูปที่11.9 หารต่อ 8255 เข้ากับ stepper motor



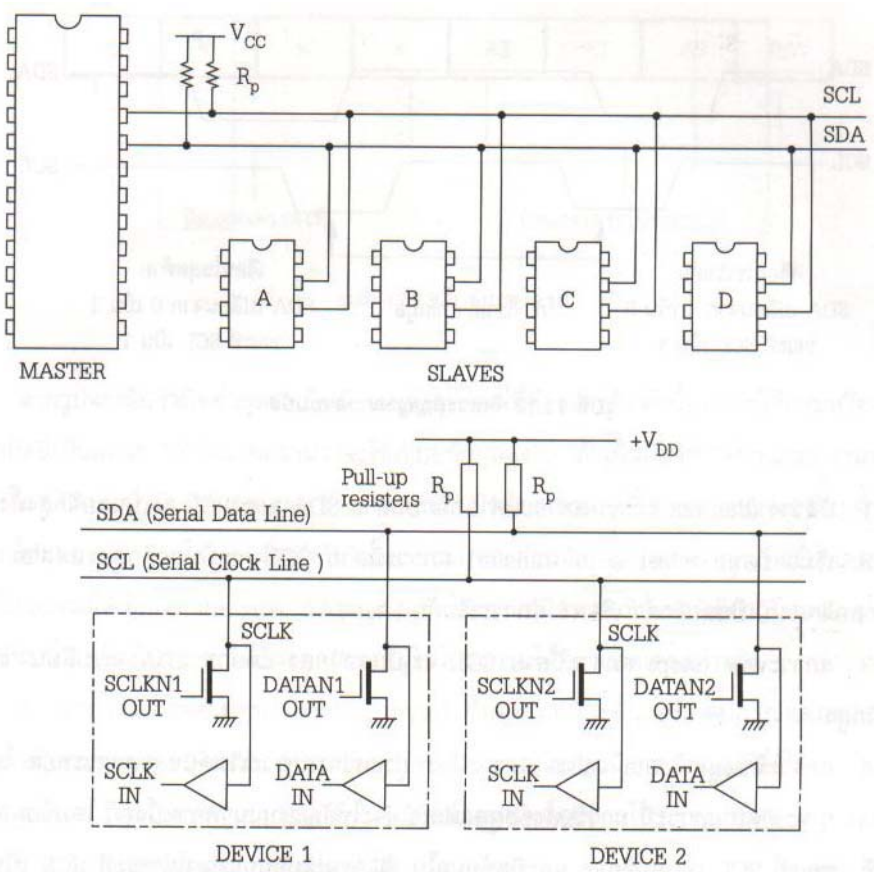
รูปที่11.10 การต่อADC804 เข้ากับ 8255

ไอซีใช้ปลี่ยนสัญญาณแอนะล็อกเป็นดิจิตอล (ADC) ตัวหนึ่งที่ใช้งานง่ายและมีราคาถูกคือเบอร์ ADC804 เมื่อวงจรภายในตัวไอซีสามารถแปลงสัญญาณแอนะล็อกเป็นดิจิตอลได้สำเร็จมันจะส่งสัญญาณ INTR เป็นลอจิก “0” ออกมา สำหรับระบบที่ต้องการแปลงสัญญาณแบบแอนะล็อกเป็นดิจิตอลนั้นสามารถนำไอซีเบอร์นี้มาต่อกับไอซีพอรีต 8255 ได้เช่นกัน ดังตัวอย่างวงจรในรูปที่ 11.10 โดยที่ ADC 804 จะรับสัญญาณแอนะล็อกเข้ามาแปลงเป็นสัญญาณดิจิตอลขนาด 8 บิต และส่งให้ทางพอร์ต PA ของ 8255 โดยใช้บิตต่ำสุด ของพอร์ต PC เป็นตัวตรวจสอบสัญญาณ INTR เมื่อระบบไมโครคอนโทรลเลอร์ต้องการอ่านข้อมูลนี้จะโปรแกรมให้ 8255 ทำงานในโหมด 0 และอ่านข้อมูลเข้ามาในระบบตามต้องการ โดยเขียนคำสั่งได้ดังนี้

```
        MOV     A, #80H          ; กำหนดไบตีควบคุมให้ PA เป็นเอาต์พุต PC เป็นอินพุต
        MOV     R1, #CRPORT      ; กำหนดแอดเดรสของพอร์ตควบคุม
        MOVX    @R1, #C_PORT     ; โปรแกรมให้ PA เป็นเอาต์พุต PC เป็นอินพุต
BACK:   MOVX    A, @R1           ; อ่านข้อมูลจากพอร์ต PC
        ANL     A, #00000001B    ; ตรวจสอบบิตต่ำสุดที่ต่ออยู่กับ INTR
        JNZ     BACK             ; วนลูปจนกว่าสัญญาณจาก INTR เป็นลอจิก "0"
; รอจนกว่าวงจรแปลงสัญญาณ ADC แปลงสัญญาณได้สำเร็จ
        MOV     R1, #A_PORT      ; กำหนดค่าแอดเดรสของพอร์ต PA
        MOVX    A, @R1           ; อ่านข้อมูลแอนะล็อกที่แปลงได้เข้ามาสู่ MCS -51
```

11.3ระบบบัสแบบ I² C และการเชื่อต่อ

จากการต่อวงจรขยายพอร์ตที่ผ่านมาจะเห็นว่าการต่อบัสข้อมูลของ MCS -51 กับชิปที่ใช้เป็นพอร์ตจะต้องใช้บัสข้อมูลแบบขนาน ซึ่งจะทำให้ต้องใช้สัญญาณในการเชื่อมต่อเป็นจำนวนมาก ปัจจุบันได้มีการพัฒนาวิธีการเชื่อมต่อชิปไอซีต่าง ๆ เข้าด้วยกันโดยใช้การสื่อสารแบบพอร์ตอนุกรม ซึ่งจะทำให้ช่วยลดสายสัญญาณที่ใช้ในการสื่อสารข้อมูลลงได้มาก ระบบบัสแบบหนึ่งที่นิยมใช้กันคือระบบบัสแบบ I² C ซึ่งย่อมาจาก คำว่า Inter – IC Communication ที่พัฒนาโดดยบริษัท Philips สำหรับใช้ในการเชื่อต่อชิปต่าง ๆ กับระบบไมโครคอนโทรลเลอร์โดยใช้สัญญาณแบบอนุกรมเพียงสองเส้น โดยเส้นหนึ่งเป็นสายข้อมูลมีชื่อว่า Serial Data Line (SDA) อีกเส้นหนึ่งเป็นสายสัญญาณนาฬิกาสำหรับกำหนดจังหวะการสื่อสารข้อมูลมีชื่อว่า Serial Clock Line (SCL) ตัวอย่างการเชื่อมต่อแสดงได้ดังรูปที่ 11.11

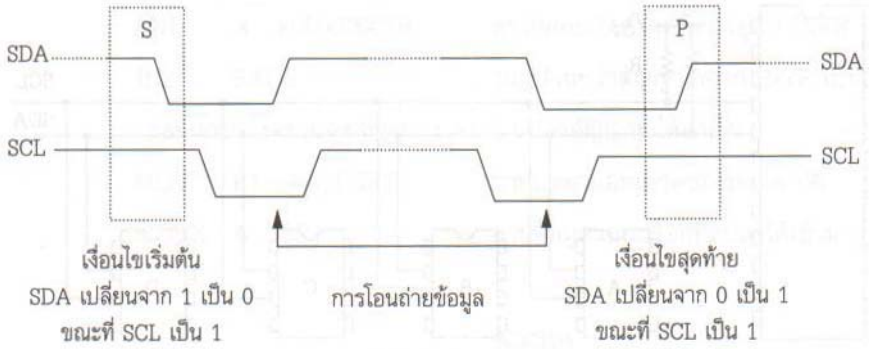


รูปที่11.11 การเชื่อมต่อด้วยระบบบัส I² C และวงจรทางเอาต์พุต

สายสัญญาณสองเส้นที่เชื่อมต่อกันสามารถต่ออุปการณ์ได้หลายตัว โดยแต่ละตัวจะมีแอดเดรสของมันเอง ระบบบัสแบบนี้สามารถส่งข้อมูลสองทิศทางด้วยความเร็วสูงถึง 100 กิโลบิตต่อวินาที และสามารถใช้กับไอซีที่ใช้แรงดันไฟฟ้าต่างกันได้ การติดต่อกับอุปกรณ์แต่ละตัวจะมีรูปแบบการรับส่งข้อมูลหรือตัวโพรโตคอล (protocol) ที่อุปกรณ์ทุกตัวรู้จัก ถ้าหากต้องการให้อุปกรณ์ตัวใดรับข้อมูล ตัวส่งจะส่งแอดเดรสของอุปกรณ์ตัวนั้นไปก่อน ถ้าหากอุปกรณ์ตัวใดมีแอดเดรสตรงกันก็รับข้อมูลนั้นไป สำหรับตัวอุปกรณ์ที่ต้องการส่งข้อมูลจะเรียกว่ามาสเตอร์ (master) ซึ่งจะเป็นตัวที่สร้างจังหวะสัญญาณต่าง ๆ บนระบบบัส ส่วนอุปกรณ์ที่ถูกควบคุมหรือเป็นตัวรับข้อมูลจะเรียกว่าสเลฟ (slave)

ถ้าหากต้องการให้ไมโครคอนโทรลเลอร์เป็นตัวส่งข้อมูล ตัวมันจะทำหน้าที่เป็นมาสเตอร์ และจะส่งสัญญาณไปบนบัส โดยจะส่งค่าแอแอดเดรสออกไปก่อน ถ้าแอดเดรสนี้ตรงกับ แอดเรสของชิปตัวใด ชิปตัวนั้นจะส่งสัญญาณตอบรับ (ACK) ออกมา และชิปตัวนั้นก็จะถูกไมโครคอนโทรลเลอร์ควบคุม และถ้าค่าแอดเดรสไม่ตรงกับชิปตัวใดก็จะไม่มีสิ่งใดเกิดขึ้น

การส่งข้อมูลต่าง ๆ จะต้องเกิดขึ้นเมื่อระบบบัสว่างเท่านั้น โดยทั่วไปแล้วสภาวะที่มีบนระบบบัสจะมีอยู่ 5 สภาสภาวะ ดังรูปที่ 11.12



รูปที่11.12 จังหวะสัญญาณเวลาบนบัส

1. บัสว่าง (bus not busy) สภาวะนี้ค่าลอจิกบนสาย SDA และ SCL จะเป็นลอจิกสูงทั้งคู่

2. เริ่มส่งข้อมูล (start data transfer) สภาวะนี้สาย SCL จะเป็นลอจิกสูง แต่สาย SDA จะเปลี่ยนจากลลอจิกสูง ไปเป็นลอจิกต่ำ เรียกว่าสภาวะเริ่มต้น (start)

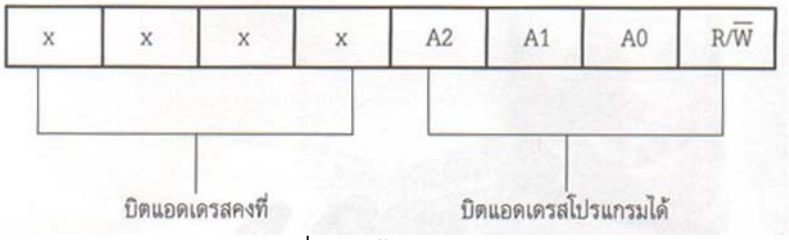
3. สภาวะหยุด (stop) สภาวะนี้สาย SCL จะเป็นลอจิกสูง แต่สาย SDA จะเปลี่ยนจากลอจิกต่ำไปลอจิกสูง

4. สภาวะมีข้อมูล (data valid) สภาวะนี้จะอยู่ระหว่างสภาวะเริ่มต้นและสภาวะหยุด โดยการรับส่งข้อมูลต่าง ๆจะเกิดในสภาวะนี้ การรับส่งข้อมูลในแต่ละบิตจะใช้สัญญาณนาฬิกาหนึ่งลูก โดยข้อมูลบน SDA จะต้องคงที่ ขณะที่ SCL เป็นลอจิกสูง และบิตข้อมูลใน SDA จะเปลี่ยนแปลงได้ขณะที่ SCL เป็นลอจิกต่ำ ถ้าหากบิตบน SDA มีการเปลี่ยนแปลงขณะที่ SCL เป็นลอจิกสูงระบบจะตีความว่าเป็นสภาวะเริ่มต้นส่งข้อมูลหรือสภาวะหยุดแทน

5. สภาวะตอบรับ (acknowledge) เมื่ออุปกรณ์มาสเตอร์ส่งข้อมูลออกมาครบหนึ่งไบต์แล้วในช่วงสัญญาณSCL ลูกที่ 9 มาสเตอร์จะส่งข้อมูลลอจิกสูงออกมา และถ้าตัวรับได้รับข้อมูลครบแล้วมันจะส่งสัญญาณตอบรับ ACK โดยทำให้ระดับลอจิกสูงบนสัญญาณ SDA ให้กลับเป็นระดับลอจิกต่ำ แต่ถ้าตัวรับได้รับข้อมูลไม่ถูกต้องตัวรับจะบังคับให้ตัวส่งหยุดในสภาวะรอ

ในการโอนถ่ายข้อมูลระหว่างสภาวะเริ่มและสภาวะหยุดสามารถโอนถ่ายข้อมูลได้ไม่จำกัดแต่เมื่อสเลฟได้รับข้อมูลแต่ละไบต์แล้ว มันจะส่งสัญญาณกลับมาทุกครั้ง

การเชื่อมต่ออุปกรณ์ต่าง ๆ ที่ใช้ระบบบัสแบบ I²C นั้นเราสามารถใช้อุปการณ์หลาย ๆ ตัวมาต่อกับไมโครคอนโทรลเลอร์ได้ แต่จะใช้สายสัญญาณทั้งหมดเพียงสองเส้นเท่านั้น ดังนั้นการติดต่อกับอุปกรณ์แต่ละตัวจะต้องระบุด้วยว่าจะต้องการติดต่อกับอุปกรณีตัวใด ขั้นตอนการเขียนข้อมูลที่ใช้ระบบบัส I²C มีสามขั้นตอนดังนี้



รูปที่11.13 ไบต์แอดเดรส

1. เขียนข้อมูลอ้างแอดเดรส (addressing) ข้อมูลไบต์แรกที่มาสเตอรีจะส่งออกไปคือข้อมูลที่ใช้อ้างแอดเดรสของอุปกรณ์ที่ต้องการติดต่อ โดยทั่วไปแล้วข้อมูลที่ใช้กำหนดแอดเดรสจะมีจำนวน 7 บิต ดังรูปที่ 11.13

จากรูปจะเห็นว่าบิตต่ำสุดจะเป้นตัวระบุว่าอุปกรณีที่ต้องการติดต่อนั้นเราจะใช้อ่านหรือเขียนข้อมูลลงไป ถ้าบิตนี้เป็นลอจิก “0” หมายความว่าจะใช้เขียนข้อมูลลงไป ถ้าเป้นลอจิก “1” หมายความว่าจะใช้อ่านข้อมูล ส่วน 7 บิตบนจะเป็นบิตแอดเดรส ซึ่งแบ่งออกเป้นบิตแอดเดรสคงที่
(Fixed address bit) ในกรณีที่มีการต่อชิปเบอร์เดียวกันหลาย ๆตัวในระบบบัสแบบ I²C นี้ จะต้องโปรแกรมให้บิตแอดเดรสที่ผู้ใช้โปรแกรมได้มีค่าแอดเดรสต่างกัน

2. การเขียนไบต์ควบคุม (Control byte) ข้อมูลไบต์นี้จะขึ้นอยู่กับอุปกรณ์แต่ละประเภท เพื่อกำหนดการทำงานต่าง ๆ ของตัวมัน อุปกรณ์บางประเภทอาจจะไม่ต้องมีการเขียนไบต์นี้ก็ได้

3. การเขียนไบต์ข้อมูล (Data byte) ที่จะโอนถ่ายในระบบ ระหว่างสภาวะเริ่มต้นและสภาวะหยุดจะมีการเขียนข้อมูลจำนวนกี่ไบต์ก็ได้